Manuel Babet

Table des matières

Babet — Manuel utilisateur **14**
- Pour démarrer . . . 14
- Trouver une fonctionnalité . . . 15
- Modules de référence . . . 17
- Organisation d'une page de module . . . 19
- Génération du PDF . . . 19
- Méthodologie de vérification . . . 19

Pour démarrer **19**
- Téléchargement . . . 19
- Compilation depuis les sources . . . 19
- Trois manières de lancer un script Lua . . . 20
  - 1. Mode dossier . . . 20
  - 2. Mode embarqué (binaire autonome) . . . 20
  - 3. Embarqué via PATH (dossier + auto-détection) . . . 21
- Invocation en ligne de commande . . . 21
- Premier script . . . 22
- Pour aller plus loin . . . 22

Modèle de sécurité **22**
- Ce contre quoi le durcissement *protège* . . . 22
- Ce contre quoi il *ne protège pas* . . . 24
- Pattern recommandé : moindre privilège . . . 24

Archive — opérations ZIP et TAR sécurisées avec gzip, xz, bzip2 ou zstd **24**
- Périmètre . . . 24
- Table des matières . . . 25
- `babet.archive.create` . . . 25
  - Format de sortie . . . 26
  - Filtres d'inclusion et d'exclusion . . . 27
- Options d'inspection/extraction et limites anti-bombe . . . 30
  - Un exemple par limite . . . 31
- `babet.archive.list` . . . 32
  - Structure retournée . . . 33
  - Champs communs à chaque entrée . . . 34
  - Champs propres au ZIP . . . 34
  - Noms binaires, chemins dangereux et normalisation . . . 34
  - Doublons et collisions . . . 35
  - Exemples d'utilisation . . . 35
  - Ce que `list()` valide — et ce qu'elle ne garantit pas . . . 37
- `babet.archive.read` . . . 37
  - Sélection par nom brut ou par index . . . 37
  - Noms dangereux et types refusés . . . 38
  - Limite mémoire `max_size` . . . 39

Intégrité et consommation de l'archive . . . 39
Utilisation dans un worker . . . 40
`babet.archive.test` . . . 40
Vérifications techniques . . . 41
Vérifications de sécurité . . . 41
Structure retournée . . . 41
Exemples d'utilisation . . . 42
`babet.archive.extract` . . . 43
Sélection par globs sûrs . . . 44
Simulation sans écriture avec `dry_run` . . . 45
Validation, entrées ignorées et intégrité . . . 45
Valeur de retour . . . 46
Exemples . . . 47
`babet.archive.extractFile` . . . 49
Règles de chemins . . . 50
Protection de la destination . . . 50
Publication atomique et nettoyage . . . 51
Permissions . . . 51
Types d'entrées pris en charge . . . 52
Erreurs et limites connues . . . 52

`babet.base64` — encodage et décodage Base64 binaires **53**
Table des matières du module . . . 53
API . . . 53
Encodage standard . . . 54
Alphabet URL-safe . . . 54
Padding . . . 55
Produire une sortie sans = . . . 55
Accepter une entrée non paddée . . . 55
Exemple combiné URL-safe sans padding . . . 55
Décodage strict . . . 56
Espaces ASCII . . . 56
Limite de sortie . . . 56
Données binaires et fichiers . . . 57
Utilisation dans un worker . . . 58
Contrat d'erreur . . . 58
Erreurs d'appel : exception Lua . . . 58
Erreurs de données : `nil, err` . . . 58
Limites et choix de conception . . . 59

Compression - flux gzip, xz, bzip2 et zstd autonomes **59**
Périmètre . . . 59
Vue d'ensemble de l'API . . . 59
`compress(source, destination, format [, opts])` . . . 60
Exemple . . . 60
`decompress(source, destination [, opts])` . . . 61
Exemple . . . 61
Flux concaténés et intégrité . . . 61
Garanties sur les fichiers et la publication . . . 61
Données binaires et mémoire . . . 62
Frontière d'exception C++ . . . 62
Workers . . . 62
Exemples d'erreur . . . 62

CURSES — interfaces utilisateur en terminal **63**
Vue d'ensemble de l'API . . . 63
Démarrer et arrêter . . . 63

Opérations d'écran . . . 64
Lecture clavier . . . 64
Processus enfants interactifs . . . 64
Workers . . . 65
Signaux . . . 65
TERM et terminfo . . . 65
Déploiement . . . 65

`argparse` — arguments de ligne de commande **65**
Table des matières du module . . . 66
Chargement et métadonnées . . . 66
API . . . 66
Déclarer des flags et des options . . . 67
Déclarer des positionnels . . . 67
Table `opts` . . . 67
Champs communs . . . 68
Champs réservés aux valeurs . . . 68
Destination . . . 68
Source des arguments . . . 68
Table globale `arg` . . . 68
Array explicite . . . 69
Formes reconnues . . . 69
Valeurs par défaut, choix et conversion . . . 69
Résultats de `parse` . . . 70
Succès normal . . . 70
Aide intégrée . . . 70
Erreur utilisateur . . . 70
Erreur de programmation . . . 71
Exemple complet . . . 71
Texte produit par `get_usage` . . . 71
Limites de la version actuelle . . . 71

EXEC - programmes externes, capture et streaming **72**
Table des matières du module . . . 72
Conventions essentielles . . . 73
Aucun shell implicite . . . 73
Une sortie non nulle n'est pas une erreur de lancement . . . 73
Les chaînes de données sont binaires . . . 73
Vue d'ensemble de l'API . . . 74
Signature et arguments . . . 74
`cmd` et recherche du programme . . . 74
`args` et absence de shell . . . 75
Utiliser explicitement un shell . . . 75
Options . . . 76
`opts.cwd` . . . 76
`opts.env` . . . 76
`opts.stdin` . . . 77
`opts.timeout` . . . 77
`opts.max_output` . . . 77
Table de résultat . . . 78
Code de sortie normal . . . 78
Fin par signal . . . 78
Timeout . . . 79
Troncature . . . 79
Exemples complets . . . 79
Capture simple . . . 79
Arguments contenant des espaces . . . 79

Traiter un code non nul . . . 79
Capturer `stderr` . . . 80
Envoyer du texte ou du binaire sur stdin . . . 80
Modifier l'environnement . . . 80
Changer de répertoire . . . 80
Borner le temps . . . 81
Borner la sortie . . . 81
Créer un helper applicatif . . . 81
Processus pilotables avec `babet.spawn` . . . 82
Signature et options . . . 82
Modes de redirection . . . 83
Méthodes du processus . . . 86
Lire stdout et stderr . . . 86
Écrire sur stdin . . . 87
Attendre, terminer et fermer . . . 87
Exemple combiné : lancer un WebDriver ou un daemon . . . 87
Exemple combiné : boucle de streaming . . . 88
Limites et prévention des blocages . . . 89
Contrat d'erreur . . . 89
Erreurs Lua levées . . . 89
Erreurs renvoyées (`nil, err`) . . . 89
Situations renvoyant une table . . . 89
Processus, signaux et sécurité . . . 90
Groupe de processus . . . 90
Masque des signaux de l'enfant . . . 90
Interaction avec `babet.signal` . . . 90
Mémoire . . . 90
Décisions et limites . . . 90

Pipelines de processus **91**
Syntaxe commune des commandes . . . 91

`babet.pipeline()` — exécution complète **91**
Signature . . . 91
Options . . . 92
Résultat . . . 92
Timeout, gros volumes et troncature . . . 93

`babet.spawnPipeline()` — streaming **93**
Signature . . . 93
Méthodes de lecture . . . 94
Écriture et fermeture de stdin . . . 94
État et identifiants . . . 95
Attente et résultat final . . . 95
Arrêt explicite . . . 95
`close()`, GC et Lua `<close>` . . . 96
Erreurs de lancement et réutilisation . . . 96

FS — fichiers, dossiers, chemins, recherche, copie, permissions et sommes de contrôle **97**
Table des matières du module . . . 97
Conventions générales . . . 98
Chemins . . . 98
Liens symboliques . . . 98
Ordre des résultats . . . 98
Valeurs de retour . . . 98
Vue d'ensemble de l'API . . . 99
Vérifier et inspecter . . . 99

Manipuler des chemins . . . 99
Agir sur le système de fichiers . . . 99
Lister et rechercher . . . 99
Copier et déplacer . . . 100
Modifier les attributs . . . 100
Existence, type et taille . . . 100
`babet.fileExists(path)` . . . 100
`babet.isFile(path)` et `babet.isfile(path)` . . . 100
`babet.isDir(path)` et `babet.isdir(path)` . . . 100
`babet.fileSize(path)` . . . 101
Répertoire courant . . . 101
`babet.currentDir()` . . . 101
`babet.chdir(path)` . . . 101
Manipulation lexicale des chemins . . . 101
`babet.getBasename(path)` . . . 102
`babet.getFilename(path)` . . . 102
`babet.getExtension(path)` . . . 102
`babet.getPath(path)` . . . 102
`babet.joinPath(...)` . . . 102
Créer, supprimer, renommer et lier . . . 103
`babet.touch(path)` . . . 103
`babet.mkdir(path)` . . . 103
`babet.remove(path)` . . . 103
`babet.rmdir(path)` . . . 104
`babet.rmdirAll(path)` . . . 104
`babet.rename(source, destination)` . . . 104
`babet.link(target, linkpath)` . . . 104
Lister, itérer et rechercher . . . 105
`babet.listFiles(path, recursive?)` . . . 105
`babet.createFileIterator(path, recursive?)` . . . 106
`babet.find(path, opts?)` . . . 107
Copier et déplacer . . . 110
`babet.copy(source, destination)` . . . 110
`babet.copyTree(source, destination, continue_on_error?)` . . . 110
`babet.moveTree(source, destination)` . . . 111
Permissions et attributs Unix . . . 112
`babet.getMode(path)` et `babet.setMode(path, mode)` . . . 112
`babet.getAttributes(path)` . . . 113
`babet.setAttributes(path, uid, gid, mode?)` . . . 113
`babet.symlinkAttr(path, uid, gid)` et `babet.symlinkattr(...)` . . . 114
Sommes de contrôle de fichiers . . . 114
CRC-32 en mémoire . . . 115
`babet.crc32(data)` . . . 115
Contrat d'erreur . . . 115
Erreurs Lua levées . . . 115
Erreurs renvoyées . . . 115
Opérations partielles . . . 115
Décisions et limites . . . 115

GUI — interface graphique GTK 4 optionnelle **116**
Disponibilité et initialisation . . . 116
API minimale des widgets . . . 117
Entry : saisie de texte sur une ligne . . . 118
DrawingArea : courbes natives et sélection au clic . . . 119
Conteneurs actualisables : Box et ScrolledWindow . . . 121
SpinButton : saisie numérique . . . 121
Calendar : choix et modification des dates . . . 122

Style CSS . . . 123
Propriétés communes des widgets . . . 123
Boucle événementielle et callbacks . . . 124
Durée de vie . . . 124
`--create-exe` . . . 124
Compléments GTK4 — canevas de notes libres (2.29.0) . . . 124
Interaction native sur les progressions et réorganisation d'une liste . . . 126
`progressBar:onClick(fonction_ou_nil)` . . . 126
`box:reorder(child, previousChild_or_nil)` . . . 126
Drag-and-drop GTK4 natif pour les listes . . . 127

`writeFileAtomic` — écriture atomique et durable d'un fichier 128
Sommaire . . . 128
API . . . 128
Valeurs par défaut . . . 129
Écriture simple . . . 129
Remplacement explicite . . . 129
Permissions . . . 130
Durabilité . . . 130
Données binaires . . . 130
Chemins et liens symboliques . . . 131
Atomicité et concurrence . . . 131
Utilisation dans un worker . . . 131
Exemple combiné Selenium . . . 132
Contrat d'erreur . . . 132
Limites . . . 132

HTTP — requêtes web synchrones, headers, corps, TLS et limites 133
Table des matières du module . . . 133
Conventions essentielles . . . 134
Client synchrone . . . 134
Statut HTTP et erreur de transport . . . 134
Corps binaires . . . 134
Options inconnues . . . 134
Vue d'ensemble de l'API . . . 134
Table d'options . . . 134
`babet.http.request(opts)` . . . 135
`babet.http.get(url, opts?)` . . . 135
`babet.http.download(url, destination, opts?)` . . . 136
`babet.http.post` et ses formes d'appel . . . 137
Sans corps . . . 137
Corps positionnel . . . 137
Corps dans `opts` . . . 137
Priorité . . . 137
URL, fragments et query . . . 137
URL absolue . . . 137
Ajouter une query . . . 137
Méthodes HTTP . . . 138
Headers de requête . . . 138
Noms . . . 139
Valeurs . . . 139
Doublons . . . 139
Casse . . . 139
Corps de requête et `Content-Type` . . . 139
Corps absent et corps vide . . . 139
JSON . . . 139
Formulaire URL-encoded . . . 140

Multipart . . . 140
`opts.timeout` . . . 140
Limite DNS . . . 140
Toujours en fournir un . . . 140
HTTPS, vérification et CA . . . 140
Vérification par défaut . . . 140
CA spécifique . . . 141
Désactiver la vérification . . . 141
`follow_redirects` . . . 141
`max_body_size` . . . 142
`max_file_size` et sécurité du fichier . . . 142
Tables de réponse . . . 142
Réponse en mémoire . . . 142
`status` . . . 143
`body` . . . 143
`headers` . . . 143
`headers_multi` . . . 143
Résultat d'un téléchargement . . . 143
Exemples complets . . . 143
GET JSON . . . 143
POST JSON authentifié . . . 144
PUT binaire . . . 144
Télécharger un artefact de release . . . 144
Gérer 404 séparément des erreurs réseau . . . 145
Requêtes parallèles avec WORKERS . . . 145
Contrat d'erreur . . . 145
Erreurs Lua levées . . . 145
Erreurs `(nil, err)` . . . 146
Pas une erreur Babet . . . 146
Sécurité et limites . . . 146

`babet.inotify` — surveillance du système de fichiers **146**
Table des matières du module . . . 147
API . . . 147
Créer un watcher . . . 147
Ajouter une surveillance . . . 147
Option `onlydir` . . . 148
Lire les événements . . . 148
Débordement de la file . . . 149
Interruption par signal . . . 149
Retirer et fermer . . . 149
Exemple . . . 149
Contrat d'erreur . . . 150
Limites . . . 150

`babet.json` — encodage et décodage JSON **150**
Table des matières du module . . . 150
API . . . 150
Correspondance des types . . . 151
Lua vers JSON . . . 151
JSON vers Lua . . . 151
`null` . . . 152
Objet vide, tableau vide et `as_array` . . . 152
Options de `encode` . . . 152
Chaînes, UTF-8 et octets NUL . . . 153
Nombres . . . 153
Ordre des objets et clés dupliquées . . . 153

Contrat d'erreur . . . 153
Limites . . . 154

`logging` — journalisation à niveaux **154**
Table des matières du module . . . 154
Émettre un message . . . 154
Garantie de non-propagation . . . 155
Format de sortie . . . 155
Seuil . . . 155
Destination . . . 155
Couleurs ANSI . . . 156
État du module . . . 156
Contrat d'erreur . . . 157
Limites . . . 157

SIGNAL — arrêt propre, rechargement et signaux POSIX **157**
Table des matières du module . . . 157
Conventions essentielles . . . 158
Les succès renvoient une seule valeur . . . 158
Les noms sont stricts . . . 158
Un callback ne reçoit aucun argument . . . 158
Vue d'ensemble de l'API . . . 158
Signaux pris en charge . . . 159
`handle(name, fn)` — installer ou remplacer . . . 159
`handle(name, nil)` — retirer un callback . . . 160
`ignore(name)` — ignorer . . . 160
`default(name)` — restaurer le comportement système . . . 160
Exécution des callbacks . . . 160
Thread principal et contexte sûr . . . 161
Ordre de dispatch . . . 161
Coalescence des occurrences . . . 161
Erreurs du callback . . . 161
Interaction avec les appels bloquants . . . 162
Interaction avec `debug.sethook` . . . 162
Exemples complets . . . 163
Arrêt propre sur `TERM` et `INT` . . . 163
Recharger une configuration avec `HUP` . . . 163
Interrompre une attente socket . . . 164
Ignorer temporairement `PIPE` . . . 164
Observer les erreurs du callback . . . 164
Contrat d'erreur . . . 165
Erreurs Lua levées . . . 165
Erreur système renvoyée . . . 165
Workers, concurrence et limites . . . 165
Fonctionnalités absentes . . . 166

SOCKET — TCP et sockets Unix, flux binaires, lignes et timeouts **166**
Table des matières du module . . . 167
Conventions essentielles . . . 167
Flux synchrones et bloquants . . . 167
Chaînes binaires . . . 167
Timeouts . . . 167
Valeurs de retour . . . 168
Vue d'ensemble de l'API . . . 168
Constructeurs . . . 168
Méthodes communes . . . 168
Méthode d'un socket d'écoute . . . 169

`babet.socket.connect(host, port, timeout?)` . . . 169
Résolution et adresses multiples . . . 169
Timeout de connexion non conservé . . . 169
Erreurs typiques . . . 169
`babet.socket.listen(host, port, backlog?)` . . . 169
Écouter uniquement en loopback . . . 169
Écouter sur toutes les interfaces . . . 170
Demander un port libre au noyau . . . 170
Backlog . . . 170
`babet.socket.connect_unix(path, timeout?)` . . . 170
`babet.socket.listen_unix(path, opts?)` . . . 170
`server:accept(timeout?)` . . . 171
`sock:set_timeout(seconds)` . . . 171
`sock:send(data)` . . . 171
`sock:recv(count, timeout?)` . . . 172
Lire exactement N octets . . . 172
EOF et timeout . . . 172
`sock:recv_line(timeout?)` . . . 173
Limite de ligne . . . 173
EOF avant LF . . . 173
Timeout avant LF . . . 173
`sock:recv_all(timeout?, max_bytes?)` . . . 173
Limite mémoire . . . 174
Mélanger `recv`, `recv_line` et `recv_all` . . . 174
`sock:peer()` et `sock:sockname()` . . . 174
`sock:close()` et nettoyage automatique . . . 175
Signaux et interruptions . . . 175
Exemples complets . . . 175
Petit serveur ligne par ligne . . . 175
Petit service local par socket Unix . . . 176
Client avec longueur préfixée . . . 176
Lire une réponse délimitée par EOF . . . 177
Contrat d'erreur . . . 177
Erreurs Lua levées . . . 177
Erreurs renvoyées . . . 177
Sécurité, performances et limites . . . 177

WEBSOCKET — client RFC 6455 pour `ws://` et `wss://` 178
Vue d'ensemble de l'API . . . 178
`babet.websocket.connect(url, opts?)` . . . 179
Options . . . 179
Exemples des options . . . 179
Négociation d'ouverture . . . 181
Envoyer du texte . . . 181
Envoyer du binaire . . . 181
Recevoir des messages . . . 181
Ping et Pong . . . 182
Fermeture . . . 182
Erreurs de protocole . . . 182
Exemple WebDriver BiDi . . . 182
Concurrence et boucles graphiques . . . 183
Sécurité et limites . . . 183

`babet.sqlite` — base de données SQL embarquée 183
Table des matières du module . . . 183
API . . . 184
Ouverture et fermeture . . . 184

Compteurs de connexion . . . 185
Sauvegarde cohérente . . . 186
Exécution directe . . . 188
Statements préparés réutilisables . . . 189
Transactions assistées . . . 190
Savepoints imbriqués . . . 193
Paramètres SQL . . . 195
Valeurs NULL explicites . . . 196
BLOB explicites . . . 197
Texte SQL et octets NUL . . . 197
Lecture des lignes et mapping de types . . . 197
Cycle de vie . . . 198
Contrat d'erreur . . . 198
Exemples complets . . . 199
Inserts préparés, NULL optionnel et BLOB . . . 199
Requête réutilisable et transaction . . . 199
Fonctions non exposées . . . 200

`babet` strings — découpage de chaînes 200
Table des matières du module . . . 200
API . . . 200
Mode séparateur . . . 201
Limiter le nombre de coupes . . . 201
Mode octets . . . 201
Chaînes binaires et octets NUL . . . 202
Chaîne vide . . . 202
Contrat d'erreur . . . 202
Limites et choix de conception . . . 203

SYS - processus, machine, environnement, exécutables, version et mémoire Lua 203
Table des matières du module . . . 203
Conventions générales . . . 204
Fonctions plates . . . 204
Chaînes strictes . . . 204
État global du processus . . . 204
Vue d'ensemble de l'API . . . 204
Constantes de version . . . 205
Identifier le processus et la machine . . . 205
`babet.pid()` . . . 205
`babet.hostname()` . . . 206
`babet.uname()` . . . 206
Rechercher un exécutable . . . 206
Recherche dans `PATH` . . . 207
Chemin direct . . . 207
Programme absent ou non exécutable . . . 207
Variables d'environnement . . . 208
`babet.env(name)` . . . 208
`babet.setenv(name, value)` . . . 208
Valeur vide et variable absente . . . 208
Interaction avec les workers . . . 209
Mémoire de la VM Lua . . . 209
`babet.getMemoryUsage()` . . . 209
`babet.getDetailedMemoryUsage()` . . . 210
Ce qui est mesuré et ce qui ne l'est pas . . . 210
Contrat d'erreur . . . 210
Décisions et limites . . . 211

babet tables — fusion et copie de tables Lua **211**
Table des matières du module . . . 211
API . . . 212
babet.mergeTables(t1, t2, ...) . . . 212
Clés entières positives : concaténation . . . 212
Toutes les autres clés : dernier écrivain gagnant . . . 212
Fusion superficielle . . . 212
babet.deepCopyTable(t) . . . 213
Valeurs et graphe partagé . . . 213
Les clés ne sont pas copiées . . . 213
Métatables et accès brut . . . 213
Profondeur maximale . . . 214
Contrat d'erreur . . . 214
Limites et choix de conception . . . 214
Hors v1 . . . 214

babet time - horloges, sommeil, dates et durées **214**
Table des matières du module . . . 215
API . . . 215
monotonic() et now() . . . 215
sleep(amount [, unit]) . . . 215
babet.time.iso([ts]) . . . 216
babet.time.parse_iso(text) . . . 216
babet.time.parse_duration(text) . . . 217
babet.time.format_duration(seconds) . . . 218
Contrat d'erreur récapitulatif . . . 218
Limites et choix de conception . . . 218

TLS — connexion chiffrée directe, STARTTLS, certificats et SNI **218**
Table des matières du module . . . 219
Conventions essentielles . . . 219
Vérification activée par défaut . . . 219
Même API de flux après le handshake . . . 219
SNI n'est pas la vérification . . . 220
Vue d'ensemble de l'API . . . 220
Options TLS détaillées . . . 220
verify . . . 220
hostname . . . 220
ca_cert et ca_path . . . 221
min_version . . . 221
timeout . . . 221
babet.socket.connect_tls(host, port, opts?) . . . 221
Hostname par défaut . . . 221
Deadline globale . . . 222
Timeout d'I/O après succès . . . 222
sock:starttls(opts?) . . . 222
Préconditions . . . 222
Hostname obligatoire avec vérification . . . 222
Consommer le plaintext avant l'upgrade . . . 223
Déroulement applicatif . . . 223
Vérification, hostname et SNI . . . 223
Autorités de certification . . . 223
CA supplémentaire, pas certificat serveur piné . . . 224
Versions TLS . . . 224
Timeouts . . . 224
connect_tls . . . 224
starttls . . . 224

I/O après le handshake . . . 224
État du socket après une erreur . . . 225
`connect_tls` . . . 225
`starttls` avant le handshake . . . 225
`starttls` après le début du handshake . . . 225
`send` après une tentative d'écriture TLS . . . 225
Exemples complets . . . 225
Client HTTPS minimal sur socket brut . . . 225
Connexion par IP avec nom de certificat . . . 226
SMTP STARTTLS simplifié . . . 226
Test local auto-signé . . . 226
Contrat d'erreur . . . 227
Erreurs Lua levées . . . 227
Erreurs `(nil, err)` . . . 227
Sécurité et limites . . . 227

`babet.toml` - décodage TOML 227
Table des matières du module . . . 228
API . . . 228
Correspondance des types . . . 228
Entiers et flottants . . . 228
Chaînes, UTF-8 et clés . . . 229
Arrays et tables . . . 229
Ambiguïté des conteneurs vides . . . 230
Dates et heures . . . 230
Ordre et informations perdues . . . 230
Contrat d'erreur . . . 231
Limites . . . 231
Non exposé en v1 . . . 231

USER - rechercher des comptes système par nom ou UID 231
Table des matières du module . . . 231
Conventions générales . . . 232
Sous-table dédiée . . . 232
Nom ou UID, sans conversion implicite . . . 232
Champs toujours présents . . . 232
Vue d'ensemble de l'API . . . 232
Table utilisateur renvoyée . . . 233
Rechercher un compte avec `get` . . . 233
Recherche par nom . . . 233
Recherche par UID . . . 234
Compte absent . . . 234
Tester l'existence avec `exists` . . . 234
Par nom . . . 235
Par UID . . . 235
Quand préférer `get` . . . 235
Validation des arguments . . . 235
NSS, workers et sécurité . . . 236
Sources de comptes . . . 236
Appels depuis des workers . . . 236
Attention aux décisions de sécurité . . . 236
Contrat d'erreur . . . 237
Décisions et limites . . . 237

WORKERS — threads OS, états Lua isolés, queues et channels 237
Table des matières du module . . . 238
Conventions essentielles . . . 239

Chaque worker possède un état Lua distinct . . . 239
Les résultats utilisent une convention proche de `pcall` . . . 239
Le résultat final est consommable une seule fois . . . 240
Vue d'ensemble de l'API . . . 240
Côté parent . . . 240
Côté worker . . . 241
`spawn(code, args?, opts?)` . . . 241
`code` . . . 241
`args` et `worker.args` . . . 242
Capacités des queues . . . 242
Transmission de channels par `opts.channels` . . . 243
Moment où `setenv`, `chdir` et `os.setlocale` sont verrouillés . . . 244
Valeurs transférables . . . 244
Scalaires et chaînes . . . 244
Tables listes et objets . . . 245
Valeurs refusées . . . 245
Copies, identité, profondeur et budgets . . . 246
Résultat final du worker . . . 246
`status()` — observer sans consommer . . . 247
`done()` — test booléen non consommant . . . 247
`join(timeout?)` — attendre et consommer . . . 247
`poll()` — tester et consommer . . . 248
Ne pas combiner `poll()` terminé puis `join()` . . . 249
Messages parent vers worker . . . 249
Messages worker vers parent . . . 249
API `worker.send`, `worker.recv` et `worker.cancelled` . . . 250
Channels directs partagés . . . 251
Créer un channel et choisir sa capacité . . . 251
`send` et `recv` . . . 251
`close` et `is_closed` . . . 252
Concurrence, ordre et durée de vie . . . 252
Timeouts et raisons d'échec . . . 253
`close()` et cycle de vie des queues . . . 253
`cancel()` et annulation coopérative . . . 254
Chargement des modules et environnement du worker . . . 254
Nombre de CPU disponibles . . . 255
Pool borné réutilisable . . . 255
Création et options . . . 256
Soumission et tâches . . . 256
Isolation et réutilisation des états . . . 256
Contre-pression et timeouts . . . 257
Fermeture, annulation et `join` . . . 257
Channels partagés dans les tâches . . . 258
Exemples complets . . . 258
Calcul simple avec `join` . . . 258
Arguments structurés . . . 258
Suivi non bloquant avec `poll` . . . 259
Worker persistant requête/réponse . . . 259
Envoyer et recevoir `nil` . . . 260
Fermeture puis drainage de l'outbox . . . 260
Contre-pression et timeout . . . 260
Parent vers worker avec un channel . . . 261
Worker vers worker sans relais du parent . . . 261
Plusieurs producteurs et consommateurs . . . 262
Annulation coopérative native . . . 263
Pool natif pour plusieurs tâches . . . 264

Interblocages à éviter . . . 265
Worker en attente de l'inbox, parent dans `join()` . . . 265
Worker bloqué sur une outbox pleine . . . 265
Producteurs bloqués sur un channel plein . . . 265
Oublier que `poll()` consomme . . . 265
GC potentiellement bloquant . . . 265
Contrat d'erreur . . . 265
Erreurs Lua levées par mauvais usage . . . 265
Échecs renvoyés par `spawn` . . . 266
Erreurs finales du worker . . . 266
Sérialisation des messages . . . 266
Garbage collector et destruction . . . 266
Signaux, environnement et concurrence . . . 267
Signaux . . . 267
`os.execute` et `io.popen` dans un worker . . . 267
Environnement et cwd . . . 268
Ressources externes . . . 268
Coût . . . 268
Fonctionnalités absentes . . . 268

Cookbook **268**
Surveiller un dossier et envoyer les nouveaux fichiers par email . . . 268
S'assurer qu'un user système existe, puis basculer dessus . . . 269
Récupérer beaucoup d'URLs en parallèle . . . 269
Utiliser WebSocket comme transport WebDriver BiDi . . . 270
Arrêt propre d'un script long-running . . . 271
Lire une config TOML, valider, persister en SQLite . . . 271
Cache avec TTL lisible, échéances en timestamps ISO . . . 272
Pretty-print d'une valeur Lua . . . 272
Hello there . . . 272

Babet — Manuel utilisateur

> *Babet*, n.m. — mot régional du sud-est de la France (Lyonnais, Forez, Dauphiné, Savoie, ainsi que la Suisse romande voisine) désignant une pomme de pin. Petit, léger, plein de graines, et capable d'allumer un feu — comme ce binaire.

Babet est un binaire Lua standalone pour le scripting et l'automatisation sous Linux. Ce manuel est organisé par besoin et par module : la table des matières ci-dessous indique non seulement le nom technique du module, mais aussi les fonctionnalités qu'il contient.

Documentation de **Babet 2.31.0**.

Babet 2.31.0 ajoute le glisser-déposer GTK4 natif avec `setDragData` et `onDrop`, une icône représentant la poignée et des callbacks protégés. Le chapitre GUI décrit les cibles imbriquées et le nettoyage après dépôt.

Babet 2.22.2 ajoute un client RFC 6455 natif via `babet.websocket`, avec `ws://` et `wss://` vérifié, négociation stricte, framing borné, Ping/Pong automatique, fragmentation, fermeture propre et disponibilité dans les workers. Le harnais d'auto-test contient désormais 45 suites atteignables.

Pour démarrer

— `Premiers pas` — installation, premier script, exécution d'un fichier ou d'un dossier, création d'un exécutable embarqué et utilisation via le `PATH`.
— `Sécurité` — modèle de menace, protections réellement fournies, limites et règles de moindre privilège.

— Cookbook — recettes complètes combinant plusieurs modules.
— Journal des modifications — notes complètes de release et historique des versions précédentes, migration, validation et limites connues.

Trouver une fonctionnalité

Je cherche à…	Chapitre à consulter	Ce que j'y trouverai
créer, supprimer, lister, rechercher, copier ou déplacer des fichiers	FS — système de fichiers	fichiers, dossiers, chemins, symlinks, `listFiles`, `find` avec confinement `xdev`, `copyTree`, permissions et checksums
publier atomiquement une configuration ou un fichier binaire	writeFileAtomic — écriture atomique	refus d'écrasement par défaut, permissions, durabilité, confinement des chemins, concurrence et workers
lire les arguments d'un script	Argparse — ligne de commande	flags, options, arguments positionnels, valeurs par défaut, choix et conversions
construire une interface terminal	CURSES — interfaces utilisateur en terminal	affichage UTF-8, clavier symbolique, resize/suspension, handoff des enfants interactifs, workers et fallbacks terminfo autonomes
construire une GUI de bureau optionnelle	GUI — interface graphique GTK 4 optionnelle	chargement paresseux de GTK 4 système, fenêtre/box/label/bouton/entry, DrawingArea, conteneurs actualisables, saisies numérique/date, CSS/classes, glisser-déposer natif, callbacks, durée de vie, boucle événementielle et exception de déploiement `--create-exe`
lancer un programme externe	Exec — processus	arguments sans shell, capture avec `exec`, streaming avec `spawn`, environnement, cwd et contrôle du processus
enchaîner plusieurs commandes	Pipelines de processus	capture complète ou streaming, stderr séparés, statuts individuels et nettoyage des groupes
créer ou inspecter/extraire des archives ZIP ou TAR	Archive — archives multi-formats sécurisées	création déterministe de ZIP et de TAR brut/gzip/xz/bzip2/zstd, listing, test intégral, extraction sélective, simulation `dry_run`, limites anti-bombe et publication atomique des fichiers
encoder ou décoder des données binaires en Base64	Base64 — données binaires	alphabet standard ou URL-safe, padding optionnel, décodage canonique strict, espaces contrôlés et limite de sortie

Je cherche à…	Chapitre à consulter	Ce que j'y trouverai
compresser ou décompresser un fichier	Compression — flux autonomes	flux gzip, xz, bzip2 et zstd, détection par contenu, limites de sortie, membres concaténés, intégrité et publication atomique
manipuler l'environnement, identifier le processus ou mesurer la VM Lua	SYS - processus et machine	version du runtime, PID, hostname, `uname`, `PATH`, `env`, `setenv` et mémoire de l'état Lua
envoyer une requête web	HTTP — client web	GET/POST et autres méthodes, query, headers validés, corps binaires, téléchargement atomique vers fichier, redirections, TLS, timeout et limites de taille
ouvrir une connexion TCP ou un socket Unix local	Socket — TCP et Unix	clients et serveurs TCP/Unix, acceptation, permissions locales, flux binaires, lignes, EOF, timeouts et nettoyage
utiliser WebSocket ou le transport WebDriver BiDi	WebSocket — client RFC 6455	`ws://`, `wss://` vérifié, masquage, fragmentation, Ping/Pong, fermeture, timeouts et limites mémoire strictes
chiffrer une connexion TCP ou faire STARTTLS	TLS — sockets sécurisées	TLS direct, STARTTLS, vérification, CA, hostname, SNI, versions, timeout et état après échec
stocker des données SQL localement	SQLite — base embarquée	ouverture en lecture/écriture ou lecture seule, clés étrangères, compteurs de changements, paramètres stricts, sauvegarde atomique cohérente, valeurs `NULL` et BLOB explicites, statements préparés et transactions assistées
encoder ou décoder du JSON	JSON — données structurées	scalaires, tableaux, objets, `null`, tableaux vides, pretty-print et erreurs
lire un fichier de configuration TOML	TOML — configuration	décodage, types TOML, tableaux, sections, dates et erreurs de parsing
surveiller un dossier	Inotify — événements fichiers	watchers, événements, lectures avec timeout, moves, cookies et fermeture
gérer les signaux Unix	SIGNAL — arrêt propre et rechargement	`TERM`/`INT`/`HUP`/`USR1`/`USR2`/`PIPE`, callbacks différés, ordre fixe, coalescence, appels interruptibles et workers
paralléliser du Lua	WORKERS — threads OS, messages et channels	états Lua isolés, `spawn`, pool persistant borné, `cpu_count`, inbox/outbox et channels directs MPMC

Je cherche à…	Chapitre à consulter	Ce que j'y trouverai
obtenir ou formater le temps	Time — horloges et durées	temps réel, monotone, sleep, ISO-8601, parsing et formatage de durées
rechercher un compte système par nom ou UID	USER - comptes système	NSS, `get`, `exists`, UID, GID principal, GECOS, home, shell et erreurs de résolution
découper ou transformer des chaînes	Strings — chaînes	`split`, séparateurs, limites et chaînes binaires
copier, fusionner ou marquer des tables	Tables — tables Lua	`mergeTables`, `deepCopyTable`, cycles et structures partagées
produire des logs	Logging — journalisation	niveaux, seuil, sortie, couleurs et comportement en cas d'erreur du sink

Modules de référence

Chaque module possède une page autonome sous `modules/`. Les intitulés développés ci-dessous permettent de comprendre leur périmètre sans avoir à ouvrir chaque fichier.

Module	Périmètre détaillé
Archive — opérations ZIP, TAR, TAR gzip, TAR xz, TAR bzip2 et TAR zstd sécurisées	Créer des ZIP, TAR, TAR gzip, TAR xz, TAR bzip2 ou TAR zstd déterministes, les inspecter, tester et extraire sélectivement avec simulation `dry_run`, ressources bornées, chemins confinés et publication atomique fichier par fichier.
Base64 — encodage et décodage binaires	Encoder ou décoder des chaînes Lua binaires avec alphabet standard ou URL-safe, padding contrôlé, validation canonique, espaces optionnels et plafond `max_output`.
Compression — flux gzip, xz, bzip2 et zstd autonomes	Compresser ou décompresser un fichier régulier avec détection automatique, expansion bornée, vérification d'intégrité, refus des symlinks et publication atomique.
Argparse — arguments de ligne de commande	Déclarer des flags, options et positionnels ; générer l'aide ; valider les choix ; convertir les valeurs.
Exec — programmes externes et processus	Exécuter sans shell avec `exec`, ou piloter stdin/stdout/stderr progressivement avec `spawn`.
Pipelines de processus	Relier plusieurs commandes sans shell, en capture complète ou en streaming, avec statuts par étape.
FS — fichiers, dossiers, chemins et attributs	Existence et types, création/suppression, chemins, listings, recherche, itérateur, copie/déplacement d'arbres, symlinks, modes Unix et checksums.
writeFileAtomic — écriture atomique et durable	Publier une chaîne Lua binaire par temporaire privé et renommage atomique, avec écrasement explicite, permissions exactes, `fsync`, confinement des chemins et support des workers.

Module	Périmètre détaillé
HTTP — requêtes web	URL, méthodes, query, headers validés, corps binaires, réponses, téléchargement atomique vers fichier, redirections, vérification TLS, timeout et limites de taille.
Inotify — surveillance du système de fichiers	Ajouter/retirer des watches, lire les événements, gérer timeouts, moves, cookies et fermeture.
JSON — encodage et décodage	Types Lua/JSON, `null`, tableaux vides, marquage de tableaux, indentation, UTF-8, cycles et limites.
Logging — journalisation	Niveaux, filtrage, destination, couleurs et appels variadiques.
SIGNAL — signaux POSIX et arrêt propre	Installer, remplacer ou retirer un callback ; ignorer/restaurer ; ordre de dispatch, coalescence, interruptions et restrictions multithread.
Socket — TCP et sockets Unix	Connexion et écoute TCP/Unix, acceptation, permissions et nettoyage inode-sensible, flux binaires, lectures bloc/ligne/EOF, adresses et timeouts.
WebSocket — client RFC 6455	Connexions clientes `ws://`/`wss://`, validation stricte de l'Upgrade, masquage client, fragmentation texte/binaire, Ping/Pong, Close, TLS, timeouts et plafonds de ressources.
SQLite — base de données embarquée	Connexions en lecture/écriture ou lecture seule, WAL/timeout, clés étrangères, compteurs de lignes, SQL direct, statements préparés réutilisables, BLOB explicites, sauvegardes WAL cohérentes, itération et transactions assistées.
Strings — manipulation de chaînes	Découpage, mode caractères, séparateurs, limite de splits et contenu binaire.
SYS - processus, machine, environnement et mémoire Lua	Constantes de version, PID, hostname, `uname`, recherche d'exécutables, lecture/modification de l'environnement, interaction avec les workers et mémoire de l'état Lua.
Tables — manipulation de tables Lua	Fusion déterministe, clés listes/maps, copie profonde, cycles et partage de sous-tables.
Time — horloges, ISO et durées	Temps réel et monotone, sommeil, formatage/parsing ISO-8601, parsing et rendu de durées.
TLS — connexions chiffrées	Connexion TLS directe, STARTTLS, vérification, CA, hostname, SNI, versions, deadlines et comportement fail-closed.
TOML — fichiers de configuration	Décodage TOML, scalaires, tableaux, tables, tableaux de tables, dates/heures et diagnostics.
USER - utilisateurs système via NSS	Recherche par nom ou UID, existence, distinction absence/erreur NSS, champs passwd, workers et limites de sécurité.
WORKERS — threads OS, files et channels	États Lua isolés, transport JSON, `status`, `done`, pool persistant borné, annulation coopérative, inbox/outbox, channels directs MPMC, fermeture, GC et interblocages.

Organisation d'une page de module

Les pages détaillées sont progressivement alignées sur la structure suivante :

1. **Périmètre** — ce que couvre le module et ce qu'il ne couvre pas ;
2. **Table des matières interne** — accès direct à chaque groupe et fonction ;
3. **Vue d'ensemble de l'API** — signatures et résultats ;
4. **Comportement détaillé** — valeurs par défaut, symlinks, récursivité, limites et effets de bord ;
5. **Exemples** — un exemple par mode ou option importante ;
6. **Contrat d'erreur** — erreurs levées et erreurs renvoyées ;
7. **Décisions et limites** — choix d'API et éléments volontairement absents.

L'objectif est qu'une fonction puisse être utilisée sans lire son code source et sans devoir deviner ses valeurs par défaut.

Génération du PDF

Le manuel peut être exporté en un seul PDF :

```
cd docs
./build_doc.sh
```

L'ordre des chapitres français est défini dans `manual_order_fr.txt`. Le script de génération ajoute ensuite la table des matières du PDF à partir des titres et sous-titres.

Méthodologie de vérification

La documentation est vérifiée module par module selon trois sources :

— l'implémentation C/C++ réellement enregistrée dans `babet` ;
— les tests de non-régression de `examples/main.lua` et `run_tests.sh` ;
— les pages française et anglaise, qui doivent décrire le même contrat.

Une divergence découverte pendant cette vérification est traitée comme une incohérence à corriger, et non comme un détail rédactionnel à masquer.

Pour démarrer

Téléchargement

Des binaires précompilés pour x86_64, aarch64 (RPi4) et armv6l (RPi0) sont disponibles sur la page des releases.

Compilation depuis les sources

Babet embarque toutes ses dépendances (Lua, OpenSSL, SQLite, miniz, libarchive, zlib, XZ Utils/liblzma, bzip2/libbz2, Zstandard/libzstd, RE2, Abseil, nlohmann/json, cpp-httplib, tomlplus-plus), donc les seuls prérequis sur ton système sont :

— un compilateur C++23 (GCC ou Clang récent)
— CMake (≥ 3.22)
— `wget`, `unzip` et `xz`

```
git clone https://github.com/Chipsterjulien/babet.git
cd babet
./build_local.sh        # télécharge les deps et compile
                        # (~5 min au premier lancement, plus vite après)
./run_tests.sh          # harness offline — doit terminer avec 0 FAIL
```

Le script de build télécharge chaque dépendance depuis une source amont épinglée, vérifie son SHA256, puis la compile statiquement. Plusieurs bibliothèques disposent d'un miroir ou d'une URL de secours explicite. Lorsqu'il n'existe aucun miroir, le script indique le nom exact de l'archive à déposer manuellement dans `downloads/` ; le même contrôle SHA256 reste appliqué avant l'extraction.

Le binaire produit est dans `test/babet`.

Trois manières de lancer un script Lua

Babet supporte trois modes de lancement :

1. Mode dossier

Pointe le binaire vers un dossier contenant `main.lua` :

```
echo 'print("hello from Babet")' > main.lua
./test/babet .
```

`require("X")` dans `main.lua` cherche `X.lua` dans le même dossier.

2. Mode embarqué (binaire autonome)

Empaquète un script et ses modules dans un binaire autonome :

```
./test/babet --create-exe . myapp
./myapp        # lance main.lua depuis le ZIP embarqué
```

Les dossiers de métadonnées de contrôle de version `.git`, `.svn` et `.hg` sont exclus automatiquement, à n'importe quelle profondeur.

En interne, Babet ajoute un ZIP à son propre binaire et y lit `main.lua` et les modules Lua empaquetés. La machine cible n'a pas besoin d'une installation séparée de Babet. Les dépendances de la plateforme restent nécessaires ; une application utilisant la GUI requiert aussi GTK4 à l'exécution. Une application générée est un artefact final : un lancement avec `--create-exe` ou `-c` est refusé avant l'exécution de son `main.lua`. Il faut utiliser le binaire Babet original pour créer un autre exécutable.

Origine des modules. Pour un module que `require` n'a pas déjà chargé, `package.preload` est consulté avant le ZIP. Le chargeur embarqué cherche `nom.lua`, puis `nom/init.lua`. Si les deux entrées sont absentes, les chargeurs Lua habituels restent actifs : `package.path` peut fournir un module Lua sur disque et `package.cpath` un module natif compatible avec l'ABI Lua. Une erreur de lecture ou de compilation d'un module trouvé dans le ZIP arrête le chargement ; elle ne déclenche pas ce repli sur disque. Ces règles s'appliquent aussi aux workers des applications générées.

`package.loadlib` reste disponible pour charger explicitement une bibliothèque native sur disque. Cette API Lua est distincte de l'API des plugins Babet, qui reste refusée dans les applications générées. L'empaquetage ne garantit donc pas que tout chargement provienne exclusivement du ZIP et ne crée pas de sandbox. Pour distribuer une application autonome, inclus ses modules Lua dans le projet empaqueté et vérifie ses besoins en ressources ou bibliothèques externes.

Le `main.lua` et les modules embarqués acceptent le BOM UTF-8 initial et la première ligne de commentaire #/shebang comme les fichiers Lua sur disque, y compris dans les workers. Les numéros de ligne sont conservés ; le bytecode Lua reste chargeable avec ces préfixes. Les modules sont lus dans l'inode de l'exécutable en cours : remplacer ou supprimer son chemin ne mélange pas les modules de l'application lancée avec ceux d'une nouvelle version.

Le démarrage nécessite un `/proc/self/exe` accessible et lisible. Une erreur d'ouverture ou de lecture de cette image arrête Babet avec le code 1 et un diagnostic ; elle ne transforme pas une application générée en CLI ou builder.

Le builder inscrit désormais dans sa copie du runtime un descripteur indiquant qu'il s'agit d'une application et précisant la position et la taille du ZIP. Cette identité est lue dans l'image chargée :

des octets ressemblant à du ZIP dans un runtime nu ne changent pas son mode. Une application dont le ZIP est tronqué, suivi de données supplémentaires, illisible ou dépourvu de `main.lua` est refusée avant toute interprétation des arguments.

Pour bénéficier de cette protection, reconstruire les applications avec le nouveau binaire Babet. Les anciens exécutables gardent leur runtime embarqué. Utiliser `--create-exe` : concaténer manuellement un ZIP au nouveau runtime ne le transforme plus en application. Si `strip` est souhaité, l'appliquer au runtime **avant** de générer l'application, puis préserver le fichier généré.

Ne pas compresser le runtime avec UPX : cela masque le descripteur dans le fichier et empêche `--create-exe` de fonctionner. `build_and_deploy.sh` conserve désormais le binaire du build, même si UPX est installé, et vérifie la création puis l'exécution d'une petite application avant de remplacer le Babet installé. Cette vérification utilise un dossier temporaire dans `build/` et fonctionne donc même si `$TMPDIR` est monté `noexec`. L'installation prépare un fichier en mode `0755` dans le dossier cible, puis remplace le chemin final par renommage atomique : un Babet déjà lancé continue avec son ancien binaire et un échec de copie laisse l'installation précédente intacte.

Chaque fichier Lua embarqué (`*.lua`, dont `main.lua` et `*/init.lua`) est limité à **16 Mio décompressés**, borne comprise. Le packaging refuse les scripts plus volumineux avant publication, indique leur nom dans l'archive et conserve une éventuelle ancienne sortie. Le ZIP terminé est également vérifié pour couvrir un fichier ayant grossi pendant l'empaquetage. La politique de taille des autres ressources reste inchangée. Le loader conserve ce contrôle pour les archives altérées qui se présentent avec un descripteur valide.

3. Embarqué via PATH (dossier + auto-détection)

Si Babet est sur `$PATH` et qu'on fait `chmod +x main.lua` après avoir ajouté une ligne shebang :

```
#!/usr/bin/env babet
print("hello")
```

```
chmod +x main.lua
./main.lua
```

Babet utilise le dossier du script comme dossier de travail, donc `require("helpers")` trouvera `helpers.lua` à côté de `main.lua`.

Invocation en ligne de commande

En plus des trois modes de lancement ci-dessus, Babet accepte quelques flags standards :

Flag	Effet
`-h`, `--help`	Affiche l'aide et sort avec code `0`.
`-V`, `--version`	Affiche `babet <version>` et sort avec code `0`.
`-c <dir> <out>`, `--create-exe <dir> <out>`	Crée un exécutable autonome nommé `<out>` en embarquant `<dir>` (qui doit contenir `main.lua`).

Tout autre argument commençant par `-` est traité comme une **option inconnue** : Babet affiche `Unknown option: ...` + un indice pour utiliser `--help`, et sort en code `1` plutôt que d'essayer de l'interpréter comme un nom de dossier. Les dossiers dont le nom commence légitimement par `-` peuvent toujours être passés via `./-dirname` (convention POSIX).

```
babet --version    # babet 2.22.0
babet --help       # usage complet
babet --bogus      # Unknown option: --bogus
                      # Try 'babet --help' for more information.
```

La même version est aussi exposée aux scripts via `babet.VERSION` (plus `babet.VERSION_MAJOR` / `VERSION_MINOR` / `VERSION_PATCH` en integer) — voir `sys`.

Premier script

Une fois le binaire compilé, essaie ceci :

```lua
-- main.lua
print("Babet dit bonjour")
print("PID :", babet.pid())
print("Hôte :", babet.hostname())

local r, err = babet.http.get("https://example.com/")
if r then
    print("Status :", r.status)
else
    print("Échec HTTP :", err)
end
```

Lance-le avec `./test/babet .` (ou un autre mode au choix).

Pour aller plus loin

— Chaque module a sa propre page sous `modules/`.
— Le module `user` est un petit exemple complet du pattern de documentation utilisé partout — lis-le comme la référence canonique.
— Pour les utilitaires de dates et durées (timestamps ISO 8601, durées lisibles comme `"5m"` ou `"2h30m"`), voir `time` — la sous-table `babet.time.*` couvre le parsing et le formatage.
— Vois `security.md` avant d'exposer des scripts Babet à quoi que ce soit qui pourrait recevoir de l'input non fiable.

Modèle de sécurité

Babet exécute des scripts Lua de confiance. Ce n'est pas une sandbox.

Les scripts s'exécutent avec la bibliothèque standard Lua complète (`luaL_openlibs`), et Babet expose en plus des primitives système comme `exec`, `remove`, `rmdirAll` et `chdir`. Un script a donc les mêmes privilèges que le processus qui le lance : il peut lire, modifier ou supprimer des fichiers, et lancer des commandes arbitraires. C'est volontaire — comme `make`, un script shell, ou n'importe quel outil de build, Babet est conçu pour exécuter du code que tu contrôles.

Ne lance que des `main.lua` et des modules `require` auxquels tu fais confiance. **N'utilise pas** Babet pour exécuter du Lua venant de sources non fiables ; il n'offre aucune isolation contre du code hostile, et n'est pas pensé pour. Si tu dois exécuter des scripts non fiables, utilise plutôt une sandbox Lua dédiée à cet usage.

Ce contre quoi le durcissement *protège*

Le travail de durcissement dans Babet protège les usages *légitimes* contre les accidents et les attaques de supply chain :

— **Confinement de `copyTree` / `moveTree`** : la racine destination est ouverte une fois, puis chaque création, copie et déplacement est effectué relativement à ce descripteur avec `openat`/`mkdirat`/`renameat` et `O_NOFOLLOW`/`AT_SYMLINK_NOFOLLOW`. Un symlink préexistant ou remplacé pendant l'opération ne peut pas rediriger une écriture vers une cible extérieure. Les gardes `is_within()` empêchent en plus de choisir une destination résolue dans la source.
— **Création et extraction d'archives confinées** : `babet.archive` refuse les chemins de source de création et de destination de sortie/extraction lorsqu'ils sont symlinkés, les objets source sélectionnés non pris en charge, les noms source sélectionnés non UTF-8, une sortie dans l'arbre source, les chemins d'entrée absolus, les composants `./..`, les backslashes, les doublons, les symlinks ZIP et les types spéciaux. Les archives à lire sont ouvertes une fois et

doivent se résoudre vers un fichier régulier, ce qui écarte notamment les FIFO bloquants. La destination est parcourue avec des descripteurs et `O_NOFOLLOW` ; chaque fichier est décompressé dans un temporaire du même répertoire, soumis à des limites anti-bombe, puis publié atomiquement. Les règles `include`/`exclude` de création réutilisent le moteur de glob sûr non récursif, élaguent les dossiers exclus avant ouverture et bornent chaque motif, le texte cumulé, les évaluations, le travail de correspondance et le nombre de motifs.

— **Confinement des flux compressés autonomes** : `babet.compression` accepte uniquement de véritables fichiers source réguliers, refuse les composants de chemin symlinkés et les destinations sur le même inode, épingle puis revérifie le descripteur source, borne la sortie décompressée, vérifie l'intégrité du codec, refuse les octets finaux parasites, prépare la sortie à côté de la destination et la publie atomiquement.

— **Écriture atomique générale** : `writeFileAtomic()` ouvre chaque parent sans suivre de symlink, refuse `..`, une destination finale symbolique ou non régulière et l'écrasement implicite. Les données sont écrites dans un temporaire privé du même dossier, les permissions sont appliquées, puis le fichier et le dossier sont synchronisés avant et après publication atomique.

— **Cleanup borné des groupes de processus** dans `babet.exec` couvre aussi la phase `chdir`/`exec`, les erreurs internes de polling et le cas où un enfant ferme ses pipes tout en continuant à tourner. Au timeout, TERM puis KILL visent le groupe entier ; Babet ne fait jamais de `waitpid` final illimité dans ces chemins d'erreur.

— **Limites de sortie** sur `babet.exec` bornent la quantité de stdout et stderr capturés en mémoire (par défaut 10 MiB chacun, configurable), pour qu'un sous-processus emballé ne puisse pas OOM le processus Babet.

— **Mises à jour locales épinglées** : `setAttributes` résout sa cible une seule fois et maintient `chown`, `chmod` et le rollback sur le même inode. `touch` crée avec `O_EXCL`, ne tronque jamais un fichier existant et épingle une cible existante avant de modifier son horodatage.

— **Staging anonyme et publication épinglée des exécutables** : `--create-exe` supprime le nom de son ZIP temporaire immédiatement après `mkstemp`, puis laisse miniz et la fusion rouvrir l'inode encore vivant via `/proc/self/fd`, jamais par un chemin temporaire remplaçable. La fusion finale épingle le dossier de sortie et publie avec `renameat()` ; modifier un parent symlinké ne peut donc pas rediriger l'exécutable.

— **Checksums des dépendances** : chaque dépendance embarquée est SHA256-pinnée dans `build_local.sh`. Un hash vide refuse la build. Un mismatch supprime le fichier téléchargé et sort en erreur. Ça protège contre un upstream compromis ou une archive Wayback Machine altérée (source de fallback).

— **Vérification TLS** est activée par défaut (`verify=true`). La désactiver demande un `verify=false` explicite par appel — pas de fallback silencieux. Pour les sockets TLS, des CA supplémentaires peuvent être passées via `ca_cert` ou `ca_path` ; HTTP expose `ca_cert`. Les variables OpenSSL `SSL_CERT_FILE` / `SSL_CERT_DIR` restent également disponibles. SNI est envoyé indépendamment de la vérification lorsqu'un hostname DNS est connu.

— **Validation des lignes HTTP** : les URL et valeurs de headers refusent CR/LF, et les noms de headers doivent respecter la grammaire HTTP `token`. Une valeur contrôlée par un utilisateur ne peut donc pas injecter un header ou une seconde ligne de requête via ces champs.

— **Durcissement WebSocket** : chaque frame cliente RFC 6455 utilise une nouvelle clé de masquage cryptographique ; les frames serveur masquées, bits RSV/opcodes réservés, UTF-8/codes Close invalides et octets de contrôle/espace dans l'autorité URL sont refusés. Les tailles de données sont bornées avant grosse allocation tandis que les frames de contrôle gardent leur plafond RFC indépendant de 125 octets. `wss://` vérifie les certificats par défaut et aucune extension de compression n'est négociée.

— **Limites réseau en mémoire** : `socket:recv_line`, `socket:recv_all` et `http.max_body_size` bornent les accumulations principales. Après un timeout de lecture socket, les octets déjà consommés restent dans un buffer commun afin de ne pas être perdus ou réordonnés si le script change de méthode.

— **Filtrage borné des noms** : les champs regex `name`, `iname` et `path` de `babet.find` utilisent RE2 avec une limite de 4096 octets et un budget mémoire de 1 Mio par expression ; les globs sûrs de `babet.find` et de la création d'archives disposent eux aussi d'une limite de 4096 octets et d'un automate non récursif. Les listes de création ajoutent des plafonds cumulés

sur le nombre, les octets et le travail de correspondance. Les références arrière et assertions d'anticipation/rétrospection ne font volontairement pas partie de la syntaxe RE2.
— **Confinement de parcours par périphérique** : `babet.find(..., { xdev = true })` conserve le point de montage étranger comme entrée observable mais ne descend pas dans ses enfants. Cela évite qu'une recherche locale parcoure accidentellement un volume monté, un partage réseau ou un pseudo-système de fichiers. La frontière est celle de `st_dev`, comme avec `find -xdev` : un sous-volume Btrfs peut être élagué, tandis qu'un bind mount du même système de fichiers ne l'est pas. Ce n'est pas une sandbox : le script peut toujours appeler `babet.find()` directement sur cet autre chemin.

Ce contre quoi il *ne protège pas*

— Un script malveillant. Babet n'a pas de sandbox. `babet.exec` n'invoque pas de shell, mais laisser une entrée non fiable choisir l'exécutable ou ses arguments peut tout de même permettre l'exécution d'un programme arbitraire ou une injection d'options. Transmettre du texte non fiable à `load` donne une exécution arbitraire de code Lua.
— Un attaquant déterminé qui a déjà obtenu une exécution de code sur la machine qui fait tourner Babet. Le durcissement rend les erreurs accidentelles visibles, pas les attaques adversariales impossibles.
— Les problèmes OS-level (exploits kernel, évasions de conteneur, escalade de privilèges). Babet est un binaire userland ordinaire.

Pattern recommandé : moindre privilège

Quand tu fais tourner Babet en service, traite-le comme n'importe quel processus non privilégié :

```
# En root, crée un user système dédié, sans shell.
useradd --system \
        --home-dir /var/lib/myapp \
        --create-home \
        --shell /usr/sbin/nologin \
        --comment "myapp Babet service" \
        myapp

# Utilise babet.user.exists("myapp") dans ton script d'install
# pour vérifier que le user est provisionné avant de lancer le
# daemon.
```

Combine ça avec des options de unit `systemd` comme `User=myapp`, `PrivateTmp=yes`, `ProtectSystem=strict`, `NoNewPrivileges=yes`, et `CapabilityBoundingSet=` (vide sauf si tu as vraiment besoin d'une capability). Le kernel fait bien plus que Babet ne peut le faire pour contenir un script qui se comporte mal ; laisse-le faire.

Archive — opérations ZIP et TAR sécurisées avec gzip, xz, bzip2 ou zstd

Périmètre

Le sous-module `babet.archive` crée, inspecte, lit en mémoire et extrait des archives **ZIP**, **TAR** et **TAR compressées avec gzip, xz, bzip2 ou zstd**, sans lancer de commande externe :

```
babet.archive.create(source_ou_sources, archive [, opts])
babet.archive.list(archive [, opts])
babet.archive.read(archive, nom_ou_index [, opts])
babet.archive.test(archive [, opts])
```

```
babet.archive.extract(archive, destination [, opts])
babet.archive.extractFile(archive, entry, destination [, opts])
```

La création, l'inspection et l'extraction ZIP continuent d'utiliser miniz. Les opérations TAR utilisent le backend libarchive lié statiquement ; gzip s'appuie sur zlib, xz sur XZ Utils/liblzma, bzip2 sur libbz2 et zstd sur Zstandard/libzstd, également liés statiquement. Les lecteurs détectent le conteneur et la compression à partir du contenu. Pour `create()`, `.tar` sélectionne un TAR brut, `.tar.gz` et `.tgz` un TAR gzip, `.tar.xz` ou `.txz` un TAR xz, `.tar.bz2`, `.tbz2` ou `.tbz` un TAR bzip2, et `.tar.zst`, `.tar.zstd` ou `.tzst` un TAR zstd. `opts.format` peut remplacer l'extension. Les flux compressés autonomes sont traités séparément par `babet.compression`.

Les six fonctions suivent le contrat habituel :

```
local result, err = babet.archive.list("backup.zip")
if not result then
    io.stderr:write(err, "\n")
end
```

Elles renvoient `(résultat, nil)` en cas de succès et `(nil, message)` en cas d'échec. Une mauvaise arité, une destination obligatoire qui n'est pas une chaîne, ou une source qui n'est ni une chaîne ni une table provoque une erreur Lua. Une liste de sources ou une table d'options invalide renvoie `(nil, message)`.

Table des matières

— babet.archive.create
— Options d'inspection/extraction et limites anti-bombe
— babet.archive.list
— babet.archive.read
— babet.archive.test
— babet.archive.extract
— babet.archive.extractFile
— Règles de chemins
— Protection de la destination
— Publication atomique et nettoyage
— Permissions
— Types d'entrées pris en charge
— Erreurs et limites connues

babet.archive.create

```
local result, err = babet.archive.create(
    source_ou_sources,
    archive
    [, opts]
)
```

Le premier argument accepte deux contrats distincts :

— une **chaîne** conserve le comportement historique : elle doit désigner un répertoire existant, et Babet archive son contenu sans inclure le nom du répertoire source lui-même ;
— une **table séquentielle non vide** (`1..n`, sans trou ni autre clé) sélectionne explicitement des fichiers réguliers et/ou des répertoires provenant éventuellement d'emplacements différents. Chaque source est placée sous son dernier composant : `/tmp/rapport.txt` devient `rapport.txt`, et `/opt/projet/docs` devient `docs/` avec tout son contenu. Une source absolue est acceptée, mais aucun composant du chemin hôte n'est exposé dans l'archive.

La liste explicite est lue par accès brut à sa partie tableau. Les métaméthodes `__len` et `__index` ne sont pas appelées : seuls les chemins réellement stockés aux indices `1..n` sont validés puis archivés.

Dans le mode liste, le dernier composant doit être stable : `.` et la racine du système de fichiers sont refusés, ainsi que tout composant `...` Deux sources ayant le même dernier composant sont une collision et font échouer l'opération, même si leurs contenus internes seraient différents. Répéter la même source est donc également refusé. L'ordre fourni par la table ne modifie pas l'ordre final : les entrées sont toujours triées par leur nom d'archive.

Dans les deux modes, le parcours utilise des descripteurs et ne suit aucun symlink. Un symlink dans le chemin d'une source de premier niveau est toujours refusé. Dans un arbre parcouru, un symlink, FIFO, socket, périphérique ou autre objet non pris en charge fait échouer l'opération lorsqu'il est sélectionné ; un objet retiré par les filtres n'est pas ouvert. Les noms d'entrées produits par `create()` doivent être en UTF-8 valide. Linux autorise des noms contenant des octets arbitraires, mais Babet refuse ceux qui ne peuvent pas être représentés de manière sûre et cohérente dans les deux formats de sortie.

Format de sortie

Sans `opts.format`, `create()` applique les règles suivantes :

- une destination terminée par `.tar` (sans distinction de casse) crée un TAR POSIX pax non compressé ;
- `.tar.gz` et `.tgz` (sans distinction de casse) créent un TAR POSIX pax compressé avec gzip ;
- `.tar.xz` et `.txz` (sans distinction de casse) créent un TAR POSIX pax compressé avec xz ;
- `.tar.bz2`, `.tbz2` et `.tbz` (sans distinction de casse) créent un TAR POSIX pax compressé avec bzip2 ;
- `.tar.zst`, `.tar.zstd` et `.tzst` (sans distinction de casse) créent un TAR POSIX pax compressé avec zstd ;
- toute autre destination conserve le comportement historique antérieur à la 2.6 et produit un ZIP, quelle que soit son extension.

`opts.format = "zip"`, `"tar"`, `"tar.gz"`, `"tar.xz"`, `"tar.bz2"` ou `"tar.zst"` impose explicitement le backend. Le format explicite prime toujours sur le suffixe. Ainsi, `format = "zip"` peut créer un ZIP nommé `backup.tar.gz`, tandis que `format = "tar"` peut créer un TAR non compressé nommé `backup.tgz`, `backup.txz`, `backup.tbz2` ou `backup.tzst`. Le format réel est déterminé par l'option explicite ou le contenu, jamais par une confiance aveugle dans l'extension.

Options de création :

Option	Défaut	Comportement
`format`	déduit	chaîne stricte : `"zip"`, `"tar"`, `"tar.gz"`, `"tar.xz"`, `"tar.bz2"` ou `"tar.zst"`
`compression_level`	`6`	ZIP/gzip/xz : `0` à `9` ; bzip2 : `1` à `9` ; zstd : `0` à `19`
`overwrite`	`false`	remplace atomiquement une archive régulière existante
`deterministic`	`true`	ordre stable et dates fixes propres au format
`include_directories`	`true`	émet les entrées de dossiers, y compris les dossiers vides
`include`	aucune	tableau dense de globs sûrs sensibles à la casse sélectionnant les chemins d'entrée
`exclude`	aucune	tableau dense de globs sûrs retirés après inclusion ; l'exclusion gagne toujours

Option	Défaut	Comportement
`max_entries`	`10000`	nombre maximal d'entrées ; plafond fixe `100000`
`max_file_size`	`256 * 1024 * 1024`	taille maximale d'un fichier source ; plafond fixe 8 Gio
`max_total_size`	`1024 * 1024 * 1024`	somme maximale des octets source ; plafond fixe 64 Gio

Pour un TAR brut, fournir explicitement `compression_level` est une erreur. Pour ZIP, TAR gzip et TAR xz, le niveau `0` est valide et le niveau `9` est le maximum. TAR bzip2 accepte les niveaux `1` à `9` ; le niveau `0` est refusé car les tailles de bloc libbz2 commencent à 100 Kio. TAR zstd accepte les niveaux `0` à `19` ; Babet exclut volontairement les niveaux ultra `20` à `22`.

Toutes les options entières exigent de vrais entiers Lua. Les options booléennes exigent de vrais booléens Lua. Les clés inconnues sont refusées. `format` est sensible à la casse et n'accepte aucun alias.

Filtres d'inclusion et d'exclusion

`include` et `exclude` sont des tableaux denses facultatifs de chaînes Lua non vides. Une liste `include` absente ou vide conserve le comportement historique et sélectionne initialement toutes les entrées prises en charge. Avec une liste `include` non vide, une entrée est sélectionnée si au moins un motif correspond. `exclude` est ensuite appliqué et gagne toujours, même lorsque le même chemin correspond aussi à `include`.

Les motifs s'appliquent au chemin complet qui sera stocké dans l'archive, avec `/` comme séparateur. La forme historique à chaîne utilise donc des chemins relatifs au répertoire source, tandis qu'une liste explicite inclut le nom final de chaque source (`docs/readme.md`, `report.txt`, etc.). La correspondance est ancrée, orientée octets et sensible à la casse :

- `*` correspond à zéro ou plusieurs octets sauf `/` ;
- `**` correspond à zéro ou plusieurs octets, y compris `/` ;
- `?` correspond à exactement un octet sauf `/` ;
- `\x` protège l'octet suivant `x`.

Un répertoire est testé sous les formes `chemin` et `chemin/`. Ainsi, `build/**` sélectionne ou exclut le dossier `build/` lui-même ainsi que tous ses descendants. Un dossier directement exclu est élagué avant son ouverture : rien de son sous-arbre n'est inspecté ni archivé. Lorsqu'un fichier profond est inclus, `include_directories = true` conserve tous ses dossiers parents nécessaires même s'ils ne correspondent pas directement à `include`. Un dossier vide correspondant à un motif n'est conservé que si les entrées de dossiers sont activées. Avec `include_directories = false`, seuls les fichiers réguliers sélectionnés sont émis et leurs parents restent implicites.

Le filtrage définit le plan réellement sélectionné. Un symlink, FIFO, socket, périphérique ou nom UTF-8 invalide qui est exclu, ou qui ne correspond pas à une liste `include` non vide, est ignoré ; le même objet fait toujours échouer la création lorsqu'il est sélectionné. Les chemins fournis comme sources explicites de premier niveau sont toujours validés avant filtrage et doivent eux-mêmes rester des fichiers réguliers ou de vrais répertoires accessibles sans symlink. Il est valide qu'aucun motif ne corresponde : Babet crée alors un ZIP ou TAR vide valide.

Le moteur de glob sûr n'effectue aucun backtracking récursif. Chaque motif est limité à 4096 octets. `include` et `exclude` sont limités ensemble à 256 motifs et 256 Kio de texte, un million d'évaluations de motifs et un budget fixe de 100 000 000 cellules de correspondance par création. Dépasser une limite provoque un échec avant publication. L'ordre des motifs n'affecte ni la sélection ni les octets d'une archive déterministe.

Tous les chemins source sont validés et chaque arbre non élagué est analysé avant la création du fichier temporaire de sortie. `max_entries`, `max_file_size` et `max_total_size` s'appliquent uniquement aux entrées retenues par les filtres. Chaque nom sélectionné est borné à 4096 octets et leur

somme à 64 Mio. Le parcours conserve des plafonds fixes de 100000 sources/objets du système de fichiers et 256 niveaux de répertoires, indépendamment de `include_directories` ; les descendants d'un dossier élagué ne sont pas visités. Les fichiers sont ensuite rouverts depuis leur descripteur source épinglé avec `O_NOFOLLOW`. Le périphérique, l'inode, la taille, la date de modification et la date de changement sont vérifiés avant et après la lecture en flux ; un fichier modifié pendant la création provoque un échec et la suppression du temporaire.

Le parent de destination doit déjà exister et ne contenir aucun symlink ni composant `..`. La destination doit être absente sauf avec `overwrite = true`, et ne peut jamais être un symlink ou un répertoire. L'archive ne peut pas être créée dans le répertoire source historique ni dans l'un des répertoires d'une liste explicite ; elle ne peut pas non plus remplacer directement un fichier sélectionné.

Le résultat indique le format sélectionné :

```
{
    files = 12,
    directories = 3,
    bytes = 987654,
    sources = 1,
    path = "backup.tar",
    format = "tar",
    compression = "none",
    compression_level = nil,
    deterministic = true,
    include_patterns = 0,
    exclude_patterns = 0,
}
```

`sources` contient le nombre de chemins fournis : `1` pour le contrat historique à répertoire unique, ou la longueur de la liste explicite. `include_patterns` et `exclude_patterns` indiquent le nombre de motifs compilés pour l'appel. Pour ZIP et TAR gzip, xz, bzip2 ou zstd, `compression_level` contient le niveau entier effectif. Pour un TAR brut, il vaut `nil`. Le champ d'archive `compression` vaut `"gzip"`, `"xz"`, `"bzip2"` ou `"zstd"` pour un TAR compressé et `"none"` sinon. La compression des entrées ZIP reste indiquée par les métadonnées par entrée renvoyées par `list()`.

Avec le mode déterministe par défaut, des contenus, un format, des versions de dépendances épinglées et des options identiques produisent des archives strictement identiques octet par octet :

- les entrées ZIP utilisent la date DOS fixe du 1er janvier 1980 à 00 :00 :00 ;
- les entrées TAR utilisent l'époque Unix zéro, les UID/GID `0`, des noms de propriétaire/groupe vides, le mode `0644` pour les fichiers et `0755` pour les répertoires ;
- un TAR gzip écrit en plus une date nulle dans l'en-tête gzip, afin que le flux externe n'introduise pas l'heure courante ;
- un flux xz ne contient aucune date courante et son preset LZMA2 est fixé par `compression_level` ;
- un flux bzip2 ne contient aucune date courante et sa taille de bloc est fixée par `compression_level` ;
- une frame zstd ne contient aucune date courante, active son checksum de frame et utilise le niveau fixé par `compression_level`.

Avec `deterministic = false`, l'ordre trié reste stable. ZIP utilise la date de modification source, qui doit être comprise entre 1980 et 2107 dans le fuseau local. TAR conserve la date de modification source avec sa précision en nanosecondes lorsqu'elle est représentable par la plateforme et n'est pas limité par la plage DOS du ZIP.

Les propriétaires, ACL, attributs étendus et bits de permission source ne sont pas copiés. TAR utilise le writer POSIX pax restreint de libarchive : les noms ordinaires utilisent des en-têtes ustar portables lorsque c'est possible, et les chemins longs ou métadonnées nécessitant une extension utilisent des en-têtes pax. ZIP64 est activé automatiquement lorsque la préanalyse indique que les champs ZIP classiques risquent d'être dépassés. Les petits ZIP restent des ZIP classiques.

La préanalyse et les contrôles empêchent les substitutions par symlink et détectent la modification de chaque fichier réellement archivé. La création n'est toutefois pas un instantané du système de fichiers : un fichier ajouté après l'analyse de son répertoire peut être absent du résultat ; un fichier déjà recensé puis supprimé ou renommé fait normalement échouer sa réouverture. Les renommages concurrents de répertoires ne sont pas signalés comme conflit transactionnel.

Avec `include_directories = false`, les répertoires vides ne peuvent pas être représentés et sont donc omis. Dans une liste explicite, le préfixe correspondant au nom du répertoire sélectionné reste présent pour ses fichiers réguliers. Les parents nécessaires sont recréés implicitement pendant l'extraction.

Exemples :

```lua
local zip, err = babet.archive.create("projet", "projet.zip", {
    compression_level = 9,
    overwrite = true,
})
assert(zip, err)

local selection, list_err = babet.archive.create({
    "bin/babet",
    "README.md",
    "docs",
}, "publication.tar.zst", {
    compression_level = 19,
})
assert(selection, list_err)
-- Entrées : babet, README.md, docs/, docs/...

local filtree, filter_err = babet.archive.create("projet", "sources.tar.zst", {
    include = { "src/**", "README.fr.md" },
    exclude = { "src/generated/**", "**/*.tmp" },
})
assert(filtree, filter_err)

local tar
tar, err = babet.archive.create("projet", "projet.tar", {
    deterministic = true,
})
assert(tar, err)

local gzip
gzip, err = babet.archive.create("projet", "projet.tar.gz", {
    compression_level = 9,
})
assert(gzip, err)

local xz
xz, err = babet.archive.create("projet", "projet.tar.xz", {
    compression_level = 9,
})
assert(xz, err)

local bzip2
bzip2, err = babet.archive.create("projet", "projet.tar.bz2", {
    compression_level = 9,
})
assert(bzip2, err)
```

```lua
local zstd
zstd, err = babet.archive.create("projet", "projet.tar.zst", {
    compression_level = 19,
})
assert(zstd, err)

local named
named, err = babet.archive.create("projet", "sauvegarde.data", {
    format = "tar.zst",
})
assert(named, err)
```

Options d'inspection/extraction et limites anti-bombe

Les options suivantes sont communes à `list()`, `read()`, `test()`, `extract()` et `extractFile()`. Elles sont appliquées à **l'archive entière**, même lorsque `read()` ou `extractFile()` ne demande qu'une seule entrée : sélectionner un petit fichier ne permet donc jamais de contourner les limites globales.

Option	Défaut	Maximum accepté	Effet
`max_entries`	`10000`	`100000`	nombre maximal d'entrées exposées par l'archive
`max_entry_size`	`256 * 1024 * 1024`	`8 * 1024^3`	taille décompressée maximale d'une entrée non répertoire
`max_total_size`	`1024 * 1024 * 1024`	`64 * 1024^3`	somme maximale des tailles décompressées des entrées non répertoire
`max_path_length`	`64 * 1024`	`1024 * 1024`	longueur maximale, en octets, d'un nom brut d'entrée inspecté
`max_total_name_bytes`	`64 * 1024 * 1024`	`64 * 1024 * 1024`	somme maximale, en octets, de tous les noms d'entrées ; cette option sert à resserrer la limite fixe
`max_compression_ratio`	`1000`	`1000000000`	rapport par entrée pour ZIP ; rapport global octets fichiers décompressés / taille archive pour TAR gzip, xz, bzip2 ou zstd ; sans effet pour TAR brut

Les cinq limites entières doivent être de **vrais entiers Lua strictement positifs**. Les flottants, même écrits `10.0`, les booléens, les chaînes numériques, zéro et les valeurs supérieures au plafond

sont refusés. `max_compression_ratio` doit être un nombre fini supérieur ou égal à `1` ; `NaN` et l'infini sont refusés. Une clé inconnue est toujours une erreur.

`max_path_length` borne la quantité de métadonnées que Babet accepte de lire ; elle ne rend pas un long chemin extractible. La politique de sécurité de l'extraction reste plus stricte : un nom de plus de **4096 octets** est signalé par `list()` avec `safe_path = false`, même lorsque sa longueur reste sous `max_path_length`. Avec la valeur par défaut, il est donc possible d'inspecter et de diagnostiquer un nom de 4097 octets sans jamais autoriser son extraction. L'appelant peut réduire `max_path_length` afin de refuser l'archive avant même la construction de la table Lua.

`max_total_name_bytes` vaut déjà son plafond fixe par défaut. Elle permet de resserrer le budget de noms pour une archive non fiable, mais pas de l'élargir. Le résultat de `list()` expose la consommation réelle dans `total_name_bytes`. Les allocations internes du lecteur ZIP miniz sont en outre plafonnées à 128 Mio par archive, de sorte qu'un répertoire central ou des commentaires ZIP excessifs ne puissent pas provoquer une allocation arbitraire avant les contrôles entrée par entrée.

L'inspection TAR fonctionne en flux : Babet lit les données de chaque entrée sans les conserver en mémoire, afin d'atteindre l'en-tête suivant et de détecter les contenus tronqués. Pour TAR gzip, une validation progressive et bornée avec zlib contrôle aussi CRC et ISIZE de chaque membre ; le bourrage nul standard reste accepté, mais une bande-annonce corrompue ou des octets finaux étrangers sont refusés. Pour TAR zstd, une validation progressive indépendante avec libzstd exige des frames complètes, accepte les frames concaténées valides et refuse corruption, troncature ou données finales étrangères. Les filtres xz et bzip2 restent entièrement consommés par libarchive.

Pour une extraction complète, l'arbre de sortie est également borné en interne à 100000 répertoires uniques, parents implicites compris, et à 64 Mio de chemins de répertoires normalisés cumulés. Ces plafonds fixes ne sont pas réglables.

Une entrée non vide qui annonce une taille compressée nulle est refusée. Les répertoires ne comptent ni dans `max_entry_size`, ni dans `max_total_size`, ni dans le rapport de compression, mais ils comptent dans `max_entries` et dans les budgets de noms.

Un exemple par limite

Limiter seulement le nombre d'entrées :

```lua
local result, err = babet.archive.test("upload.zip", {
    max_entries = 2000,
})
assert(result, err)
```

Limiter seulement la taille d'une entrée :

```lua
local result, err = babet.archive.test("upload.zip", {
    max_entry_size = 64 * 1024 * 1024,
})
assert(result, err)
```

Limiter seulement la taille totale annoncée :

```lua
local result, err = babet.archive.test("upload.zip", {
    max_total_size = 512 * 1024 * 1024,
})
assert(result, err)
```

Limiter seulement la longueur d'un nom brut :

```lua
local result, err = babet.archive.test("upload.zip", {
    max_path_length = 8192,
})
assert(result, err)
```

Limiter seulement le budget cumulé des noms :

```lua
local result, err = babet.archive.test("upload.zip", {
    max_total_name_bytes = 2 * 1024 * 1024,
})
assert(result, err)
```

Limiter seulement le rapport de compression :

```lua
local result, err = babet.archive.test("upload.zip", {
    max_compression_ratio = 200,
})
assert(result, err)
```

Combiner toutes les protections pour une archive reçue d'un tiers :

```lua
local result, err = babet.archive.test("upload.zip", {
    max_entries = 2000,
    max_entry_size = 64 * 1024 * 1024,
    max_total_size = 512 * 1024 * 1024,
    max_path_length = 8192,
    max_total_name_bytes = 2 * 1024 * 1024,
    max_compression_ratio = 200,
})
assert(result, err)
```

Les options propres à la création ou à l'écriture, telles que `overwrite`, `preserve_permissions`, `format`, `dry_run`, `include` ou `exclude`, ne sont pas acceptées par `list()`, `read()` ou `test()`. `read()` accepte en plus `max_size`, documentée dans sa section ; cette option est refusée par les cinq autres fonctions.

babet.archive.list

```lua
local info, err = babet.archive.list(archive [, opts])
```

`archive` est le chemin d'un fichier ZIP, TAR brut, TAR gzip, TAR xz, TAR bzip2 ou TAR zstd. Le format et la compression sont détectés par le **contenu**, jamais par l'extension : un TAR zstd nommé `sauvegarde.bin` est reconnu, tandis qu'un fichier nommé `sauvegarde.tar.zst` mais contenant autre chose est refusé. Les suffixes usuels `.tar`, `.tar.gz`, `.tgz`, `.tar.xz`, `.txz`, `.tar.bz2`, `.tbz`, `.tbz2`, `.tar.zst`, `.tar.zstd` et `.tzst` ne sont donc que des conventions de nommage.

Le chemin source peut être un symlink vers un fichier régulier. Babet ouvre sa cible une seule fois et conserve ce même descripteur pendant toute l'analyse. Les répertoires, FIFO, sockets et périphériques sont refusés avant l'analyse du format ; l'ouverture non bloquante empêche notamment qu'un FIFO sans écrivain suspende l'appel.

Babet essaie d'abord le lecteur ZIP miniz. Si le fichier épinglé n'est pas un ZIP, le même descripteur est rembobiné puis confié au lecteur TAR libarchive, avec uniquement les filtres intégrés `none`, `gzip`, `xz`, `bzip2` et `zstd`. Aucun décompresseur externe n'est lancé. Les archives ZIP fractionnées et les archives multivolumes ne sont pas prises en charge. Une archive standard vide est valide et renvoie `count = 0`.

`list()` ne crée **aucun fichier, aucun dossier et aucun fichier temporaire**. Elle conserve l'ordre physique/logique des entrées tel qu'il apparaît dans l'archive : aucun tri, aucune déduplication et aucune réorganisation ne sont effectués. `entries[1]` représente toujours la première entrée rencontrée, `entries[2]` la deuxième, etc. Cette stabilité permet aux champs `index`, `duplicate_of` et `conflict_with` de référencer directement la table retournée.

La fonction renvoie exactement deux valeurs : `(info, nil)` en cas de succès et `(nil, message)` en cas d'échec.

Structure retournée

```
{
    format = "zip",           -- "zip" ou "tar"
    compression = "none",     -- "none", "gzip", "xz", "bzip2" ou "zstd"
    entries = {
        {
            index = 1,
            name = "docs/readme.txt", -- nom brut, chaîne Lua binaire
            path = "docs/readme.txt", -- chemin normalisé si safe_path == true
            valid_utf8 = true,
            type = "file",
            size = 1234,

            compressed_size = 530,     -- ZIP uniquement, sinon nil
            crc32 = 305419896,          -- ZIP uniquement, sinon nil
            compression_method = 8,    -- ZIP uniquement, sinon nil
            encrypted = false,
            supported = true,

            safe_path = true,
            extractable = true,
            reason = nil,

            unix_mode = 420,            -- 0644, ou nil
            mtime = 1750000000,         -- secondes Unix, ou nil
            mtime_nsec = nil,           -- TAR uniquement, ou nil
            uid = nil,                  -- TAR uniquement, ou nil
            gid = nil,                  -- TAR uniquement, ou nil
            sparse = false,
            link_target = nil,

            duplicate = false,
            duplicate_of = nil,
            conflict = false,
            conflict_with = nil,
            conflict_reason = nil,
        },
    },
    count = 1,
    total_size = 1234,
    archive_size = 666,
    total_name_bytes = 19,
    duplicates = 0,
    conflicts = 0,
    zip64 = false,             -- nil pour TAR
}
```

`format` décrit le conteneur. Pour ZIP, `compression` vaut toujours `"none"`, car la compression est choisie entrée par entrée et apparaît dans `compression_method`. Pour TAR, `compression` vaut `"none"`, `"gzip"`, `"xz"`, `"bzip2"` ou `"zstd"` selon le filtre détecté.

`count` est le nombre d'éléments de `entries`. `total_size` est la somme des tailles déclarées des entrées non répertoire. `archive_size` est la taille du fichier d'archive épinglé. `total_name_bytes` est la somme exacte des longueurs en octets des champs `name`. `zip64` est un booléen pour ZIP et vaut `nil` pour TAR.

Champs communs à chaque entrée

- `index` est l'indice Lua 1-based de l'entrée dans l'ordre de l'archive ;
- `name` conserve le nom brut sous forme de chaîne Lua binaire ;
- `path` est la forme normalisée prévue pour l'extraction. Un `/` terminal de répertoire est retiré. Ce champ n'est utilisable comme chemin que lorsque `safe_path == true` ; il peut être vide pour une entrée dangereuse ;
- `valid_utf8` indique si `name` est un encodage UTF-8 strictement valide. Babet ne convertit, ne remplace et ne normalise jamais les octets ;
- `type` vaut `file`, `directory`, `symlink`, `hardlink`, `fifo`, `character_device`, `block_device`, `socket` ou `unsupported` ;
- `size` est la taille décompressée déclarée. Pour un répertoire ou un objet spécial elle vaut normalement `0`, selon les métadonnées réellement lues ;
- `safe_path` ne juge que le nom : chemin relatif, séparateur `/`, composants non vides, absence de `.` et `..`, absence de préfixe de lecteur Windows, absence d'octet NUL, type cohérent avec un `/` terminal et longueur maximale extractible de 4096 octets ;
- `extractable` juge l'entrée isolément : chemin sûr, type autorisé, absence de chiffrement et méthode prise en charge. Il ne tient volontairement pas compte d'un doublon ou d'une collision avec une autre entrée ; consulter aussi `duplicate` et `conflict` ;
- `reason` vaut `nil` lorsque l'entrée est isolément extractible, sinon une raison stable telle que `symlink entries are refused` ou `entry path contains '.' or '..'` ;
- `unix_mode` contient les bits `0000` à `7777` lorsqu'ils sont disponibles, sinon `nil`. La valeur décimale `420` correspond à `0644` ;
- `mtime` est l'heure de modification en secondes depuis l'époque Unix lorsque le backend peut la fournir, sinon `nil` ;
- `mtime_nsec` est la partie nanoseconde d'un timestamp TAR lorsque celui-ci est présent. Elle vaut `nil` pour ZIP ;
- `uid` et `gid` sont les identifiants numériques TAR lorsqu'ils sont explicitement présents, sinon `nil`. Ils valent toujours `nil` pour ZIP ;
- `sparse` signale un fichier TAR creux ;
- `link_target` contient la cible brute d'un symlink ou d'un hard link TAR lorsque libarchive l'expose, sinon `nil`.

Pour ZIP, `mtime` provient du timestamp DOS de l'entrée converti par miniz en `time_t`. Le format ZIP ne stocke ni fuseau horaire ni nanosecondes : la valeur est donc une information de calendrier locale à interpréter avec prudence. Pour TAR, `mtime` peut être négatif ou supérieur à la plage DOS et `mtime_nsec` conserve la précision disponible. L'absence d'une métadonnée est toujours représentée par `nil`, jamais par une valeur inventée.

Champs propres au ZIP

`compressed_size`, `crc32` et `compression_method` reprennent les métadonnées du répertoire central ZIP. `encrypted` signale le drapeau de chiffrement et `supported` indique si miniz reconnaît la méthode de compression. Ces champs permettent l'inspection, mais `crc32` n'est pas une preuve d'intégrité : `list()` ne décompresse pas tous les payloads ZIP. Une archive peut donc avoir un répertoire central lisible tout en contenant une donnée compressée dont le CRC ne sera détecté qu'à la lecture/extraction. Pour une validation complète des en-têtes locaux, des données et des CRC, utiliser `archive.test()`.

Pour TAR, `compressed_size`, `crc32` et `compression_method` valent `nil`, `encrypted` vaut `false`, et `supported` indique que l'en-tête a pu être interprété. La compression éventuelle concerne le flux TAR entier et apparaît dans le champ global `compression`.

Noms binaires, chemins dangereux et normalisation

Les noms ZIP sont des chaînes d'octets issues du répertoire central. Les noms TAR sont les chaînes natives fournies par libarchive après traitement des préfixes ustar, noms longs GNU et champs pax `path`. Les comparaisons sont exactes, orientées octets et sensibles à la casse. Aucune normalisation

Unicode n'est appliquée : deux représentations Unicode visuellement identiques restent deux noms distincts. Un nom UTF-8 invalide est conservé et signalé par `valid_utf8 = false` ; il n'est pas automatiquement déclaré dangereux si les autres règles de chemin sont respectées.

Les chemins absolus, les composants `.` ou `..`, les doubles `/`, les backslashes, les préfixes `C:`, les noms vides, les octets NUL et les noms de plus de 4096 octets sont signalés par `safe_path = false` et une `reason` explicite. `list()` continue à inspecter les autres entrées tant que les limites de métadonnées restent respectées. Elle ne tente jamais de corriger silencieusement un chemin dangereux.

Doublons et collisions

Les diagnostics sont attachés à la **deuxième occurrence et aux suivantes** :

- `duplicate = true` signifie que le même champ `name`, octet pour octet, a déjà été rencontré. `duplicate_of` contient l'indice de la première occurrence ;
- `conflict = true` signifie que le chemin de sortie sûr et normalisé de cette entrée ne peut pas coexister avec une entrée antérieure. `conflict_with` contient l'indice antérieur retenu et `conflict_reason` vaut actuellement `duplicate output path` ou `file/directory path conflict` ;
- `duplicates` compte les occurrences exactes supplémentaires ;
- `conflicts` compte les occurrences supplémentaires qui rendraient le plan d'extraction ambigu ou impossible.

Un doublon exact dont le chemin est sûr est donc à la fois un `duplicate` et un `conflict`. Deux noms bruts différents, par exemple `docs/` et `docs`, ne sont pas des doublons, mais ils entrent en collision après normalisation et le second porte `conflict = true`. Une entrée dangereuse peut être signalée comme doublon brut, mais elle n'entre pas dans le calcul des collisions de sortie puisqu'elle n'a pas de chemin d'extraction sûr.

`list()` ne transforme pas ces diagnostics en erreur : son rôle est précisément de permettre l'examen d'une archive suspecte. `extract()` continue en revanche à refuser les doublons et collisions avant toute écriture.

Exemple de diagnostic :

```lua
local info, err = babet.archive.list("upload.zip")
assert(info, err)

for _, entry in ipairs(info.entries) do
    if not entry.safe_path then
        print(entry.index, entry.name, entry.reason)
    elseif entry.duplicate then
        print(entry.index, "duplique l'entrée", entry.duplicate_of)
    elseif entry.conflict then
        print(entry.index, entry.conflict_reason,
            "avec l'entrée", entry.conflict_with)
    end
end
```

Exemples d'utilisation

Afficher simplement le contenu dans l'ordre de l'archive :

```lua
local info, err = babet.archive.list("backup.tar.zst")
assert(info, err)

print(info.format, info.compression, info.count)
for _, entry in ipairs(info.entries) do
    print(entry.index, entry.type, entry.path, entry.size)
end
```

Exploiter les métadonnées disponibles sans supposer qu'elles existent :

```
local info, err = babet.archive.list("package.zip")
assert(info, err)

for _, entry in ipairs(info.entries) do
    if entry.mtime ~= nil then
        print(entry.name, os.date("%Y-%m-%d %H:%M:%S", entry.mtime))
    end
    if entry.unix_mode ~= nil then
        print(string.format("mode=%04o", entry.unix_mode))
    end
    if entry.uid ~= nil then
        print("uid/gid", entry.uid, entry.gid)
    end
end
```

Refuser soi-même toute archive ambiguë avant de poursuivre un traitement :

```
local info, err = babet.archive.list("upload.zip")
assert(info, err)

if info.duplicates ~= 0 or info.conflicts ~= 0 then
    error("archive ambiguë")
end

for _, entry in ipairs(info.entries) do
    if not entry.safe_path or not entry.extractable then
        error((entry.reason or "entrée refusée") .. ": " .. entry.name)
    end
end
```

Traiter correctement un nom non UTF-8 :

```
local info, err = babet.archive.list("legacy.zip")
assert(info, err)

for _, entry in ipairs(info.entries) do
    if not entry.valid_utf8 then
        -- La chaîne reste binaire : afficher les octets plutôt que la passer
        -- à une interface qui exige du texte UTF-8.
        local bytes = {}
        for i = 1, #entry.name do
            bytes[#bytes + 1] = string.format("%02x", entry.name:byte(i))
        end
        print(entry.index, table.concat(bytes))
    end
end
```

Utiliser `list()` dans un worker :

```
local worker, err = babet.workers.spawn([[
local info, list_err = babet.archive.list(worker.args.archive, {
    max_entries = 5000,
})
if not info then error(list_err) end
return {
    format = info.format,
    compression = info.compression,
    count = info.count,
```

```
    duplicates = info.duplicates,
    conflicts = info.conflicts,
}
]], { archive = "backup.tar.zst" })
assert(worker, err)

local joined, result = worker:join()
assert(joined, result)
print(result.format, result.count)
```

La fonction et tous ses champs ont le même comportement dans l'état Lua principal, en mode dossier, en mode embarqué et dans les workers.

Ce que `list()` valide — et ce qu'elle ne garantit pas

Pour ZIP, Babet valide la structure nécessaire à l'ouverture et à la lecture du répertoire central, les nombres et tailles annoncés, les chemins et les limites. Les données compressées des fichiers ne sont pas toutes lues : un CRC de payload invalide peut donc rester invisible à ce stade.

Pour TAR et TAR compressé, atteindre l'en-tête suivant impose de consommer les données de chaque entrée ; Babet détecte ainsi de nombreuses troncatures, en-têtes invalides et erreurs du flux externe. Cette lecture complète ne change pas le rôle de l'API : `list()` retourne un inventaire, tandis que `archive.test()` transforme la lecture complète et les règles de sécurité agrégées en un verdict strict.

Une archive techniquement lisible mais contenant des chemins dangereux, liens ou objets spéciaux est volontairement retournée avec ses diagnostics. Une archive dont le format est inconnu, dont la structure ne peut pas être lue, qui dépasse une limite ou dont le flux TAR est tronqué renvoie `(nil, message)`.

`babet.archive.read`

```
local data, err = babet.archive.read(
    archive,
    nom_brut_ou_index
    [, opts]
)
```

`read()` charge **un fichier régulier** dans une chaîne Lua binaire, sans créer de destination, de fichier temporaire ni de répertoire. Elle reconnaît par le contenu les mêmes formats que `list()` : ZIP, TAR brut, TAR gzip, TAR xz, TAR bzip2 et TAR zstd.

Le succès renvoie exactement `(data, nil)`. `data` peut être vide et peut contenir des octets NUL ou du texte non UTF-8. Une erreur renvoie exactement `(nil, message)`.

Sélection par nom brut ou par index

Le deuxième argument accepte deux formes strictes :

- une **chaîne** recherche le nom brut exact, octet pour octet et avec respect de la casse ;
- un **entier** sélectionne l'index 1-based exposé dans `list().entries[i].index`.

Un flottant tel que `2.0`, un booléen ou une table n'est pas converti silencieusement en index. Un nom ne peut pas contenir d'octet NUL. Les noms longs GNU et pax d'un TAR sont ceux déjà décodés et exposés par `list()`.

Lecture directe par nom :

```
local manifest, err = babet.archive.read(
    "package.tar.zst",
    "manifest.json"
```

```
)
assert(manifest, err)
```

Lecture par index obtenu sans supposer l'ordre de l'archive :

```
local info, err = babet.archive.list("package.zip")
assert(info, err)

local target
for _, entry in ipairs(info.entries) do
    if entry.name == "assets/logo.bin" then
        target = entry.index
        break
    end
end
assert(target, "assets/logo.bin absent")

local logo
logo, err = babet.archive.read("package.zip", target)
assert(logo, err)
```

La sélection par nom est refusée si le même nom brut apparaît plusieurs fois : Babet ne choisit jamais silencieusement la première ou la dernière occurrence. Après inspection de `duplicate` et `duplicate_of`, un index explicite peut lire l'occurrence voulue :

```
local info, err = babet.archive.list("duplicated.zip")
assert(info, err)

for _, entry in ipairs(info.entries) do
    if entry.name == "settings.json" then
        print(entry.index, entry.duplicate, entry.duplicate_of)
    end
end

-- Choix explicite de la deuxième occurrence.
local data
data, err = babet.archive.read("duplicated.zip", 7)
assert(data, err)
```

L'index désambiguïse uniquement l'entrée à lire. Il ne supprime, ne trie et ne réécrit aucune entrée de l'archive.

Noms dangereux et types refusés

Comme `read()` n'utilise jamais le nom comme chemin de destination, un fichier régulier nommé `../config`, `/absolute` ou avec un nom non UTF-8 peut être lu par son nom brut ou son index. Aucune normalisation ni « réparation » n'est appliquée. Le script doit consulter `valid_utf8`, `safe_path`, `duplicate` et `conflict` dans `list()` avant de réutiliser ce nom dans un autre contexte :

```
local info, err = babet.archive.list("untrusted.zip")
assert(info, err)

local entry = assert(info.entries[1])
print(entry.name, entry.valid_utf8, entry.safe_path)

local data
data, err = babet.archive.read("untrusted.zip", entry.index)
assert(data, err)
```

Cette tolérance est sûre uniquement parce qu'aucune écriture n'a lieu. Les règles strictes de confinement restent inchangées pour `extract()` et `extractFile()`.

Les répertoires, symlinks, hard links, FIFO, sockets, périphériques, types inconnus, entrées ZIP chiffrées, méthodes ZIP non prises en charge et fichiers TAR sparse sont refusés. `read()` ne suit jamais un lien contenu dans l'archive.

Limite mémoire `max_size`

`read()` ajoute une option propre :

Option	Défaut	Maximum accepté	Effet
`max_size`	`8 * 1024 * 1024`	`256 * 1024 * 1024`	nombre maximal d'octets renvoyés dans la chaîne Lua

`max_size` doit être un véritable entier Lua entre `1` et `268435456`. La limite est inclusive : une entrée produisant exactement `max_size` octets est acceptée.

```
local config, err = babet.archive.read(
    "package.tar.gz",
    "config.json",
    { max_size = 1024 * 1024 }
)
assert(config, err)
```

Babet refuse tôt une taille annoncée supérieure à `max_size`, puis applique la même limite au callback qui reçoit les octets réellement décompressés. Un ZIP mensonger qui annonce une petite taille mais produit davantage est donc arrêté avant que le tampon borné ne puisse être dépassé. La vérification porte sur les octets binaires, pas sur un nombre de caractères Unicode.

`max_size` ne remplace pas `max_entry_size` :

- `max_size` borne uniquement la valeur Lua demandée ;
- `max_entry_size`, `max_total_size`, `max_entries`, `max_path_length`, `max_total_name_bytes` et `max_compression_ratio` continuent de protéger l'archive entière.

Exemple combiné pour une archive non fiable :

```
local manifest, err = babet.archive.read(
    "upload.tar.xz",
    "manifest.json",
    {
        max_size = 512 * 1024,
        max_entries = 2000,
        max_entry_size = 64 * 1024 * 1024,
        max_total_size = 512 * 1024 * 1024,
        max_path_length = 8192,
        max_total_name_bytes = 2 * 1024 * 1024,
        max_compression_ratio = 200,
    }
)
assert(manifest, err)
```

Intégrité et consommation de l'archive

Pour ZIP, Babet inspecte les métadonnées de l'archive, puis décompresse intégralement l'entrée sélectionnée avec miniz. Une erreur de méthode, de décompression, de taille produite ou de CRC fait échouer la lecture.

Pour TAR, la première passe inspecte et consomme toute l'archive. La seconde passe compare chaque en-tête au plan inspecté, transmet seulement les blocs du fichier choisi au tampon mémoire borné et consomme toutes les autres entrées jusqu'à la fin. Les validations supplémentaires des flux gzip et zstd restent appliquées. Une corruption ou une troncature ailleurs dans le TAR peut donc faire échouer la lecture du petit fichier demandé.

`read()` n'est pas un alias de `test()` : elle valide complètement la donnée sélectionnée et les structures nécessaires à sa lecture, tandis que `test()` décompresse tous les payloads ZIP et applique en plus le verdict global de sécurité sur toutes les entrées.

Utilisation dans un worker

L'API est disponible dans les workers. Une chaîne UTF-8 ordinaire peut être renvoyée directement. Pour une donnée binaire contenant NUL ou des octets non UTF-8, le worker doit la traiter sur place ou renvoyer un résultat sérialisable, par exemple sa taille et son CRC :

```lua
local worker, err = babet.workers.spawn([[
local data, read_err = babet.archive.read(
    worker.args.archive,
    worker.args.index,
    { max_size = worker.args.max_size }
)
if not data then error(read_err) end
return {
    size = #data,
    crc32 = babet.crc32(data),
}
]], {
    archive = "assets.tar.bz2",
    index = 3,
    max_size = 4 * 1024 * 1024,
})
assert(worker, err)

local joined, result = worker:join()
assert(joined, result)
print(result.size, result.crc32)
```

Cette restriction au retour du worker vient du contrat de sérialisation des workers, pas de `archive.read()` : la lecture elle-même reste entièrement binaire.

`babet.archive.test`

```lua
local result, err = babet.archive.test(archive [, opts])
```

`test()` vérifie intégralement une archive **sans rien extraire et sans rien écrire**. Elle accepte les mêmes formats détectés par contenu que `list()` : ZIP, TAR brut, TAR gzip, TAR xz, TAR bzip2 et TAR zstd. Elle accepte exactement les mêmes six limites anti-bombe documentées plus haut et refuse toute autre option.

Le contrat est volontairement strict : un succès signifie que l'archive est à la fois **techniquement valide** et **entièrement sûre selon les règles d'extraction de Babet**. Une archive lisible mais contenant un chemin dangereux, un doublon, une collision, un symlink, un hard link, un fichier sparse, un objet spécial, une entrée chiffrée ou une méthode ZIP non prise en charge renvoie donc `(nil, message)`. `test()` ne renvoie pas une table `warnings` ni un champ `safe = false` : pour obtenir un inventaire détaillé d'une archive suspecte sans la valider, utiliser `list()`.

Le chemin source suit le même contrat que `list()` : un symlink vers un fichier régulier est accepté, tandis qu'un répertoire, FIFO, socket ou périphérique est refusé sans blocage. Le fichier ouvert est

épinglé par descripteur pendant toute la vérification. L'extension ne participe jamais au choix du backend.

Vérifications techniques

Pour un ZIP, Babet contrôle notamment :

- les métadonnées EOCD et ZIP64, avec refus des archives fractionnées ou multivolumes ;
- le répertoire central et les limites annoncées ;
- l'en-tête local de **chaque** entrée, y compris les fichiers vides et les répertoires ;
- l'identité du nom local et du nom central, les drapeaux, la méthode, les tailles, le CRC, les champs ZIP64 locaux et les data descriptors, avec ou sans signature ;
- les bornes de chaque zone locale, l'absence de chevauchement entre entrées et l'absence de recouvrement avec le répertoire central ;
- la décompression complète de chaque entrée non répertoire, la taille réellement produite et son CRC.

Pour TAR, le lecteur consomme l'intégralité des données et vérifie les en-têtes, checksums TAR disponibles, tailles, troncatures, padding et données finales. Pour TAR gzip, xz, bzip2 ou zstd, le flux de compression est lui aussi consommé jusqu'à sa fin ; les erreurs d'intégrité, frames ou membres tronqués et octets finaux étrangers sont refusés selon les garanties du codec et du backend.

Une archive vide correctement formée est valide. Une erreur de lecture, un format inconnu, une structure ambiguë, une limite dépassée ou une incohérence entre métadonnées et données provoque `(nil, message)`.

Vérifications de sécurité

Après la validation technique, Babet applique aux entrées, dans leur ordre d'archive, les mêmes règles que l'extraction réelle :

- chemin relatif non vide utilisant uniquement `/`, sans `.` ni `..`, sans double séparateur, backslash, préfixe de lecteur ou octet NUL ;
- longueur extractible maximale de 4096 octets ;
- uniquement des fichiers réguliers et répertoires ordinaires ;
- aucun symlink, hard link, FIFO, socket, périphérique, objet non pris en charge ou fichier sparse ;
- aucun doublon exact, doublon de chemin de sortie ou conflit fichier/dossier ;
- aucune entrée ZIP chiffrée ou utilisant une méthode non prise en charge.

Le premier diagnostic est déterministe : Babet conserve l'ordre de lecture de l'archive et arrête la vérification au premier échec selon les phases ci-dessus. Un succès garantit donc que `files + directories == entries`.

Structure retournée

```
{
    format = "tar",
    compression = "zstd",
    entries = 42,
    files = 35,
    directories = 7,
    total_size = 12345678,
    archive_size = 3456789,
    total_name_bytes = 812,
    zip64 = nil,
}
```

- `format` vaut `"zip"` ou `"tar"` ;

— `compression` vaut `"none"`, `"gzip"`, `"xz"`, `"bzip2"` ou `"zstd"` ;
— `entries`, `files` et `directories` comptent les entrées validées ;
— `total_size` est la somme des tailles décompressées des fichiers réguliers ;
— `archive_size` est la taille en octets du fichier archive épinglé ;
— `total_name_bytes` est la somme des longueurs brutes des noms ;
— `zip64` est un booléen pour ZIP et `nil` pour TAR.

La fonction renvoie exactement `(result, nil)` en cas de succès et `(nil, message)` en cas d'échec.

Exemples d'utilisation

Vérifier simplement une archive avant de la conserver :

```
local result, err = babet.archive.test("upload.zip")
if not result then
    io.stderr:write("archive refusée : ", err, "\n")
    return 1
end

print(result.format, result.entries, result.total_size)
```

Vérifier un TAR compressé, même lorsque son extension est trompeuse :

```
local result, err = babet.archive.test("sauvegarde.bin")
assert(result, err)
assert(result.format == "tar")
assert(result.compression == "zstd")
```

Resserrer uniquement le nombre d'entrées :

```
local result, err = babet.archive.test("upload.zip", {
    max_entries = 500,
})
assert(result, err)
```

Resserrer uniquement la taille d'un fichier :

```
local result, err = babet.archive.test("upload.zip", {
    max_entry_size = 16 * 1024 * 1024,
})
assert(result, err)
```

Resserrer uniquement la taille totale :

```
local result, err = babet.archive.test("upload.zip", {
    max_total_size = 128 * 1024 * 1024,
})
assert(result, err)
```

Resserrer uniquement la longueur des noms :

```
local result, err = babet.archive.test("upload.zip", {
    max_path_length = 4096,
})
assert(result, err)
```

Resserrer uniquement le budget cumulé des noms :

```
local result, err = babet.archive.test("upload.zip", {
    max_total_name_bytes = 1024 * 1024,
})
assert(result, err)
```

Resserrer uniquement le rapport de compression :

```lua
local result, err = babet.archive.test("upload.zip", {
    max_compression_ratio = 100,
})
assert(result, err)
```

Combiner toutes les limites pour une archive non fiable :

```lua
local result, err = babet.archive.test("upload.zip", {
    max_entries = 500,
    max_entry_size = 16 * 1024 * 1024,
    max_total_size = 128 * 1024 * 1024,
    max_path_length = 4096,
    max_total_name_bytes = 1024 * 1024,
    max_compression_ratio = 100,
})
assert(result, err)
```

Distinguer une inspection diagnostique d'un verdict strict :

```lua
local info, list_err = babet.archive.list("upload.zip")
assert(info, list_err)

local verified, test_err = babet.archive.test("upload.zip")
if not verified then
    print("contenu visible mais archive refusée :", test_err)
end
```

Utiliser `test()` dans un worker :

```lua
local worker, err = babet.workers.spawn([[
local result, test_err = babet.archive.test(worker.args.archive, {
    max_entries = 2000,
    max_total_size = 512 * 1024 * 1024,
})
if not result then error(test_err) end
return result
]], { archive = "backup.tar.zst" })
assert(worker, err)

local joined, result = worker:join()
assert(joined, result)
print(result.format, result.compression, result.entries)
```

Le comportement et le contrat de retour sont identiques en mode dossier, en mode embarqué, en mode embarqué via `PATH` et dans les workers.

babet.archive.extract

```lua
local result, err = babet.archive.extract(
    archive,
    destination
    [, opts]
)
```

`archive` peut être un ZIP, un TAR brut, un TAR gzip, un TAR xz, un TAR bzip2 ou un TAR zstd. Le format et la compression sont détectés par le contenu, pas par l'extension.

Options acceptées :

Option	Défaut	Comportement
`overwrite`	`false`	autorise le remplacement atomique d'un fichier régulier existant
`dry_run`	`false`	prévisualise l'extraction et vérifie les données sélectionnées sans modifier le système de fichiers
`preserve_permissions`	`false`	conserve les bits Unix ordinaires `rwx` lorsqu'ils sont disponibles
`include`	aucune	tableau dense de globs sûrs sélectionnant les chemins internes normalisés
`exclude`	aucune	tableau dense de globs sûrs retirés après inclusion ; l'exclusion gagne toujours
`max_entries`	`10000`	nombre maximal d'entrées analysées ; plafond fixe `100000`
`max_entry_size`	`256 * 1024 * 1024`	taille décompressée maximale d'une entrée ; plafond fixe 8 Gio
`max_total_size`	`1024 * 1024 * 1024`	somme maximale des tailles déclarées ; plafond fixe 64 Gio
`max_path_length`	`64 * 1024`	longueur maximale d'un nom brut ; plafond fixe 1 Mio
`max_total_name_bytes`	`64 * 1024 * 1024`	somme maximale des longueurs de noms ; plafond fixe 64 Mio
`max_compression_ratio`	`1000`	rapport décompressé/compressé maximal, entre 1 et `1000000000`

Les booléens doivent être de vrais booléens Lua, les limites entières de vrais entiers Lua strictement positifs et `max_compression_ratio` un nombre fini. Les clés inconnues, les arguments supplémentaires et les options propres à `create()` sont refusés. `dry_run`, `include` et `exclude` ne sont acceptées que par `extract()`, pas par `list()`, `read()`, `test()` ni `extractFile()`.

Sélection par globs sûrs

`include` et `exclude` réutilisent exactement le même moteur borné que `archive.create()`. Il n'existe pas de second moteur ni de variante propre à l'extraction.

Une liste `include` absente ou vide conserve le comportement historique et sélectionne initialement toutes les entrées. Avec une liste `include` non vide, une entrée est retenue si au moins un motif correspond. `exclude` est appliqué ensuite et gagne toujours, même lorsqu'un chemin correspond aussi à `include`.

Les motifs sont ancrés sur le **chemin interne normalisé complet** de l'entrée, avec `/` comme séparateur. Les comparaisons sont orientées octets et sensibles à la casse :

— `*` correspond à zéro ou plusieurs octets sauf `/` ;

— ** correspond à zéro ou plusieurs octets, y compris `/` ;
— ? correspond à exactement un octet sauf `/` ;
— `\x` protège l'octet suivant `x`.

Un répertoire est testé sous les formes `chemin` et `chemin/`. Exclure `src/generated` ou `src/generated/**` exclut donc le répertoire et coupe tout son sous-arbre, même si l'archive ne contient pas d'entrée de répertoire explicite. Les seuls parents nécessaires aux fichiers retenus sont créés ; un répertoire explicite non sélectionné n'est pas créé. Un répertoire vide n'est restauré que si son entrée explicite est sélectionnée.

Les mêmes limites que pour la création sont appliquées : chaque motif est limité à 4096 octets ; `include` et `exclude` sont limités ensemble à 256 motifs et 256 Kio de texte, un million d'évaluations et un budget fixe de 100 000 000 cellules de correspondance par appel. Le graphe normalisé utilisé pour propager l'exclusion des répertoires est en outre plafonné à 100000 chemins de répertoire uniques et à 64 Mio de texte cumulé. Dépasser une limite fait échouer l'opération avant toute publication. L'ordre des motifs ne modifie pas la sélection.

Simulation sans écriture avec `dry_run`

Avec `dry_run = true`, Babet construit exactement le même plan qu'une extraction réelle, mais n'appelle aucune opération de création, d'écriture, de chmod, de renommage, de suppression ni de publication. L'archive est épinglée et analysée, les mêmes limites et filtres sont appliqués, les entrées sélectionnées sont validées, puis la destination est parcourue uniquement en lecture avec `openat`/`fstatat` et `O_NOFOLLOW`. Aucun dossier racine, parent implicite, fichier temporaire ou entrée finale n'est créé.

Le contrôle de destination conserve la politique réelle :

— une racine absente est acceptée et signalée par `would_create_destination = true` ;
— une racine ou un parent existant qui est un symlink ou un objet non dossier est refusé ;
— une entrée sélectionnée absente compte dans `would_create` ;
— un fichier régulier existant compte dans `would_overwrite` uniquement avec `overwrite = true` ; avec `overwrite = false`, l'appel échoue comme l'extraction réelle ;
— un répertoire explicite sélectionné et déjà présent compte dans `would_skip`, car l'extraction réelle le conserve et ne modifie pas son mode ;
— un conflit fichier/répertoire ou une cible finale symlinkée reste une erreur.

`would_create`, `would_overwrite` et `would_skip` comptent seulement les **entrées explicites sélectionnées dans l'archive**. Leur somme vaut donc `entries`. La racine de destination et les parents implicites nécessaires ne sont pas inclus dans ces compteurs ; la racine dispose de son booléen séparé. Lorsque des filtres actifs ne sélectionnent rien, le comportement reste celui de l'extraction réelle : la destination n'est pas inspectée et `would_create_destination` vaut `false`. À l'inverse, une archive vide sans filtre prévisualise bien la création éventuelle de sa racine.

La simulation vérifie réellement les données qui seraient extraites. Pour ZIP, chaque fichier régulier sélectionné est décompressé vers un consommateur nul et son CRC est contrôlé ; les payloads ZIP non sélectionnés restent ignorés, comme pour l'extraction sélective réelle. Pour TAR, la seconde passe consomme le flux complet brut ou compressé et compare de nouveau les en-têtes au plan initial, sans transmettre les données au système de fichiers. `archive.test()` reste l'API à utiliser lorsqu'il faut vérifier tous les payloads ZIP, y compris ceux qui seraient ignorés.

Le résultat décrit un **instantané**. Un autre processus peut modifier la destination après le retour de `dry_run` ; la simulation n'est donc jamais une garantie qu'une extraction ultérieure réussira ni qu'elle produira les mêmes compteurs. L'extraction réelle recommence tous les contrôles et épingle les descripteurs nécessaires avant toute publication.

Validation, entrées ignorées et intégrité

Babet épingle d'abord le fichier d'archive et inspecte **toutes les entrées et les métadonnées nécessaires au plan** afin d'appliquer les limites globales avant d'ouvrir la destination. La sélection

est ensuite calculée sur les noms normalisés. Seules les entrées retenues participent au plan de sortie :

- un doublon, une collision fichier/répertoire, un lien ou un objet spécial présent uniquement hors sélection n'empêche pas l'extraction ;
- une entrée dangereuse sélectionnée reste refusée avant toute écriture ;
- avec une liste `include` non vide, un chemin dangereux sans forme normalisée ne peut correspondre à aucun motif et reste hors sélection ;
- avec seulement `exclude`, le comportement historique « tout sélectionner » reste actif : un chemin dangereux ne peut pas être rendu sûr par un motif d'exclusion et fait donc échouer l'appel ;
- les collisions déjà présentes sur le disque mais situées hors sélection ne sont ni remplacées ni modifiées.

Cette politique permet d'extraire une partie sûre d'une archive mixte sans présenter `exclude` comme un mécanisme de réparation de chemins invalides. Toutes les limites d'entrées, de tailles, de noms et de rapport de compression continuent à porter sur l'archive inspectée entière, pas seulement sur les fichiers retenus.

Pour ZIP, les payloads ignorés ne sont pas décompressés et leur CRC n'est donc pas vérifié par l'extraction sélective. Pour TAR, le lecteur doit avancer dans le flux jusqu'aux en-têtes suivants, y compris dans un flux compressé, mais `extract()` n'est pas l'API de vérification complète. Appeler `archive.test()` avant l'extraction lorsqu'il faut garantir l'intégrité de **toutes** les entrées, y compris celles qui seront ignorées.

Avant toute publication, Babet refuse les entrées retenues non extractibles, les sorties retenues dupliquées, les conflits fichier/répertoire retenus et les objets existants incompatibles dans la destination. Les fichiers sélectionnés sont préparés dans la destination sécurisée puis publiés atomiquement. Pour TAR, le même fichier épinglé est relu depuis le début et chaque en-tête est comparé au plan issu de la première passe.

Lorsque des filtres actifs ne sélectionnent rien, l'appel réussit sans créer la destination. Sans filtre, une archive vide conserve le comportement historique et crée le répertoire racine de destination.

Valeur de retour

La fonction renvoie exactement `(result, nil)` ou `(nil, message)` :

```
{
    entries = 28,     -- entrées réellement sélectionnées
    files = 24,       -- fichiers réguliers sélectionnés
    directories = 4, -- répertoires explicites sélectionnés
    skipped = 15,     -- entrées de l'archive non sélectionnées
    bytes = 987654,   -- octets décompressés des fichiers sélectionnés
    path = "restauration",
}
```

Avec `dry_run = true`, la même table reçoit en plus :

```
{
    dry_run = true,
    would_create = 20,
    would_overwrite = 3,
    would_skip = 5,
    would_create_destination = false,
}
```

Ces champs supplémentaires sont absents d'une extraction réelle afin de ne pas modifier son contrat historique. `directories` ne compte pas les parents implicites créés pour atteindre un fichier. `entries == files + directories` pour une extraction réussie, puisque les liens et objets spéciaux

sélectionnés sont refusés. En simulation réussie, `would_create + would_overwrite + would_skip == entries`. `skipped` vaut `0` lorsqu'aucun filtre n'est actif.

Exemples

Prévisualiser une extraction vers une destination absente :

```lua
local plan, err = babet.archive.extract("backup.zip", "restore", {
    dry_run = true,
})
assert(plan, err)
assert(plan.dry_run == true)
assert(plan.would_create_destination == true)
assert(not babet.fileExists("restore"))
```

Prévisualiser les remplacements autorisés dans une destination existante :

```lua
local plan, err = babet.archive.extract("backup.tar.zst", "restore", {
    dry_run = true,
    overwrite = true,
    preserve_permissions = true,
})
assert(plan, err)
print(plan.would_create, plan.would_overwrite, plan.would_skip)
```

Sans `overwrite = true`, un fichier final déjà présent reste une erreur :

```lua
local plan, err = babet.archive.extract("backup.zip", "restore", {
    dry_run = true,
})
assert(plan == nil and type(err) == "string")
```

Combiner simulation, sélection et limites :

```lua
local plan, err = babet.archive.extract("project.tar.xz", "output", {
    dry_run = true,
    include = { "src/**", "docs/**", "README.md" },
    exclude = { "src/generated/**", "**/*.tmp" },
    overwrite = true,
    preserve_permissions = true,
    max_entries = 20000,
    max_total_size = 2 * 1024 * 1024 * 1024,
})
assert(plan, err)
assert(plan.would_create + plan.would_overwrite + plan.would_skip
    == plan.entries)
assert(not babet.fileExists("output"))
```

Extraire seulement deux zones précises :

```lua
local result, err = babet.archive.extract("project.tar.zst", "output", {
    include = {
        "src/**",
        "README.md",
    },
})
assert(result, err)
print(result.entries, result.skipped)
```

Exclure les fichiers temporaires tout en conservant le reste :

```lua
local result, err = babet.archive.extract("backup.zip", "restore", {
    exclude = {
        "**/*.tmp",
        "cache/**",
    },
})
assert(result, err)
```

Combiner inclusion et exclusion, avec priorité aux exclusions :

```lua
local result, err = babet.archive.extract("project.tar.xz", "output", {
    include = {
        "src/**",
        "docs/**",
        "README.md",
    },
    exclude = {
        "src/generated/**",
        "**/*.tmp",
    },
    overwrite = false,
    preserve_permissions = true,
    max_total_size = 2 * 1024 * 1024 * 1024,
})
assert(result, err)
```

Utiliser ? et protéger un caractère * littéral :

```lua
local result, err = babet.archive.extract("assets.zip", "assets", {
    include = {
        "icons/icon-?.png", -- icon-a.png, mais pas icon-large.png
        "docs/file\\*.txt", -- nom contenant réellement un astérisque
    },
})
assert(result, err)
```

Gérer une sélection vide sans créer de dossier :

```lua
local result, err = babet.archive.extract("package.zip", "unused", {
    include = { "does-not-exist/**" },
})
assert(result, err)
assert(result.entries == 0 and result.skipped > 0)
assert(not babet.fileExists("unused"))
```

Utiliser les filtres dans un worker :

```lua
local worker, err = babet.workers.spawn([[
local result, extract_err = babet.archive.extract(
    worker.args.archive,
    worker.args.destination,
    {
        include = { "docs/**", "README.md" },
        exclude = { "docs/drafts/**" },
    }
)
if not result then error(extract_err) end
return result
]], {
    archive = "package.tar.gz",
```

```
    destination = "documentation",
})
assert(worker, err)

local joined, result = worker:join()
assert(joined, result)
print(result.files, result.skipped)
```

Prévisualiser dans un worker sans créer la destination :

```
local worker, err = babet.workers.spawn([[
local plan, extract_err = babet.archive.extract(
    worker.args.archive, worker.args.destination, {
        dry_run = true,
        include = { "manifest.json", "docs/**" },
    })
if not plan then error(extract_err) end
return plan
]], {
    archive = "package.tar.gz",
    destination = "preview-only",
})
assert(worker, err)

local joined, plan = worker:join()
assert(joined, plan)
assert(plan.dry_run == true)
assert(not babet.fileExists("preview-only"))
```

Le contrat, les limites et la sélection sont identiques en mode dossier, en mode embarqué, en mode embarqué via `PATH` et dans les workers.

babet.archive.extractFile

```
local result, err = babet.archive.extractFile(
    archive,
    entry,
    destination
    [, opts]
)
```

`archive` peut être un ZIP, un TAR brut, un TAR gzip, un TAR xz, un TAR bzip2 ou un TAR zstd ; la détection repose sur le contenu et non sur l'extension. Le ZIP reste traité par miniz. Le TAR utilise la même source épinglée et la même lecture libarchive en deux passes que l'extraction complète.

`entry` est le **nom brut exact** exposé par `list()`. La recherche est sensible à la casse et échoue si le nom est absent ou apparaît plusieurs fois. Pour TAR, les noms longs GNU et les enregistrements pax `path` sont recherchés après leur décodage par libarchive, exactement tels qu'ils figurent dans `list().entries[i].name`.

Seule une entrée de type fichier régulier peut être sélectionnée. Les fichiers TAR sparse, symlinks, hard links, répertoires et types spéciaux sont refusés lorsqu'ils sont sélectionnés. Le chemin de l'entrée n'est pas reproduit : le contenu est écrit exactement dans le chemin `destination` fourni par l'appelant.

```
local result, err = babet.archive.extractFile(
    "package.tar",
    "assets/logo.bin",
    "cache/current-logo.bin",
```

```
    { overwrite = true }
)
```

Résultat :

```
{
    bytes = 4096,
    path = "cache/current-logo.bin",
    entry = "assets/logo.bin",
}
```

Les limites anti-bombe restent calculées sur l'ensemble de l'archive. Pour TAR, la première passe consomme et valide tous les flux de données. La seconde compare chaque en-tête avec le plan inspecté et parcourt l'archive jusqu'à sa fin avant de publier le fichier temporaire sélectionné. Les données des autres fichiers réguliers sont ignorées ; un chemin dangereux, un type spécial ou un fichier sparse non sélectionné n'empêche donc pas de choisir un fichier régulier sûr. En revanche, toute donnée malformée ou tronquée ailleurs dans l'archive fait échouer l'opération sans publier la destination.

Règles de chemins

Pour une extraction complète, chaque nom d'entrée doit respecter toutes les règles suivantes :

— entre 1 et 4096 octets ;
— aucun octet NUL ;
— aucun `\` ;
— pas de `/` initial ;
— pas de préfixe de lecteur comme `C:` dans le premier composant ;
— aucun composant vide (`a//b`) ;
— aucun composant `.` ou `..` ;
— un `/` terminal uniquement pour une véritable entrée répertoire.

Les chemins restent relatifs à la destination. Babet ne transforme jamais un nom Windows en chemin Linux et ne « nettoie » jamais silencieusement une traversée : l'archive est refusée. Avec `extractFile()`, les mêmes règles s'appliquent au nom de l'entrée sélectionnée ; les noms dangereux non sélectionnés ne deviennent jamais des chemins de destination.

`read()` constitue volontairement l'exception : comme elle n'écrit rien, elle peut sélectionner un fichier régulier au nom dangereux par son nom brut ou son index. Elle ne normalise jamais ce nom et ne l'utilise jamais comme chemin.

Les doublons sont détectés après normalisation du `/` terminal des répertoires. Une archive contenant deux sorties identiques, ou `node` comme fichier puis `node/child`, est refusée avant extraction.

Protection de la destination

La sécurité ne repose pas seulement sur une comparaison de chaînes. Babet parcourt la destination composant par composant avec des descripteurs de répertoire et refuse de suivre les symlinks :

— un symlink dans le chemin racine de destination est refusé ;
— un parent symlinké sous la destination est refusé ;
— une cible finale symlinkée est refusée, même avec `overwrite = true` ;
— un fichier ne peut remplacer qu'un fichier régulier ;
— une entrée répertoire ne peut utiliser qu'un répertoire existant ou nouveau.

Les répertoires déjà présents sont acceptés s'ils sont réels et restent inchangés. Babet ne modifie pas leurs permissions.

Le chemin de destination lui-même est fourni par le script et peut être absolu ou relatif. Les règles sur `..` décrites plus haut concernent les noms non fiables contenus dans l'archive.

Publication atomique et nettoyage

`create()` écrit le ZIP, TAR brut, TAR gzip ou TAR xz complet dans un temporaire unique de mode `0600`, placé dans le répertoire de destination. Son nom interne court ne reprend pas le nom de sortie, afin qu'une destination valide proche de la limite `NAME_MAX` reste créable. Après finalisation du répertoire central ZIP par miniz ou du flux TAR par libarchive, Babet vide les tampons, exécute `fsync`, passe le fichier en `0644`, le publie atomiquement, puis synchronise le répertoire parent. Sans écrasement, la publication par hard link refuse toute cible apparue entre-temps ; avec écrasement, un `rename` du même répertoire remplace atomiquement la cible. Toute erreur antérieure à la publication supprime le temporaire et laisse l'ancienne destination intacte. Si le `fsync` final du répertoire parent échoue, la nouvelle archive est déjà publiée atomiquement, mais l'appel renvoie une erreur car la persistance après panne ne peut pas être confirmée.

Chaque fichier extrait est écrit progressivement vers un fichier temporaire unique créé dans son répertoire final avec le mode `0600`. Les noms temporaires sont choisis de façon à ne jamais entrer en collision avec une sortie déclarée par l'archive. L'écriture est bornée par la taille annoncée. Pour ZIP, miniz vérifie la décompression et le CRC. Pour TAR, libarchive fournit les blocs en flux et Babet vérifie leur ordre, leurs bornes et la taille finale annoncée. Pour un TAR enveloppé dans gzip, Babet valide en plus le CRC et l'ISIZE de chaque membre gzip avec zlib avant toute publication. Le temporaire reçoit ensuite ses permissions finales.

Tous les fichiers d'une extraction complète sont préparés avant le début de la publication : une erreur de lecture, de décompression ou de CRC ne publie donc aucun fichier. `extractFile()` conserve lui aussi son fichier sélectionné en temporaire jusqu'à la fin de la lecture vérifiée de toute l'archive. Les temporaires sont supprimés et les nouveaux répertoires encore vides créés **sous** la destination sont retirés. Les composants du chemin racine de destination qui ont dû être créés peuvent rester présents.

La publication est atomique **fichier par fichier** :

— sans écrasement, Babet utilise une création qui échoue si la cible existe ;
— avec écrasement, le temporaire du même répertoire remplace atomiquement le fichier régulier cible.

L'opération entière n'est toutefois pas une transaction multi-fichiers. Une erreur système très tardive pendant la publication ou la mise en permissions des répertoires peut survenir après la publication de fichiers précédents. Ces fichiers peuvent alors rester présents, et un ancien fichier déjà remplacé n'est pas restauré.

Pour l'extraction, la publication atomique protège contre un fichier partiellement visible, mais ne garantit pas la persistance après une panne brutale : les fichiers et répertoires extraits ne sont pas tous synchronisés par `fsync`. `create()` applique la séquence plus forte de synchronisation du fichier et du parent décrite plus haut.

Permissions

Par défaut :

```
fichiers                                  0644
répertoires finaux                        0755
temporaires                                0600
répertoires d'archive pendant la préparation 0700
composants créés du chemin destination 0755 (soumis à l'umask)
```

Avec `preserve_permissions = true`, Babet conserve uniquement les neuf bits ordinaires propriétaire/groupe/autres (`0777`) des modes Unix annoncés par le ZIP ou le TAR. Les bits setuid, setgid et sticky sont systématiquement supprimés.

Les permissions d'un répertoire créé sont appliquées seulement après l'extraction de son contenu, afin qu'un mode final restrictif comme `0000` n'empêche pas l'opération en cours. Les répertoires préexistants ne sont jamais rechmodés.

Les UID, GID, ACL, attributs étendus et dates ne sont pas restaurés.

Types d’entrées pris en charge

Type	`create()`	`list()`	`read()`	`extract()`	`extractFile()`
fichier régulier	pris en charge	inspecté	pris en charge, sauf TAR sparse	pris en charge, sauf TAR sparse	pris en charge, sauf TAR sparse
répertoire	pris en charge si activé	inspecté	non sélectionnable	pris en charge	non sélectionnable
fichier TAR sparse	jamais créé	identifié en TAR	refusé	refusé	refusé s’il est sélectionné
symlink	refusé s’il est sélectionné	identifié	refusé	refusé	refusé
hard link	jamais créé	identifié en TAR	refusé	refusé	refusé s’il est sélectionné
FIFO, socket, périphérique, type inconnu	refusé s’il est sélectionné	identifié si exposé	refusé	refusé	refusé
entrée chiffrée	jamais créée	identifiée en ZIP	refusée	refusée	refusée
méthode de compression ZIP inconnue	jamais créée	identifiée	refusée	refusée	refusée

Pendant la création, un symlink ou objet spécial retiré par les filtres est ignoré plutôt qu’ouvert. Les entrées de répertoires ne sont émises qu’avec `include_directories = true`.

Pour la création de fichiers réguliers, ZIP, gzip et xz acceptent les niveaux `0` à `9` ; bzip2 accepte `1` à `9` ; zstd accepte `0` à `19` ; un TAR brut stocke les octets source sans compression.

Une entrée ZIP qui ressemble à un hard link mais est encodée comme fichier régulier est traitée comme un fichier régulier indépendant ; aucun lien n’est créé.

Erreurs et limites connues

Les causes courantes d’échec sont notamment :

- source absente ou dangereuse, symlink source, nom source non UTF-8, type source non pris en charge, format de création invalide, option réservée au ZIP utilisée pour TAR ou date hors de la plage ZIP ;
- archive de sortie dans la source, parent de destination dangereux ou symlink cible ;
- fichier d’archive absent, illisible, non régulier, non pris en charge ou malformé ;
- données TAR tronquées, données finales qui ne constituent pas un TAR, contenu attaché à une entrée TAR non régulière ou changement détecté entre l’inspection et l’extraction ;
- métadonnées incohérentes ou CRC invalide ;
- limite anti-bombe dépassée ;
- `max_size` dépassée pendant une lecture en mémoire ;
- nom dupliqué ambigu pour `read()` sans index explicite ;
- chemin absolu, traversée, doublon ou conflit ;
- entrée chiffrée, symlink ou type non pris en charge ;
- destination existante sans `overwrite = true` ;
- symlink ou type dangereux dans la destination ;
- manque d’espace, permissions insuffisantes ou erreur d’E/S.

Le module est synchrone : l'appel ne revient qu'après la fin de la création, de l'inspection, de la lecture ou de l'extraction. Il n'expose pas encore de progression, d'annulation ou de timeout.

Les six fonctions publiques `create()`, `list()`, `read()`, `test()`, `extract()` et `extractFile()` partagent la même frontière d'exception C++. Un échec d'allocation interne devient `(nil, "archive: out of memory")` ; une autre exception interne devient `(nil, "archive: internal failure")` ou `(nil, "archive: unknown internal failure")`. Aucune exception C++ ne traverse la frontière Lua.

Le fichier d'archive ne doit pas être modifié simultanément. Une troncature ou modification ZIP provoque normalement une erreur de lecture, décompression ou CRC. Le TAR est parcouru en flux par libarchive ; les TAR concaténés sont lus jusqu'à leur fin et les données tronquées ou finales qui ne constituent pas un TAR sont refusées. Aucun des deux backends ne fournit d'instantané concurrent. La destination ne doit pas non plus être réorganisée simultanément par un autre processus. Les parcours refusent les symlinks et revérifient les types avant publication, mais un acteur qui renomme activement des répertoires peut faire échouer l'opération et rendre le nettoyage de ses temporaires seulement « meilleur effort ».

La création TAR brute, TAR gzip, TAR xz, TAR bzip2 et TAR zstd est prise en charge. Les flux compressés autonomes restent volontairement hors de cette API d'archive et sont traités par `babet.compression`.

Les archives multi-disques ne constituent pas une cible prise en charge. ZIP64 est accepté lorsque miniz peut le lire, tout en restant soumis aux limites configurées.

`babet.base64` — encodage et décodage Base64 binaires

Le module `babet.base64` implémente les alphabets Base64 de la RFC 4648 sans lancer de programme externe. Les entrées et sorties sont des chaînes Lua **binaires** : les octets NUL, les données non UTF-8 et toutes les valeurs de `0` à `255` sont conservés exactement.

Le décodeur est volontairement strict. Il vérifie l'alphabet choisi, la position du padding, la longueur du dernier groupe et les bits inutilisés de la dernière valeur. Une représentation non canonique n'est donc pas acceptée silencieusement.

Table des matières du module

— API
— Encodage standard
— Alphabet URL-safe
— Padding
— Décodage strict
— Espaces ASCII
— Limite de sortie
— Données binaires et fichiers
— Utilisation dans un worker
— Contrat d'erreur
— Limites et choix de conception

API

Fonction	Contrat
`babet.base64.encode(data [, opts])`	Renvoie `texte, nil` ou `nil, err`
`babet.base64.decode(text [, opts])`	Renvoie `données, nil` ou `nil, err`

`data` et `text` doivent être de vraies chaînes Lua. Un nombre n'est jamais converti implicitement en texte.

Options d'encodage :

```
{
    url_safe = false,
    padding = true,
}
```

Options de décodage :

```
{
    url_safe = false,
    allow_unpadded = false,
    ignore_whitespace = false,
    max_output = nil,
}
```

Les options inconnues sont refusées. Les booléens doivent être de vrais booléens Lua ; `max_output` doit être un entier positif ou nul.

Encodage standard

Sans option, `encode()` utilise l'alphabet Base64 standard : lettres, chiffres, `+` et `/`, avec padding canonique `=` lorsque le dernier groupe contient moins de trois octets.

```
local encoded, err = babet.base64.encode("hello")
assert(encoded, err)
assert(encoded == "aGVsbG8=")
```

Les vecteurs classiques de la RFC 4648 sont respectés :

```
assert(babet.base64.encode("")       == "")
assert(babet.base64.encode("f")      == "Zg==")
assert(babet.base64.encode("fo")     == "Zm8=")
assert(babet.base64.encode("foo")    == "Zm9v")
assert(babet.base64.encode("foob")   == "Zm9vYg==")
assert(babet.base64.encode("fooba")  == "Zm9vYmE=")
assert(babet.base64.encode("foobar") == "Zm9vYmFy")
```

`encode()` n'ajoute aucun retour à la ligne et ne produit pas le format MIME à 76 colonnes.

Alphabet URL-safe

`url_safe = true` sélectionne l'alphabet Base64 URL/file-name safe de la RFC 4648 : `-` remplace `+` et `_` remplace `/`.

```
local binary = string.char(251, 255)
local encoded, err = babet.base64.encode(binary, {
    url_safe = true,
})
assert(encoded, err)
assert(encoded == "-_8=")
```

Le décodage doit utiliser le même alphabet :

```
local decoded, err = babet.base64.decode("-_8=", {
    url_safe = true,
})
assert(decoded, err)
assert(decoded == string.char(251, 255))
```

Les alphabets ne sont pas mélangés automatiquement :

```
assert(babet.base64.decode("-_8=") == nil)
assert(babet.base64.decode("+/8=", { url_safe = true }) == nil)
```

Ce choix strict évite qu'une entrée soit acceptée sous plusieurs formes alors qu'un protocole attend un alphabet précis.

Padding

Produire une sortie sans =

`padding = false` retire uniquement les caractères = finaux. Le contenu utile reste identique.

```
local encoded, err = babet.base64.encode("hello", {
    padding = false,
})
assert(encoded, err)
assert(encoded == "aGVsbG8")
```

Lorsque la longueur source est un multiple de trois, aucun padding n'est nécessaire même avec l'option par défaut :

```
assert(babet.base64.encode("foo") == "Zm9v")
```

Accepter une entrée non paddée

Le décodeur refuse par défaut un dernier groupe incomplet sans = :

```
local decoded, err = babet.base64.decode("aGVsbG8")
assert(decoded == nil)
assert(type(err) == "string")
```

Active explicitement `allow_unpadded` lorsque le protocole utilise ce format :

```
local decoded, err = babet.base64.decode("aGVsbG8", {
    allow_unpadded = true,
})
assert(decoded, err)
assert(decoded == "hello")
```

`allow_unpadded` n'assouplit pas les autres règles : un groupe d'un seul symbole reste tronqué et des bits finaux non nuls restent refusés.

Exemple combiné URL-safe sans padding

C'est la forme fréquente dans des identifiants de protocole ou certains champs JSON :

```
local source = string.char(0, 1, 2, 251, 255)

local token, err = babet.base64.encode(source, {
    url_safe = true,
    padding = false,
})
assert(token, err)

local restored
restored, err = babet.base64.decode(token, {
    url_safe = true,
    allow_unpadded = true,
})
assert(restored, err)
assert(restored == source)
```

Décodage strict

Le décodeur vérifie notamment :

- chaque caractère appartient exactement à l'alphabet sélectionné ;
- = apparaît uniquement à la fin ;
- il existe au maximum deux caractères de padding ;
- la longueur paddée est un multiple de quatre ;
- une fin non paddée contient deux ou trois symboles, jamais un seul ;
- les bits inutilisés du dernier symbole sont nuls.

Ainsi, ces entrées sont refusées :

```
local invalid = {
    "%%%",   -- caractères hors alphabet
    "AA=A",  -- padding au milieu
    "A===",  -- trop de padding
    "Zg=",   -- longueur paddée invalide
    "Zh==",  -- bits inutilisés non nuls
    "Zm9=",  -- bits inutilisés non nuls
}

for _, text in ipairs(invalid) do
    local value, err = babet.base64.decode(text)
    assert(value == nil)
    assert(type(err) == "string")
end
```

Les positions signalées par les erreurs `invalid character`, `invalid padding` ou `non-zero trailing bits` sont comptées à partir de **1**, en octets dans la chaîne d'entrée originale.

Espaces ASCII

Par défaut, tout espace est un caractère invalide :

```
local value, err = babet.base64.decode("Zm9v\nYmFy")
assert(value == nil)
```

`ignore_whitespace = true` ignore les six espaces ASCII classiques : espace, tabulation, saut de ligne, tabulation verticale, saut de page et retour chariot.

```
local value, err = babet.base64.decode(" Zm9v\tYmFy\r\n", {
    ignore_whitespace = true,
})
assert(value, err)
assert(value == "foobar")
```

L'option ne supprime aucun autre octet Unicode ou de contrôle. Elle n'autorise pas non plus un mauvais padding ou un alphabet incorrect.

Lorsque des espaces sont ignorés, une position d'erreur continue de désigner la position dans la chaîne originale, espaces compris.

Limite de sortie

`max_output` borne la taille exacte du résultat décodé. La limite est contrôlée après la validation lexicale et canonique complète de la valeur Base64, puis avant l'allocation de la chaîne de sortie décodée.

```
local value, err = babet.base64.decode("Zm9v", {
    max_output = 2,
```

```
})
assert(value == nil)
assert(err == "base64: decoded output exceeds max_output")
```

La limite est inclusive :

```
local value, err = babet.base64.decode("Zm9v", {
    max_output = 3,
})
assert(value, err)
assert(value == "foo")
```

`max_output = 0` accepte uniquement une sortie vide :

```
assert(babet.base64.decode("", { max_output = 0 }) == "")
assert(babet.base64.decode("Zg==", { max_output = 0 }) == nil)
```

Pour une valeur Base64 non fiable, fixe une limite adaptée au protocole :

```
local png, err = babet.base64.decode(response.value, {
    max_output = 32 * 1024 * 1024,
})
assert(png, err)
```

La limite porte sur la sortie décodée, pas sur la longueur du texte Base64.

Données binaires et fichiers

Base64 n'impose aucun UTF-8. Une chaîne Lua contenant des octets arbitraires est acceptée :

```
local original = "\0\1\2abc\255"
local encoded = assert(babet.base64.encode(original))
local decoded = assert(babet.base64.decode(encoded))
assert(decoded == original)
```

Pour encoder un fichier :

```
local file = assert(io.open("capture.png", "rb"))
local data = assert(file:read("*a"))
assert(file:close())

local text, err = babet.base64.encode(data)
assert(text, err)
```

Pour écrire une valeur décodée :

```
local data, err = babet.base64.decode(text, {
    max_output = 32 * 1024 * 1024,
})
assert(data, err)

local ok
ok, err = babet.writeFileAtomic("capture-restored.png", data, {
    overwrite = true,
    permissions = tonumber("600", 8),
})
assert(ok, err)
```

`babet.writeFileAtomic()` publie les octets décodés depuis un fichier temporaire privé créé dans le même dossier et n'expose jamais une image partiellement écrite.

Utilisation dans un worker

Le sous-module est enregistré dans chaque état Lua worker :

```lua
local job = assert(babet.workers.spawn([[
    local encoded = assert(babet.base64.encode("hello"))
    local decoded = assert(babet.base64.decode(encoded))
    return decoded == "hello"
]]))

local ok, result = job:join()
assert(ok and result == true)
```

Le transport actuel de `worker.args`, du résultat et des files worker repose toujours sur JSON et refuse les chaînes contenant un octet NUL. Base64 peut transformer une donnée binaire en texte transportable, mais cette représentation augmente sa taille d'environ un tiers.

```lua
local encoded = assert(babet.base64.encode(binary_data))
local job = assert(babet.workers.spawn([[
    return assert(babet.base64.decode(worker.args.encoded))
]], { encoded = encoded }))
```

Attention : si le résultat décodé contient un NUL, il ne peut pas être renvoyé directement par le transport JSON actuel du worker. Le worker doit le traiter, l'écrire dans un fichier, ou le réencoder avant de le renvoyer.

Contrat d'erreur

Erreurs d'appel : exception Lua

Les erreurs de programmation lèvent une erreur Lua :

- nombre d'arguments incorrect ;
- donnée ou texte non chaîne ;
- options non table ;
- clé d'option non chaîne ;
- option inconnue ;
- booléen d'option mal typé ;
- `max_output` non entier ou négatif.

```lua
local ok, err = pcall(function()
    babet.base64.decode("Zm9v", { max_output = -1 })
end)
assert(ok == false)
```

Erreurs de données : `nil, err`

Une entrée Base64 invalide ou une limite dépassée renvoie `nil, err` :

```lua
local value, err = babet.base64.decode("%%%")
assert(value == nil)
assert(type(err) == "string")
assert(err:find("base64:", 1, true) == 1)
```

Les principales formes de message sont :

- `base64: invalid character at byte N;`
- `base64: invalid padding at byte N;`
- `base64: truncated input;`
- `base64: non-zero trailing bits at byte N;`
- `base64: decoded output exceeds max_output;`

— `base64: out of memory` ;
— `base64: encoded output too large` ou `decoded output too large`.

Les diagnostics `internal ... mismatch` et `unexpected internal error` sont des garde-fous : ils ne doivent pas apparaître pour une entrée utilisateur valide ou invalide ordinaire. Signale-les comme un défaut de Babet s'ils surviennent.

Ne dépends que des raisons documentées lorsque ton script doit distinguer une limite d'une donnée invalide. Le texte exact des erreurs d'allocation ou des positions peut être enrichi dans une version future.

Limites et choix de conception

Le module traite une chaîne complète en mémoire. Il ne fournit pas encore :

— d'encodeur ou décodeur en streaming ;
— de lecture/écriture directe de fichiers ;
— de retours à la ligne MIME ;
— de détection automatique entre alphabet standard et URL-safe ;
— de mode permissif acceptant un padding ou des bits finaux non canoniques.

Ces absences sont volontaires. L'API reste petite, déterministe et adaptée aux captures Selenium, jetons, payloads JSON, petites ressources binaires et échanges avec des outils externes.

Compression - flux gzip, xz, bzip2 et zstd autonomes

Périmètre

`babet.compression` compresse ou décompresse les octets d'un seul fichier régulier. Il traite des **flux autonomes**, et non des archives :

— gzip (`.gz` par convention) ;
— xz (`.xz`) ;
— bzip2 (`.bz2`) ;
— Zstandard (`.zst`).

Un flux autonome ne contient ni nom d'entrée portable, ni arborescence, ni permissions, ni métadonnées d'archive. Utilisez `babet.archive` pour les ZIP et les TAR, notamment `.tar.gz`, `.tar.xz`, `.tar.bz2` et `.tar.zst`.

Vue d'ensemble de l'API

```
local ok, err = babet.compression.compress(
    source, destination, format [, opts]
)

local ok, err = babet.compression.decompress(
    source, destination [, opts]
)
```

Les deux fonctions renvoient :

— `true, nil` en cas de succès ;
— `nil, "message"` en cas d'erreur opérationnelle ou d'option.

Un mauvais type pour un argument obligatoire, un octet NUL intégré ou une arité incorrecte provoque une erreur Lua. Les options inconnues sont refusées.

compress(source, destination, format [, opts])

`source`, `destination` et `format` sont des chaînes Lua strictes. Les chemins ne doivent être ni vides ni contenir d'octet NUL.

`format` doit être exactement l'une de ces valeurs :

`"gzip"` `"xz"` `"bzip2"` `"zstd"`

L'extension du fichier n'est ni examinée ni ajoutée automatiquement. Cet appel est donc valide :

```
local ok, err = babet.compression.compress(
    "donnees.bin",
    "donnees.compressees",
    "zstd"
)
```

La table facultative accepte :

Champ	Type	Défaut	Signification
`overwrite`	booléen	`false`	Remplacer atomiquement un fichier destination régulier existant.
`level`	entier Lua	selon le format	Choisir explicitement le niveau de compression.

`level` est un entier Lua strict. Les flottants, chaînes numériques et valeurs hors limites sont refusés avant l'ouverture du fichier source. Les niveaux acceptés et les valeurs utilisées lorsque le champ est absent sont :

Format	Minimum	Maximum	Défaut
gzip	0	9	6
xz	0	9	6
bzip2	1	9	9
zstd	1	22	3

Un niveau plus élevé privilégie généralement le taux de compression au prix de plus de temps processeur et parfois de davantage de mémoire. Pour gzip, le niveau `0` produit un flux valide sans compression DEFLATE. Les niveaux xz les plus élevés peuvent consommer beaucoup de mémoire ; ils ne doivent être choisis qu'explicitement.

Exemple

```
local ok, err = babet.compression.compress(
    "base.dump",
    "base.dump.zst",
    "zstd",
    { overwrite = true, level = 10 }
)

if not ok then
    io.stderr:write(err, "\n")
    return 1
end
```

`decompress(source, destination [, opts])`

Le format est détecté à partir des octets du flux et non de son extension. Renommer `fichier.gz` en `fichier.bin` ne change donc pas son décodage. Les données non compressées et les formats non pris en charge sont refusés.

La table facultative accepte :

Champ	Type	Défaut	Signification
`overwrite`	booléen	`false`	Remplacer atomiquement un fichier destination régulier existant.
`max_output_size`	entier Lua	1 Gio	Nombre maximal d'octets décompressés acceptés.

`max_output_size` doit être compris entre `1` et `68719476736` octets (64 Gio). La limite est contrôlée avant chaque écriture. En cas de dépassement, le fichier temporaire est supprimé et la destination n'est pas publiée.

Exemple

```
local ok, err = babet.compression.decompress(
    "base.dump.zst",
    "base.dump",
    {
        overwrite = true,
        max_output_size = 4 * 1024 * 1024 * 1024,
    }
)

if not ok then
    io.stderr:write(err, "\n")
    return 1
end
```

Flux concaténés et intégrité

Les membres gzip, flux xz, membres bzip2 et frames zstd valides concaténés sont décodés dans leur ordre au sein du même fichier destination.

Les décodeurs vérifient les informations d'intégrité obligatoires ainsi que tout checksum facultatif présent dans le flux. Les frames zstd produites par Babet activent le checksum de contenu facultatif. Une frame zstd externe qui ne le contient pas reste lisible, mais le format ne peut alors pas détecter toute altération possible des données. Babet refuse :

— les flux tronqués ;
— les checksums ou frames corrompus ;
— les octets arbitraires ajoutés après le dernier membre ou la dernière frame ;
— les entrées dont la signature ne correspond à aucun format pris en charge.

Garanties sur les fichiers et la publication

Le module applique une politique fail-closed :

— la source doit être un véritable fichier régulier ;
— une source qui est un symlink est refusée ;
— les parents de la source et de la destination doivent déjà exister et sont ouverts composant par composant avec `O_NOFOLLOW` ;
— les composants `..` sont refusés dans ces chemins parents ;
— un symlink ou un objet non régulier en destination est refusé ;
— la source et la destination ne doivent pas désigner le même inode, y compris via un lien physique ;
— le descripteur source est épinglé et sa taille ainsi que ses timestamps sont revérifiés avant publication ;
— la sortie est écrite progressivement dans un temporaire de mode `0600` situé dans le dossier destination ;
— le fichier terminé est synchronisé, placé en mode `0644`, puis publié atomiquement ;
— avec `overwrite = false`, la publication ne peut pas remplacer une destination apparue concurremment ;
— les temporaires sont supprimés lors des échecs antérieurs à la publication.

Le module n'utilise jamais de shell et ne déduit jamais le nom de destination de métadonnées compressées.

Données binaires et mémoire

Les entrées et sorties sont des flux d'octets. Les octets NUL et les contenus binaires arbitraires sont préservés.

La compression et la décompression utilisent des buffers de travail bornés de 64 Kio. L'entrée ou la sortie complète n'est chargée ni dans l'état Lua ni dans une chaîne C++ unique. `max_output_size` borne en plus l'expansion lors de la décompression. Le décodeur xz possède un plafond mémoire interne de 256 Mio et le décodeur zstd refuse les fenêtres supérieures à 128 Mio.

Frontière d'exception C++

Les deux fonctions publiques passent par la frontière commune Lua/C++. Cela ne modifie aucun retour ordinaire et empêche une exception C++ inattendue de traverser les frames C de Lua :

— échec d'allocation : `(nil, "compression: out of memory");`
— autre exception standard : `(nil, "compression: internal failure");`
— exception inconnue : `(nil, "compression: unknown internal failure").`

Le descripteur source et la sortie non publiée restent possédés par des gardes RAII pendant l'opération. Sur l'un de ces échecs, les descripteurs sont fermés et le temporaire du dossier destination est supprimé avant le retour du diagnostic.

Workers

`babet.compression` est enregistré dans chaque état Lua worker. Plusieurs workers peuvent traiter des fichiers différents en parallèle. Les règles ordinaires de collision du système de fichiers restent applicables lorsque plusieurs workers visent la même destination.

Exemples d'erreur

```lua
local ok, err = babet.compression.compress(
    "entree.bin", "entree.bin.gz", "zip"
)
-- nil, "compression: format must be 'gzip', 'xz', 'bzip2', or 'zstd'"

ok, err = babet.compression.decompress(
    "gros.gz", "gros.bin", { max_output_size = 1024 }
```

```
)
-- nil, "compression: decompressed output exceeds opts.max_output_size"
```

Une opération en échec doit toujours être traitée avant d'utiliser la destination.

CURSES — interfaces utilisateur en terminal

`babet.curses` fournit une petite API d'interface terminal UTF-8 basée sur **ncursesw** lié statiquement. Elle vise les applications interactives plein écran tout en conservant le principe de déploiement de Babet : un seul fichier.

La première API reste volontairement petite : un seul écran pour le processus, pas de fenêtres, panels, menus, forms, pointeurs ncurses ou constantes numériques exposés à Lua.

Vue d'ensemble de l'API

```
babet.curses.start()                  -- true
babet.curses.stop()                   -- true (sûr même si inactif)
babet.curses.clear()                  -- true
babet.curses.refresh()                -- true
local lignes, colonnes = babet.curses.size()
babet.curses.move(ligne, colonne)     -- true ; coordonnées à partir de 1
babet.curses.write(texte)             -- true ; UTF-8 valide obligatoire
local touche, err = babet.curses.readKey([timeout_secondes])
```

Les appels lèvent une erreur Lua si les arguments sont invalides ou si le terminal/ncurses échoue. Pour `readKey(timeout)`, un délai expiré normal renvoie `nil, "timeout"`. Une lecture bloquante interrompue sans touche livrée peut renvoyer `nil, "interrupted"`.

Démarrer et arrêter

```
assert(babet.curses.start())
assert(babet.curses.clear())
assert(babet.curses.move(1, 1))
assert(babet.curses.write("Bonjour é Ω"))
assert(babet.curses.refresh())
local touche = babet.curses.readKey()
assert(babet.curses.stop())
```

`start()` exige que `stdin` et `stdout` soient **le même TTY**, que `TERM` soit non vide et qu'une locale `LC_CTYPE` UTF-8 soit disponible. Il doit être appelé sur le thread principal de Babet. Une seconde session simultanée est refusée.

Un `start()` réussi active à la demande le hook Lua partagé des signaux et du terminal sur le thread Lua principal et la coroutine appelante. Les nouvelles coroutines héritent du hook de leur créatrice ; les autres coroutines déjà existantes ne sont pas modifiées. Démarre curses avant de créer les coroutines qui doivent servir les événements pendant des boucles Lua pures. Le hook reste installé après `stop()` ; voir son contrat et son coût.

`stop()` est idempotent. Si un enfant interactif possède alors le terminal, `stop()` termine la session curses logique **sans reprendre le TTY à l'enfant**. Le gestionnaire de processus normal restaurera ensuite le terminal parent quand l'enfant se terminera ou s'arrêtera.

La fin normale, une erreur Lua non interceptée, une exception C++ maîtrisée et les terminaisons par défaut `SIGINT`/`SIGTERM`/`SIGHUP` restaurent le terminal avant la sortie de Babet. Un crash fatal ou `SIGKILL` ne peut offrir cette garantie ; la commande `reset` reste le recours manuel classique après un terminal endommagé.

Dans le CLI et les exécutables générés, `os.exit(...)` restaure aussi curses, même sans demander la fermeture Lua. Les codes de sortie sont conservés. Voir le contrat de sortie pour les finaliseurs, les workers actifs et les callbacks natifs.

Opérations d'écran

`size()` renvoie `(lignes, colonnes)` sous forme d'entiers positifs.

`move(ligne, colonne)` utilise des coordonnées **à partir de 1**. Une position hors écran lève une erreur.

`write(texte)` écrit de l'UTF-8 valide à la position courante. Les NUL intégrés et l'UTF-8 invalide sont refusés. L'affichage n'est pas automatiquement poussé : utiliser `refresh()` lorsque la mise à jour doit devenir visible.

`clear()` efface l'écran standard et `refresh()` publie les changements en attente.

Lecture clavier

```lua
local touche, err = babet.curses.readKey(0.5)
if not touche and err == "timeout" then
    -- aucune entrée pendant 500 ms
elseif touche == "up" then
    -- flèche haut
elseif touche == "resize" then
    local lignes, colonnes = babet.curses.size()
end
```

Les caractères ordinaires sont renvoyés comme chaînes Lua UTF-8. Les touches spéciales utilisent des noms symboliques stables plutôt que les nombres ncurses :

`up`, `down`, `left`, `right`, `home`, `end`, `page_up`, `page_down`, `backspace`, `delete`, `insert`, `enter`, `resize` et `f1` à `f63`. Une autre touche spéciale reconnue par ncurses renvoie `"special"`.

`SIGWINCH` est coalescé puis traité sur le thread principal. Une fois la nouvelle taille appliquée, `readKey()` renvoie l'événement symbolique `"resize"`.

Si un callback de signal appelle `curses.stop()` pendant `readKey`, la lecture renvoie `nil, "interrupted"`, même sans timeout. Elle n'accède plus à l'écran libéré, y compris si une touche avait été lue juste avant le callback.

Processus enfants interactifs

Babet conserve **un seul propriétaire du terminal interactif à la fois**. Depuis le thread principal, un enfant interactif fonctionne naturellement pendant une session curses :

```lua
assert(babet.curses.start())

local p = assert(babet.spawn("sh", {}, {
    stdin = "inherit",
    stdout = "inherit",
    stderr = "inherit",
}))
local resultat = assert(p:wait())
p:close()

-- curses est à nouveau restauré sur le thread principal
assert(babet.curses.write("enfant terminé"))
assert(babet.curses.refresh())
assert(babet.curses.stop())
```

Avant de céder le TTY, Babet sauvegarde le mode programme curses et quitte curses. Le gestionnaire terminal existant donne alors le premier plan au groupe de processus de l'enfant. À sa fin ou à son arrêt, le thread moniteur n'effectue que la récupération POSIX ; la restauration ncurses elle-même est différée sur le thread principal.

Un processus interactif arrêté peut être repris au premier plan avec `process:resume(true)` par le même mécanisme.

Workers

Les appels ncurses sont réservés au thread principal. Dans un worker :

- toute opération `babet.curses.*` est refusée ;
- un `babet.spawn()` interactif héritant du terminal est refusé tant que la session curses principale est active ;
- les processus non interactifs utilisant pipes/fichiers/null restent permis.

Cela évite tout accès ncurses multithread et toute ambiguïté de propriétaire du TTY.

Signaux

Babet reste propriétaire de la politique des signaux pendant curses :

- `SIGWINCH` devient un événement `"resize"` différé ;
- Ctrl-Z (`SIGTSTP`) restaure le terminal avant la suspension, puis curses est réactivé après `SIGCONT` ;
- les handlers `babet.signal` de `INT`, `TERM` et `HUP` restent prioritaires ;
- si ces signaux gardent leur action système par défaut, Babet restaure curses sur le thread principal avant de relancer le signal avec son action par défaut.

Aucune fonction ncurses n'est appelée depuis un handler de signal fatal.

TERM et terminfo

Babet préfère la base terminfo du système. Son ncursesw statique embarque aussi les fallbacks suivants :

```
linux, vt100, xterm, xterm-256color,
screen, screen-256color, tmux, tmux-256color
```

Les terminaux courants restent ainsi utilisables même sans arborescence terminfo système, y compris dans un exécutable généré autonome. Babet ne remplace jamais silencieusement un `TERM` inconnu. Un `TERM` absent/inconnu ou un terminfo inutilisable devient une erreur Lua normale au lieu de laisser ncurses quitter le processus.

Déploiement

ncursesw est lié statiquement au runtime Babet normal. `--create-exe` n'a donc besoin ni de compilateur, ni de linker, ni de `libncursesw.so`, ni de dossier terminfo externe lors de la création ou à l'exécution. Une application générée embarque le même runtime curses que le vrai binaire Babet.

`argparse` — arguments de ligne de commande

`argparse` est un module Lua pur embarqué dans Babet. Il construit un parseur déclaratif pour les flags, les options à valeur et les arguments positionnels :

```lua
local argparse = require("argparse")
local parser = argparse("mon-script", "Description facultative")
```

Le module ne fait ni `print` ni `os.exit`. Il renvoie l'aide et les erreurs au script, qui décide comment les présenter et quel code de sortie utiliser.

Table des matières du module

— Chargement et métadonnées
— API
— Déclarer des flags et des options
— Déclarer des positionnels
— Table `opts`
  — Champs communs
  — Champs réservés aux valeurs
  — Destination
— Source des arguments
  — Table globale `arg`
  — Array explicite
— Formes reconnues
— Valeurs par défaut, choix et conversion
— Résultats de `parse`
  — Succès normal
  — Aide intégrée
  — Erreur utilisateur
  — Erreur de programmation
— Exemple complet
— Texte produit par `get_usage`
— Limites de la version actuelle

Chargement et métadonnées

```lua
local argparse = require("argparse")

print(argparse._VERSION)      -- "babet argparse 1.1.0"
print(argparse._DESCRIPTION)  -- description en anglais
```

`require("argparse")` renvoie une table appelable. Le constructeur accepte zéro, un ou deux arguments :

```lua
argparse()                          -- programme affiché : "prog"
argparse("outil")                   -- nom du programme
argparse("outil", "Description") -- nom et description
```

`prog` et `description` doivent être des chaînes lorsqu'ils sont fournis. Un argument supplémentaire ou un mauvais type lève une erreur Lua : il s'agit d'une erreur de programmation, pas d'une erreur de ligne de commande.

API

Méthode	Résultat	Rôle
`parser:flag(spec [, opts])`	`parser`	Déclare un flag sans valeur.
`parser:option(spec [, opts])`	`parser`	Déclare une option qui consomme une valeur.
`parser:argument(name [, opts])`	`parser`	Déclare un argument positionnel.
`parser:parse([src])`	`res, err`	Parse `src` ou la table globale `arg`.
`parser:get_usage()`	chaîne	Génère le texte d'aide.

Les méthodes du builder sont chaînables :

```
local parser = argparse("copie")
    :flag("-v --verbose")
    :option("-o --output")
    :argument("source")
```

Les arités sont strictes. Par exemple, `parser:option("-o", {}, "extra")` et `parser:get_usage(true)` lèvent une erreur Lua.

Déclarer des flags et des options

`spec` est une seule chaîne contenant un ou plusieurs noms séparés par des espaces :

```
parser:flag("-v")
parser:flag("--verbose")
parser:flag("-v --verbose")
parser:option("-o --output")
```

Chaque nom doit :

- commencer par un ou deux tirets ;
- contenir au moins un caractère après les tirets ;
- ne pas contenir `=` ;
- ne pas être dupliqué dans la même spec ni dans le parseur.

Les noms suivants sont réservés et ne peuvent pas être déclarés :

- `-` : traité comme un positionnel littéral ;
- `--` : termine le parsing des options ;
- `-h` et `--help` : aide intégrée.

Un nom à trois tirets comme `---verbose` est également refusé.

Toutes les validations sont effectuées avant de modifier le parseur. Si une erreur du builder est interceptée par `pcall`, aucune option ni aucun alias partiel n'est conservé.

Déclarer des positionnels

```
parser:argument("input")
parser:argument("output", { required = false })
```

Le nom doit être une chaîne non vide, sans espace et sans tiret initial. Les positionnels sont attribués strictement de gauche à droite.

Un positionnel est requis par défaut. Il devient optionnel si :

- `required = false` est indiqué ; ou
- une valeur `default` est fournie sans préciser `required`.

Tous les positionnels requis doivent précéder les positionnels optionnels. Le builder refuse donc ce cas ambigu :

```
parser
    :argument("facultatif", { required = false })
    :argument("requis") -- erreur Lua
```

Table `opts`

`opts` doit être une table ou `nil`. Les champs inconnus et les champs dont le type est incorrect lèvent immédiatement une erreur Lua. Cela permet de détecter un nom mal orthographié comme `hlep` au lieu de `help`.

Champs communs

Champ	Type	Effet
`help`	chaîne	Texte ajouté par `get_usage()`.
`default`	toute valeur Lua	Valeur utilisée lorsque l'élément est absent.
`dest`	chaîne non vide	Clé utilisée dans la table résultat.
`required`	booléen	Exige la présence de l'élément.

Ces quatre champs sont acceptés par `flag`, `option` et `argument`.

Champs réservés aux valeurs

Champ	Type	Effet
`choices`	array dense de chaînes	Autorise uniquement certaines valeurs brutes.
`convert`	fonction	Convertit la chaîne brute en une valeur Lua.

`choices` et `convert` sont acceptés par `option` et `argument`, mais pas par `flag`, puisqu'un flag ne consomme aucune valeur.

Le builder copie l'array `choices`. Une modification ultérieure de la table fournie ne change donc pas le parseur.

Destination

Sans `dest`, la destination est :

1. le premier nom long sans `--`, s'il existe ;
2. sinon le premier nom court sans son premier `-`.

```
parser:option("-o --output") -- destination : "output"
parser:flag("-v")            -- destination : "v"
```

Une destination doit être unique dans tout le parseur. Les destinations `help` et `usage` sont réservées au résultat de l'aide intégrée.

```
parser:flag("-q", { dest = "quiet" })
```

Source des arguments

Table globale `arg`

Sans argument, `parse()` lit la table globale `arg` aux indices `1`, `2`, et ainsi de suite jusqu'au premier trou :

```
local result, err = parser:parse()
```

Les indices `0` et négatifs sont ignorés. `parse(nil)` est équivalent à `parse()`.

Chaque valeur rencontrée doit être une chaîne. Si `arg` n'est pas une table, le parseur utilise une liste vide.

Array explicite

Pour les tests ou une source construite par le programme :

```
local result, err = parser:parse({ "-v", "input.txt" })
```

La table doit être un array dense indexé de `1` à `n`, sans trou, sans clé supplémentaire et composé uniquement de chaînes. Une source invalide est une erreur de parsing et renvoie `(nil, err)` ; elle ne lève pas d'exception.

Formes reconnues

Une option à valeur accepte :

```
--long valeur
--long=valeur
-s valeur
-s=valeur
```

Une valeur inline vide est valide :

```
parser:parse({ "--output=" }) -- output == ""
```

Les options restent actives après un positionnel tant que `--` n'a pas été rencontré :

```
input.txt --verbose
```

`--` termine le parsing des options. Tous les tokens suivants deviennent des positionnels, même s'ils commencent par `-` :

```
parser:parse({ "--", "-5" })
```

Avant `--`, un token comme `-5` est interprété comme un nom d'option. Pour un nombre négatif fourni comme valeur d'une option, aucun séparateur n'est nécessaire :

```
parser:parse({ "--count", "-5", "input.txt" })
```

Le token suivant une option à valeur est toujours consommé littéralement, même s'il ressemble à une option ou s'il vaut `--` :

```
parser:parse({ "--output", "--verbose" })
-- output == "--verbose" ; le flag verbose n'est pas activé
```

Le token isolé `-` est un positionnel. Les options répétées sont acceptées : la dernière valeur gagne. Un flag répété reste simplement à `true`.

Les formes suivantes ne sont pas prises en charge :

— options courtes agglomérées, par exemple `-abc` pour trois flags ;
— valeur courte collée, par exemple `-ofile.txt` ;
— positionnels variadiques.

Un nom déclaré littéralement `-abc` reste toutefois une option normale si la spec contient exactement ce nom ; il n'est jamais décomposé.

Valeurs par défaut, choix et conversion

L'ordre appliqué à une valeur fournie par l'utilisateur est :

1. contrôle de `choices` sur la chaîne brute ;
2. appel de `convert` avec cette chaîne.

```
parser:option("-n --count", {
    choices = { "1", "2", "3" },
    convert = tonumber,
})
```

Les éléments de `choices` sont toujours des chaînes. Si la valeur brute ne figure pas dans la liste, `parse()` renvoie `(nil, err)` sans appeler le convertisseur.

Un convertisseur peut renvoyer :

— toute valeur non `nil`, y compris `false`, pour réussir ;
— `nil` pour échouer avec le message générique `invalid value` ;
— `nil, "raison"` pour fournir une raison ;
— lever une erreur, qui est interceptée et devient `conversion error`.

Les valeurs `default` sont des valeurs Lua déjà prêtes : elles ne sont ni contrôlées par `choices` ni passées à `convert`. Elles gardent leur type et, pour une table, leur identité. Une option `required = true` reste requise même si un `default` est présent.

Pour un flag :

— absent, il vaut `default` si ce champ est défini, sinon `false` ;
— présent, il vaut toujours `true`.

Une option ou un positionnel absent avec un défaut `nil` n'ajoute pas de clé observable dans la table résultat, conformément au comportement des tables Lua.

Résultats de `parse`

Succès normal

```lua
local result, err = parser:parse()
-- result : table
-- err    : nil
```

La table est indexée par les destinations déclarées.

Aide intégrée

Avant `--`, `-h` ou `--help` arrête immédiatement le parsing et renvoie :

```lua
{
    help = true,
    usage = "...",
}, nil
```

Les options requises, les positionnels manquants et les tokens qui suivent ne sont pas vérifiés. Après `--`, `-h` et `--help` sont de simples positionnels.

L'aide est un succès et n'est pas ajoutée à une table de résultat normale. Les destinations `help` et `usage` sont donc réservées pour éviter toute ambiguïté dans le code appelant.

Erreur utilisateur

Les erreurs de ligne de commande renvoient `(nil, message)` :

```
unknown option '--bad'
option '--output' requires a value
flag '-v' does not take a value
missing required option '--output'
missing required argument 'input'
argument 'mode': invalid choice 'other'
option '--count': invalid value
unexpected argument 'extra'
```

`parse()` intercepte les erreurs levées par `convert`. Une valeur utilisateur ne fait donc pas remonter l'exception du convertisseur.

Erreur de programmation

Le constructeur et le builder lèvent une erreur Lua pour :

- une mauvaise arité ;
- un type incorrect ;
- une spec vide ou inutilisable ;
- un nom ou une destination réservé ;
- un nom ou une destination dupliqué ;
- un champ `opts` inconnu ;
- un array `choices` invalide ;
- un positionnel requis placé après un positionnel optionnel.

La mauvaise arité de `parse` est également une erreur de programmation. En revanche, une table `src` mal formée renvoie `(nil, err)`.

Exemple complet

```lua
local argparse = require("argparse")

local parser = argparse("convert", "Convertit un fichier.")
    :flag("-v --verbose", { help = "Plus de détails" })
    :option("-o --output", { default = "out.txt" })
    :option("-n --count", {
        choices = { "1", "2", "3" },
        convert = tonumber,
        default = 1,
    })
    :argument("input")

local args, err = parser:parse()
if args and args.help then
    print(args.usage)
    return
end
if not args then
    io.stderr:write("erreur : ", err, "\n")
    io.stderr:write(parser:get_usage(), "\n")
    os.exit(2)
end

print(args.input, args.output, args.count, args.verbose)
```

Texte produit par `get_usage`

`get_usage()` renvoie une chaîne et n'écrit rien. Les titres et le message de l'aide intégrée sont fixes et en anglais : `Usage`, `Arguments`, `Options` et `show this help`. Il n'existe pas encore d'API de traduction, de largeur de colonne ou de groupes visuels.

Le texte affiche les positionnels et les aides déclarées, mais n'ajoute pas automatiquement les valeurs par défaut, les choix ni le caractère requis des options.

Limites de la version actuelle

Ne sont pas exposés :

- les sous-commandes ;
- les positionnels variadiques (`nargs`) ;

— les groupes mutuellement exclusifs ;
— les options répétables accumulées dans un array ;
— les options courtes agglomérées ;
— la personnalisation ou la localisation du texte d'aide ;
— une option permettant de désactiver l'aide intégrée.

EXEC - programmes externes, capture et streaming

Babet propose deux API complémentaires pour lancer un programme externe sans shell implicite :

— `babet.exec` envoie éventuellement une entrée complète, capture `stdout` et `stderr` en mémoire, puis attend la fin du programme ;
— `babet.spawn` renvoie immédiatement un objet processus pilotable, afin de lire et écrire progressivement sans accumuler automatiquement les flux en RAM.

Le module EXEC couvre :

— la recherche d'un programme dans `PATH` ;
— la construction sûre de `argv`, sans parsing shell ;
— un répertoire de travail spécifique au processus enfant ;
— la fusion d'un environnement personnalisé avec celui de Babet ;
— l'envoi d'une chaîne binaire sur l'entrée standard ;
— la capture binaire de `stdout` et `stderr` ;
— un timeout couvrant le lancement, les échanges et l'attente finale ;
— une limite de mémoire distincte pour chaque flux de sortie ;
— l'arrêt du groupe de processus enfant en cas de timeout.

`babet.exec` ne fournit pas de redirection directe vers un fichier : il capture `stdout` et `stderr` en mémoire. Pour lancer un processus pilotable, `babet.spawn` permet de configurer séparément `stdin`, `stdout` et `stderr` avec les modes `pipe`, `inherit` ou `null`, ainsi que de rediriger `stdout` et `stderr` vers un fichier.

`babet.exec` ne fournit pas non plus d'interpréteur de commandes. Pour relier plusieurs programmes sans shell, utilise `babet.pipeline()` ou `babet.spawnPipeline()`.

Table des matières du module

— Conventions essentielles
— Vue d'ensemble de l'API
— Signature et arguments
  — `cmd` et recherche du programme
  — `args` et absence de shell
  — Utiliser explicitement un shell
— Options
  — `cwd`
  — `env`
  — `stdin`
  — `timeout`
  — `max_output`
— Table de résultat
  — Code de sortie normal
  — Fin par signal
  — Timeout
  — Troncature
— Exemples complets
  — Capture simple
  — Arguments contenant des espaces
  — Traiter un code non nul

    - Capturer `stderr`
    - Envoyer du texte ou du binaire sur stdin
    - Modifier l'environnement
    - Changer de répertoire
    - Borner le temps
    - Borner la sortie
    - Créer un helper applicatif
- Processus en streaming avec `babet.spawn`
    - Signature et options
    - Méthodes du processus
    - Lire stdout et stderr
    - Écrire sur stdin
    - Attendre, terminer et fermer
    - Exemple de boucle de streaming
    - Limites et prévention des blocages
- Contrat d'erreur
- Processus, signaux et sécurité
- Décisions et limites

Conventions essentielles

Aucun shell implicite

Cette forme :

```
local result, err = babet.exec("printf", { "%s\n", "hello world" })
```

lance directement `printf`. Chaque élément de la table `args` devient un slot séparé de `argv`. Les espaces, `$`, `*`, `;`, `>`, les guillemets et les autres caractères spéciaux ne sont pas interprétés par un shell.

Cette propriété évite la plupart des problèmes de quoting et des injections de commandes.

Une sortie non nulle n'est pas une erreur de lancement

Un programme qui démarre correctement puis renvoie le code `1`, `2` ou `127` produit normalement une table de résultat. L'appelant décide si ce code est acceptable.

```
local result, err = babet.exec("grep", { "needle", "data.txt" })
assert(result, err)

if result.code == 0 then
    print("motif trouvé")
elseif result.code == 1 then
    print("aucune correspondance")
else
    error(result.stderr)
end
```

`assert(babet.exec(...))` vérifie seulement que Babet a pu lancer et suivre le programme. Il ne vérifie pas `result.code`.

Les chaînes de données sont binaires

`opts.stdin`, `result.stdout` et `result.stderr` sont des chaînes Lua binary-safe. Elles peuvent contenir des octets NUL et ne sont pas limitées à du texte UTF-8.

Les chaînes transmises à des API POSIX qui attendent une chaîne C - `cmd`, les éléments de `args`, `cwd`, les noms et valeurs de variables d'environnement - ne peuvent en revanche pas contenir d'octet NUL.

Vue d'ensemble de l'API

```
local result, err = babet.exec(cmd, args?, opts?)
```

Élément	Type	Défaut	Rôle
`cmd`	string stricte	obligatoire	programme ou chemin à exécuter
`args`	séquence de strings	aucun argument	éléments `argv[1]..argv[n]`
`opts.cwd`	string	répertoire courant hérité	répertoire de travail de l'enfant
`opts.env`	table string -> string	environnement hérité	variables ajoutées ou remplacées
`opts.stdin`	string binaire	stdin fermé immédiatement	données envoyées à l'enfant
`opts.timeout`	nombre fini > 0	aucune limite	budget global en secondes
`opts.max_output`	integer `1..2 Gio`	10 Mio	cap par flux capturé

Résultats principaux :

Situation	Résultat
programme lancé et terminé	`(result, nil)`
programme lancé puis timeout	`(result, nil)` avec `timed_out = true`
code de sortie non nul	`(result, nil)` avec `code ~= 0`
programme introuvable ou `cwd` invalide	`(nil, err)`
`args` ou `opts` invalides	`(nil, err)`
`cmd` absent ou non string	erreur Lua levée

Signature et arguments

`cmd` et recherche du programme

`cmd` doit être une véritable chaîne Lua.

```
local result, err = babet.exec("git", { "status", "--short" })
```

Si `cmd` ne contient pas de `/`, Babet construit d'abord l'environnement final de l'enfant dans le parent, puis recherche le programme avec la valeur de `PATH` réellement transmise. Un `PATH` remplacé dans `opts.env` contrôle donc à la fois la recherche initiale et les sous-commandes lancées ensuite.

```
local result = assert(babet.exec("my-private-tool", {}, {
    env = { PATH = "/opt/private/bin:/usr/bin:/bin" },
}))
```

Lorsque l'environnement effectif ne contient aucune variable `PATH`, Babet utilise le chemin de recherche POSIX fourni par le système (`_CS_PATH`, avec `/bin:/usr/bin` comme repli). Il n'ajoute pas cette valeur à l'environnement de l'enfant.

Chaque composante vide de `PATH` désigne le répertoire de travail effectif de l'enfant. Une composante relative est elle aussi résolue depuis ce répertoire, après application éventuelle de `opts.cwd`.

```
local result = assert(babet.exec("tool", {}, {
    cwd = "/opt/my-app",
    env = { PATH = "bin:/usr/bin:/bin" },
}))
-- recherche d'abord /opt/my-app/bin/tool
```

Babet poursuit la recherche après `ENOENT` ou `ENOTDIR`. Un candidat présent mais non exécutable (`EACCES`) n'empêche pas un candidat ultérieur de réussir ; si aucun candidat ne fonctionne et qu'au moins un accès a été refusé, l'erreur finale reste `EACCES` plutôt que d'être transformée en « introuvable ».

Si `cmd` contient un `/`, `PATH` n'est pas consulté. Un chemin absolu est transmis directement à `execve()` ; un chemin relatif est interprété depuis le répertoire de l'enfant.

```
local result = assert(babet.exec("./tool", {}, {
    cwd = "/opt/my-app/bin",
}))
```

La liste ordonnée des candidats, `argv` et `envp` sont préparés avant `fork()`. L'enfant n'alloue rien et ne reparcourt pas `PATH` : il essaie uniquement ces chemins avec `execve()`, en poursuivant après `ENOENT`, `ENOTDIR` ou `EACCES`. Un fichier peut néanmoins être supprimé ou remplacé avant l'un de ces appels ; Babet renvoie alors `(nil, err)` avec l'erreur système observée. Un fichier exécutable dont le format n'est pas reconnu renvoie également une erreur de lancement (`ENOEXEC`) : Babet ne tente jamais de le relancer implicitement avec un shell. Lors d'une migration depuis Babet 2.16.x, ajoutez donc un shebang aux scripts exécutables qui en sont dépourvus, par exemple `#!/bin/sh`.

`args` et absence de shell

`args` est facultatif. Il doit être une table représentant une séquence dense `1..n` de chaînes.

```
local result = assert(babet.exec("cp", {
    "--",
    "fichier avec espaces.txt",
    "destination/",
}))
```

Babet lit uniquement les entrées brutes de la séquence dense `1..n`. Les métaméthodes `__len` et `__index` ne sont pas invoquées. Pour un comportement prévisible :

- utilise des indices consécutifs à partir de `1` ;
- ne laisse aucun trou ;
- ne place pas de métadonnées dans la même table ;
- utilise uniquement des chaînes.

```
-- Correct
local args = { "-c", "print('ok')" }

-- Incorrect : valeur numérique
local result, err = babet.exec("lua", { "-e", 42 })
-- result == nil ; err indique que tous les arguments doivent être strings
```

`argv[0]` est automatiquement défini à `cmd`. Le premier élément de `args` devient `argv[1]`.

Utiliser explicitement un shell

Les opérateurs `|`, `>`, `&&`, `;`, les variables `$NAME` et les globs ne sont interprétés que si tu lances volontairement un shell.

```
local result = assert(babet.exec("sh", {
    "-c",
    "printf '%s\n' \"$HOME\" | sed 's#^#home=#'",
}))
```

Les données incorporées dans une commande shell doivent alors être quotées avec les règles du shell. Pour des valeurs non fiables, préfère les transmettre via `args`, `stdin` ou `env` plutôt que de les concaténer dans le script `-c`.

Options

Le troisième argument est facultatif. S'il est fourni, il doit être une table. Les champs inconnus sont actuellement ignorés ; une faute de frappe comme `timeot = 5` ne déclenche donc pas d'erreur. Utilise exactement les noms ci-dessous.

opts.cwd

Définit le répertoire courant du programme enfant sans modifier celui du processus Babet.

```
local result, err = babet.exec("pwd", {}, {
    cwd = "/var/tmp",
})
assert(result, err)
print(result.stdout) -- /var/tmp
```

Le `chdir` est réalisé dans l'enfant avant le lancement du programme. Il affecte notamment :

- les chemins relatifs utilisés par le programme ;
- un `cmd` relatif contenant `/` ;
- les composantes relatives du `PATH` utilisé pour la recherche.

Si le dossier n'existe pas ou n'est pas accessible, Babet renvoie `(nil, err)`. Le répertoire courant de Babet reste inchangé.

opts.env

`env` est une table dont chaque clé et chaque valeur doit être une chaîne. L'environnement est **fusionné** avec celui du processus Babet.

```
local result = assert(babet.exec("sh", {
    "-c", "printf '%s' \"$APP_MODE\"",
}, {
    env = { APP_MODE = "production" },
}))

assert(result.stdout == "production")
```

Règles :

- une clé fournie remplace la variable héritée de même nom ;
- une clé absente reste héritée ;
- une valeur `""` crée ou conserve une variable définie mais vide ;
- il n'existe pas d'option pour supprimer complètement une variable ;
- une clé doit être non vide et ne pas contenir `=` ;
- clés et valeurs refusent les octets NUL ;
- les types autres que string sont refusés.

Définie mais vide n'est pas la même chose qu'absente :

```
local result = assert(babet.exec("sh", { "-c", [[
    if [ "${TOKEN+x}" = x ] && [ -z "$TOKEN" ]; then
        printf 'defined-empty'
    fi
]] }, {
    env = { TOKEN = "" },
}))

assert(result.stdout == "defined-empty")
```

L'environnement est préparé dans le parent avant `fork`, ce qui évite d'appeler `setenv` dans un enfant issu d'un processus multithreadé.

opts.stdin

`stdin` est une chaîne envoyée intégralement à l'entrée standard du programme.

```
local result = assert(babet.exec("sha256sum", { "-" }, {
    stdin = "hello world\n",
}))
print(result.stdout)
```

La chaîne est binary-safe :

```
local payload = "A\0B\0C"
local result = assert(babet.exec("cat", {}, { stdin = payload }))
assert(result.stdout == payload)
```

Si `stdin` est absent ou `nil`, Babet ferme immédiatement le pipe d'entrée. Le programme enfant observe donc un EOF. Une chaîne vide produit elle aussi un EOF, après l'ouverture normale du pipe.

Babet écrit et lit les trois pipes concurremment pour éviter les deadlocks avec un programme qui produit beaucoup de sortie avant de consommer toute son entrée.

opts.timeout

`timeout` est un nombre fini strictement positif exprimé en secondes.

```
local result = assert(babet.exec("sleep", { "30" }, {
    timeout = 0.5,
}))

assert(result.timed_out)
```

Le budget commence avant la préparation du lancement et couvre :

1. la préparation de l'environnement ;
2. `fork` ;
3. le `chdir` de l'enfant ;
4. l'appel `exec` ;
5. l'envoi de stdin ;
6. la capture de stdout et stderr ;
7. l'attente finale du processus.

La résolution interne est la milliseconde. Une valeur inférieure à `0.001` se comporte donc comme une échéance quasiment immédiate et ne doit pas être utilisée pour une temporisation précise.

La valeur doit rester inférieure ou égale à `INT_MAX` millisecondes, soit environ 24,8 jours. Les valeurs nulles, négatives, NaN, infinies ou trop grandes sont refusées avec `(nil, err)`.

À l'expiration après le lancement :

1. Babet envoie `SIGTERM` au groupe de processus ;
2. il laisse jusqu'à 2 secondes pour une sortie propre ;
3. il envoie `SIGKILL` si nécessaire ;
4. il draine encore les sorties pendant une fenêtre bornée.

opts.max_output

`max_output` limite le nombre d'octets conservés **pour chaque flux**.

```
local result = assert(babet.exec("yes", {}, {
    timeout = 0.2,
    max_output = 4096,
```

```
}))

assert(#result.stdout == 4096)
assert(result.stdout_truncated)
```

Valeurs :

- défaut : `10 * 1024 * 1024`, soit 10 Mio ;
- minimum : 1 octet ;
- maximum : `2 * 1024 * 1024 * 1024`, soit 2 Gio ;
- type : nombre entier Lua.

La limite est indépendante : jusqu'à 10 Mio de stdout **et** 10 Mio de stderr par défaut.

Une fois la limite atteinte, Babet continue à lire et à jeter le surplus. Cette étape est essentielle : arrêter de lire remplirait le pipe et pourrait bloquer le programme enfant. Les premiers octets sont conservés ; les derniers sont perdus.

Table de résultat

Après un lancement réussi, le premier résultat est toujours une table avec les six champs suivants :

```
{
    stdout = "...",
    stderr = "...",
    code = 0,
    timed_out = false,
    stdout_truncated = false,
    stderr_truncated = false,
}
```

Champ	Type	Signification
`stdout`	string binaire	premiers octets capturés sur la sortie standard
`stderr`	string binaire	premiers octets capturés sur la sortie d'erreur
`code`	integer	code normal, convention signal, ou `-1` si indisponible
`timed_out`	boolean	Babet a déclenché l'arrêt pour dépassement du budget
`stdout_truncated`	boolean	stdout a dépassé `max_output`
`stderr_truncated`	boolean	stderr a dépassé `max_output`

Code de sortie normal

Quand le programme appelle `exit(n)` ou retourne depuis `main`, `code` vaut le statut POSIX `0..255`.

```
local result = assert(babet.exec("sh", { "-c", "exit 7" }))
assert(result.code == 7)
assert(not result.timed_out)
```

Fin par signal

Quand le processus est terminé par un signal, Babet utilise la convention :

`code = 128 + numéro_du_signal`

Par exemple, `SIGKILL` vaut généralement 9 sous Linux, donc `code == 137`. Cette convention est pratique mais ne remplace pas un champ séparé indiquant le signal exact ; aucun tel champ n'existe actuellement.

Timeout

`timed_out == true` signifie que Babet a atteint son échéance et a commencé la procédure d'arrêt. Le champ `code` dépend de la façon dont le programme s'est terminé :

- code volontaire si le programme intercepte `SIGTERM` et sort proprement ;
- souvent 143 pour `SIGTERM` ;
- souvent 137 après `SIGKILL` ;
- exceptionnellement `-1` si le statut n'a pas pu être obtenu dans la fenêtre de nettoyage bornée.

Ne teste donc pas uniquement `code == 137` pour reconnaître un timeout : utilise `timed_out`.

La sortie produite avant l'échéance est conservée dans la limite de `max_output`.

Troncature

Les flags `stdout_truncated` et `stderr_truncated` indiquent uniquement que le flux correspondant a dépassé `max_output`.

```
if result.stdout_truncated then
    print("stdout incomplet : augmente max_output ou redirige vers un fichier")
end
```

Dans un cas noyau extrêmement pathologique où un descendant échappe au groupe et maintient un pipe ouvert après `SIGKILL`, Babet peut abandonner le drainage pour ne pas rester bloqué indéfiniment. Cette cause rare n'est pas représentée par les flags de troncature, qui restent réservés au cap mémoire.

Exemples complets

Capture simple

```
local result, err = babet.exec("git", { "rev-parse", "HEAD" })
assert(result, err)

if result.code ~= 0 then
    error("git a échoué : " .. result.stderr)
end

local commit = result.stdout:gsub("%s+$", "")
print(commit)
```

Arguments contenant des espaces

```
local filename = "rapport juillet 2026.txt"
local result = assert(babet.exec("printf", {
    "nom=<%s>\n",
    filename,
}))

assert(result.stdout == "nom=<rapport juillet 2026.txt>\n")
```

Aucun guillemet supplémentaire n'est nécessaire autour de `filename` : la frontière d'argument est déjà représentée par la table Lua.

Traiter un code non nul

```
local result, err = babet.exec("diff", {
    "--brief",
    "config.old",
```

```
    "config.new",
})
assert(result, err)

if result.code == 0 then
    print("fichiers identiques")
elseif result.code == 1 then
    print("fichiers différents")
else
    error("diff impossible : " .. result.stderr)
end
```

Capturer `stderr`

```
local result = assert(babet.exec("sh", {
    "-c", "printf out; printf err >&2",
}))

assert(result.stdout == "out")
assert(result.stderr == "err")
```

Envoyer du texte ou du binaire sur stdin

```
local csv = "name,score\nAlice,18\nBob,15\n"
local result = assert(babet.exec("sort", {}, { stdin = csv }))
print(result.stdout)
```

```
local bytes = string.char(0x00, 0x01, 0xFE, 0xFF)
local result = assert(babet.exec("cat", {}, { stdin = bytes }))
assert(result.stdout == bytes)
```

Modifier l'environnement

```
local result = assert(babet.exec("sh", { "-c", [[
    printf 'mode=%s lang=%s' "$APP_MODE" "$LANG"
]] }, {
    env = {
        APP_MODE = "test",
        LANG = "C",
    },
}))

print(result.stdout)
```

Seules les clés fournies sont remplacées. Les autres variables, notamment `HOME`, restent héritées.

Changer de répertoire

```
local result = assert(babet.exec("find", {
    ".", "-maxdepth", "1", "-type", "f",
}, {
    cwd = "/var/log",
}))

print(result.stdout)
```

Le CWD du script principal n'est pas modifié.

Borner le temps

```
local result, err = babet.exec("sh", {
    "-c", "printf started; sleep 30",
}, {
    timeout = 1.0,
})
assert(result, err)

if result.timed_out then
    print("commande interrompue après le budget")
    print("sortie disponible :", result.stdout)
end
```

Borner la sortie

```
local result = assert(babet.exec("sh", {
    "-c", "yes log-line | head -n 100000",
}, {
    max_output = 64 * 1024,
}))

if result.stdout_truncated then
    print("seuls les 64 premiers Kio ont été conservés")
end
```

Pour des gigaoctets, ne capture pas en mémoire. Lance explicitement un shell et redirige vers un fichier choisi avec soin :

```
local result = assert(babet.exec("sh", {
    "-c", "my-command > /var/tmp/my-command.log 2>&1",
}, {
    timeout = 600,
}))
```

Créer un helper applicatif

```
local function run_checked(cmd, args, opts)
    local result, err = babet.exec(cmd, args, opts)
    if not result then
        return nil, err
    end
    if result.timed_out then
        return nil, string.format("%s: timeout", cmd)
    end
    if result.code ~= 0 then
        local detail = result.stderr ~= "" and result.stderr or result.stdout
        return nil, string.format(
            "%s: code %d: %s",
            cmd,
            result.code,
            detail
        )
    end
    return result
end

local result, err = run_checked("git", { "status", "--short" }, {
```

```
    cwd = "/srv/project",
    timeout = 10,
})
assert(result, err)
print(result.stdout)
```

Processus pilotables avec `babet.spawn`

`babet.spawn` utilise le même moteur de lancement sans shell que `babet.exec`, mais n'attend pas la fin du programme. Il renvoie un userdata représentant le processus enfant :

```
local process, err = babet.spawn("yt-dlp", {
    "--newline",
    "https://example.invalid/video",
})
assert(process, err)
```

Par défaut, stdin, stdout et stderr sont raccordés à trois pipes non bloquants, ce qui conserve exactement le comportement historique. Chaque flux peut aussi être hérité, raccordé à `/dev/null`, ou, pour les sorties, envoyé directement dans un fichier. stderr peut enfin être fusionné dans stdout.

Signature et options

```
local process, err = babet.spawn(command, args?, opts?)
```

Élément	Type	Défaut	Rôle
`command`	string stricte	obligatoire	programme ou chemin à lancer
`args`	array dense de strings	aucun argument	`argv[1]..argv[n]`
`opts.cwd`	string	répertoire courant hérité	répertoire de l'enfant
`opts.env`	table string -> string	environnement hérité	variables ajoutées/remplacées
`opts.launch_timeout`	nombre fini > 0	aucune échéance générale	borne la préparation, le transfert terminal, `chdir` et `exec`
`opts.stdin`	`"pipe"`, `"inherit"` ou `"null"`	`"pipe"`	source de l'entrée standard
`opts.stdout`	mode ou table fichier	`"pipe"`	destination de la sortie standard
`opts.stderr`	mode, `"stdout"` ou table fichier	`"pipe"`	destination de la sortie d'erreur

Les options inconnues sont refusées. `spawn` n'accepte pas les options `timeout` et `max_output` de `exec` : la durée de vie est pilotée avec `wait()`/`terminate()`, et les sorties raccordées à des pipes sont lues progressivement.

`launch_timeout` couvre uniquement la préparation et la phase de lancement jusqu'à l'établissement du nouvel exécutable. Il ne limite pas la durée totale du programme. Une erreur d'ouverture d'un fichier de redirection ou de résolution du programme survient avant `fork()` : aucun enfant n'est alors créé.

La recherche dans `PATH`, les chemins contenant `/`, la fusion de `env` et l'absence de shell implicite suivent les mêmes règles que `exec`.

Modes de redirection

`"pipe"` Le mode par défaut crée un pipe non bloquant entre le parent et l'enfant :

```
local process = assert(babet.spawn("cat", {}, {
    stdin = "pipe",
    stdout = "pipe",
    stderr = "pipe",
}))

assert(process:write("bonjour\n"))
assert(process:close_stdin())
print(assert(process:read_stdout(64 * 1024, 1)))
```

Les méthodes `write`, `close_stdin`, `read_stdout` et `read_stderr` ne sont disponibles que pour les flux configurés avec `"pipe"`.

`"inherit"` L'enfant réutilise directement le descripteur standard correspondant de Babet :

```
local process = assert(babet.spawn("make", { "-j4" }, {
    stdin = "inherit",
    stdout = "inherit",
    stderr = "inherit",
}))
assert(process:wait())
```

Cette forme convient lorsqu'un programme doit interagir avec le terminal ou respecter une redirection appliquée extérieurement au processus Babet.

Lorsque `stdin = "inherit"` désigne le terminal de contrôle et que Babet en possède le premier plan, l'enfant conserve son groupe de processus séparé mais Babet lui transfère le terminal avant de l'autoriser à exécuter le programme. Le transfert est synchronisé : l'enfant ne peut pas tenter sa première lecture comme groupe d'arrière-plan et recevoir `SIGTTIN`. Les enfants interactifs directs lisent normalement et `Ctrl+C` est envoyé au groupe enfant de premier plan. La chaîne réelle Babet → Yaourt → Pacman est également validée avec Babet 2.15.0 et les versions suivantes.

Babet 2.16.0 ajoute un contrat de contrôle de jobs volontairement limité et explicite :

- `wait()` et `state()` observent les états suspendu et repris ;
- lorsque `Ctrl+Z` suspend l'enfant, Babet sauvegarde son `termios`, restaure le groupe de premier plan et le `termios` du parent, puis `wait()` renvoie `(nil, "stopped")` ;
- `is_running()` reste à `true` pendant la suspension puisque l'enfant n'est pas terminé ;
- `resume()` restaure le `termios` enfant, lui rend le premier plan puis envoie `SIGCONT` ;
- `resume(false)` envoie `SIGCONT` sans lui rendre le terminal. Ce mode vise les jobs raccordés à des pipes ; un processus en arrière-plan qui tente de lire le terminal peut être suspendu de nouveau par le noyau.

```
local process = assert(babet.spawn("outil-interactif", {}, {
    stdin = "inherit",
    stdout = "inherit",
    stderr = "inherit",
}))

local result, err = process:wait()
if not result and err == "stopped" then
    assert(process:state() == "stopped")

    -- Le parent possède le terminal et peut dialoguer avec l'utilisateur.
    print("enfant suspendu")
```

```
    -- Restaure le terminal enfant, le premier plan, puis envoie SIGCONT.
    assert(process:resume())
    result, err = process:wait()
end
assert(result, err)
```

Il ne s'agit volontairement pas d'une API complète de shell. Babet ne maintient pas de table de jobs, n'implémente pas des commandes telles que `fg`/`bg` et ne transfère pas le terminal à `pipeline()` ou `spawnPipeline()`.

La restitution après la fin définitive est asynchrone. Chaque transfert interactif crée un moniteur natif qui attend la fin de l'enfant direct sans le récolter. Sous Linux, il utilise un pidfd lorsque disponible, puis `waitid(..., WNOWAIT)` comme fallback. Le moniteur possède un descripteur de terminal dupliqué ; le registre natif sérialisé conserve la copie de l'état à restaurer. Le moniteur n'appelle jamais Lua et aucun handler `SIGCHLD` n'effectue d'opération terminal. Le parent peut donc relire son terminal après la fin de l'enfant avant même d'appeler `wait()`, `is_running()`, `state()` ou `close()` :

```
local process <close> = assert(babet.spawn("outil-interactif-court", {}, {
    stdin = "inherit",
    stdout = "inherit",
    stderr = "inherit",
}))

-- D'autres traitements Lua peuvent avoir lieu. Dès la fin de l'enfant, le
-- moniteur natif restaure le groupe parent et son termios.
babet.sleep(0.5)
local answer = io.read("*l")

-- Le statut final est toujours récolté et mis en cache normalement.
local result = assert(process:wait())
```

Les transferts de terminal sont sérialisés dans Babet. Si un enfant interactif se termine et qu'un autre `spawn()` interactif commence avant que le moniteur détaché ne soit ordonnancé, le nouveau lancement constate la fin définitive de l'enfant direct précédent avec `waitid(..., WNOHANG | WNOWAIT)` sans le récolter, restaure exactement le groupe de premier plan et le `termios` parent sauvegardés, puis effectue le nouveau transfert. Un ancien moniteur ne restaure que tant que son propre groupe enfant possède encore le terminal : il ne peut donc pas le reprendre à un enfant lancé ensuite.

Une réservation encore active dans un autre thread n'est jamais attendue sans limite. Babet attend au plus deux secondes, ou seulement le temps restant de `launch_timeout` lorsqu'il est plus court. Si la réservation n'est toujours pas libérée, `spawn()` renvoie `(nil, "terminal handoff is busy")` avant tout `fork()` : aucun enfant n'est lancé silencieusement sans terminal. La réservation initiale poursuit son propre rollback ou transfert normal ; une fois libérée, un lancement interactif suivant peut de nouveau acquérir le terminal et retrouve le `termios` parent sauvegardé.

Le moniteur suit l'enfant direct, pas un job shell complet. Si cet enfant se termine après avoir laissé des descendants dans son groupe de processus, Babet rend le terminal au parent dès la fin de l'enfant direct. Les descendants qui tentent ensuite de lire le terminal sont en arrière-plan et peuvent recevoir `SIGTTIN`. Cette limite est volontaire : Babet ne maintient pas de table de jobs shell et n'attend pas des descendants arbitraires.

Un retour `"timeout"` de `wait(timeout)` ne tue toujours pas l'enfant et ne retire pas le terminal à un enfant actif au premier plan. Il faut ensuite utiliser `wait()`, `terminate()`, `kill()` ou `close()` selon le besoin. Un enfant suspendu est différent : `wait()` renvoie immédiatement `"stopped"` et Babet a déjà rendu le terminal au parent.

Si stdin hérité est un pipe, un fichier ou un autre descripteur non terminal, aucun transfert de groupe de premier plan ni moniteur asynchrone n'est créé. stdout et stderr peuvent toujours être

capturés ou redirigés indépendamment.

`"null"` Le flux est raccordé à `/dev/null` :

```
local process = assert(babet.spawn("programme-silencieux", {}, {
    stdin = "null",  -- EOF immédiat pour l'enfant
    stdout = "null",
    stderr = "null",
}))
```

Redirection vers un fichier stdout et stderr acceptent une table stricte :

```
{
    file = "/var/tmp/application.log", -- obligatoire, sans octet NUL
    append = true,                     -- false par défaut : troncature
    permissions = tonumber("600", 8), -- 0600 par défaut, création seulement
}
```

- `append = false` tronque un fichier régulier existant ;
- `append = true` écrit avec `O_APPEND` ;
- `permissions` doit être un entier de `0000` à `0777` et s'applique seulement lorsque Babet crée le fichier ;
- la destination finale ne peut pas être un lien symbolique ;
- après ouverture, Babet vérifie avec `fstat()` qu'il s'agit d'un fichier régulier ; dossiers, FIFO, sockets et périphériques sont refusés ;
- le descripteur est préparé avant `fork()` et n'est conservé que par l'enfant.

Exemple stdout seul :

```
local process = assert(babet.spawn("generate-report", {}, {
    stdout = {
        file = "report.log",
        permissions = tonumber("640", 8),
    },
}))
```

Exemple stderr seul en ajout :

```
local process = assert(babet.spawn("worker", {}, {
    stderr = {
        file = "errors.log",
        append = true,
    },
}))
```

Fusion de stderr dans stdout `stderr = "stdout"` duplique le descripteur 2 vers le stdout déjà configuré. Cela fonctionne avec un pipe, l'héritage, `/dev/null` ou un fichier :

```
local process = assert(babet.spawn("driver", {}, {
    stdout = {
        file = "driver.log",
        append = true,
    },
    stderr = "stdout",
}))
```

Pour envoyer les deux flux dans un même fichier, cette fusion est préférable à deux tables fichier indépendantes : elle utilise un seul descripteur et un seul offset d'écriture.

Méthodes du processus

Méthode	Retour	Rôle
`read_stdout([max [, timeout]])`	`data, nil` ou `nil, reason`	lit stdout en pipe
`read_stderr([max [, timeout]])`	`data, nil` ou `nil, reason`	lit stderr en pipe
`write(data [, timeout])`	`count, nil` ou `nil, reason`	écrit sur stdin en pipe
`close_stdin()`	`true, nil` ou `nil, reason`	ferme stdin en pipe
`is_running()`	booléen ou `nil, err`	vrai jusqu'à la fin définitive
`state()`	état ou `nil, err`	inspecte le cycle de vie
`pid()`	entier	renvoie le PID enfant
`wait([timeout])`	`result, nil` ou `nil, reason`	attend ou signale arrêt/timeout
`resume([foreground])`	`true, nil` ou `nil, reason`	reprend un enfant suspendu
`terminate([grace])`	`result, nil` ou `nil, err`	SIGTERM puis SIGKILL borné
`kill()`	`result, nil` ou `nil, err`	SIGKILL sur le groupe
`close()`	`true, nil`	finalise le handle processus

Les raisons courtes sont :

- `"timeout"` : aucun progrès avant la fin du délai ;
- `"stopped"` : `wait()` a observé un enfant suspendu et rendu le terminal ;
- `"closed"` : handle fermé, pipe fermé ou EOF selon la méthode ;
- `"interrupted"` : interruption Babet traitée ;
- `"not_piped"` : la méthode vise un flux configuré autrement que `"pipe"` ;
- `"not_stopped"` : `resume()` vise un enfant qui n'est pas suspendu ;
- `"not_interactive"` : une reprise au premier plan est demandée sans terminal ;
- `"exited"` : `resume()` est demandé après la fin définitive.

Les autres diagnostics utilisent un préfixe `process:` ou `spawn:`.

La table renvoyée par `wait`, `terminate` et `kill` contient :

```
{
    code = 0,          -- code normal, ou 128 + signal
    exited = true,     -- fin normale via exit/_exit
    signaled = false,  -- fin causée par un signal
    signal = nil,      -- numéro du signal si signaled == true
}
```

Un second appel à `wait()` après la fin renvoie le même résultat : le statut est mis en cache après le `waitpid` initial.

Lire stdout et stderr

Les lectures sur un pipe sont binary-safe. Par défaut, elles lisent au plus 64 Kio et sont non bloquantes (`timeout = 0`). La taille maximale par appel est 16 Mio.

```
local chunk, err = process:read_stdout(64 * 1024, 0.25)

if chunk then
    io.write(chunk)
elseif err == "timeout" then
    -- rien de disponible pendant 250 ms
elseif err == "closed" then
    -- EOF définitif sur stdout
elseif err == "not_piped" then
    -- stdout est hérité, nul ou redirigé vers un fichier
```

```
else
    error(err)
end
```

`"closed"` indique l'EOF du flux concerné, pas nécessairement la fin du processus. Un enfant peut fermer stdout puis continuer à travailler.

Lorsque stdout et stderr sont tous deux des pipes, il faut les drainer tous les deux. Lire un seul flux tandis que l'autre se remplit peut bloquer l'enfant. Une sortie dirigée vers un fichier, héritée ou nulle ne possède pas ce risque de remplissage d'un pipe parent.

Écrire sur stdin

Avec `stdin = "pipe"`, `write` accepte une chaîne Lua binaire, y compris des octets NUL. Le pipe est non bloquant : l'appel peut n'écrire qu'une partie de la chaîne. La valeur renvoyée sert à reprendre à l'octet suivant.

```
local function write_all(process, data)
    local offset = 1
    while offset <= #data do
        local written, err = process:write(data:sub(offset), 1)
        assert(written, err)
        offset = offset + written
    end
end

write_all(process, payload)
assert(process:close_stdin())
```

Une chaîne vide renvoie `(0, nil)`. Après `close_stdin`, `write` renvoie `(nil, "closed")`. Avec `stdin = "inherit"` ou `stdin = "null"`, `write` et `close_stdin` renvoient `(nil, "not_piped")`.

Attendre, terminer et fermer

`wait()` sans argument attend la fin du processus. `wait(timeout)` rend `(nil, "timeout")` si le délai expire, sans envoyer de signal ni rendre l'objet inutilisable.

```
local result, err = process:wait(0)
if not result and err == "timeout" then
    -- le processus tourne encore
end
```

`terminate(grace_period)` envoie SIGTERM à tout le groupe. Si le groupe ne se termine pas avant la fin de la période de grâce, Babet envoie SIGKILL. La grâce par défaut est de deux secondes. `kill()` envoie directement SIGKILL.

`close()` est idempotent. Si l'enfant tourne encore, il est terminé et réapé de façon bornée, puis tous les descripteurs encore détenus par le parent sont fermés. Le garbage collector et la métaméthode Lua `__close` effectuent le même nettoyage :

```
local process <close> = assert(babet.spawn("long-job"))
-- nettoyage automatique en quittant la portée
```

Exemple combiné : lancer un WebDriver ou un daemon

```
local process, err = babet.spawn("bin/geckodriver", {
    "--port=4444",
}, {
    stdin = "null",
    stdout = {
```

```
        file = "logs/geckodriver.log",
        append = true,
        permissions = tonumber("600", 8),
    },
    stderr = "stdout",
})
assert(process, err)

local result, wait_err = process:wait(0)
if not result and wait_err == "timeout" then
    print("geckodriver est lancé avec le PID", process:pid())
end
```

Aucun shell n'est impliqué, les arguments restent séparés, stdin ne peut pas bloquer en attente de données et les logs ne remplissent aucun pipe parent.

Exemple combiné : boucle de streaming

```
local process = assert(babet.spawn("yt-dlp", {
    "--newline",
    "-f", "bestvideo+bestaudio/best",
    url,
}))

local stdout_open, stderr_open = true, true
while stdout_open or stderr_open do
    if stdout_open then
        local data, read_err = process:read_stdout(64 * 1024, 0.1)
        if data then
            io.write(data)
            io.flush()
        elseif read_err == "closed" then
            stdout_open = false
        elseif read_err ~= "timeout" then
            error(read_err)
        end
    end

    if stderr_open then
        local data, read_err = process:read_stderr(64 * 1024, 0)
        if data then
            io.stderr:write(data)
            io.stderr:flush()
        elseif read_err == "closed" then
            stderr_open = false
        elseif read_err ~= "timeout" then
            error(read_err)
        end
    end
end

local result, wait_err = process:wait(5)
assert(result, wait_err)
process:close()
```

Limites et prévention des blocages

— `wait()` ne draine pas les sorties configurées en `"pipe"`. Pour un enfant bavard, draine les pipes, redirige les logs vers un fichier, utilise `"inherit"`/`"null"`, ou choisis `babet.exec` pour une petite sortie bornée.
— Écrire une grosse entrée à un programme qui réémet simultanément beaucoup de données exige d'alterner écriture et lecture.
— Un objet `babet.spawn()` ne peut pas être raccordé après coup à un autre processus. Pour un pipeline linéaire, utilise `babet.spawnPipeline()`.
— Les tables fichier ne sont pas disponibles pour stdin dans cette version.
— Babet ne crée pas les répertoires parents des fichiers de redirection.
— Les fichiers de redirection sont ouverts avant `fork()`. Si une erreur plus tardive de `fork()` ou de lancement survient, un fichier peut donc déjà avoir été créé ou tronqué, même si aucun processus utilisable n'est renvoyé.
— `close()` et `terminate()` ciblent le groupe créé au lancement. Un descendant qui change volontairement de groupe ou de session peut échapper à ce contrôle.

Contrat d'erreur

Erreurs Lua levées

Seule la signature du premier argument utilise une erreur Lua :

```
babet.exec()   -- lève : commande manquante
babet.exec(42) -- lève : cmd doit être une string
```

Utilise `pcall` uniquement si une mauvaise signature peut provenir de données non maîtrisées.

Erreurs renvoyées `(nil, err)`

Les validations suivantes ne lèvent pas ; elles renvoient `(nil, err)` :

— `args` n'est ni une table ni `nil` ;
— un élément de `args` n'est pas une string ;
— `opts` n'est ni une table ni `nil` ;
— `cwd`, `stdin`, `timeout`, `max_output` ou `env` ont un mauvais type ;
— une clé d'environnement est vide, contient `=` ou un NUL ;
— une valeur d'environnement contient un NUL ;
— timeout nul, négatif, non fini ou trop grand ;
— `max_output` nul, négatif, fractionnaire ou supérieur à 2 Gio ;
— programme introuvable ou non exécutable ;
— réservation du terminal interactif occupée trop longtemps ;
— `cwd` invalide ;
— échec de `pipe`, `fork`, `poll`, `waitpid` ou autre erreur interne système.

Après un `fork()` réussi, Babet conserve la propriété des descripteurs et du groupe de processus jusqu'au transfert vers un objet `spawn()` ou jusqu'à la récupération finale par `exec()`. Si une exception C++ interne survient pendant cette période — par exemple lors d'une allocation de tampon — le chemin d'urgence ferme les descripteurs, envoie immédiatement `SIGKILL` au groupe et tente de récupérer l'enfant pendant une fenêtre courte avant de renvoyer `nil, err`. Ce nettoyage d'exception n'utilise pas la période de grâce de `terminate()`, réservée aux arrêts explicitement demandés.

```
local result, err = babet.exec("echo", "hello")
assert(result == nil)
print(err) -- args must be a table
```

Situations renvoyant une table

Ces situations ne sont pas des erreurs de l'API :

— le programme renvoie un code non nul ;
— le programme écrit sur `stderr` ;
— le timeout expire après ou pendant le lancement ;
— la sortie dépasse `max_output`.

Processus, signaux et sécurité

Groupe de processus

L'enfant crée son propre groupe. En cas de timeout, Babet signale le groupe et pas seulement le PID direct. Cela couvre normalement les descendants lancés par un shell ou par le programme.

Un descendant qui appelle volontairement `setsid` ou change de groupe peut échapper à cette stratégie. `babet.exec` n'est pas un sandbox et ne remplace pas systemd, les cgroups, les namespaces ou les limites de ressources.

Masque des signaux de l'enfant

`exec`, `spawn`, `pipeline` et `spawnPipeline` lancent les programmes externes avec un masque de signaux vide, depuis le thread principal comme depuis un worker. Le masque interne du worker ne bloque donc pas `SIGTERM`, `SIGINT` ou `SIGPIPE` dans l'enfant. Le masque du thread appelant reste inchangé.

Cela ne modifie pas les règles POSIX sur les dispositions héritées : un signal explicitement ignoré peut rester ignoré après `execve`. Cette politique concerne les quatre API Babet, pas les fonctions Lua standard `os.execute` et `io.popen`.

Interaction avec `babet.signal`

`babet.exec` n'est pas signal-aware. Un signal géré reçu pendant l'appel ne fait pas renvoyer `(nil, "interrupted")`. Le handler C marque le signal comme en attente ; le callback Lua ne pourra être dispatché qu'après le retour dans la VM Lua, généralement via le hook d'instructions du module SIGNAL.

Utilise `opts.timeout` pour borner une commande longue. Pour interrompre une commande à la demande, conçois un protocole externe ou lance-la via un service supervisé.

Mémoire

`stdin` est conservé en mémoire par le script, et stdout/stderr sont capturés en mémoire dans les limites choisies. Le défaut autorise environ 20 Mio de capture au total, plus les allocations du programme et des chaînes Lua.

Décisions et limites

— Linux/POSIX uniquement : l'implémentation utilise `fork`, `execve`, `poll`, les groupes de processus et les signaux.
— Aucun shell implicite.
— Un appel `exec` reste une commande unique ; les pipelines natifs utilisent `babet.pipeline()` ou `babet.spawnPipeline()`.
— Un seul bloc `stdin`, pas de producteur streaming.
— Capture complète retournée à la fin, pas de callback ligne par ligne.
— Pas de redirection native de stdin/stdout/stderr vers des fichiers ou fd.
— Pas de champ séparé `signal` dans la table de résultat.
— Pas de modification/unset complet de l'environnement : `env` fusionne et une chaîne vide reste une variable définie.
— Les champs inconnus de `opts` sont ignorés.

— La liste ordonnée des candidats de `PATH`, `argv` et `envp` sont préparés dans le parent ; après `fork()`, l'enfant ne parcourt cette liste qu'avec `execve()`.
— Les callbacks de `babet.signal` ne sont pas dispatchés pendant l'appel.

Pour les règles de sécurité générales, consulte Sécurité.

Pipelines de processus

Babet fournit deux API complémentaires pour relier plusieurs programmes par des pipes POSIX, sans passer par un shell :

— `babet.pipeline()` exécute le pipeline jusqu'à sa fin et capture les flux ;
— `babet.spawnPipeline()` rend immédiatement un objet pilotable en streaming.

Dans les deux cas, stdout d'une étape est connecté directement à stdin de la suivante. Le parent n'intercepte que stdin de la première étape, stdout de la dernière et stderr de chaque étape séparément.

Syntaxe commune des commandes

```
local commands = {
    { "printf", { "orange\npomme\norange\n" } },
    { "grep", { "orange" } },
    { "sort", { "-u" } },
}
```

`commands` doit être un tableau dense de 2 à 32 étapes. Chaque étape prend l'une des formes suivantes :

```
{ command }
{ command, args }
{ command, args, opts }
```

Pour fournir `opts` sans argument, utilise explicitement `{ command, {}, opts }` ; les tableaux troués sont refusés.

— `command` est une chaîne non vide sans octet NUL ;
— `args` est un tableau dense de chaînes sans octet NUL ;
— `opts.cwd` définit le répertoire de travail de l'étape ;
— `opts.env` ajoute ou remplace des variables d'environnement pour l'étape.

Aucune chaîne n'est interprétée par un shell : jokers, redirections, variables, guillemets et opérateurs comme `|`, `&&` ou `>` n'ont aucun sens spécial.

Les options globales `cwd` et `env` servent de valeurs par défaut. Un `cwd` local remplace le `cwd` global. L'environnement local est fusionné après l'environnement global et gagne en cas de nom identique. Toutes les commandes sont validées avant le premier `fork()`.

Comme pour `exec()` et `spawn()`, lorsqu'une commande ne contient pas `/`, sa recherche utilise le `PATH` de l'environnement final de l'étape, après fusion de l'environnement hérité, des options globales et des options locales. Une composante vide ou relative de `PATH` est interprétée depuis le `cwd` effectif de cette étape. Toutes les listes ordonnées de candidats, tous les `argv` et tous les `envp` sont préparés avant le premier `fork()` ; chaque enfant ne parcourt ensuite que sa liste avec `execve()`, sans analyser `PATH` ni allouer.

`babet.pipeline()` — exécution complète

Signature

```
result, err = babet.pipeline(commands [, opts])
```

Exemple :

```
local result, err = babet.pipeline({
    { "printf", { "orange\npomme\norange\n" } },
    { "grep", { "orange" } },
    { "sort", { "-u" } },
}, {
    timeout = 10,
    max_output = 16 * 1024 * 1024,
})

assert(result, err)
print(result.stdout)
```

Options

Option	Type	Description
`cwd`	chaîne	Répertoire de travail par défaut de toutes les étapes.
`env`	table chaîne vers chaîne	Variables fusionnées avec l'environnement de Babet.
`stdin`	chaîne binaire	Données écrites progressivement dans la première étape, puis stdin est fermé.
`timeout`	nombre fini strictement positif, au plus `10^12`	Délai global en secondes, lancement, E/S et attente finale compris.
`max_output`	entier de 1 à 2 Gio	Plafond distinct pour stdout final et chaque stderr ; défaut : 10 Mio.

Les clés inconnues sont refusées. `stdin`, stdout et stderr sont binaires et peuvent contenir des octets NUL. Le timeout a une résolution interne d'une milliseconde ; les valeurs supérieures à `10^12` secondes sont refusées.

Résultat

Après le lancement effectif de toutes les étapes, une terminaison non nulle ou un signal ne constitue pas une erreur d'API. Babet renvoie :

```
{
    stdout = "...",
    stderr = { "stderr étape 1", "stderr étape 2", ... },
    stages = {
        {
            launched = true,
            code = 0,
            exited = true,
            signaled = false,
            signal = nil,
        },
        -- ...
    },
    code = 0,
```

```
    all_succeeded = true,
    failed_index = nil,
    timed_out = false,
    stdout_truncated = false,
    stderr_truncated = { false, false, ... },
}
```

`code` est celui de la dernière étape, comme pour un pipeline Unix. Il vaut le code de sortie normal ou `128 + numéro_du_signal`. Dans le cas pathologique où un statut reste indisponible après un nettoyage borné consécutif à un timeout, il vaut `-1` pour l'étape concernée. `all_succeeded` n'est vrai que si toutes les étapes ont réussi. `failed_index` indique la première étape dont le code n'est pas zéro, même si la dernière réussit.

Une étape intermédiaire non nulle n'arrête pas artificiellement les autres. Les pipes conservent leur comportement POSIX normal ; un producteur amont peut notamment recevoir `SIGPIPE` si le consommateur ferme son entrée.

Timeout, gros volumes et troncature

Le timeout de `pipeline()` couvre le lancement, l'écriture de stdin, le drainage de stdout et stderr et l'attente finale. S'il expire avant que toutes les étapes aient terminé `chdir` et `exec`, l'appel renvoie `nil, err` avec le message `"pipeline: launch timed out"` après rollback des étapes déjà créées. Après un lancement complet, l'expiration produit une table de résultat avec `timed_out == true` : Babet ferme stdin, envoie `SIGTERM` aux groupes de toutes les étapes, puis `SIGKILL` aux survivants et récupère les enfants directs.

Les descripteurs sont non bloquants et pilotés avec `poll()`. stdin, stdout final et tous les stderr progressent simultanément, ce qui évite les deadlocks classiques avec de gros volumes. Après `max_output`, les octets supplémentaires sont jetés mais le flux continue d'être drainé ; le drapeau de troncature correspondant est alors vrai.

`babet.spawnPipeline()` — streaming

Signature

```
pipeline, err = babet.spawnPipeline(commands [, opts])
```

Options globales :

Option	Type	Description
`cwd`	chaîne	Répertoire de travail par défaut de toutes les étapes.
`env`	table chaîne vers chaîne	Variables fusionnées avec l'environnement de Babet.
`launch_timeout`	nombre fini strictement positif	Délai couvrant uniquement la création et le lancement complet des étapes, au plus `INT_MAX` millisecondes (environ 24,8 jours).

`spawnPipeline()` n'accepte ni `stdin`, ni `timeout`, ni `max_output` : le script écrit et lit progressivement et choisit lui-même ses limites. Un succès renvoie un userdata et `nil` ; une erreur de validation ou de lancement renvoie `nil, err` après nettoyage des étapes déjà créées.

Cette API est destinée aux commandes reliées par des pipes, pas aux programmes interactifs. L'entrée standard de la première étape est toujours le pipe piloté par Babet ; aucun transfert du terminal

de premier plan n'est effectué. Une étape qui ouvre explicitement `/dev/tty` et tente d'y lire peut donc être suspendue par les règles POSIX des groupes d'arrière-plan. Pour un outil qui doit dialoguer directement avec l'utilisateur, utilise `babet.spawn()` avec `stdin`, `stdout` et `stderr` en mode `"inherit"`.

```
local pipeline, err = babet.spawnPipeline({
    { "cat" },
    { "tr", { "a-z", "A-Z" } },
})
assert(pipeline, err)

local written, write_err = pipeline:write("bonjour\n")
assert(written, write_err)
assert(pipeline:close_stdin())

local chunks = {}
while true do
    local data, read_err = pipeline:read_stdout(64 * 1024, 1)
    if data then
        chunks[#chunks + 1] = data
    elseif read_err == "closed" then
        break
    elseif read_err ~= "timeout" then
        error(read_err)
    end
end

local status, wait_err = pipeline:wait(5)
assert(status, wait_err)
assert(status.code == 0)
print(table.concat(chunks))
```

Méthodes de lecture

```
data, err = pipeline:read_stdout([max_bytes [, timeout]])
data, err = pipeline:read_stderr(stage [, max_bytes [, timeout]])
```

- `max_bytes` vaut 64 Kio par défaut et doit être compris entre 1 et 16 Mio ;
- `timeout` est exprimé en secondes, vaut zéro par défaut, doit être fini et positif ou nul, et ne peut pas dépasser `INT_MAX` millisecondes, soit environ 24,8 jours ;
- `stage` est un indice de 1 au nombre d'étapes ;
- une arité invalide, un indice invalide ou un `max_bytes` invalide lève une erreur Lua, comme les méthodes de `babet.spawn()` ;
- un timeout invalide renvoie `nil, err` ;
- un succès renvoie une chaîne binaire non vide et `nil` ;
- aucune donnée immédiatement disponible renvoie `nil, "timeout"` ;
- la fin du flux, ou une lecture après sa fermeture, renvoie `nil, "closed"`.

Chaque appel lit au plus un bloc. Il n'existe aucune capture cachée : le script est responsable de conserver, limiter ou jeter les données reçues. Les stderr restent indépendants ; il faut donc drainer ceux susceptibles de produire assez de données pour remplir leur pipe.

Écriture et fermeture de stdin

```
count, err = pipeline:write(data [, timeout])
ok, err = pipeline:close_stdin()
```

`write()` accepte une chaîne binaire. Il peut n'écrire qu'une partie de la chaîne ; `count` indique le

nombre d'octets réellement transmis. Une chaîne vide renvoie `0, nil`. Après timeout, l'objet reste réutilisable. Une entrée fermée par le script ou par le processus aval renvoie `nil, "closed"` sans livrer `SIGPIPE` au processus Babet.

`close_stdin()` ferme l'entrée de la première étape et est idempotente. Elle doit être appelée lorsque plus aucune donnée ne sera écrite, sinon un programme attendant EOF peut ne jamais terminer.

Exemple d'écriture complète :

```
local offset = 1
while offset <= #data do
    local count, err = pipeline:write(data:sub(offset), 1)
    if count then
        offset = offset + count
    elseif err ~= "timeout" then
        error(err)
    end
    -- Pendant un gros transfert, drainer aussi stdout et les stderr.
end
assert(pipeline:close_stdin())
```

État et identifiants

```
running = pipeline:is_running([stage])
pids = pipeline:pids()
```

Sans indice, `is_running()` est vrai tant qu'au moins une étape directe n'a pas été récupérée. Avec un indice, il décrit uniquement cette étape. L'appel interroge les statuts sans blocage et ne consomme pas le résultat final : `wait()` reste utilisable ensuite.

`pids()` renvoie les PID directs dans l'ordre des étapes. Ils sont fournis pour le diagnostic ; le script ne doit pas supposer qu'ils restent valides après la fin du pipeline.

Attente et résultat final

```
status, err = pipeline:wait([timeout])
```

Sans timeout, `wait()` attend tous les enfants directs. Avec un timeout fini et positif ou nul, `nil, "timeout"` laisse le pipeline intact et réutilisable. Une fois tous les statuts acquis, les appels suivants sont idempotents et renvoient le même résultat :

```
{
    stages = {
        { launched = true, code = 0, exited = true,
          signaled = false, signal = nil },
        -- ...
    },
    code = 0,
    all_succeeded = true,
    failed_index = nil,
}
```

`wait()` **ne draine pas** stdout ou stderr. Un pipeline qui remplit un pipe peut donc rester bloqué avant de terminer. Il faut lire les flux pendant l'exécution, exactement comme avec `babet.spawn()`.

Arrêt explicite

```
status, err = pipeline:terminate([grace_period])
status, err = pipeline:kill()
```

`terminate()` envoie `SIGTERM` à chaque groupe de processus, laisse `grace_period` secondes — 2 secondes par défaut — aux processus pour terminer, puis envoie `SIGKILL` aux survivants. La période de grâce doit tenir dans `INT_MAX` millisecondes, soit environ 24,8 jours. `kill()` envoie directement `SIGKILL`. Les deux méthodes ferment stdin et renvoient la même table de statuts que `wait()` lorsque tous les enfants directs ont été récupérés.

Chaque étape possède son propre groupe. Avant de récupérer le processus direct, Babet supprime les descendants encore présents dans ce groupe, y compris lors d'une terminaison normale où la commande a quitté sans attendre un enfant lancé en arrière-plan. Le PID du leader reste ainsi réservé jusqu'au signal et ne peut pas être recyclé vers un processus étranger.

Pour `terminate()`, tous les membres du groupe reçoivent `SIGTERM` dès le début de la période de grâce. Les leaders directs déjà terminés restent volontairement non récupérés pendant toute cette période : leur PID, également utilisé comme PGID, ne peut ainsi pas être recyclé avant l'éventuel `SIGKILL` final envoyé au groupe. Une étape qui crée volontairement une nouvelle session ou un nouveau groupe de processus échappe à cette garantie POSIX. Un script qui connaît ses processus comme coopératifs peut choisir une grâce plus courte, par exemple `pipeline:terminate(0.05)`.

`close()`, GC et Lua `<close>`

```
ok, err = pipeline:close()
```

`close()` est idempotente. Elle ferme les descripteurs, tente `SIGTERM`, utilise `SIGKILL` si nécessaire et récupère les enfants directs. Après fermeture, les lectures et écritures renvoient `nil, "closed"`, et `is_running()` renvoie `false`. Un statut déjà entièrement acquis reste accessible par `wait()`.

Le même nettoyage est appelé par `__gc` et `__close`. L'usage recommandé est :

```
local pipeline <close> = assert(babet.spawnPipeline({
    { "producteur" },
    { "consommateur" },
}))
```

Le GC constitue un filet de sécurité, pas un mécanisme de synchronisation : utilisez `wait()`, `terminate()` ou `close()` explicitement lorsque le moment du nettoyage compte. Si le pipeline est déjà terminé et qu'une fermeture immédiate est importante, appelez d'abord `wait()` (ou constatez `is_running() == false`) avant `close()` : Babet peut alors utiliser le chemin rapide des pipelines déjà récupérés. Un `close()` appelé sans avoir récupéré un pipeline seulement terminé peut encore consommer le délai de grâce normal de 0,5 s afin de sécuriser le nettoyage des groupes de processus.

Erreurs de lancement et réutilisation

Les erreurs de validation, de pipe, de `fork`, de `chdir` ou d'`exec`, ainsi qu'un `launch_timeout`, renvoient `nil, err`. Si une étape échoue pendant le lancement, toutes celles déjà créées sont arrêtées et récupérées ; aucun objet partiellement valide n'est exposé. Le lanceur applique lui-même la limite publique de 32 étapes, indépendamment de la validation effectuée par le binding Lua.

Après le premier `fork()`, puis pendant la capture synchrone de `pipeline()`, Babet conserve un propriétaire de secours pour les descripteurs et les enfants directs. Si une exception C++ interne survient, le nettoyage d'urgence ferme les flux, envoie immédiatement `SIGKILL` uniquement aux groupes dont le leader n'a pas encore été récupéré, puis tente une récupération bornée. La période de grâce de `terminate()` n'est pas utilisée pendant un déroulement de pile. Pour `spawnPipeline()`, les ressources ne sont transférées au userdata qu'après un lancement entièrement réussi.

Les erreurs non destructives `"timeout"` et `"interrupted"` autorisent un nouvel appel. `"closed"` signifie que le flux ou l'objet concerné ne peut plus servir pour cette opération. Une erreur système plus grave est renvoyée sous forme de texte et doit être traitée comme telle.

FS — fichiers, dossiers, chemins, recherche, copie, permissions et sommes de contrôle

Les fonctions de ce chapitre sont exposées directement dans la table globale `babet`. Il n'existe pas de sous-table `babet.fs` : ce namespace plat est historique et fait désormais partie de l'API stable.

Le module FS couvre :

- la vérification de l'existence et du type d'un chemin ;
- la création, l'écriture atomique et la suppression de fichiers et de dossiers ;
- le changement de répertoire courant ;
- la manipulation lexicale des chemins ;
- le listing, l'itération et la recherche récursive ;
- la copie et le déplacement d'arborescences ;
- les liens symboliques ;
- les permissions, propriétaires et groupes Unix ;
- les sommes de contrôle CRC-32, MD5, SHA et BLAKE2.

Table des matières du module

- Conventions générales
- Vue d'ensemble de l'API
- Existence, type et taille
  - `fileExists`
  - `isFile` / `isfile`
  - `isDir` / `isdir`
  - `fileSize`
- Répertoire courant
  - `currentDir`
  - `chdir`
- Manipulation lexicale des chemins
  - `getBasename`
  - `getFilename`
  - `getExtension`
  - `getPath`
  - `joinPath`
- Créer, supprimer, renommer et lier
  - `touch`
  - `writeFileAtomic`
  - `mkdir`
  - `remove`
  - `rmdir`
  - `rmdirAll`
  - `rename`
  - `link`
- Lister, itérer et rechercher
  - `listFiles`
  - `createFileIterator`
  - `find`
- Copier et déplacer
  - `copy`
  - `copyTree`
  - `moveTree`
- Permissions et attributs Unix
  - `getMode` / `setMode`
  - `getAttributes` / `setAttributes`
  - `symlinkAttr` / `symlinkattr`

— Sommes de contrôle de fichiers
— CRC-32 en mémoire
— Contrat d’erreur
— Décisions et limites

Conventions générales

Chemins

Les chemins sont des chaînes Lua. Ils peuvent être relatifs ou absolus.

Babet ne réalise pas automatiquement :

— l’expansion de `~` ;
— l’expansion de variables comme `$HOME` ;
— l’expansion des globs `*.lua` ;
— la normalisation générale de `.` et `..` dans les fonctions lexicales.

Par exemple :

```lua
local home = assert(babet.env("HOME"))
local config = babet.joinPath(home, ".config", "mon-app")
```

Un octet NUL dans un chemin est toujours refusé. Selon la fonction, une mauvaise signature lève une erreur Lua, tandis qu’une erreur du système de fichiers est généralement renvoyée sous la forme `(nil, err)`.

Liens symboliques

Il n’existe pas une règle unique valable pour toutes les fonctions :

— `isFile`, `isDir`, `fileSize`, `getMode`, `getAttributes` et les checksums suivent un lien valide vers sa cible ;
— `remove` supprime le lien lui-même, y compris s’il est cassé ;
— `rmdir` et `rmdirAll` refusent une racine symbolique ;
— `symlinkAttr` modifie le propriétaire et le groupe du lien lui-même ;
— `listFiles`, `createFileIterator` et `find` ne descendent pas dans un lien symbolique vers un dossier ;
— `copyTree` et `moveTree` refusent une racine source ou destination symbolique, mais gèrent les liens présents à l’intérieur de l’arbre.

Chaque fonction détaille ci-dessous son comportement exact.

Ordre des résultats

`listFiles`, `createFileIterator` et `find` suivent l’ordre fourni par le système de fichiers. Cet ordre n’est pas trié et ne doit pas être considéré comme stable.

Trie explicitement le résultat lorsque l’ordre compte :

```lua
local files = assert(babet.listFiles("documents", true))
table.sort(files)
```

Valeurs de retour

La convention la plus fréquente est :

```lua
local value, err = fonction(...)
if not value then
    print(err)
end
```

Une action réussie renvoie généralement `(true, nil)`. Quelques fonctions pures ou méthodes d'itérateur ont une forme différente ; ces exceptions sont signalées dans leur section.

Vue d'ensemble de l'API

Vérifier et inspecter

Fonction	Résultat en cas de succès
`babet.fileExists(path)`	`(boolean, nil)` — vrai uniquement pour un fichier régulier
`babet.isFile(path)`	`(boolean, nil)` — même prédicat fonctionnel que `fileExists`
`babet.isDir(path)`	`(boolean, nil)` — vrai pour un dossier
`babet.fileSize(path)`	`(integer, nil)` — taille d'un fichier régulier en octets
`babet.currentDir()`	`(string, nil)` — répertoire courant
`babet.getMode(path)`	`(integer, nil)` — mode Unix `0000..07777`
`babet.getAttributes(path)`	`(table, nil)` — `{mode, owner, group}`

Manipuler des chemins

Fonction	Résultat en cas de succès
`babet.getBasename(path)`	`(string, nil)`
`babet.getFilename(path)`	`(string, nil)`
`babet.getExtension(path)`	`(string, nil)`
`babet.getPath(path)`	`(string, nil)`
`babet.joinPath(a, b, ...)`	`string`
`babet.joinPath({a, b, ...})`	`string`

Agir sur le système de fichiers

Fonction	Résultat en cas de succès
`babet.touch(path)`	`(true, nil)`
`babet.writeFileAtomic(path, data, opts?)`	`(true, nil)` — publication binaire atomique et durable ; chapitre détaillé
`babet.mkdir(path)`	`(true, nil)` — récursif et idempotent
`babet.remove(path)`	`(true, nil)` — fichier régulier ou symlink
`babet.rmdir(path)`	`(true, nil)` — dossier réel et vide
`babet.rmdirAll(path)`	`(true, nil)` — dossier réel, suppression récursive
`babet.rename(source, destination)`	`(true, nil)`
`babet.link(target, linkpath)`	`(true, nil)` — crée un symlink
`babet.chdir(path)`	`(true, nil)`

Lister et rechercher

Fonction	Résultat en cas de succès
`babet.listFiles(path, recursive?)`	`(table, nil)`
`babet.createFileIterator(path, recursive?)`	`(iterator, nil)`
`babet.find(path, opts?)`	`(table, nil)`

Copier et déplacer

Fonction	Résultat en cas de succès
`babet.copy(source, destination)`	`(true, nil)` — un fichier
`babet.copyTree(source, destination, continue_on_error?)`	`(true, nil)` — un arbre
`babet.moveTree(source, destination)`	`(true, nil)` — un arbre

Modifier les attributs

Fonction	Résultat en cas de succès
`babet.setMode(path, mode)`	`(true, nil)`
`babet.setAttributes(path, uid, gid, mode?)`	`(true, nil)`
`babet.symlinkAttr(path, uid, gid)`	`(true, nil)`

Existence, type et taille

`babet.fileExists(path)`

Vérifie si `path` désigne un **fichier régulier**.

```
local exists, err = babet.fileExists("rapport.txt")
```

Résultats :

- fichier régulier : `(true, nil)` ;
- dossier, lien cassé ou chemin absent : `(false, nil)` ;
- lien valide vers un fichier régulier : `(true, nil)` ;
- véritable erreur d'inspection : `(nil, err)`.

Malgré son nom historique, `fileExists` ne répond pas « vrai » pour tous les types de chemins. Pour un dossier, utilise `isDir`.

```
local is_file = assert(babet.fileExists("archive.tar"))
local is_dir  = assert(babet.isDir("sauvegardes"))
```

`babet.isFile(path)` et `babet.isfile(path)`

`isFile` est le nom canonique. `isfile` est un alias déprécié conservé pour compatibilité.

La fonction teste le même type de chemin que `fileExists` : un fichier régulier, en suivant un éventuel symlink valide.

```
local regular, err = babet.isFile("programme.lua")
if err then
    print("Inspection impossible :", err)
elseif regular then
    print("C'est un fichier régulier")
end
```

Un chemin inexistant renvoie `(false, nil)` et non une erreur.

`babet.isDir(path)` et `babet.isdir(path)`

`isDir` est le nom canonique. `isdir` est un alias déprécié.

```
local directory, err = babet.isDir("assets")
```

Résultats :

— dossier réel : `(true, nil)` ;
— lien valide vers un dossier : `(true, nil)` ;
— fichier, lien cassé ou chemin absent : `(false, nil)` ;
— erreur d'accès ou d'inspection : `(nil, err)`.

`babet.fileSize(path)`

Renvoie la taille d'un fichier régulier en octets.

```
local size, err = babet.fileSize("video.mp4")
if not size then
    print("Taille inaccessible :", err)
else
    print(size, "octets")
end
```

La fonction suit un symlink valide vers un fichier. Elle refuse les dossiers et les autres types de fichiers.

```
local size, err = babet.fileSize("documents")
-- size == nil ; err indique que le chemin n'est pas un fichier régulier
```

Répertoire courant

`babet.currentDir()`

Renvoie le répertoire de travail courant du processus.

```
local cwd, err = babet.currentDir()
assert(cwd, err)
print(cwd)
```

Le résultat est le chemin courant tel que fourni par le système de fichiers, généralement absolu.

`babet.chdir(path)`

Change le répertoire courant de tout le processus.

```
local previous = assert(babet.currentDir())
assert(babet.chdir("/tmp"))
print(assert(babet.currentDir()))
assert(babet.chdir(previous))
```

Le chemin est résolu vers sa forme canonique : les composants `.` et `..` et les symlinks sont résolus avant le changement.

Le répertoire courant est partagé par tous les threads. Pour cette raison, `chdir` est définitivement refusé dès que le premier `babet.workers.spawn()` a été effectué, même si le worker est déjà terminé. La première tentative de chargement GTK via `gui.available()` ou `gui.init()` déclenche la même règle, même si elle échoue. Configure le répertoire avant ces appels ; `babet.currentDir()` reste disponible en lecture. Voir le contrat GUI.

```
assert(babet.chdir("/srv/mon-app")) -- à faire avant tout spawn
local worker = assert(babet.workers.spawn("return true"))

local ok, err = babet.chdir("/tmp")
-- ok == nil ; err explique que chdir est interdit après workers.spawn
```

Manipulation lexicale des chemins

Ces fonctions traitent uniquement des chaînes. Elles ne vérifient pas que le chemin existe.

`babet.getBasename(path)`

Renvoie la dernière composante, extension comprise.

```
local name = assert(babet.getBasename("/var/log/app.log"))
print(name) -- app.log
```

Un chemin qui ne possède pas de dernière composante exploitable, comme `/`, renvoie `(nil, err)`.

`babet.getFilename(path)`

Renvoie la dernière composante sans sa dernière extension.

```
print(assert(babet.getFilename("report.tar.gz"))) -- report.tar
print(assert(babet.getFilename("README")))        -- README
```

`babet.getExtension(path)`

Renvoie la dernière extension, point compris.

```
print(assert(babet.getExtension("report.tar.gz"))) -- .gz
```

Un nom sans extension renvoie `(nil, err)`, pas une chaîne vide.

```
local ext, err = babet.getExtension("README")
-- ext == nil
```

`babet.getPath(path)`

Renvoie le dossier parent lexical.

```
print(assert(babet.getPath("/var/log/app.log"))) -- /var/log
```

Un nom simple sans composante parent, par exemple `app.log`, renvoie `(nil, err)` plutôt que `"."`.

`babet.joinPath(...)`

Deux formes sont acceptées :

```
local path = babet.joinPath("var", "lib", "mon-app", "data.db")
```

```
local path = babet.joinPath({ "var", "lib", "mon-app", "data.db" })
```

Il faut au moins deux segments non vides. En cas de succès, `joinPath` renvoie **une seule valeur**, la chaîne assemblée. En cas d'échec, elle renvoie `(nil, err)`.

Dans la forme table, Babet lit directement les entrées réellement stockées dans la partie tableau, dans l'ordre brut `1..n`. Les métaméthodes `__len` et `__index` ne sont pas appelées et ne peuvent donc pas fabriquer de segments. Une entrée brute manquante est refusée au lieu d'être fournie par `__index`. Les clés qui n'appartiennent pas à la partie tableau ne constituent pas des segments et sont ignorées.

La fonction ne fait qu'ajuster le séparateur `/` à la jonction des segments :

```
print(babet.joinPath("/var/", "/log", "app.log"))
-- /var/log/app.log
```

Elle ne normalise pas les composants `.` ou `..` et ne traite pas un segment absolu ultérieur comme un nouveau point de départ :

```
print(babet.joinPath("base", "..", "autre")) -- base/../autre
print(babet.joinPath("base", "/autre"))      -- base/autre
```

Ne mélange pas la forme table et les arguments séparés ; lorsque le premier argument est une table, son contenu constitue la liste complète des segments.

Créer, supprimer, renommer et lier

`babet.touch(path)`

Crée un fichier vide lorsque le chemin n'existe pas, ou met à jour la date de dernière modification d'un chemin existant.

```
assert(babet.touch("cache/ready.flag"))
```

Le dossier parent doit déjà exister : `touch` ne le crée pas.

```
local ok, err = babet.touch("absent/sous/dossier/fichier.txt")
-- ok == nil si absent/sous/dossier n'existe pas
```

Pour créer d'abord les parents :

```
assert(babet.mkdir("cache/images"))
assert(babet.touch("cache/images/index.dat"))
```

Sur un chemin existant, la fonction met à jour son horodatage ; elle peut donc également toucher un dossier, comme la commande Unix `touch`. Un symlink final valide est suivi vers sa cible.

`touch` n'ouvre jamais un chemin existant dans un mode tronquant. Le parent et la cible existante sont épinglés avant l'opération : si un fichier apparaît pendant une course de création, il est repris sans perdre son contenu. Un symlink final pendant est refusé ; Babet ne crée pas sa cible absente à travers un chemin de lien encore mutable.

`babet.mkdir(path)`

Crée un dossier avec une sémantique équivalente à `mkdir -p` :

- la création est **toujours récursive** ;
- les parents manquants sont créés ;
- rappeler la fonction sur un dossier existant réussit ;
- un fichier ou un autre non-dossier bloquant le chemin produit `(nil, err)`.

```
assert(babet.mkdir("data/cache/images"))
-- crée data, data/cache et data/cache/images si nécessaire

assert(babet.mkdir("data/cache/images"))
-- succès également lorsque le dossier existe déjà
```

Il n'existe **aucune option pour désactiver la récursivité**. Pour exiger que le parent existe déjà, vérifie-le avant l'appel :

```
local parent = "data/cache"
assert(babet.isDir(parent), "le parent doit déjà exister")
assert(babet.mkdir(babet.joinPath(parent, "images")))
```

Les nouveaux dossiers utilisent les permissions `0777` filtrées par l'umask du processus. Le mode du dossier source n'entre pas en jeu.

`babet.remove(path)`

Supprime uniquement :

- un fichier régulier ;
- un lien symbolique, valide ou cassé.

```
assert(babet.remove("temp.txt"))

-- le lien est supprimé, pas sa cible
assert(babet.link("fichier-important.txt", "raccourci"))
assert(babet.remove("raccourci"))
```

La fonction refuse les dossiers, FIFO, sockets, devices et autres types spéciaux.

```lua
local ok, err = babet.remove("mon-dossier")
-- ok == nil ; utiliser rmdir ou rmdirAll
```

babet.rmdir(path)

Supprime un **vrai dossier vide**.

```lua
assert(babet.mkdir("cache-vide"))
assert(babet.rmdir("cache-vide"))
```

La fonction refuse :

- un dossier non vide ;
- un fichier ;
- un symlink, même s'il pointe vers un dossier ou se termine par `/` ou `/.` ;
- un chemin absent.

```lua
local ok, err = babet.rmdir("dossier-non-vide")
```

babet.rmdirAll(path)

Supprime récursivement un **vrai dossier racine** et tout son contenu.

```lua
assert(babet.rmdirAll("build-temp"))
```

La racine doit exister et être un dossier réel. Un fichier ou un symlink vers un dossier est refusé, y compris avec un suffixe `/`, `/.` ou `/./`.

Les symlinks situés *à l'intérieur* du dossier supprimé sont supprimés comme des entrées ; leur cible extérieure n'est pas parcourue.

```lua
local is_dir = babet.isDir("build-temp")
if is_dir then
    assert(babet.rmdirAll("build-temp"))
end
```

babet.rename(source, destination)

Renomme ou déplace une entrée sur le même système de fichiers.

```lua
assert(babet.rename("ancien.txt", "nouveau.txt"))
```

La fonction accepte les fichiers, dossiers et symlinks, y compris les liens cassés. Elle ne crée pas le dossier parent de destination.

Sous Linux, elle suit la sémantique de `rename(2)` : une destination existante peut être remplacée lorsque les types sont compatibles. Une traversée de systèmes de fichiers échoue ; `rename` n'effectue pas de fallback copie + suppression.

```lua
local ok, err = babet.rename("/tmp/a.txt", "/autre-montage/a.txt")
-- peut échouer avec cross-device link
```

Pour déplacer une arborescence avec fallback cross-filesystem, utilise `moveTree`.

babet.link(target, linkpath)

Crée toujours un **lien symbolique**, jamais un lien dur.

La cible est enregistrée exactement telle qu'elle est fournie :

```lua
assert(babet.link("../shared/config.toml", "app/config.toml"))
```

Une cible relative reste relative, et une cible inexistante est autorisée :

```lua
assert(babet.link("future-file.txt", "dangling-link"))
```

Contrairement à `touch` et `copy`, `link` crée récursivement les dossiers parents explicites de `linkpath` :

```
assert(babet.link("../../data.db", "a/b/c/database"))
-- crée a/, a/b/ et a/b/c/ si nécessaire
```

Le chemin du lien ne doit pas déjà être occupé.

Lister, itérer et rechercher

Les trois API ne renvoient pas les chemins sous la même forme :

Fonction	Forme des chemins renvoyés
`listFiles(root, ...)`	relatifs à `root`
`createFileIterator(root, ...)`	tels que produits depuis `root` : préfixés par le chemin fourni
`find(root, ...)`	tels que produits depuis `root` : préfixés par le chemin fourni

`babet.listFiles(path, recursive?)`

Renvoie une table séquentielle contenant uniquement les fichiers réguliers. Les dossiers ne sont jamais inclus.

Listing non récursif — comportement par défaut

```
local files, err = babet.listFiles("assets")
assert(files, err)

for _, relative_path in ipairs(files) do
    print(relative_path)
end
```

Seuls les fichiers directement contenus dans `assets` sont renvoyés.

Listing récursif

```
local files, err = babet.listFiles("assets", true)
assert(files, err)

table.sort(files)
for _, relative_path in ipairs(files) do
    print(relative_path)
end
```

Les chemins sont relatifs à la racine demandée :

```
logo.png
icons/open.png
icons/close.png
```

et non :

```
assets/logo.png
assets/icons/open.png
```

Symlinks

— un symlink valide vers un fichier régulier est inclus sous le nom du lien ;
— un lien cassé est ignoré ;
— un symlink vers un dossier n'est ni inclus comme fichier, ni parcouru ;
— une boucle de symlinks de dossiers ne provoque donc pas de récursion infinie.

```lua
local files = assert(babet.listFiles("collection", true))
-- un lien collection/externe -> /autre/dossier n'est pas suivi
```

Le second argument est facultatif et vaut `false` par défaut. Lorsqu'il est présent, il doit être un booléen Lua strict : nombres et chaînes ne sont pas convertis. La fonction accepte exactement un ou deux arguments.

`babet.createFileIterator(path, recursive?)`

Crée un véritable userdata paresseux possédant deux méthodes :

— `iterator:next()` — `(chemin, nil)`, `(nil, err)` ou `(nil, nil)` à la fin ;
— `iterator:close()` — libère explicitement l'itérateur natif.

Seul l'itérateur de dossier est créé au départ. L'arbre avance d'une entrée à chaque appel à `next()` : un gros parcours n'est plus copié dans un `std::vector` avant le premier résultat et la mémoire ne croît plus avec le nombre total de fichiers.

Non récursif

```lua
local iterator, err = babet.createFileIterator("assets")
assert(iterator, err)

while true do
    local path, next_err = iterator:next()
    assert(not next_err, next_err)
    if path == nil then
        break
    end
    print(path) -- assets/logo.png, etc.
end

iterator:close()
```

Récursif

```lua
local iterator = assert(babet.createFileIterator("assets", true))
while true do
    local path, next_err = iterator:next()
    assert(not next_err, next_err)
    if not path then break end
    print(path)
end
iterator:close()
```

L'ouverture de la racine peut toujours échouer immédiatement et renvoie `(nil, err)`. Une erreur rencontrée plus tard pendant l'inspection ou l'avancement du dossier est différée jusqu'à l'appel à `next()` correspondant, qui renvoie `(nil, err)`. La fin normale possède le résultat distinct `(nil, nil)`.

Comportement des symlinks :

— lien valide vers un fichier régulier : inclus sous le chemin du lien ;
— lien cassé, boucle ou cible inaccessible : ignoré ;

— lien vers un dossier : non parcouru ;
— véritable erreur d'inspection d'une entrée ou d'avancement : renvoyée par `next()`.

Le code existant qui ne lit que la première valeur reste compatible, mais il ne peut pas distinguer la fin d'une erreur de parcours différée ; le nouveau code doit contrôler la seconde valeur. Après `close()`, appeler `next()` lève une erreur Lua. Oublier `close()` n'est pas fatal : le garbage collector finit par libérer l'objet. L'argument facultatif `recursive` doit être un booléen Lua strict et le constructeur accepte exactement un ou deux arguments.

`babet.find(path, opts?)`

Recherche récursivement les entrées d'un dossier et renvoie une table de chemins.

Les deux appels suivants sont équivalents :

```
local entries = assert(babet.find("src"))
local entries = assert(babet.find("src", nil))
```

Sans option, fichiers **et** dossiers sont inclus. La racine elle-même n'est pas incluse.

Options

Option	Type	Défaut	Effet
`type`	string	aucun filtre	`"f"` pour fichiers, `"d"` pour dossiers
`glob`	string	absent	glob borné sur le nom complet, sensible à la casse
`iglob`	string	absent	glob borné sur le nom complet, casse ASCII ignorée
`path_glob`	string	absent	glob borné sur le chemin complet renvoyé, sensible à la casse
`path_iglob`	string	absent	glob borné sur le chemin complet renvoyé, casse ASCII ignorée
`name`	string	absent	regex RE2 sur le nom complet, sensible à la casse
`iname`	string	absent	regex RE2 sur le nom complet, insensible à la casse
`path`	string	absent	regex RE2 recherchée dans le chemin complet
`mindepth`	integer	`0`	profondeur minimale incluse
`maxdepth`	integer	sans limite pratique	profondeur maximale incluse
`xdev`	boolean	`false`	ne descend pas dans un dossier dont `st_dev` diffère de celui de la racine

Toutes les options présentes sont combinées avec un **ET logique**. Un motif vide agit comme un filtre absent, conformément au comportement historique des champs regex.

Filtres glob sûrs Utilise les champs glob pour les recherches ordinaires sur les noms de fichiers, en particulier lorsque le motif peut provenir d'une personne non fiable :

```
local lua_files = assert(babet.find("src", {
    type = "f",
    glob = "*.lua",
}))

local images = assert(babet.find("assets", {
    type = "f",
    iglob = "*.jpg",
}))
```

Le langage glob est volontairement réduit :

— * : zéro ou plusieurs octets sauf /;
— ** : zéro ou plusieurs octets, y compris /;
— ? : exactement un octet sauf /;
— \x : octet littéral x.

La correspondance est ancrée sur la totalité du nom ou du chemin. Les crochets, accolades et extensions glob du shell n'ont aucune signification particulière et sont comparés littéralement. Chaque motif est limité à 4096 octets. Le moteur n'utilise aucun backtracking récursif : son travail reste borné de façon polynomiale par la taille du motif et du candidat.

`path_glob` et `path_iglob` comparent le chemin complet renvoyé en utilisant / comme séparateur. Il faut employer ** lorsqu'un joker doit traverser des dossiers :

```
local tests = assert(babet.find(".", {
    type = "f",
    path_glob = "**/tests/**/*.lua",
}))
```

Les variantes insensibles à la casse ne replient que les lettres ASCII. Les octets UTF-8 hors ASCII sont comparés exactement, ce qui rend le comportement indépendant de la locale du processus.

Expressions régulières RE2 bornées Les champs historiques `name`, `iname` et `path` utilisent désormais le moteur RE2 lié statiquement. `name` et `iname` correspondent au nom complet :

```
local images = assert(babet.find("assets", {
    type = "f",
    iname = ".*\\.(png|jpg|jpeg)$",
}))
```

`path` effectue une recherche regex partielle dans le chemin complet :

```
local tests = assert(babet.find(".", {
    type = "f",
    path = "/tests/",
}))
```

Chaque regex est limitée à 4096 octets et compilée avec un budget mémoire RE2 de 1 Mio. La correspondance reste linéaire : lorsque le cache DFA atteint son budget, RE2 utilise son NFA borné au lieu d'un backtracking exponentiel. Babet active le mode Latin-1 de RE2 afin que chaque octet d'un chemin Linux reste traitable, y compris pour un nom qui n'est pas un UTF-8 valide. `iname` désactive la sensibilité à la casse dans RE2, sans consulter la locale du processus.

RE2 refuse volontairement les constructions qui nécessitent du backtracking ou un état non régulier, notamment les références arrière et les assertions de regard avant/arrière. Un tel motif fait

échouer tout l'appel avec `(nil, err)`. Utilise les globs sûrs pour les filtres simples et RE2 lorsqu'un regroupement, une alternative, une classe de caractères ou une répétition bornée est vraiment utile.

Limiter la profondeur La profondeur `0` correspond aux enfants directs de la racine :

```
local direct_children = assert(babet.find("src", {
    maxdepth = 0,
}))
```

Les enfants d'un sous-dossier sont à la profondeur `1` :

```
local one_level_below = assert(babet.find("src", {
    mindepth = 1,
    maxdepth = 1,
}))
```

`mindepth` filtre uniquement les résultats ; les dossiers moins profonds sont tout de même traversés pour atteindre les niveaux demandés.

Arborescences vivantes et dossiers disparus `babet.find()` parcourt l'arborescence avec une pile explicite d'itérateurs de dossiers. Avant d'ouvrir un dossier enfant, Babet avance l'itérateur du parent vers le frère suivant. Si le dossier disparaît dans cet intervalle, son `ENOENT` reste local à ce sous-arbre disparu : la recherche l'ignore et reprend depuis le parent déjà positionné.

La même règle s'applique lorsqu'un dossier en cours d'énumération disparaît : ce cadre devenu inutilisable est retiré et le parcours reprend dans son parent. Ce comportement vaut avec ou sans `xdev`, conserve l'ordre préfixe pour les entrées encore présentes et ne transforme pas les autres erreurs en succès partiel. Les erreurs de permission, d'entrée/sortie, les boucles de symlinks et toute erreur autre qu'`ENOENT` font toujours échouer l'appel complet avec `(nil, err)`.

Rester sur le système de fichiers de la racine `xdev = true` mémorise le champ Linux `st_dev` de la racine réellement parcourue, puis empêche la descente dans tout dossier appartenant à un autre périphérique. L'option est un booléen strict et vaut `false` par défaut : son absence ne change donc aucun parcours historique.

```
local local_entries = assert(babet.find("/srv/application", {
    xdev = true,
}))
```

Comme avec `find -xdev`, le point de montage étranger lui-même reste visible et peut correspondre à `type`, `name`, `path`, aux globs et aux limites de profondeur. Seuls ses enfants sont élagués. Par exemple, cette recherche peut renvoyer le dossier monté `/srv/application/cache`, mais aucun fichier placé sous ce montage :

```
local local_logs = assert(babet.find("/srv/application", {
    xdev = true,
    type = "f",
    path_iglob = "**/*.log",
    maxdepth = 8,
}))
```

La comparaison porte sur le périphérique de la racine, même lorsque le chemin de racine est un symlink valide vers un dossier. Les symlinks rencontrés dans l'arborescence ne sont toujours pas suivis. La comparaison est strictement fondée sur `st_dev`, comme `find -xdev` : un sous-volume Btrfs peut donc être élagué, tandis qu'un bind mount du même système de fichiers conserve le même `st_dev` et n'est pas élagué.

Si un candidat disparaît pendant l'inspection supplémentaire de `xdev`, la même règle limitée à `ENOENT` s'applique et le sous-arbre disparu est ignoré. Une erreur lors de l'inspection de la racine, ou

toute autre erreur lors de l'inspection d'un point de descente, fait échouer l'appel avec `(nil, err)`.

Symlinks et erreurs Le parcours ne descend pas dans les symlinks de dossiers. Les tests de type suivent néanmoins une cible valide : un symlink vers un fichier peut donc correspondre à `type = "f"`, et un symlink vers un dossier peut apparaître avec `type = "d"` sans que son contenu soit parcouru.

Une regex invalide ou trop longue, un glob invalide ou trop long, ou une erreur de parcours fait échouer tout l'appel avec `(nil, err)`. Il n'existe pas de mode « continuer malgré les erreurs » pour `find`.

Copier et déplacer

`babet.copy(source, destination)`

Copie un seul fichier avec `std::filesystem::copy_file` et écrase une destination fichier existante.

```
assert(babet.copy("config/default.toml", "config/local.toml"))
```

Le dossier parent de destination doit déjà exister :

```
assert(babet.mkdir("backup/config"))
assert(babet.copy("config/app.toml", "backup/config/app.toml"))
```

Comportement important :

— un symlink source valide est suivi et donne un nouveau fichier régulier ;
— un symlink destination valide est suivi : la cible du lien est écrasée ;
— un dossier source ou destination n'est pas accepté ;
— les permissions ordinaires du fichier source sont copiées selon la sémantique du système ; owner, group et horodatages ne sont pas garantis ;
— l'opération ne bénéficie pas du confinement renforcé de `copyTree`.

Utilise `copyTree` pour une arborescence ou lorsque la destination doit être protégée contre les redirections par symlink.

`babet.copyTree(source, destination, continue_on_error?)`

Copie récursivement un dossier source dans un dossier destination.

```
assert(babet.copyTree("site", "backup/site"))
```

Valeur par défaut de `continue_on_error` Le troisième argument est facultatif et vaut **`true` par défaut**. Lorsqu'il est présent, il doit être un booléen Lua strict ; la fonction accepte exactement deux ou trois arguments.

Cela signifie qu'en cas d'erreur limitée à une entrée :

1. un avertissement est écrit sur `stderr` ;
2. la copie tente de poursuivre avec les autres entrées ;
3. l'appel final renvoie `(nil, "completed with warnings ...")`.

Le succès partiel n'est donc jamais présenté comme un succès total.

```
local ok, err = babet.copyTree("source", "destination")
if not ok then
    print(err) -- peut signaler des warnings après une copie partielle
end
```

Mode strict Passe explicitement `false` pour arrêter la copie à la première erreur :

```
local ok, err = babet.copyTree("source", "destination", false)
assert(ok, err)
```

Passe explicitement `true` lorsque tu souhaites rendre le choix visible dans le code :

```
local ok, err = babet.copyTree("source", "destination", true)
```

Destination et fusion

- la destination est créée récursivement si elle n'existe pas ;
- si elle existe, le contenu est fusionné ;
- les fichiers réguliers existants sont remplacés ;
- un symlink déjà présent dans la destination, comme composant intermédiaire ou comme fichier final, est refusé ;
- les deux arborescences doivent être disjointes : la destination ne peut être ni la source elle-même, ni un de ses descendants, ni un de ses ancêtres ;
- la racine source et la racine destination ne doivent pas être des symlinks.

Métadonnées Lorsqu'un nouveau fichier est créé :

- les bits ordinaires `rwx` du fichier source sont conservés ;
- `setuid`, `setgid` et sticky sont retirés ;
- le nouvel inode appartient à l'utilisateur et au groupe déterminés par le système pour le processus appelant ;
- les horodatages ne sont pas conservés.

Les dossiers créés utilisent `0777` filtré par l'umask. Le mode, owner, group et les dates des dossiers sources ne sont pas reproduits.

Symlinks internes Les liens symboliques présents dans l'arbre sont recréés :

- les cibles relatives restent identiques ;
- les liens cassés restent des liens cassés ;
- une cible absolue située à l'intérieur de la source est réécrite pour viser l'élément correspondant dans la destination ;
- une cible absolue extérieure reste inchangée.

Les symlinks de dossiers ne sont jamais parcourus comme des dossiers.

`babet.moveTree(source, destination)`

Déplace récursivement une arborescence.

```
assert(babet.moveTree("staging/site", "public/site"))
```

Chemin rapide Lorsque la destination n'existe pas, que le déplacement reste sur le même filesystem et qu'aucun lien absolu interne ne doit être réécrit, Babet renomme l'arbre complet. L'opération est alors rapide et conserve les inodes et leurs métadonnées.

Fusion ou traversée de filesystem Lorsque la destination existe, que le déplacement traverse un filesystem, ou qu'un lien interne doit être réécrit, Babet utilise un fallback entrée par entrée :

- scan complet de la source avant la première modification ;
- création des dossiers de destination ;
- création des symlinks de destination ;
- déplacement des autres entrées ;
- suppression des seuls liens scannés, puis des dossiers devenus vides, du plus profond jusqu'à la racine.

Le nettoyage final ne supprime jamais récursivement le contenu restant. Un fichier ajouté après le scan reste dans la source : le dossier non vide fait échouer le déplacement, même si d'autres entrées ont déjà été transférées. Les identités des liens et dossiers sont revérifiées avant leur suppression ; cela ne rend pas l'ensemble de l'opération transactionnel face aux mutations concurrentes.

Dans ce fallback, un fichier copié vers un nouvel inode conserve ses bits `rwx` ordinaires, mais pas les bits spéciaux, l'owner, le group ou les dates. Les dossiers créés suivent l'umask.

Entre deux filesystems, seuls les fichiers réguliers bénéficient du fallback copie puis suppression. Une FIFO est refusée sans attendre un écrivain. Sur un même filesystem, son déplacement par renommage reste possible.

Avant une ouverture en lecture pour copier, Babet vérifie l'inode au moyen d'un descripteur Linux `O_PATH`, qui n'ouvre pas les périphériques pour leurs opérations d'entrée/sortie. Seul un fichier régulier est ensuite ouvert via `/proc/self/fd`, en conservant le même inode même si son chemin a été remplacé. Cette étape partagée par `copyTree` et le fallback de `moveTree` nécessite procfs ; son échec refuse la copie sans publier le fichier de destination.

Après une copie EXDEV, `moveTree` garde l'inode source épinglé et compare son identité avec le chemin source avant la suppression. Un remplacement détecté provoque une erreur en conservant le remplaçant et la copie déjà publiée. Ce contrôle ne rend pas `lstat` puis `unlink` atomiques : il reste nécessaire d'éviter les modifications concurrentes de l'arbre pendant le déplacement.

Garde-fous

- la source doit être un vrai dossier, pas un symlink ;
- la destination racine ne doit pas être un symlink ;
- la destination ne peut être ni la source, ni un descendant, ni un ancêtre de la source, y compris après résolution des chemins ;
- un symlink préexistant dans la destination de fusion est refusé ;
- les liens internes utilisent les mêmes règles de réécriture que `copyTree`.

`moveTree` réduit les risques d'état partiel en scannant l'arbre avant de le modifier et en préparant les symlinks en premier. Il n'est toutefois pas une transaction générale : une erreur survenant après plusieurs déplacements de fichiers peut laisser une partie des entrées dans la source et une partie dans la destination. Le message d'erreur doit alors être traité comme nécessitant une vérification manuelle des deux arbres.

Permissions et attributs Unix

Les modes sont des entiers. Lua ne possède pas de littéral `0o755` : utilise une chaîne octale lorsque l'API l'accepte, ou `tonumber("755", 8)`.

```lua
local mode_755 = tonumber("755", 8)
```

`babet.getMode(path)` et `babet.setMode(path, mode)`

`getMode` renvoie les permissions ordinaires et les bits spéciaux dans l'intervalle `0000..07777`.

```lua
local mode, err = babet.getMode("run.sh")
assert(mode, err)
print(string.format("%04o", mode))
```

`setMode` accepte deux formes :

```lua
assert(babet.setMode("run.sh", "755"))
```

```lua
assert(babet.setMode("run.sh", tonumber("755", 8)))
```

Une chaîne est interprétée en base 8. Un nombre est pris comme valeur entière directe. Les valeurs non entières, négatives ou supérieures à `07777` sont refusées.

Les bits spéciaux peuvent être demandés explicitement :

```lua
assert(babet.setMode("outil", "4755"))
local mode = assert(babet.getMode("outil"))
assert(mode == tonumber("4755", 8))
```

Ces fonctions suivent un symlink vers sa cible.

babet.getAttributes(path)

Renvoie uniquement les trois champs suivants :

```lua
local attrs, err = babet.getAttributes("fichier.txt")
assert(attrs, err)

print(attrs.mode)  -- integer 0000..07777
print(attrs.owner) -- UID
print(attrs.group) -- GID
```

La table ne contient ni taille, ni mtime, ni type de fichier.

La fonction suit un symlink vers sa cible.

babet.setAttributes(path, uid, gid, mode?)

Modifie le propriétaire et le groupe, puis éventuellement le mode.

Sans mode :

```lua
local attrs = assert(babet.getAttributes("fichier.txt"))
assert(babet.setAttributes(
    "fichier.txt",
    attrs.owner,
    attrs.group
))
```

Avec mode :

```lua
assert(babet.setAttributes(
    "fichier.txt",
    1000,
    1000,
    tonumber("640", 8)
))
```

Contrairement à `setMode`, le quatrième argument doit être un **entier** ; une chaîne comme `"640"` n'est pas acceptée.

Validation avant modification :

— UID et GID doivent être des entiers non négatifs dans la plage de `uid_t` et `gid_t` ;
— le mode doit être compris entre `0` et `07777`.

POSIX ne fournit pas d'opération atomique combinant `chown` et `chmod`. Babet résout le chemin une seule fois, épingle cette cible, puis effectue la lecture des métadonnées, le `chown`, le `chmod` éventuel et tout rollback sur le même inode. Un remplacement du chemin pendant l'appel ne peut donc pas rediriger la phase suivante vers une nouvelle cible. Si le `chmod` échoue après un `chown` réussi, Babet tente de restaurer l'owner, le group et le mode d'origine ; une restauration incomplète est signalée dans le message d'erreur.

L'appel suit un symlink final, mais la cible sélectionnée au début reste épinglée même si le lien est modifié concurremment. Les droits nécessaires dépendent du système et de l'identité du processus ; changer l'owner exige généralement les privilèges root.

`babet.symlinkAttr(path, uid, gid)` et `babet.symlinkattr(...)`

`symlinkAttr` est le nom canonique. `symlinkattr` est un alias déprécié.

La fonction applique `lchown` et ne suit donc pas un lien symbolique :

```
assert(babet.symlinkAttr("raccourci", 1000, 1000))
```

Elle modifie l'UID et le GID du lien lui-même, pas ceux de sa cible. Elle ne modifie pas le mode.

UID et GID sont validés comme dans `setAttributes`.

Sommes de contrôle de fichiers

Toutes les fonctions suivantes lisent le fichier en flux et renvoient une chaîne hexadécimale en minuscules :

Fonction	Algorithme	Longueur hex	Usage conseillé
`babet.crc32sum(path)`	CRC-32 IEEE	8	détection d'erreurs accidentelles uniquement
`babet.md5sum(path)`	MD5	32	compatibilité legacy, pas sécurité
`babet.sha1sum(path)`	SHA-1	40	compatibilité legacy, pas sécurité
`babet.sha256sum(path)`	SHA-256	64	intégrité cryptographique courante
`babet.sha384sum(path)`	SHA-384	96	intégrité cryptographique
`babet.sha512sum(path)`	SHA-512	128	intégrité cryptographique
`babet.sha3_256sum(path)`	SHA3-256	64	intégrité cryptographique
`babet.sha3_384sum(path)`	SHA3-384	96	intégrité cryptographique
`babet.sha3_512sum(path)`	SHA3-512	128	intégrité cryptographique
`babet.blake2b512sum(path)`	BLAKE2b-512	128	intégrité cryptographique
`babet.blake2s256sum(path)`	BLAKE2s-256	64	intégrité cryptographique

Exemple :

```
local digest, err = babet.sha256sum("release.tar.gz")
assert(digest, err)
print(digest)
```

Vérification avec une valeur attendue :

```
local expected = "..."
local actual = assert(babet.sha256sum("release.tar.gz"))
assert(actual == expected, "checksum incorrect")
```

Contrat commun :

- seuls les fichiers réguliers sont acceptés ;
- un symlink valide vers un fichier régulier est suivi ;
- les dossiers, FIFO, sockets, devices et pseudo-fichiers non réguliers sont refusés ;

— une erreur de lecture renvoie `(nil, err)` et ne produit jamais le digest trompeur d'un contenu partiel ;
— le contenu n'est pas chargé entièrement en mémoire.

CRC-32, MD5 et SHA-1 ne doivent pas servir à authentifier des données face à un attaquant. Pour un nouveau contrôle d'intégrité, préfère SHA-256, SHA-3 ou BLAKE2.

CRC-32 en mémoire

`babet.crc32(data)`

Calcule le CRC-32 d'une chaîne Lua déjà en mémoire. Les chaînes Lua étant binary-safe, les octets NUL sont acceptés.

```
print(babet.crc32("abc")) -- 352441c2
print(babet.crc32(""))    -- 00000000

local bytes = "a\0b\0c"
local digest = babet.crc32(bytes)
```

En cas de succès, cette fonction renvoie une seule chaîne, sans seconde valeur `nil`. Un mauvais nombre ou type d'arguments lève une erreur Lua.

Contrat d'erreur

Erreurs Lua levées

Une mauvaise signature ou un type incompatible lève généralement une erreur Lua :

```
babet.fileSize({})       -- erreur Lua
babet.mkdir("a", true)  -- erreur Lua : mkdir accepte exactement un argument
```

Les chemins contenant un octet NUL sont refusés avant tout appel système.

Erreurs renvoyées

Les échecs du système de fichiers sont normalement renvoyés :

```
local ok, err = babet.remove("absent.txt")
if not ok then
    print(err)
end
```

Les prédicats `fileExists`, `isFile` et `isDir` traitent un chemin simplement absent comme une réponse normale `(false, nil)`. Une vraie erreur d'inspection reste `(nil, err)`.

Opérations partielles

— `copyTree` en mode continuation peut copier certaines entrées puis renvoyer une erreur de synthèse indiquant des warnings ;
— `moveTree` peut laisser un état partagé entre source et destination si une erreur survient tard dans le fallback entrée par entrée ;
— `setAttributes` tente un rollback si la phase `chmod` échoue après `chown`, mais signale explicitement si ce rollback n'est pas complet.

Décisions et limites

— **Namespace plat historique** : `babet.fileExists`, pas `babet.fs.fileExists`.
— **Noms canoniques** : `isFile`, `isDir` et `symlinkAttr`. Les formes `isfile`, `isdir` et `symlinkattr` sont dépréciées.

— **`mkdir` toujours récursif** : l'API ne possède pas de variante non récursive.
— **`copy` pour un fichier, `copyTree` pour un arbre** : aucune surcharge implicite selon le type de la source.
— **Le glob est un filtre de `find`, pas une fonction autonome** : utilise `glob`, `iglob`, `path_glob` ou `path_iglob` dans `babet.find`.
— **Pas de tri automatique** : appelle `table.sort` lorsqu'un ordre stable est nécessaire.
— **Pas de préservation complète des métadonnées dans les copies** : les permissions ordinaires des fichiers sont prises en charge, mais pas les propriétaires, groupes, dates ou modes des dossiers.
— **Pas d'I/O asynchrone** : utilise les workers pour paralléliser des traitements indépendants.

GUI — interface graphique GTK 4 optionnelle

`babet.gui` est l'API graphique de bureau optionnelle de Babet. Elle se distingue volontairement de `babet.curses` : le toolkit graphique **n'est pas lié** dans Babet. Sous Linux, le premier et seul backend supporté est GTK 4, chargé paresseusement depuis le système cible lorsqu'un script demande la GUI.

Un script qui n'utilise jamais `babet.gui` conserve le contrat de déploiement habituel de Babet. Une application GUI générée reste exactement un fichier, mais GTK 4 doit déjà être installé sur la machine cible.

Disponibilité et initialisation

```
local gui = babet.gui

local disponible, pourquoi = gui.available()
if not disponible then
    print(pourquoi)
    return
end

local ok, err = gui.init()
assert(ok, err)
```

`available()` charge et valide l'étroite surface de symboles GTK/GLib/Cairo sans initialiser l'affichage. Dès qu'un DSO GTK a été ouvert avec succès, il reste résident jusqu'à la fin du processus, y compris si la validation d'un symbole requis échoue. Le résultat du chargement est mémorisé : les appels suivants ne rouvrent pas GTK. `init()` empêche GTK de modifier la locale globale du processus puis utilise son chemin d'initialisation récupérable. Ces deux appels sont réservés au thread OS principal de Babet.

La première tentative de chargement GTK, via `available()` ou `init()`, verrouille définitivement les mutations de l'état partagé : `babet.setenv` et `babet.chdir` renvoient `nil, err`, et `os.setlocale(locale, catégorie)` lève une erreur si `locale` n'est pas `nil`. Configure donc l'environnement, le répertoire courant et la locale **avant** ces appels. `babet.env`, `babet.currentDir` et `os.setlocale(nil, catégorie)` restent consultables.

Le verrouillage précède `dlopen`, car le chargement et l'initialisation de GTK peuvent exécuter du code natif et créer des threads indépendants des workers Babet. Il reste actif même si GTK est absent, si un symbole manque ou si l'affichage ne peut pas être initialisé, ainsi qu'après fermeture des fenêtres ou recréation d'un contexte d'embedding. Un appel refusé avant le chargement (arguments invalides, mauvais thread) ne déclenche pas ce verrouillage.

La protection couvre les points d'entrée Lua de Babet. Un hôte ou un plugin natif doit aussi coordonner ses propres mutations et threads ; Babet n'intercepte pas les appels directs à la libc. Voir les précautions de GLib sur les threads et l'état global.

Si GTK 4 manque, le diagnostic nomme le runtime absent et donne des exemples d'installation Debian/Ubuntu, Arch Linux et Fedora. Si GTK est installé mais qu'aucun affichage graphique n'est initialisable, `init()` renvoie un diagnostic spécifique au lieu de terminer le processus.

API minimale des widgets

```lua
local gui = babet.gui
assert(gui.init())

local fenetre = gui.window {
    title = "Babet GUI",
    width = 480,
    height = 240,
}

local colonne = gui.box {
    orientation = "vertical", -- ou "horizontal"
    spacing = 8,
}

local texte = gui.label("Prêt")
local bouton = gui.button("Lancer")

assert(fenetre:add(colonne))
assert(colonne:add(texte))
assert(colonne:add(bouton))

assert(bouton:onClick(function()
    assert(texte:setText("Cliqué"))
    assert(gui.quit())
end))

assert(fenetre:show())
assert(gui.run())
assert(fenetre:close())
```

La première surface reste volontairement petite :

- `babet.gui.available()`
- `babet.gui.init()`
- `babet.gui.setCss(css_ou_nil)`
- `babet.gui.window([options])`
- `babet.gui.box([options])`
- `babet.gui.label(texte)`
- `babet.gui.button(texte)`
- `babet.gui.entry([options])`
- `babet.gui.drawingArea([options])`
- `babet.gui.scrolledWindow()`
- `babet.gui.spinButton([options])`
- `babet.gui.calendar([options])`
- `conteneur:add(enfant)` pour fenêtres, boxes et ScrolledWindow
- `box:remove(enfant)` / `box:clear()`
- `scrolledWindow:remove(enfant)` / `scrolledWindow:clear()`
- `label:setText(texte)` / `button:setText(texte)` / `entry:setText(texte)`
- `button:onClick(fonction)`
- `drawingArea:onClick(fonction_ou_nil)`
- `spinButton:getValue()` / `spinButton:setValue(nombre)` / `spinButton:onChanged(fonction_ou_nil)`

— `calendar:getDate()` / `calendar:setDate(année, mois, jour)` / `calendar:onChanged(fonction_ou_nil)`
— `calendar:markDay(jour)` / `calendar:unmarkDay(jour)` / `calendar:clearMarks()` / `calendar:onViewChanged(fonction_ou_nil)`
— propriétés communes : `setMargins`, `setHExpand`, `setVExpand`, `setVisible`, `setSensitive`, `addClass`, `removeClass`
— `window:show()` / `window:close()`
— `babet.gui.run()` / `babet.gui.quit()`

Les options de `window` acceptent actuellement `title`, `width` entier positif et `height` entier positif. Les options de `box` acceptent `orientation = "vertical"|"horizontal"` et `spacing` entier positif ou nul. Les chaînes transmises à GTK refusent les NUL embarqués, car le toolkit attend des chaînes C.

Entry : saisie de texte sur une ligne

```lua
local saisie = assert(gui.entry {
    text = "",              -- par défaut : vide
    placeholder = "Nom",  -- par défaut : vide
    editable = true,         -- par défaut : true
})
assert(colonne:add(saisie))

assert(saisie:onChanged(function()
    print("Texte courant :", saisie:getText())
end))
assert(saisie:onActivate(function()
    print("Texte validé :", saisie:getText())
end))
```

Créer le champ après `gui.init()` et conserver sa fenêtre pendant la boucle événementielle. `entry()` et `entry(nil)` utilisent les valeurs par défaut. Les options sont lues directement dans la table, sans appeler `__index`.

Méthode	Contrat
`entry:getText()`	Renvoie une seule chaîne Lua : une copie du texte courant.
`entry:setText(texte)`	Remplace le texte. Si la valeur finale diffère, appelle `onChanged` exactement une fois et synchroniquement avec le texte final ; un texte identique ne déclenche aucun callback.
`entry:setPlaceholder(texte)`	Définit l'indication du champ vide et sans focus ; `""` la supprime.
`entry:setEditable(booléen)`	Autorise/interdit la saisie utilisateur ; `setText` reste utilisable par le programme.
`entry:onChanged(fonction_ou_nil)`	Remplace le callback de changement de texte ; `nil` explicite le retire.
`entry:onActivate(fonction_ou_nil)`	Remplace le callback de validation, normalement déclenché par Entrée ; `nil` explicite le retire.

Les setters et les enregistrements de callbacks renvoient `true, nil`. La création renvoie un widget, ou `nil, diagnostic` en cas d'échec runtime contrôlé. Un argument incorrect ou un widget détruit/de mauvais type lève une erreur Lua. Le texte et l'indication doivent être des chaînes UTF-8 adaptées à GTK, sans NUL embarqué ; les nombres ne sont pas convertis en chaînes. `editable` exige un véritable booléen. Ces opérations ne convertissent pas les nombres et ne valident pas les données métier.

Les callbacks ne reçoivent aucun argument : capturer le champ et utiliser `getText()` pour lire sa valeur. Les deux callbacks sont indépendants et leur enregistrement ne les déclenche pas. Pour un `setText()` programmatique, Babet masque volontairement la rafale interne suppression/insertion de GTK puis émet au plus un `onChanged` après le retour du setter natif : exactement un si le texte final diffère de la copie initiale, aucun si le texte est identique. Le callback voit donc la valeur finale et jamais une chaîne vide intermédiaire due à l'implémentation de GTK. Cette notification synthétique s'exécute néanmoins avant le retour de `setText()`, sur son thread Lua appelant, y compris une coroutine reprise sur le thread OS principal. Les modifications effectuées par l'utilisateur continuent de suivre les événements `changed` ordinaires de GTK. Cela inclut leurs états intermédiaires : par exemple, remplacer une sélection en tapant peut produire brièvement un texte vide. Seul `entry:setText()` applique la coalescence décrite ci-dessus. Les événements traités par `gui.run()` utilisent le thread Lua principal. Un callback ne peut pas faire de `yield` à travers GTK. Les erreurs, y compris une tentative de yield, sont signalées et contenues. Un callback peut se remplacer, se retirer ou fermer sa fenêtre ; les états natif et Lua restent valides jusqu'au retour d'un setter en cours. Modifier le texte depuis `onChanged` peut déclencher une nouvelle notification : éviter une mise à jour récursive inconditionnelle, ou retirer temporairement le callback.

Voir `examples/gui_entry/main.lua` pour un exemple complet : texte en direct, validation par Entrée et bascule en lecture seule.

```
babet examples/gui_entry
babet --create-exe examples/gui_entry ./gui-entry-app
./gui-entry-app
```

DrawingArea : courbes natives et sélection au clic

`DrawingArea` permet de dessiner avec Cairo et de recevoir un clic utilisateur sans dépendre d'une bibliothèque de graphiques externe.

```
local zone = assert(gui.drawingArea {width = 640, height = 300})
assert(colonne:add(zone))
assert(zone:onDraw(function(ctx, largeur, hauteur)
    assert(ctx:setSourceRGB(1, 1, 1))
    assert(ctx:rectangle(0, 0, largeur, hauteur))
    assert(ctx:fill())
    assert(ctx:setSourceRGB(0.1, 0.4, 0.8))
    assert(ctx:setLineWidth(2))
    assert(ctx:moveTo(20, hauteur - 20))
    assert(ctx:lineTo(largeur - 20, 20))
    assert(ctx:stroke())
    assert(ctx:setFontSize(14))
    assert(ctx:text(20, 22, "Courbe native"))
end))
-- Depuis le callback d'un bouton/Entry, après modification des données :
assert(zone:queueDraw())

assert(zone:onClick(function(x, y, bouton, nb_pressions)
    print("clic", x, y, bouton, nb_pressions)
end))
-- nil retire seulement le callback de clic :
assert(zone:onClick(nil))
```

`drawingArea()` et `drawingArea(nil)` utilisent 320 x 200 pixels logiques. Les options `width` et `height` sont des entiers strictement positifs au plus égaux à `INT_MAX`. Elles demandent une taille de contenu, sans fixer une taille maximale ni une allocation constante. Les options sont lues sans `__index`. GTK transmet la taille réellement allouée à `onDraw(ctx, largeur, hauteur)`.

`zone:onDraw(fonction_ou_nil)` remplace le callback ; `nil` explicite le retire. L'inscription et le retrait demandent un redessin. `zone:queueDraw()` demande un redessin ultérieur, sans appeler immé-

diatement le callback. GTK peut regrouper plusieurs demandes. Ces méthodes sont réservées aux DrawingArea et renvoient `true, nil`.

`zone:onClick(fonction_ou_nil)` remplace ou retire le callback de clic. Le callback reçoit exactement `x`, `y`, `bouton`, `nb_pressions` : les deux premières valeurs sont des nombres Lua exprimés dans les coordonnées d'allocation du widget, `bouton` est un entier (`1` pour le bouton principal) et `nb_pressions` est le compteur de pressions consécutives de GTK (`1` pour un clic simple, `2` pour la seconde pression d'un double-clic, etc.). Une fonction Lua qui ne déclare que trois paramètres reste compatible, Lua ignorant les arguments supplémentaires. Contrairement à `onDraw`, ce callback est un callback d'événement ordinaire : il peut modifier des widgets et appeler `queueDraw()`. Les erreurs sont protégées et signalées sur stderr comme pour les autres callbacks GUI.

Le contexte est emprunté et valide **uniquement pendant cet appel à `onDraw`**. Un contexte conservé provoque une erreur Lua après le retour, une erreur ou une tentative de suspension du callback. Le dessin s'exécute sur le thread Lua principal. Une coroutine reprise synchroniquement sur le même thread OS peut utiliser le contexte tant que le callback est actif. Les workers ne le peuvent pas. La création des arguments et l'appel utilisateur sont protégés contre les erreurs Lua, y compris le manque de mémoire. Les erreurs sont signalées sur stderr ; la boucle reste utilisable.

Dans `onDraw`, dessiner à partir de données déjà prêtes. Créer ou modifier un widget, inscrire un callback ou appeler `queueDraw()` est refusé, même sur un autre widget. Préparer les données et demander le redessin depuis un callback de saisie. `entry:getText()` et `gui.quit()` restent autorisés ; `quit()` demande seulement l'arrêt de la boucle après le retour de GTK. Les finalizers déclenchés pendant le dessin attendent le retour de GTK avant de détruire les widgets. Ces règles suivent les restrictions de GTK pendant le dessin.

Toutes les méthodes ci-dessous renvoient `true, nil`. Les mauvais arguments, les contextes expirés et les erreurs Cairo lèvent une erreur Lua. Les paramètres numériques doivent être de vrais nombres Lua finis ; les chaînes numériques, NaN et les infinis sont refusés.

Méthode du contexte	Effet
`ctx:newPath()`	Efface le chemin courant.
`ctx:moveTo(x, y)`	Commence un sous-chemin au point indiqué.
`ctx:lineTo(x, y)`	Ajoute un segment.
`ctx:closePath()`	Referme le sous-chemin vers son début.
`ctx:rectangle(x, y, width, height)`	Ajoute un rectangle ; dimensions signées acceptées.
`ctx:arc(x, y, radius, start, finish)`	Ajoute un arc ; rayon positif ou nul, angles en radians, sens des angles croissants.
`ctx:stroke()`	Trace puis efface le chemin avec la couleur et l'épaisseur courantes.
`ctx:fill()`	Remplit puis efface le chemin avec la couleur courante.
`ctx:setLineWidth(width)`	Épaisseur strictement positive.
`ctx:setSourceRGB(r, g, b)`	Couleur opaque ; composantes entre 0 et 1 inclus.
`ctx:setSourceRGBA(r, g, b, a)`	Couleur avec alpha ; toutes les composantes entre 0 et 1 inclus.
`ctx:setFontSize(size)`	Taille de police strictement positive.
`ctx:text(x, y, text)`	Texte positionné par sa ligne de base ; UTF-8 valide, sans NUL.

Les coordonnées sont des pixels logiques GTK, origine en haut à gauche et Y croissant vers le bas. Le contexte conserve le découpage et l'échelle de GTK. Babet sauvegarde et restaure l'état graphique Cairo autour du callback et efface le chemin de dessin. `arc` peut rejoindre le point courant ; appeler `newPath()` pour un arc indépendant. `text` utilise l'API texte minimale de Cairo et déplace le point courant : terminer un chemin avant de l'annoter. Cela convient aux légendes d'un graphique, pas

au texte riche, à la mise en page multiligne ni aux écritures nécessitant une composition complexe. Aucun pointeur natif, surface persistante, exporteur d'image ou binding Cairo général n'est exposé.

Voir `examples/gui_drawing/main.lua` : courbe illustrative de poids avec intervalles de dates inégaux, bornes Y automatiques et bouton de changement des données. Cette démonstration de dessin ne sauvegarde pas de mesures.

```
babet examples/gui_drawing
babet --create-exe examples/gui_drawing ./gui-drawing-app
./gui-drawing-app
```

Conteneurs actualisables : Box et ScrolledWindow

Une `Box` peut maintenant retirer un enfant précis ou vider tous ses enfants :

```
assert(colonne:remove(ancien_widget))
assert(colonne:clear())
```

`remove(enfant)` exige que l'enfant appartienne réellement au conteneur. `clear()` retire tous les enfants. Dans les deux cas, un enfant dont le handle Lua est encore conservé reste valide et peut être ajouté de nouveau à une autre `Box` ou à un `ScrolledWindow`. Le parent cesse de le garder vivant côté Lua. Cela permet de reconstruire une liste ou un historique sans remplacer toute la fenêtre. `Window` ne propose volontairement pas `remove()`/`clear()` dans cette surface.

`gui.scrolledWindow()` crée une zone défilante GTK native. Elle accepte un seul enfant logique avec `add(enfant)` ; ajouter un second enfant avant d'avoir retiré ou vidé le premier est une erreur. Le widget enfant peut lui-même être une Box contenant autant de lignes que nécessaire :

```
local defilement = assert(gui.scrolledWindow())
local historique = assert(gui.box {orientation = "vertical", spacing = 4})
assert(defilement:add(historique))
assert(colonne:add(defilement))

-- Après une modification de la base :
assert(historique:clear())
for _, ligne in ipairs(nouvelles_lignes) do
    assert(historique:add(gui.label(ligne)))
end
```

SpinButton : saisie numérique

```
local poids = assert(gui.spinButton {
    min = 30,
    max = 250,
    step = 0.1,
    value = 91.4,
    digits = 1,
})

assert(poids:onChanged(function()
    print("Poids :", poids:getValue())
end))
assert(poids:setValue(90.8))
```

Sans options, les valeurs sont `min = 0`, `max = 100`, `step = 1`, `value = 0` et `digits = 0`. Si une plage personnalisée est fournie sans `value`, la valeur initiale est `min`, comme avec le constructeur GTK natif. `min`, `max`, `step` et `value` doivent être des nombres Lua finis ; `step` doit être strictement positif et une valeur initiale explicitement fournie doit être comprise entre les bornes. `digits` est un entier de 0 à 20. Les options sont lues directement, sans `__index`. Le SpinButton est configuré en saisie numérique.

`getValue()` renvoie un nombre Lua. `setValue(nombre)` délègue la valeur à GTK, qui la contraint à la plage du SpinButton. `onChanged(fonction_ou_nil)` remplace ou retire le callback. Un changement programmatique peut déclencher le callback synchroniquement avant le retour de `setValue()`. Comme pour Entry, le widget et son état logique sont épinglés jusqu'au retour de GTK ; une fermeture de la fenêtre depuis le callback ne crée donc pas de pointeur pendant.

Calendar : choix et modification des dates

```lua
local date = assert(gui.calendar {year = 2026, month = 10, day = 7})

assert(date:onChanged(function()
    local annee, mois, jour = date:getDate()
    print(annee, mois, jour)
end))

local joursParMois = { ["2026-10"] = {7, 18}, ["2025-12"] = {31} }
local function actualiserMarqueurs()
    local annee, mois = date:getDate() -- seulement l'année et le mois affichés
    assert(date:clearMarks())
    local cle = string.format("%04d-%02d", annee, mois)
    for _, jour in ipairs(joursParMois[cle] or {}) do
        assert(date:markDay(jour))
    end
end
assert(date:onViewChanged(actualiserMarqueurs))
actualiserMarqueurs() -- affichage initial
assert(date:setDate(2025, 12, 31)) -- les dates passées sont autorisées
actualiserMarqueurs() -- setDate() n'appelle pas onViewChanged()
```

`calendar()` et `calendar(nil)` utilisent la date sélectionnée par défaut par GTK. Pour fixer une date à la construction, `year`, `month` et `day` doivent être fournis ensemble et être des entiers formant une date grégorienne valide, avec une année comprise entre 1 et 9999. `getDate()` renvoie exactement trois valeurs `année, mois, jour`. `setDate(année, mois, jour)` accepte aussi les dates passées. `onChanged(fonction_ou_nil)` suit le même contrat protégé et remplaçable que le SpinButton et Entry.

Depuis Babet **2.30.0**, Calendar expose aussi `markDay`, `unmarkDay`, `clearMarks` et `onViewChanged`.

`markDay(jour)` place le marqueur visuel GTK natif sur un numéro de jour ; `unmarkDay(jour)` le retire et `clearMarks()` efface tous les marqueurs. `jour` doit être un entier de 1 à 31. Les marqueurs restent attachés aux **numéros de jour** quand le mois affiché change : marquer le 5 marque aussi le 5 des mois suivants. Pour des données datées, effacer et reconstruire les marqueurs pour chaque mois affiché.

`onViewChanged(fonction_ou_nil)` est appelé lors d'une navigation native de l'utilisateur vers le mois ou l'année précédent(e)/suivant(e) : flèches de l'en-tête, molette, clavier ou sélection d'un jour grisé d'un mois voisin. **`setDate()` ne le déclenche pas**, même s'il change de mois ou d'année. Après un `setDate()` réussi, appeler explicitement la même fonction de rafraîchissement, comme ci-dessus ; sinon les anciens marqueurs restent sur le nouveau mois.

La navigation GTK peut aussi ajuster le jour sélectionné **sans `onChanged()`**. Par exemple, avec GTK 4.14, passer au mois suivant depuis le 31 janvier 2024 sélectionne le 29 février et déclenche seulement `onViewChanged`. Sélectionner un jour grisé peut produire deux étapes : `onViewChanged` voit d'abord le nouveau mois avec un jour intermédiaire (29 février dans cet exemple), puis `onChanged` voit le jour sélectionné (par exemple le 1er février).

Pour reconstruire les marqueurs dans `onViewChanged`, n'utiliser que **l'année et le mois** de `getDate()` : son jour n'est pas nécessairement la sélection finale de l'utilisateur. Si l'application suit la date complète, relire `getDate()` dans les deux callbacks et au moment de valider une opération. Ne

pas supposer que `onChanged` signale à lui seul tous les changements de date, ni qu'il suit chaque navigation. Babet transmet ces signaux natifs GTK sans ajouter de notification de sélection.

L'implémentation utilise l'API Calendar disponible depuis les premières versions de GTK 4 plutôt que d'exiger les setters apparus beaucoup plus tard ; le runtime reste donc compatible avec les distributions GTK 4 antérieures à 4.20.

Style CSS

Babet peut installer une feuille CSS GTK au niveau de l'application et associer des classes CSS aux widgets :

```
assert(gui.setCss([[
.card { background: #ffffff; border-radius: 12px; padding: 12px; }
.primary { background: #2563eb; color: white; }
]]))

local panneau = assert(gui.box())
assert(panneau:addClass("card"))
local bouton = assert(gui.button("Enregistrer"))
assert(bouton:addClass("primary"))
assert(bouton:removeClass("primary"))
assert(gui.setCss(nil))
```

`gui.setCss(css_ou_nil)` remplace la feuille de style de l'application pour l'état Lua propriétaire de la GUI. `nil` la retire. `widget:addClass(nom)` et `widget:removeClass(nom)` fonctionnent sur tous les widgets vivants ; le nom est fourni sans point initial. Les diagnostics de parsing CSS restent ceux de GTK. À partir de GTK 4.12, Babet utilise `gtk_css_provider_load_from_string()` ; avec GTK 4.0 à 4.10, il se replie sur l'ancienne API `gtk_css_provider_load_from_data()`. Aucun de ces symboles n'est une dépendance directe à l'édition de liens.

Propriétés communes des widgets

Tous les widgets vivants exposent les méthodes suivantes :

Méthode	Effet
`widget:setMargins(marge)`	Applique la même marge entière positive ou nulle sur les quatre côtés.
`widget:setMargins(top, end_, bottom, start_)`	Définit séparément les quatre marges GTK, dans cet ordre.
`widget:setHExpand(booléen)`	Autorise/désactive l'expansion horizontale.
`widget:setVExpand(booléen)`	Autorise/désactive l'expansion verticale.
`widget:setVisible(booléen)`	Affiche ou masque le widget.
`widget:setSensitive(booléen)`	Active ou désactive les interactions utilisateur avec le widget.
`widget:addClass(nom)`	Ajoute une classe CSS GTK au widget.
`widget:removeClass(nom)`	Retire une classe CSS GTK du widget.

Les marges sont des entiers compris entre 0 et `INT_MAX` et les quatre autres méthodes exigent de vrais booléens Lua. Elles renvoient `true, nil`. Comme les autres mutations GTK, ces appels sont refusés pendant `onDraw`. Les lectures `entry:getText()`, `spinButton:getValue()` et `calendar:getDate()` restent des opérations non mutantes.

Boucle événementielle et callbacks

Les opérations GTK sont réservées au thread principal. Une session GUI vivante et une session `babet.curses` active sont mutuellement exclusives. `gui.run()` possède la boucle interactive principale jusqu'à la destruction de toutes les fenêtres ou l'appel à `gui.quit()`. `gui.run()` doit être lancé depuis le thread Lua principal lui-même ; un appel depuis une coroutine Lua est refusé même si cette coroutine est reprise sur le thread OS principal de Babet.

Babet installe une petite source de réveil GLib bornée afin de continuer à servir ses callbacks de signaux Unix différés pendant que GTK possède la boucle. Les workers peuvent poursuivre des calculs non graphiques, mais ne doivent jamais appeler directement les méthodes GUI.

Les callbacks Lua des boutons, Entry, SpinButton et Calendar sont exécutés sous `lua_pcall`. Une erreur est signalée sur stderr avec le préfixe `babet.gui callback error:` et ne traverse jamais la pile C de GTK ; la boucle événementielle reste utilisable.

Durée de vie

Chaque méthode valide le handle Lua. Les parents Lua gardent leurs enfants Lua vivants, tandis que GTK devient propriétaire du widget natif après `add()`. `remove()` et `clear()` reprennent une référence native avant le détachement GTK, puis retirent la racine Lua du parent : un handle enfant encore détenu par le script reste valide et réutilisable. Le signal GTK `destroy` invalide le handle Babet correspondant. Toute utilisation ultérieure produit une erreur Lua contrôlée plutôt qu'un déréférencement de pointeur natif périmé.

La collecte d'un widget non parenté libère sa référence de construction. La collecte d'une fenêtre top-level détruit cette fenêtre. À la fermeture de l'état Lua, les callbacks GUI sont neutralisés avant `lua_close()`.

Les callbacks appartiennent au handle Lua du widget. Capturer ce même widget dans son callback ne le conserve pas indéfiniment : un cycle widget/callback devenu inaccessible peut être collecté. Un parent vivant conserve bien ses handles enfants et leurs callbacks.

`--create-exe`

Aucune commande spéciale n'est nécessaire :

```
babet --create-exe ./mon-projet-gui mon-appli-gui
```

Le résultat est un seul fichier applicatif. Contrairement à une application Babet sans GUI, ce fichier dépend volontairement du runtime GTK 4 installé sur le Linux cible. Aucun `babet-gtk.so`, DSO temporaire extrait, compilateur ou linker n'est nécessaire au lancement ni au moment du `--create-exe`.

Cette exception limitée à la GUI ne modifie pas l'autonomie des scripts et applications générées qui n'utilisent pas `babet.gui`.

Compléments GTK4 — canevas de notes libres (2.29.0)

Ces composants restent **génériques** et ne définissent ni projet, ni objectif, ni base de données. Les positions et contenus sont sauvegardés par l'application.

Construction	Méthodes supplémentaires	Usage
`gui.fixed()`	`canvas:put(child, x, y)`, `canvas:move(child, x, y)`, `canvas:remove(child)`, `canvas:clear()`	Widgets placés librement (coordonnées GTK en pixels logiques).

Construction	Méthodes supplémentaires	Usage
`gui.textView {text = "...", editable = true}`	`getText()`, `setText()`, `setEditable(bool)`, `onChanged(fn ou nil)`	Éditeur multiligne GTK.
`gui.checkButton {label = "...", active = false}`	`getActive()`, `setActive(bool)`, `setText(label)`, `onChanged(fn ou nil)`	Cases à cocher.
`gui.progressBar()`	`setFraction(0..1)`, `getFraction()`, `setShowText(bool)`, `setText(label)`, `onClick(fn ou nil)`	Progression calculée ou manuelle, contrôlée par l'application.
`gui.box()`	`reorder(child, previousChild ou nil)`	Changement d'ordre natif, sans démonter les enfants.
`gui.expander {label = "...", expanded = false}`	`add(child)`, `remove(child)`, `clear()`, `setExpanded(bool)`, `getExpanded()`, `setText(label)`	Bloc repliable, **un seul enfant** GTK (utiliser `gui.box()` pour plusieurs éléments).
Widget non interactif : label, box, drawingArea, fixed, progressBar	`onDrag(function(phase, x, y) ... end ou nil)`	Geste de déplacement, déclenché sur un **élément-poignée**.
Tous les widgets	`setSizeRequest(width, height)`	Taille minimale demandée (dimension `-1` : automatique).

`onDrag` déclenche les phases `"begin"` (coordonnées locales du début), puis `"update"` et `"end"` (**déplacements relatifs au début, mesurés dans le repère stable de la surface GTK même si la poignée bouge**). Le déplacement n'est **pas automatique** : appeler `canvas:move(bloc, x, y)` depuis le callback. Il faut accrocher `onDrag` à un élément non interactif dédié (par exemple un label-titre) : boutons, éditeurs de texte, cases à cocher et expanders possèdent leurs propres gestes GTK et ne sont pas acceptés. Les callbacks sont exécutés de façon protégée ; ils ne doivent pas lever ou céder la main à GTK.

Les coordonnées de `put` et `move` doivent être finies, entre -10 000 000 et 10 000 000. Le canevas `gui.fixed()` doit être dimensionné ou avoir assez de contenu pour produire une zone défilable ; `GtkFixed` n'effectue pas de placement automatique. Un enfant retiré reste réutilisable.

Exemple exécutable : examples/gui_free_notes/main.lua.

```lua
local gui = babet.gui
assert(gui.init())
local window = gui.window {title="Notes", width=700, height=500}
local canvas = gui.fixed()
local content = gui.box {orientation="vertical"}
local handle = gui.label("Déplacer")
local editor = gui.textView {text="Mon idée"}
content:add(handle)
content:add(editor)
window:add(canvas)
canvas:put(content, 50, 40)
local ox, oy = 50, 40
handle:onDrag(function(phase, x, y)
    if phase == "begin" then ox, oy = 50, 40 end
    if phase == "update" then canvas:move(content, ox + x, oy + y) end
end)
window:show()
gui.run()
```

Le mini-exemple ci-dessus est illustratif (il faudrait mémoriser les dernières coordonnées si plusieurs déplacements sont réalisés). La démonstration complète le fait correctement.

Interaction native sur les progressions et réorganisation d'une liste

Babet expose **le véritable `GtkProgressBar`** et le réordonnancement natif **`gtk_box_reorder_child_after()`** ; une application n'a pas à redessiner une progression ni à détruire/recréer ses lignes.

`progressBar:onClick(fonction_ou_nil)`

- Fonction facultative, ou `nil` pour désactiver le callback ; renvoie `true, nil`.
- Appel sur un **appui du bouton principal**, avec quatre arguments : `(x, y, button, n_press)` ; les coordonnées sont locales au widget.
- `n_press` est le compteur de clics successifs propre à GTK, **pas** le nombre total de validations de l'application. Conserver son propre compteur.
- La progression n'augmente **jamais automatiquement** : `setFraction(0..1)` reste sous le contrôle du programme, pour les modes manuel et automatique.
- Le callback, protégé par Babet, peut être remplacé ou supprimer la fenêtre.
- `progressBar:setText()` concerne le texte **dans la barre** ; un intitulé à droite est un `gui.label()` distinct, dans une `gui.box{orientation="horizontal"}`.
- Une barre de progression n'est pas un bouton activable au clavier. Si elle sert d'action d'avancement et pas seulement d'indicateur, prévoir aussi un vrai bouton (ou un contrôle équivalent accessible au clavier).

```
local bar = assert(gui.progressBar())
local title = assert(gui.label("The graphics"))
local row = assert(gui.box {orientation = "horizontal", spacing = 8})
assert(row:add(bar))
assert(row:add(title))
assert(bar:setHExpand(true))
assert(bar:setShowText(true))
local steps, total = 0, 4
assert(bar:onClick(function(x, y, button, n_press)
    steps = math.min(total, steps + 1)
    assert(bar:setFraction(steps / total))
    assert(bar:setText(tostring(steps * 25) .. " %"))
    if steps == total then assert(title:addClass("completed")) end
end))
```

Pour barrer les libellés avec le rendu typographique GTK, utiliser le CSS :

```
label.completed, checkbutton.completed label {
    text-decoration-line: line-through;
    opacity: 0.6;
}
```

Dans une checklist : `check:onChanged(function() ... check:getActive()` permet d'ajouter ou retirer `check:addClass("completed")` / `check:removeClass("completed")`. À la fin d'une progression manuelle, appliquer la même classe sur le libellé voisin. Aucune API spécialisée de « checklist » n'est inscrite dans le moteur Babet.

`box:reorder(child, previousChild_or_nil)`

`box` doit être un `gui.box`, `child` doit déjà lui appartenir et `previousChild` doit appartenir à **la même boîte**. Le deuxième argument est obligatoire : pour placer `child` au début, il faut passer `nil` explicitement. `nil` place `child` au début ; une autre ligne le place juste après elle. La modification est native **sans `remove()`, `add()` ni reparentage** : les callbacks et l'état des widgets restent intacts, et

la table des enfants Lua conserve le même ordre. Une demande invalide est rejetée sans modifier la liste ; une demande qui ne change rien est acceptée. Par exemple :

```
-- [first, middle, last] : déplacer la dernière ligne au deuxième rang.
assert(list:reorder(last, first))
-- Placer une ligne tout en haut.
assert(list:reorder(last, nil))
```

Drag-and-drop GTK4 natif pour les listes

Depuis Babet **2.31.0**, le DND natif est disponible en complément de `onDrag()` :

- `poignee:setDragData(identifiant_ou_nil)` installe un **`GtkDragSource`** sur un widget non interactif (`label`, `box`, `drawingArea`, `fixed` ou `progressBar`). La chaîne UTF-8 sert de donnée de déplacement ; `nil` désactive la source sans reconstruire le widget. La chaîne doit être un UTF-8 valide et les octets NUL internes sont refusés afin de garder une sémantique texte exacte. L'icône de glisser montre le widget-poignée, pas l'identifiant transporté, avec un point d'accroche fixe en (10, 10).
- `cible:onDrop(fonction_ou_nil)` installe un **`GtkDropTarget`** natif sur tout widget non-fenêtre. Le callback reçoit `(phase, payload, x, y)` avec les phases `"enter"`, `"motion"`, `"drop"` et `"leave"`. Pour `leave`, `payload`, `x` et `y` valent `nil`. Pendant `enter`/`motion`, certains backends peuvent ne pas avoir encore préchargé la chaîne ; `payload` peut donc être `nil`. Lors de `drop`, la chaîne est disponible.
- Le callback accepte le dépôt par défaut (`nil` ou toute valeur vraie). Un retour explicite `false`, ou une erreur du callback, refuse cette notification. Un refus pendant `enter` ou `motion` **n'annule pas définitivement le geste** : d'autres mouvements, `leave`, et éventuellement `drop` peuvent suivre. Vérifier de nouveau l'identifiant et la position finale dans `drop`, puis renvoyer `false` si l'opération est interdite. Les callbacks restent protégés.
- `onDrop(nil)` désactive **la cible native** et son callback Lua, et annule le survol/préchargement en cours. Un enfant désactivé n'intercepte plus les dépôts destinés à une cible parente ou au traitement de texte natif du widget. `onDrop(fonction)` réactive ensuite le même contrôleur.
- Une désactivation depuis un callback de dépôt annule aussi le transfert en cours ; même une réactivation immédiate ne prend effet qu'au geste suivant. GTK peut terminer le callback en cours sans accéder à des données libérées. Cela n'annule pas les modifications du modèle déjà effectuées par le callback : désactiver avant d'appliquer une modification si l'objectif est de l'annuler.
- Le transfert possède sa propre donnée : modifier ou effacer `setDragData()` pendant un geste concerne les gestes suivants, pas l'identifiant déjà transféré.
- L'action GTK annoncée est **MOVE**. Babet transporte seulement l'identifiant ; l'application garde son modèle, choisit la position et appelle ensuite `box:reorder()`.

Utiliser une poignée non interactive dédiée. Éviter de combiner `onDrag()` et `setDragData()` sur le même widget : `onDrag` peut démarrer puis perdre la séquence lorsque le glisser-déposer natif prend la main au seuil de déplacement.

Avec des cibles imbriquées, le parent reçoit aussi `enter`/`motion` pendant le survol de l'enfant. Si le parent accepte le survol, l'enfant peut recevoir `drop` même après avoir renvoyé `false` pendant le survol. **Toujours revalider le dépôt final.** Un `false` dans le `drop` de l'enfant ne transmet pas le dépôt au parent ; seule la désactivation de l'enfant par `onDrop(nil)` le rend transparent.

Ne pas attendre `leave` après un dépôt. GTK peut terminer un dépôt accepté ou refusé sans émettre `leave`, y compris sur le parent. Dans chaque callback `drop`, nettoyer les indicateurs de toutes les cibles concernées avant de valider ou de modifier le modèle. Garder le nettoyage dans `leave` pour la sortie du pointeur ou un geste annulé :

```
local rows = { ["task-42"] = rowA }
local function clearDropIndicators()
    for _, widget in ipairs({list, rowA, rowB}) do
        assert(widget:removeClass("drop-target"))
```

```
    end
end
assert(handleA:setDragData("task-42"))
assert(rowB:onDrop(function(phase, payload, x, y)
    if phase == "enter" or phase == "motion" then
        assert(rowB:addClass("drop-target"))
    elseif phase == "leave" then
        assert(rowB:removeClass("drop-target"))
    elseif phase == "drop" then
        clearDropIndicators() -- nettoyage commun à tous les callbacks drop
        local moved = rows[payload]
        if not moved or moved == rowB then return false end
        assert(list:reorder(moved, rowB)) -- place moved juste après rowB
    end
end))
```

Si le parent `list` possède aussi un callback `onDrop`, appeler cette même fonction dans sa branche `drop`, y compris lorsque le dépôt est refusé.

Pour placer avant une cible, l'application passe à `reorder()` la ligne qui précède cette cible dans son propre modèle (ou `nil` pour la première place). Le repère d'insertion et la persistance restent donc des décisions de l'UI, mais le geste et le dépôt eux-mêmes utilisent bien `GtkDragSource` et `GtkDropTarget`, sans simulation avec `onDrag()`.

`writeFileAtomic` — écriture atomique et durable d'un fichier

`babet.writeFileAtomic()` publie une chaîne Lua binaire dans un fichier régulier sans exposer de contenu partiellement écrit sous le nom final. Les données sont écrites dans un temporaire privé du même dossier, les permissions finales sont appliquées, puis le temporaire est publié atomiquement.

Cette fonction est adaptée aux configurations, jetons, états JSON, petits fichiers binaires et captures déjà présentes en mémoire. Elle ne remplace pas les API de streaming comme `babet.http.download()` pour les très gros contenus.

Sommaire

— API
— Valeurs par défaut
— Écriture simple
— Remplacement explicite
— Permissions
— Durabilité
— Données binaires
— Chemins et liens symboliques
— Atomicité et concurrence
— Utilisation dans un worker
— Exemple combiné Selenium
— Contrat d'erreur
— Limites

API

```
local ok, err = babet.writeFileAtomic(path, data, opts?)
```

Argument	Type	Rôle
`path`	chaîne	destination relative ou absolue, sans octet NUL
`data`	chaîne	contenu binaire complet, octets NUL autorisés
`opts`	table ou `nil`	options strictes

Résultats :

- succès : `true, nil` ;
- erreur de système de fichiers ou de publication : `nil, err` ;
- mauvaise arité, mauvais type ou option inconnue : erreur Lua levée.

Valeurs par défaut

```
{
    overwrite = false,
    permissions = tonumber("644", 8),
    durable = true,
}
```

Option	Défaut	Effet
`overwrite`	`false`	refuse une destination existante
`permissions`	`0644`	permissions exactes du nouveau fichier final
`durable`	`true`	synchronise le fichier temporaire puis le dossier parent

Les clés inconnues et les clés non textuelles sont refusées. Les booléens ne sont pas convertis depuis `0`, `1` ou une chaîne. `permissions` doit être un entier compris entre `0` et `0777`.

Écriture simple

```
local ok, err = babet.writeFileAtomic("state.json", [[{"ready":true}]])
assert(ok, err)
```

La destination ne devient visible qu'après l'écriture complète du temporaire. Si `state.json` existe déjà, l'appel échoue et conserve son contenu.

La fonction ne crée pas les dossiers parents :

```
assert(babet.mkdir("runtime"))
assert(babet.writeFileAtomic("runtime/state.json", "{}"))
```

Remplacement explicite

```
local ok, err = babet.writeFileAtomic("state.json", new_state, {
    overwrite = true,
})
assert(ok, err)
```

Avec `overwrite = true` :

- une destination absente est créée ;
- un fichier régulier existant est remplacé par renommage dans le même dossier ;
- un lien symbolique, dossier, FIFO, socket ou périphérique est refusé ;

— l'ancien fichier reste intact après toute erreur antérieure à la publication.

Sans écrasement, Babet utilise une publication qui échoue atomiquement si une autre tâche gagne la course et crée la destination en premier.

Permissions

```
local ok, err = babet.writeFileAtomic("token.bin", token, {
    permissions = tonumber("600", 8),
})
assert(ok, err)
```

Le temporaire est toujours créé avec un mode privé `0600` ou plus restrictif à cause de l'`umask`. Avant publication, Babet applique ensuite exactement les permissions demandées avec `fchmod()`. Les bits setuid, setgid et sticky ne peuvent pas être demandés : la plage autorisée est `0000..0777`.

Lors d'un remplacement, les permissions du nouveau fichier proviennent de l'option et non de l'ancien fichier.

Durabilité

Avec le défaut `durable = true`, la séquence est :

1. création d'un temporaire unique dans le dossier destination ;
2. écriture complète en gérant `EINTR` et les écritures partielles ;
3. application des permissions ;
4. `fsync()` du temporaire ;
5. publication atomique ;
6. `fsync()` du dossier parent.

Cette séquence vise la conservation après une panne brutale, selon les garanties du système de fichiers et du matériel.

Pour un cache reconstructible où la performance prime :

```
local ok, err = babet.writeFileAtomic("cache/index.bin", cache, {
    overwrite = true,
    durable = false,
})
assert(ok, err)
```

`durable = false` conserve la publication atomique, mais ne confirme pas la persistance après une panne ou une coupure d'alimentation.

Si la publication réussit puis que la synchronisation du dossier échoue, l'appel renvoie une erreur explicite indiquant que la destination est déjà publiée atomiquement mais que sa persistance n'a pas pu être confirmée.

Données binaires

`data` est une chaîne Lua binaire. Aucun contrôle UTF-8 n'est effectué :

```
local payload = "\0\1\2PNG\255\254"
assert(babet.writeFileAtomic("capture.bin", payload))
```

Une chaîne vide crée un fichier régulier de taille nulle.

La totalité des données doit déjà être en mémoire. Pour un téléchargement ou une transformation de plusieurs gigaoctets, préfère une API qui écrit en streaming.

Chemins et liens symboliques

Les chemins peuvent être relatifs ou absolus. Ils sont interprétés littéralement : aucune expansion de ~, de `$HOME`, du shell ou des globs n'est effectuée.

Pour confiner l'écriture :

- chaque composant parent est ouvert avec `openat()` et `O_NOFOLLOW` ;
- un composant parent symbolique est refusé ;
- `..` est refusé ;
- le dernier composant symbolique est refusé ;
- les dossiers parents absents ne sont pas créés ;
- la destination doit être absente ou être un fichier régulier lorsque `overwrite = true`.

```
local ok, err = babet.writeFileAtomic("link/config.json", "{}", {
    overwrite = true,
})
assert(not ok)
print(err)
```

Un processus disposant lui-même du droit d'écriture sur le dossier parent reste dans le même périmètre de confiance : POSIX ne fournit pas de renommage conditionnel portable lié à un inode précis. Babet ne suit aucun lien final et réinspecte la destination immédiatement avant publication, mais les permissions du dossier restent la frontière de sécurité contre un acteur hostile concurrent.

Atomicité et concurrence

Deux lecteurs n'observent jamais un mélange de l'ancien et du nouveau contenu : ils voient l'un ou l'autre fichier complet.

Avec `overwrite = false`, plusieurs producteurs peuvent tenter la même création :

```
local ok, err = babet.writeFileAtomic("once.dat", value)
if not ok and err:find("already exists", 1, true) then
    -- Un autre producteur a gagné.
end
```

Un seul appel publie la destination. Les autres échouent sans la remplacer.

Avec `overwrite = true`, le dernier renommage réussi détermine le contenu final. L'ordre entre deux writers concurrents n'est pas défini.

Utilisation dans un worker

La fonction est disponible dans chaque état Lua worker :

```
local job = assert(babet.workers.spawn([[
    local ok, err = babet.writeFileAtomic(worker.args.path, worker.args.data, {
        overwrite = true,
        permissions = tonumber("600", 8),
    })
    assert(ok, err)
    return true
]], {
    path = "runtime/result.bin",
    data = "worker-result",
}))

assert(job:join())
```

Le contenu transmis dans `worker.args` reste soumis au sérialiseur JSON des workers. Une chaîne binaire avec NUL ne peut donc pas être passée directement par `worker.args` ; elle peut être produite dans le worker, conservée en Base64 ou lue depuis une autre source.

Exemple combiné Selenium

Décoder une capture WebDriver puis la publier sans fichier partiel :

```
local png, err = babet.base64.decode(response.value, {
    max_output = 32 * 1024 * 1024,
})
assert(png, err)

assert(babet.mkdir("captures"))
local ok
ok, err = babet.writeFileAtomic("captures/latest.png", png, {
    overwrite = true,
    permissions = tonumber("640", 8),
    durable = true,
})
assert(ok, err)
```

Les lecteurs de `latest.png` voient soit l'ancienne capture complète, soit la nouvelle, jamais une image partiellement décodée ou écrite.

Contrat d'erreur

Erreurs Lua levées :

```
babet.writeFileAtomic()                     -- arité
babet.writeFileAtomic(42, "data")           -- path non chaîne
babet.writeFileAtomic("x", {})              -- data non chaîne
babet.writeFileAtomic("x", "data", 42)      -- opts non table
babet.writeFileAtomic("x", "data", {
    permissions = "600",
})
babet.writeFileAtomic("x", "data", {
    unknown = true,
})
```

Erreurs renvoyées sous forme `nil, err` :

- parent absent, inaccessible, symbolique ou non dossier ;
- destination existante sans `overwrite` ;
- destination finale symbolique ou non régulière ;
- échec de création, écriture, `fchmod`, `fsync`, fermeture ou publication ;
- nom temporaire unique impossible à allouer.

Après une erreur antérieure à la publication, l'ancienne destination reste intacte et le temporaire est supprimé au mieux.

Limites

La première version ne fournit pas :

- création automatique des dossiers parents ;
- ajout à un fichier ;
- écriture depuis un objet `io.file` ou un descripteur ;
- API de streaming ;

— verrouillage applicatif entre plusieurs writers ;
— rotation ou plafond automatique de taille ;
— publication atomique de plusieurs fichiers comme une transaction unique.

Pour plusieurs fichiers liés, publie un nouveau dossier/version puis change un pointeur applicatif, ou utilise SQLite lorsqu'une transaction est nécessaire.

HTTP — requêtes web synchrones, headers, corps, TLS et limites

`babet.http` est un client HTTP/HTTPS synchrone construit sur cpp-httplib et OpenSSL. Il convient aux APIs web, webhooks, appels REST simples, réponses en mémoire bornées et téléchargements directs vers un fichier.

Le module couvre :

— HTTP et HTTPS ;
— GET, HEAD, OPTIONS, POST, PUT, PATCH et DELETE ;
— paramètres de query encodés et normalisés ;
— headers de requête ;
— corps texte ou binaire ;
— réponses binaires ;
— headers répétés ;
— redirections optionnelles ;
— vérification TLS activée par défaut ;
— timeout de connexion et budget global ;
— taille maximale de réponse en mémoire ;
— téléchargement GET en streaming avec remplacement atomique du fichier final.

Il ne fournit pas d'upload en streaming, de callbacks de réception, de session persistante exposée, de cookies, de multipart/form-data, de helper de formulaire URL-encoded, de proxy, de serveur HTTP, de WebSocket, ni d'HTTP/2 ou HTTP/3.

Table des matières du module

— Conventions essentielles
— Vue d'ensemble de l'API
— `request(opts)`
— `get(url, opts?)`
— `download(url, destination, opts?)`
— `post` et ses formes d'appel
— URL, fragments et query
— Méthodes HTTP
— Headers de requête
— Corps de requête et Content-Type
— Timeouts
— TLS et CA
— Redirections
— Taille maximale de réponse
— Taille et sécurité du téléchargement
— Tables de réponse
— Exemples complets
— Contrat d'erreur
— Sécurité et limites

Conventions essentielles

Client synchrone

L'appel bloque le thread courant jusqu'à la réponse ou l'erreur. Pour lancer plusieurs requêtes indépendantes en parallèle, utilise des workers, chacun avec sa propre requête.

Statut HTTP et erreur de transport

Une réponse HTTP reçue est un succès de transport, même avec un statut 404 ou 500 :

```
local response, err = babet.http.get(url, { timeout = 10 })
assert(response, err)

if response.status >= 200 and response.status < 300 then
    print(response.body)
else
    io.stderr:write("HTTP ", response.status, "\n", response.body)
end
```

Un échec DNS, TCP, TLS, timeout ou limite de corps renvoie `(nil, err)`.

Corps binaires

Le corps de requête et `response.body` sont binary-safe :

```
local response = assert(babet.http.post(url, "AB\0CD", {
    timeout = 10,
}))
assert(#response.body >= 0)
```

Les URL, méthodes, noms de headers, chemins de CA et valeurs destinées à une API C ne peuvent pas contenir de NUL. Les URL et headers refusent aussi CR/LF pour empêcher l'injection de lignes HTTP.

Options inconnues

Les champs inconnus de la table sont actuellement ignorés. Cette faute ne produit donc pas d'erreur :

```
local r = babet.http.get(url, { timeot = 5 }) -- typo : pas de timeout
```

Utilise exactement les noms documentés et centralise éventuellement la construction des options dans ton application.

Vue d'ensemble de l'API

```
local response, err = babet.http.request(opts)
local response, err = babet.http.get(url, opts?)
local result, err = babet.http.download(url, destination, opts?)
local response, err = babet.http.post(url)
local response, err = babet.http.post(url, body)
local response, err = babet.http.post(url, opts)
local response, err = babet.http.post(url, body, opts)
local response, err = babet.http.post(url, nil, opts)
```

Table d'options

Champ	Type	Défaut	Rôle
`url`	string	obligatoire pour `request`	URL absolue `http://` ou `https://`
`method`	string	`"GET"`	méthode supportée, insensible à la casse
`headers`	table	`{}`	noms string, valeurs string ou number
`body`	string binaire	absent	corps, méthodes à corps uniquement
`query`	table	absente	clés string, valeurs string ou number
`timeout`	nombre fini `> 0`	défauts internes	secondes, connexion + budget global
`verify`	booléen strict	`true`	vérification du certificat HTTPS
`ca_cert`	string	autorités par défaut d'OpenSSL	fichier PEM utilisé à la place des autorités par défaut pour cette requête
`follow_redirects`	booléen strict	`false`	suit les redirections
`max_body_size`	entier `1..2 Gio`	`64 MiB`	cap mémoire pour `request`/`get`/`post`
`max_file_size`	entier positif	`8 Gio`	cap du fichier reçu par `download`

Les wrappers `get` et `post` remplacent toujours `url` et `method`, même si la table fournie contient d'autres valeurs.

`babet.http.request(opts)`

Forme générale pour toutes les méthodes :

```
local response, err = babet.http.request({
    url = "https://api.example.com/items/42",
    method = "DELETE",
    headers = {
        ["Authorization"] = "Bearer " .. token,
    },
    timeout = 10,
})
```

Le premier argument doit être une table. `opts.url` doit être présent et être une string stricte. Les erreurs dans les champs renvoient `(nil, err)` plutôt que de lever, sauf mauvais type du premier argument.

`babet.http.get(url, opts?)`

Raccourci qui copie superficiellement `opts`, puis impose :

```
opts.url = url
opts.method = "GET"
```

```
local response, err = babet.http.get(
    "https://api.example.com/health",
    { timeout = 5 }
)
```

Le second argument doit être absent, `nil` ou une table.

Un corps est interdit pour GET, y compris via `opts.body` :

```lua
local response, err = babet.http.get(url, { body = "x" })
-- nil, "http: body not allowed for GET"
```

babet.http.download(url, destination, opts?)

Télécharge une réponse GET directement dans un fichier, sans accumuler le corps dans Lua ni dans une chaîne C++ :

```lua
local result, err = babet.http.download(
    "https://example.com/archive.tar.gz",
    "downloads/archive.tar.gz",
    {
        timeout = 120,
        follow_redirects = true,
        max_file_size = 4 * 1024 * 1024 * 1024,
    }
)
assert(result, err)
assert(result.saved, "HTTP " .. result.status)
print(result.path, result.bytes)
```

L'appel reste synchrone, mais les chunks reçus sont écrits progressivement dans un fichier temporaire placé dans le même dossier. La destination finale est remplacée par un unique `renameat` seulement après réception complète de la réponse finale, synchronisation et fermeture du temporaire.

Le wrapper impose toujours GET. Il accepte les options communes `headers`, `query`, `timeout`, `verify`, `ca_cert` et `follow_redirects`. `body` est refusé. `max_file_size` remplace `max_body_size` pour cet appel :

- entier Lua strictement positif ;
- défaut : 8 Gio ;
- configurable jusqu'à `math.maxinteger` ;
- appliqué aux octets réellement transmis au receiver fichier.

Seule une réponse finale 2xx est validée. Une réponse 1xx, 3xx non suivie, 4xx ou 5xx reçue n'est pas une erreur de transport : `download` renvoie une table avec `saved = false`, supprime le temporaire et laisse une éventuelle destination existante strictement inchangée.

Le contrat du chemin est volontairement strict :

- le dossier parent doit déjà exister ;
- Babet ne crée pas les dossiers parents ;
- les composants `..` sont refusés ;
- les composants parents qui sont des symlinks sont refusés ;
- un symlink exactement à la destination est remplacé comme inode, sans suivre ni modifier sa cible ;
- le temporaire est créé dans le même dossier avec `O_EXCL` et supprimé après toute erreur ;
- une destination existante reste intacte après erreur DNS, TCP, TLS, timeout, limite de taille, statut HTTP non-2xx ou écriture disque.

Le renommage atomique empêche les lecteurs d'observer un fichier final partiel. L'API ne fournit pas de reprise, callback de progression, durabilité garantie du dossier parent après crash, ni préservation des métadonnées de l'ancien fichier. Le nouveau fichier est créé avec le mode `0666` filtré par l'umask du processus.

`babet.http.post` et ses formes d'appel

Sans corps

```
local response, err = babet.http.post(url)
local response, err = babet.http.post(url, { timeout = 5 })
local response, err = babet.http.post(url, nil, { timeout = 5 })
```

Dans ces formes, aucun champ `body` n'est ajouté par le wrapper. La requête POST peut donc être envoyée sans corps explicite.

Corps positionnel

```
local response, err = babet.http.post(url, "payload", {
    timeout = 5,
})
```

Le corps doit être une string. Un nombre, un booléen ou une table à cette position lève une erreur Lua, sauf qu'une table au deuxième argument est interprétée comme `opts`.

Corps dans `opts`

```
local response, err = babet.http.post(url, {
    body = "payload",
    timeout = 5,
})
```

Priorité

Si un corps positionnel et `opts.body` sont fournis, le corps positionnel gagne :

```
local response = assert(babet.http.post(url, "used", {
    body = "ignored",
    timeout = 5,
}))
```

URL, fragments et query

URL absolue

Babet accepte uniquement les schemes `http` et `https` :

```
https://example.com:8443/path?existing=1#fragment
```

L'URL doit contenir un host non vide. Les littéraux IPv6 entre crochets sont pris en charge :

```
local r, err = babet.http.get("http://[::1]:8080/", { timeout = 2 })
```

Le fragment `#...` n'est jamais envoyé au serveur.

Babet n'effectue pas une validation RFC exhaustive du host avant de le passer à cpp-httplib. Les erreurs de résolution ou de syntaxe restantes apparaissent au transport.

Ajouter une query

```
local response = assert(babet.http.get("https://example.com/search", {
    query = {
        q = "lua socket",
        page = 2,
    },
```

```
    timeout = 10,
}))
```

Les clés doivent être des strings. Les valeurs peuvent être des strings ou des numbers. Babet protège les délimiteurs et les octets non ASCII, puis cpp-httplib 0.45.0 normalise la query avant l'envoi. Le format effectivement envoyé suit donc les conventions suivantes :

- l'espace devient `+` ;
- `+` devient `%2B` ;
- `/` et `?` restent littéraux dans une valeur ;
- les octets UTF-8 non ASCII sont percent-encodés en `%HH` ;
- `&`, `=` et `%` sont percent-encodés lorsqu'ils appartiennent à une clé ou une valeur.

Par exemple, `q = "lua socket"` devient `q=lua+socket`, tandis que `path = "a/b"` reste `path=a/b`.

Si l'URL contient déjà `?a=b`, les nouveaux paramètres sont ajoutés avec `&`. La query déjà présente est elle aussi normalisée par cpp-httplib : `%20` peut devenir `+` et `%2F` peut devenir `/`. Le fragment `#...` est supprimé avant l'envoi.

L'ordre d'itération d'une table Lua n'est pas garanti : ne signe pas une URL en supposant l'ordre produit par `query`. Pour une signature canonique, construis toi-même une URL déjà ordonnée et tiens compte de la normalisation décrite ci-dessus.

Les NUL présents dans une clé ou une valeur sont envoyés sous la forme `%00`, car la query est traitée comme des octets. Assure-toi que le serveur accepte ce contenu.

Méthodes HTTP

Méthodes supportées :

- GET ;
- HEAD ;
- OPTIONS ;
- POST ;
- PUT ;
- PATCH ;
- DELETE.

La casse est normalisée : `method = "put"` devient PUT.

```
local response = assert(babet.http.request({
    url = "https://api.example.com/items/42",
    method = "PATCH",
    body = '{"enabled":true}',
    headers = { ["Content-Type"] = "application/json" },
    timeout = 10,
}))
```

GET, HEAD et OPTIONS refusent un champ `body`. POST, PUT, PATCH et DELETE acceptent un corps absent, vide ou non vide.

Il n'existe pas de raccourci `put`, `patch` ou `delete` dans la version actuelle : utilise `request`.

Headers de requête

```
headers = {
    ["Accept"] = "application/json",
    ["Authorization"] = "Bearer " .. token,
    ["X-Retry"] = 3,
}
```

Noms

Une clé doit être une string non vide composée uniquement de caractères HTTP `token` : lettres, chiffres et `!#$%&'*+-.^_`|~`.

Sont notamment refusés :

- espace ou tabulation ;
- `:` ;
- CR/LF ;
- NUL ;
- clé non string.

Cette validation empêche l'injection d'un second header ou d'une ligne de requête.

Valeurs

Une valeur peut être une string ou un number. Les nombres sont convertis en texte avec la conversion Lua.

NUL, CR et LF sont refusés. Les booléens, tables et autres types sont refusés.

Doublons

Une table Lua ne peut contenir qu'une valeur par clé exacte. Le binding ne fournit pas de forme permettant d'émettre plusieurs headers de requête portant le même nom. Pour les listes combinables, construis une valeur séparée par des virgules lorsque le standard du header l'autorise. Ne combine pas ainsi `Cookie` ou d'autres champs sans vérifier leur syntaxe.

Casse

Les noms sont transmis avec la casse fournie. HTTP traite les noms de headers sans tenir compte de la casse. Pour détecter `Content-Type`, Babet compare en minuscules.

Corps de requête et `Content-Type`

Corps absent et corps vide

Ces deux cas sont distincts :

```
-- Aucun champ body.
babet.http.post(url, { timeout = 5 })

-- Corps explicitement présent, mais vide.
babet.http.post(url, "", { timeout = 5 })
```

Lorsque `body` existe sur une méthode à corps et qu'aucun `Content-Type` n'est fourni, Babet utilise :

`application/octet-stream`

Avec un corps absent, aucun Content-Type par défaut n'est ajouté.

JSON

```
local payload = assert(babet.json.encode({
    name = "babet",
    enabled = true,
}))

local response = assert(babet.http.post(url, payload, {
    headers = {
```

```
        ["Content-Type"] = "application/json",
        ["Accept"] = "application/json",
    },
    timeout = 10,
}))
```

Formulaire URL-encoded

Babet ne fournit pas de helper `form`. Encode explicitement le corps et le header :

```
local body = "user=julien&active=1" -- encoder chaque champ si non fiable
local response = assert(babet.http.post(url, body, {
    headers = {
        ["Content-Type"] = "application/x-www-form-urlencoded",
    },
    timeout = 10,
}))
```

La table `query` ne doit pas être détournée pour produire automatiquement un corps de formulaire : elle modifie uniquement l'URL.

Multipart

Aucun constructeur multipart n'est fourni. Pour des uploads complexes ou du streaming de gros fichiers, utilise un outil spécialisé via `babet.exec`, par exemple `curl`, en transmettant les valeurs sensibles par arguments ou fichiers plutôt que par concaténation shell.

`opts.timeout`

```
local response, err = babet.http.get(url, { timeout = 10 })
```

Le timeout doit être un nombre fini strictement positif. Les valeurs positives inférieures à 1 ms sont arrondies à 1 ms. Le maximum représentable est environ `INT_MAX` millisecondes.

Babet configure deux protections cpp-httplib :

- timeout de connexion ;
- timeout maximal global de la requête.

La première limite atteinte gagne.

Limite DNS

Comme les sockets bruts, la résolution DNS synchrone peut se produire hors du budget effectivement contrôlé par cpp-httplib. Un résolveur bloqué peut donc faire dépasser la durée demandée.

Toujours en fournir un

Sans `opts.timeout`, Babet laisse les valeurs internes de cpp-httplib et du système s'appliquer. Pour tout serveur distant, passe explicitement une limite.

HTTPS, vérification et CA

Vérification par défaut

```
local response = assert(babet.http.get("https://example.com/", {
    timeout = 10,
}))
```

`verify = true` par défaut. Le certificat et le hostname de l'URL sont vérifiés par cpp-httplib/OpenSSL.

CA spécifique

```
local response = assert(babet.http.get("https://internal.example/", {
    ca_cert = "/etc/myapp/internal-ca.pem",
    timeout = 10,
}))
```

Avec un `ca_cert` non vide, HTTP utilise les autorités de ce fichier **à la place des autorités par défaut** pour la requête. Sans `ca_cert`, ou avec `ca_cert = ""`, il charge les autorités par défaut d'OpenSSL, en tenant compte de `SSL_CERT_FILE` et `SSL_CERT_DIR`. Le fichier explicite n'est pas ajouté aux autorités par défaut. Cette règle s'applique aussi à `http.download`.

Cette politique diffère de `socket.connect_tls` et `starttls`, ainsi que de `websocket.connect`, qui **ajoutent** leurs CA personnalisées aux autorités déjà chargées. HTTP n'expose pas `ca_path` et n'effectue pas leur recherche supplémentaire des emplacements de CA connus des distributions.

Exemple : si A appartient aux autorités par défaut et si le fichier fourni contient uniquement B, HTTP accepte un serveur signé par B et rejette un serveur signé uniquement par A. TLS socket et WSS continuent d'accepter A et B. Dans tous les cas, `verify = true` vérifie aussi l'identité du serveur. La configuration d'un appel ne rend pas B fiable pour les appels suivants.

Choisir un fichier de CA n'est pas du pinning d'un certificat serveur ou d'une clé : plusieurs certificats signés par une autorité du fichier peuvent être acceptés, à condition de réussir les autres vérifications TLS.

Désactiver la vérification

```
local response = assert(babet.http.get("https://127.0.0.1:8443/", {
    verify = false,
    timeout = 3,
}))
```

Réserve cette forme aux tests contrôlés. Il n'existe pas d'option `hostname` HTTP séparée : le host de l'URL est utilisé.

`follow_redirects`

Par défaut, une redirection reste visible :

```
local r = assert(babet.http.get(url, { timeout = 10 }))
if r.status == 301 or r.status == 302 or r.status == 307 or r.status == 308 then
    print(r.headers.location)
end
```

Pour suivre automatiquement :

```
local r = assert(babet.http.get(url, {
    follow_redirects = true,
    timeout = 10,
}))
```

La table de réponse décrit alors la réponse finale. Babet n'expose pas l'historique des hops ni une option `max_redirects` ; la limite interne de cpp-httplib s'applique.

Lorsqu'une redirection change de schéma, d'hôte ou de port, cpp-httplib 0.45 retire automatiquement les headers `Host`, `Authorization` et `Proxy-Authorization` avant la requête suivante. Une redirection de même origine conserve `Authorization`. Les autres headers ajoutés par le script, notam-

ment `Cookie` ou un secret personnalisé, ne bénéficient pas d'une suppression générale : évite donc de suivre une destination non fiable avec de tels headers.

Avant d'activer le suivi sur une URL non fiable, considère aussi :

— le passage HTTPS vers HTTP ;
— l'accès à une adresse interne ou à un service de métadonnées cloud ;
— la destination finale réellement autorisée par ton application.

La suite de tests locale vérifie qu'un header `Authorization` n'est pas transmis lors d'une redirection vers une autre origine.

`max_body_size`

La réponse est entièrement accumulée en mémoire avant d'être renvoyée.

```
local response, err = babet.http.get(url, {
    timeout = 30,
    max_body_size = 8 * 1024 * 1024,
})
```

— défaut : 64 MiB ;
— minimum : 1 octet ;
— maximum : 2 GiB ;
— type : entier Lua strictement positif.

Dès que le flux dépasserait la limite, Babet annule la réception et renvoie :

```
nil, "http: response body exceeds max_body_size"
```

Aucun corps partiel n'est exposé. Le header `Content-Length` n'est pas la seule protection : la limite s'applique aux chunks réellement reçus, y compris avec un transfert chunked ou une longueur absente/trompeuse.

Babet prend en charge les trois cadrages de corps HTTP/1.1 utilisés ici : `Content-Length`, `Transfer-Encoding: chunked` et corps terminé par la fermeture de connexion. Babet 2.9.1 corrige spécifiquement les deux derniers avec cpp-httplib 0.45.0 ; les limites portent toujours sur les octets réellement livrés au receiver de Babet.

Pour un gros payload GET, utilise `babet.http.download` au lieu d'augmenter cette limite en mémoire.

`max_file_size` et sécurité du fichier

`max_file_size` s'applique uniquement à `download`. La limite porte sur les octets du corps reçus, pas seulement sur le `Content-Length` annoncé. Si le flux dépasserait le plafond, Babet annule, supprime le temporaire, préserve la destination existante et renvoie :

```
nil, "http: response body exceeds max_file_size"
```

Le défaut de 8 Gio est un garde-fou, pas une recommandation d'accepter tout fichier de cette taille. Choisis la plus petite limite compatible avec l'artefact attendu. Un manque d'espace ou une erreur du système de fichiers renvoie aussi `(nil, err)` avec la même garantie de nettoyage.

Tables de réponse

Réponse en mémoire

```
{
    status = 200,
    body = "...",
    headers = {
        ["content-type"] = "application/json",
```

```
        ["set-cookie"] = "b=2",
    },
    headers_multi = {
        ["content-type"] = { "application/json" },
        ["set-cookie"] = { "a=1", "b=2" },
    },
}
```

`status`

Nombre entier du statut HTTP reçu.

`body`

Chaîne binaire complète, éventuellement vide. Pour HEAD, le corps est vide, même si `Content-Length` décrit la taille qu'aurait eue une réponse GET.

`headers`

Table avec noms en minuscules. Si un header apparaît plusieurs fois, la dernière occurrence rencontrée gagne. Cette vue est pratique pour les headers simples mais insuffisante pour `Set-Cookie`.

`headers_multi`

Chaque nom en minuscules pointe toujours vers une séquence de toutes les occurrences, même lorsqu'il n'y en a qu'une.

```
for _, cookie in ipairs(response.headers_multi["set-cookie"] or {}) do
    print(cookie)
end
```

L'ordre des occurrences suit celui fourni par cpp-httplib. L'ordre des noms dans `pairs()` n'est pas garanti.

Résultat d'un téléchargement

```
{
    status = 200,
    saved = true,
    bytes = 123456,
    path = "downloads/archive.tar.gz",
    headers = { ... },
    headers_multi = { ... },
}
```

- `status`, `headers` et `headers_multi` gardent le sens décrit ci-dessus ;
- `saved` vaut vrai uniquement si le statut final est 2xx et si le commit atomique a réussi ;
- `bytes` indique le nombre d'octets de corps reçus, y compris pour un non-2xx ;
- `path` existe uniquement lorsque `saved` vaut vrai ;
- aucun champ `body` n'est créé volontairement.

Exemples complets

GET JSON

```
local response, err = babet.http.get("https://api.example.com/v1/status", {
    headers = { ["Accept"] = "application/json" },
    query = { verbose = 1 },
```

```
    timeout = 10,
    max_body_size = 1024 * 1024,
})
assert(response, err)

if response.status ~= 200 then
    error("HTTP " .. response.status .. ": " .. response.body)
end

local data, decode_err = babet.json.decode(response.body)
assert(data, decode_err)
```

POST JSON authentifié

```
local body = assert(babet.json.encode({ title = "hello" }))
local response, err = babet.http.post(
    "https://api.example.com/v1/items",
    body,
    {
        headers = {
            ["Authorization"] = "Bearer " .. token,
            ["Content-Type"] = "application/json",
            ["Accept"] = "application/json",
        },
        timeout = 15,
        max_body_size = 2 * 1024 * 1024,
    }
)
assert(response, err)
```

PUT binaire

```
local file = assert(io.open("image.bin", "rb"))
local payload = file:read("a")
file:close()

local response = assert(babet.http.request({
    url = "https://upload.example.com/blob/42",
    method = "PUT",
    body = payload,
    headers = { ["Content-Type"] = "application/octet-stream" },
    timeout = 30,
    max_body_size = 1024 * 1024,
}))
```

Le fichier est lu entièrement en mémoire avant l'appel. Cette recette ne convient pas à un très gros fichier.

Télécharger un artefact de release

```
assert(babet.mkdir("downloads"))
local result, err = babet.http.download(
    "https://example.com/releases/tool.tar.gz",
    "downloads/tool.tar.gz",
    {
        timeout = 120,
        follow_redirects = true,
```

```
        max_file_size = 512 * 1024 * 1024,
    }
)
assert(result, err)
if not result.saved then
    error("statut HTTP inattendu " .. result.status)
end
print(result.bytes, "octets enregistrés dans", result.path)
```

Gérer 404 séparément des erreurs réseau

```
local response, err = babet.http.get(url, { timeout = 5 })
if not response then
    io.stderr:write("transport impossible: ", err, "\n")
elseif response.status == 404 then
    print("ressource absente")
elseif response.status >= 400 then
    io.stderr:write("erreur HTTP ", response.status, "\n")
else
    print(response.body)
end
```

Requêtes parallèles avec WORKERS

```
local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = assert(babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, {
            timeout = 10,
            max_body_size = 1024 * 1024,
        })
        if not r then error(err) end
        return { status = r.status, size = #r.body }
    ]], { url = url }))
end

for i, job in ipairs(jobs) do
    local ok_result, value = job:join()
    if ok_result then
        print(urls[i], value.status, value.size)
    else
        io.stderr:write(urls[i], ": ", value, "\n")
    end
end
```

Contrat d’erreur

Erreurs Lua levées

Mauvais type des arguments positionnels :

```
babet.http.request("not a table")
babet.http.get(42)
babet.http.get(url, "not a table")
babet.http.post(url, 42)
```

```lua
babet.http.download(url, 42)
babet.http.download(url, "fichier", "pas une table")
```

Erreurs `(nil, err)`

— `opts.url` absent ou non string ;
— URL sans scheme/host, scheme non supporté, CR/LF ou NUL ;
— méthode non supportée ;
— corps sur GET/HEAD/OPTIONS ;
— champ d'option au mauvais type ou hors plage ;
— query invalide ;
— nom ou valeur de header invalide ;
— DNS, connexion, TLS, envoi, lecture ou timeout ;
— corps de réponse dépassant `max_body_size` ou `max_file_size` ;
— chemin de destination ou erreur du système de fichiers ;
— échec d'allocation interne converti en `(nil, "http: out of memory")` ;
— autre exception interne convertie en `(nil, "http: internal failure")`.

Pas une erreur Babet

Un statut 1xx, 3xx, 4xx ou 5xx reçu produit une table normale. Pour `download`, seul un 2xx est enregistré ; les autres statuts renvoient `saved = false`. L'application définit sa propre politique.

Les opérations réseau internes, y compris le handshake et la fermeture TLS, sont protégées contre `SIGPIPE` sans remplacer le gestionnaire de signal de l'application. Une rupture de transport remonte comme une erreur HTTP ; une erreur de fermeture best-effort ne retire pas une réponse déjà reçue.

Sécurité et limites

— **SSRF.** Une URL contrôlée par un utilisateur peut viser loopback, le LAN, des sockets metadata cloud ou des services administratifs. Valide scheme, host, port et redirections selon ton modèle de menace.
— **Headers sensibles.** N'inclus pas un token dans les logs. Réfléchis à son comportement en cas de redirection inter-domaines.
— **TLS.** Garde `verify = true`. Une CA privée est préférable à `verify = false`.
— **Limites.** Réduis `max_body_size` ou `max_file_size` au minimum attendu.
— **Compression.** Selon les capacités compilées de cpp-httplib et les headers, le corps peut être traité par la bibliothèque ; la limite Babet porte sur les octets livrés au receiver.
— **Streaming ciblé.** `request`, `get` et `post` gardent la réponse en mémoire. `download` streame uniquement une réponse GET vers un fichier ; les uploads restent en mémoire.
— **Validation de l'artefact.** Le remplacement atomique évite un fichier final partiel, mais l'appelant doit encore valider statut, taille attendue et somme cryptographique.
— **Pas de session exposée.** Chaque appel construit un nouveau client ; aucun cookie jar ou pool de connexions n'est garanti à travers les appels.
— **Pas de formulaire/multipart automatique.** Encode explicitement ou utilise un outil spécialisé.
— **DNS synchrone.** Le resolver peut dépasser le timeout demandé.
— **API HTTP uniquement.** Pour un protocole custom, utilise SOCKET/TLS.

`babet.inotify` — surveillance du système de fichiers

`babet.inotify` expose une interface minimale au mécanisme Linux `inotify(7)`. Les notifications proviennent du noyau : il n'est pas nécessaire de parcourir périodiquement les dossiers surveillés.

Le module est **spécifique à Linux** et la surveillance n'est pas récursive.

Table des matières du module

— API
— Créer un watcher
— Ajouter une surveillance
  — Option `onlydir`
— Lire les événements
  — Débordement de la file
  — Interruption par signal
— Retirer et fermer
— Exemple
— Contrat d'erreur
— Limites

API

Fonction	Résultat
`babet.inotify.new()`	`watcher` | `(nil, err)`
`w:add(path, events [, opts])`	`wd` entier | `(nil, err)`
`w:read([timeout])`	tableau d'événements | `(nil, "timeout")` | `(nil, "interrupted")` | `(nil, err)`
`w:remove(wd)`	`(true, nil)` | `(nil, err)`
`w:close()`	`(true, nil)`

Les signatures sont strictes : les arguments supplémentaires lèvent une erreur Lua. `close()` est idempotente et le ramasse-miettes ferme aussi un watcher oublié.

Créer un watcher

```lua
local watcher, err = babet.inotify.new()
if not watcher then
    error(err)
end
```

Chaque appel crée une instance indépendante. Le descripteur inotify est ouvert avec `IN_NONBLOCK` et `IN_CLOEXEC` : `read()` pilote lui-même l'attente, et le descripteur n'est pas transmis aux programmes lancés par `exec`.

Ajouter une surveillance

```lua
local wd, err = watcher:add(path, events [, opts])
```

`path` doit être une chaîne sans octet NUL. Il peut désigner un dossier ou un fichier existant.

`events` est une liste stricte et non vide, indexée de `1` à `N`, sans trou ni clé supplémentaire. Chaque élément doit être l'un des noms suivants :

Nom	Événement demandé
`"access"`	lecture ou accès au fichier
`"modify"`	contenu modifié
`"attrib"`	attributs ou métadonnées modifiés
`"close_write"`	descripteur ouvert en écriture fermé
`"close_nowrite"`	descripteur non ouvert en écriture fermé
`"close"`	combinaison de `close_write` et `close_nowrite`
`"open"`	fichier ouvert

Nom	Événement demandé
`"moved_from"`	entrée déplacée hors du dossier surveillé
`"moved_to"`	entrée déplacée dans le dossier surveillé
`"move"`	combinaison de `moved_from` et `moved_to`
`"create"`	entrée créée dans le dossier surveillé
`"delete"`	entrée supprimée du dossier surveillé
`"delete_self"`	cible surveillée supprimée
`"move_self"`	cible surveillée déplacée

Option `onlydir`

```lua
local wd = assert(watcher:add(path, { "create" }, {
    onlydir = true,
}))
```

`onlydir` doit être un booléen lorsqu'elle est présente :

- `true` ajoute le masque Linux `IN_ONLYDIR` et refuse donc une cible qui n'est pas un dossier ;
- `false` conserve le comportement normal ;
- les autres champs de `opts` sont actuellement ignorés.

Ajouter de nouveau un watch sur la même cible suit la sémantique native d'inotify : le masque existant est remplacé, car `IN_MASK_ADD` n'est pas utilisé.

Lire les événements

```lua
local events, err = watcher:read([timeout])
```

Le timeout est exprimé en secondes et accepte les valeurs décimales :

- argument omis ou `nil` : attente illimitée ;
- `0` : lecture non bloquante des événements déjà disponibles ;
- valeur positive : attente bornée.

Un timeout négatif, NaN, infini ou trop grand renvoie `(nil, err)`. Un argument d'un mauvais type lève une erreur Lua.

Une lecture réussie renvoie une liste de tables :

```lua
{
    {
        wd = 1,
        name = "photo.jpg",
        events = { create = true, close_write = true },
        is_dir = false,
        cookie = 0,
    },
}
```

- `wd` est le watch descriptor renvoyé par `add` ;
- `name` est le nom de l'entrée concernée, ou `""` pour un événement portant sur la cible surveillée elle-même ;
- `events` contient un booléen vrai pour chaque bit reçu ;
- `is_dir` reflète `IN_ISDIR` ;
- `cookie` permet d'apparier `moved_from` et `moved_to` ; hors déplacement, il vaut généralement `0`.

En plus des événements demandables par `add`, le noyau peut produire :

- `events.ignored` lorsque le watch est retiré, automatiquement ou par `remove` ;
- `events.unmount` lorsque le système de fichiers surveillé est démonté.

Un appel peut renvoyer plusieurs événements, car le module lit et décode un lot complet de la file noyau.

Débordement de la file

Lorsque la file inotify déborde, certains événements sont perdus. Le module renvoie explicitement :

```
{
    wd = -1,
    name = "",
    events = { overflow = true },
    is_dir = false,
    cookie = 0,
}
```

Le programme doit alors rescanner les chemins surveillés pour reconstruire un état fiable.

Interruption par signal

Si un signal géré par `babet.signal` arrive pendant l'attente, son callback Lua est exécuté puis `read()` renvoie `(nil, "interrupted")`.

Retirer et fermer

```
assert(watcher:remove(wd))
assert(watcher:close())
```

Après `remove(wd)`, le noyau place normalement un événement `ignored` dans la file. `remove` sur un identifiant invalide renvoie `(nil, err)`.

Après `close`, `add`, `read` et `remove` renvoient une erreur indiquant que le watcher est fermé. `close` peut être appelée plusieurs fois.

Exemple

```
local watcher = assert(babet.inotify.new())
local wd = assert(watcher:add("/srv/incoming", {
    "close_write",
    "moved_to",
}, {
    onlydir = true,
}))

while true do
    local events, err = watcher:read()
    if not events then
        if err == "interrupted" then
            break
        end
        error(err)
    end

    for _, event in ipairs(events) do
        if event.events.overflow then
            rescan_directory()
        elseif event.wd == wd then
            process_entry(event.name)
        end
```

```
    end
end

watcher:close()
```

Contrat d'erreur

- — mauvais nombre ou mauvais type d'arguments : erreur Lua ;
- — valeur invalide, erreur système ou watcher fermé : `(nil, err)` ;
- — `read` sans événement avant l'échéance : `(nil, "timeout")` ;
- — signal géré pendant `read` : `(nil, "interrupted")` ;
- — `remove` et `close` réussis : `(true, nil)` ;
- — exception C++ inattendue : `(nil, "inotify: out of memory")`, `(nil, "inotify: internal failure")` ou `(nil, "inotify: unknown internal failure")`.

Les erreurs système sont préfixées par `"inotify: "` et conservent la description fournie par le système.

La fabrique et toutes les méthodes du userdata passent par cette frontière. `__gc` utilise un chemin `catch (...)` dédié et non levant : le nettoyage du ramasse-miettes ne tente jamais d'allouer ni de renvoyer une erreur pendant le traitement d'une exception.

Limites

- — Linux uniquement ;
- — pas de surveillance récursive ;
- — pas de `IN_DONT_FOLLOW`, `IN_ONESHOT` ni `IN_MASK_ADD` ;
- — une instance peut contenir plusieurs watches et plusieurs instances indépendantes peuvent coexister dans le même processus.

`babet.json` — encodage et décodage JSON

Le module `babet.json` s'appuie sur **nlohmann/json 3.11.3**. Il convertit les valeurs Lua en texte JSON et inversement, avec deux sentinelles pour représenter sans ambiguïté `null` et le tableau vide `[]`.

Table des matières du module

- — API
- — Correspondance des types
  - — Lua vers JSON
  - — JSON vers Lua
- — `null`
- — Objet vide, tableau vide et `as_array`
- — Options de `encode`
- — Chaînes, UTF-8 et octets NUL
- — Nombres
- — Ordre des objets et clés dupliquées
- — Contrat d'erreur
- — Limites

API

Élément	Contrat
`babet.json.encode(value [, opts])`	Renvoie `texte, nil` ou `nil, err`
`babet.json.decode(text)`	Renvoie `valeur, nil` ou `nil, err`
`babet.json.null`	Sentinelle représentant le `null` JSON
`babet.json.empty_array`	Sentinelle constante qui s'encode en `[]`
`babet.json.as_array(t)`	Marque `t` comme tableau JSON et renvoie la même table

`encode` accepte exactement un ou deux arguments. `decode` exige exactement **une chaîne Lua** : un nombre n'est pas converti implicitement en texte. `as_array` exige exactement une table.

Correspondance des types

Lua vers JSON

Lua	JSON
`nil` passé directement à `encode`	`null`
`babet.json.null`	`null`
`babet.json.empty_array`	`[]`
`babet.sqlite.NULL`	refusé ; utiliser `babet.json.null`
booléen	booléen
entier Lua	nombre entier JSON
flottant Lua fini	nombre JSON flottant
chaîne UTF-8 valide	chaîne JSON
table vide non marquée	objet vide `{}`
clés entières exactement `1..n`	tableau JSON
clés uniquement de type chaîne	objet JSON

Dans une table Lua, affecter `nil` supprime la clé avant même l'encodage. Utilise `babet.json.null` pour conserver explicitement une clé dont la valeur JSON doit être `null`.

`babet.sqlite.NULL` n'est volontairement pas un alias de null JSON. C'est une sentinelle lightuserdata réservée au bind SQLite : `encode` renvoie donc une erreur qui nomme explicitement `babet.sqlite.NULL`. Utilise `babet.json.null` à la frontière JSON.

Une table mixant clés chaînes et entières, un tableau à trous, ou une table contenant une clé non représentable (`0`, entier négatif, flottant, booléen, table, etc.) ne peut pas être convertie : `encode` renvoie alors `nil, err`.

JSON vers Lua

JSON	Lua
`null`	`babet.json.null`
booléen	booléen
entier signé dans la plage Lua	entier Lua
entier non signé supérieur à `math.maxinteger`	flottant Lua
nombre JSON flottant	flottant Lua
chaîne	chaîne Lua
tableau	table séquentielle `1..n`
objet	table à clés chaînes

Un entier JSON trop grand pour `lua_Integer` est converti en flottant plutôt que de reboucler en valeur négative. Cette conversion peut perdre de la précision, comme toute conversion vers un nombre IEEE 754.

`null`

Lua ne peut pas conserver une valeur `nil` dans une table. La sentinelle partagée `babet.json.null` comble cette différence :

```
local J = babet.json

local value = assert(J.decode([[{"answer":null}]]))
assert(value.answer == J.null)

local text = assert(J.encode({ answer = J.null }))
-- text représente {"answer":null}
```

Teste la sentinelle par identité avec ==. C'est une table spéciale, mais elle ne doit pas être utilisée comme une table de données.

Objet vide, tableau vide et `as_array`

Une table Lua vide est ambiguë. Babet choisit l'objet JSON par défaut :

```
local J = babet.json

assert(J.encode({}) == "{}")
assert(J.encode(J.empty_array) == "[]")
assert(J.encode(J.as_array({})) == "[]")
```

`empty_array` est une sentinelle partagée destinée aux valeurs que l'on ne souhaite pas modifier. `as_array(t)` est préférable pour un tableau construit dynamiquement : il marque la table avec une métatable interne, la laisse mutable et renvoie exactement la même table.

```
local tags = {}
-- table.insert(tags, ...) peut ne jamais être appelé

local text = assert(J.encode({ tags = J.as_array(tags) }))
-- {"tags":[]}
```

Points importants :

- `as_array` est idempotent ;
- le marquage **remplace toute métatable préexistante** sur la table ;
- la forme n'est vérifiée qu'au moment de `encode` : une table marquée contenant des clés chaînes ou des trous produit `nil, err` ;
- `as_array(J.null)` et `as_array(J.empty_array)` lèvent une erreur Lua ;
- seul un tableau JSON **vide** décodé reçoit automatiquement ce marquage. Un tableau non vide décodé puis entièrement vidé devient une table vide ordinaire et se ré-encode donc en `{}` ; appelle `as_array(t)` pour conserver `[]` dans ce cas ;
- les écritures ordinaires sur les sentinelles sont refusées. Comme pour toute table Lua, `rawset` contourne les métaméthodes : ne modifie pas les sentinelles.

Options de `encode`

La table optionnelle ne contient qu'une option interprétée :

```
local compact = assert(babet.json.encode({ a = 1 }))
local pretty  = assert(babet.json.encode({ a = 1 }, { indent = 4 }))
```

— `indent = 0..256` active la mise en forme avec retours à la ligne ;
— `indent = 0` n'ajoute aucun espace d'indentation, mais conserve les retours à la ligne de la sortie formatée ;
— option absente, `nil` ou entier négatif : sortie compacte ;
— une valeur non entière ou supérieure à `256` lève une erreur Lua ;
— les autres champs de la table d'options sont ignorés. En particulier, l'ancienne option documentée par erreur `pretty` n'existe pas.

Chaînes, UTF-8 et octets NUL

Les chaînes JSON doivent être en UTF-8 valide. Les caractères Unicode sont conservés dans la sortie ; Babet ne force pas leur conversion en séquences `\uXXXX`.

Les chaînes Lua peuvent contenir des octets NUL. Ils sont binary-safe côté Lua et sont échappés conformément au JSON lors de l'encodage. Cela vaut aussi pour les clés d'objets :

```
local J = babet.json
local original = "a\0b"
local text = assert(J.encode({ [original] = original }))
local back = assert(J.decode(text))
assert(back[original] == original)
```

Une chaîne ou une clé contenant une séquence UTF-8 invalide fait échouer l'encodage avec `nil, err`. Un texte JSON contenant une chaîne UTF-8 invalide est également refusé par `decode`.

Nombres

Babet préserve le sous-type Lua lorsque le JSON le permet : `42` devient un entier Lua, tandis que `42.0` et `1e3` deviennent des flottants. À l'encodage, un flottant Lua comme `3.0` reste un nombre JSON flottant.

`NaN`, `+Inf` et `-Inf` ne font pas partie du standard JSON et sont refusés avec `nil, err`.

Ordre des objets et clés dupliquées

L'ordre des membres d'un objet JSON n'a pas de signification sémantique et ne doit pas être utilisé comme contrat. Le backend actuel les émet généralement dans l'ordre lexicographique des clés, mais un appelant doit comparer les données décodées, pas le texte brut, lorsque l'ordre n'est pas imposé par un format externe.

Lors du décodage d'un objet JSON contenant plusieurs fois la même clé, la dernière valeur remplace les précédentes.

Contrat d'erreur

Les erreurs de conversion ou de parsing sont renvoyées et ne lèvent pas d'exception Lua :

```
local value, err = babet.json.decode("{ invalide")
if not value then
    print(err) -- préfixé par "json:"
end
```

Cela comprend notamment :

— JSON invalide, données supplémentaires après le document ou UTF-8 invalide ;
— table mixte, à trous ou avec des clés non représentables ;
— fonction, userdata (dont `babet.sqlite.NULL`) ou thread impossible à encoder ;
— `NaN` ou infinité ;
— cycle ou profondeur d'imbrication excessive.

Les erreurs d'utilisation de l'API lèvent en revanche une erreur Lua :

— mauvaise arité ;
— argument de `decode` qui n'est pas une chaîne ;
— second argument de `encode` qui n'est ni une table ni `nil` ;
— `opts.indent` non entier ou supérieur à `256` ;
— argument invalide pour `as_array`.

Les erreurs de parsing fournies par nlohmann/json indiquent la position de l'échec, généralement sous forme de ligne et de colonne.

Limites

La profondeur maximale de conversion est de **1000 niveaux** dans les deux sens. Cette limite sert également de garde-fou contre les tables cycliques. Le module construit le document complet en mémoire : il ne propose pas d'API de streaming.

`logging` — journalisation à niveaux

`logging` est un module Lua pur embarqué dans Babet. Il se charge avec `require("logging")` et ne fait pas partie de la table `babet`.

```lua
local log = require("logging")
```

Il expose cinq niveaux :

Niveau	Valeur
`log.TRACE`	10
`log.DEBUG`	20
`log.INFO`	30
`log.WARN`	40
`log.ERROR`	50

Il n'existe pas de niveau `fatal`.

Table des matières du module

— Émettre un message
  — Garantie de non-propagation
— Format de sortie
— Seuil
— Destination
— Couleurs ANSI
— État du module
— Contrat d'erreur
— Limites

Émettre un message

```lua
log.trace(...)
log.debug(...)
log.info(...)
log.warn(...)
log.error(...)
```

Chaque fonction accepte zéro ou plusieurs valeurs et ne renvoie aucune valeur. Un message est émis lorsque le niveau de la fonction est supérieur ou égal au seuil courant.

Le seuil initial est `log.INFO` : `trace` et `debug` sont donc filtrés par défaut.

Les arguments sont convertis avec `tostring` puis joints par un espace :

```
log.info("user=", 42, "active=", true)
-- ... [INFO ] user= 42 active= true
```

Un appel sans argument produit une ligne avec un message vide. Les octets NUL et les retours à la ligne contenus dans le message ne sont ni supprimés ni échappés. Un message multiligne produit donc plusieurs lignes physiques, mais seul le début de l'appel porte le préfixe du logger.

Garantie de non-propagation

Les cinq fonctions d'émission ne lèvent jamais d'erreur. Toute la chaîne est protégée :

- conversion des arguments par `tostring` ;
- création de l'horodatage par `os.date` ;
- appel de la méthode `write` du sink.

Si une de ces opérations échoue, le message est perdu silencieusement et l'exécution du script continue. Les arguments d'un message filtré ne sont pas convertis par `tostring`.

Format de sortie

Sans couleur, une ligne suit ce format :

```
YYYY-MM-DD HH:MM:SS [LEVEL] message\n
```

Exemple :

```
2026-07-13 21:45:02 [WARN ] tentative 3 sur 5
```

L'horodatage utilise **l'heure locale** de la machine. Il n'inclut ni fuseau, ni millisecondes. Les labels occupent cinq caractères : `[TRACE]`, `[DEBUG]`, `[INFO ]`, `[WARN ]`, `[ERROR]`.

Seuil

```
log.set_level(level)
local level = log.get_level()
```

`set_level` exige exactement un argument et ne renvoie aucune valeur. `get_level` n'accepte aucun argument et renvoie le seuil courant.

Le niveau peut être :

- un nom parmi `"trace"`, `"debug"`, `"info"`, `"warn"`, `"error"`, sans distinction de casse ;
- un nombre fini, ce qui permet un seuil intermédiaire comme `25.5`.

```
log.set_level("DEBUG")
log.set_level(log.WARN)
log.set_level(25.5)
```

Les chaînes numériques ne sont pas converties et les espaces ne sont pas retirés : `"30"` et `" info "` sont refusés. `NaN`, `+inf` et `-inf` sont également refusés.

Destination

```
log.set_output(out)
local out = log.get_output()
```

La destination initiale est `io.stderr`.

`set_output` exige exactement une table ou un userdata exposant une méthode `write` appelable. Une méthode fournie par `__index` est acceptée. La méthode est ensuite appelée avec la syntaxe `out:write(line)`.

```
local file, err = io.open("app.log", "a")
if not file then
    log.error("ouverture impossible :", err)
else
    log.set_output(file)
end
```

Le module :

- n'écrit rien pendant `set_output` ;
- ne ferme jamais la destination ;
- ne la vide jamais avec `flush` ;
- ignore la valeur renvoyée par `write` ;
- avale les erreurs d'écriture pendant l'émission.

Un fichier fermé possède encore une méthode `write` et peut donc être installé ; les écritures suivantes échoueront silencieusement. La gestion de la durée de vie du sink appartient au script.

`get_output` n'accepte aucun argument et renvoie exactement l'objet installé.

Couleurs ANSI

```
log.set_color(true)
local enabled = log.get_color()
```

Les couleurs sont désactivées par défaut. `set_color` exige exactement un booléen et ne renvoie aucune valeur. `get_color` n'accepte aucun argument et renvoie un booléen.

Quand les couleurs sont actives :

- `trace` : ANSI dim ;
- `debug` : cyan ;
- `info` : aucune couleur ;
- `warn` : jaune ;
- `error` : rouge.

Le reset ANSI est écrit avant le `\n`. Le module ne détecte pas si la sortie est un terminal : l'activation est toujours explicite.

État du module

`require("logging")` renvoie la même table tant que `package.loaded.logging` n'est pas supprimé. Le seuil, le sink et le réglage de couleur sont donc partagés par tous les utilisateurs du module dans **un même état Lua**.

Chaque worker Babet possède son propre état Lua : ses réglages de logging sont indépendants de ceux du thread principal et des autres workers.

Les valeurs initiales sont :

```
log.set_level(log.INFO)
log.set_output(io.stderr)
log.set_color(false)
```

Le module expose également :

```
log._VERSION      -- "babet logging 1.1.0"
log._DESCRIPTION  -- description textuelle
```

Contrat d'erreur

Les setters et getters lèvent une erreur Lua en cas de mauvaise arité ou de valeur invalide. Ils ne renvoient pas `(nil, err)`.

Les fonctions `trace`, `debug`, `info`, `warn` et `error`, elles, ne lèvent jamais et ne renvoient aucune valeur.

Limites

Le module ne fournit pas :

— de logger nommé ou hiérarchique ;
— de seuil par module ;
— de destinations multiples ;
— de format personnalisé ou JSON ;
— de fuseau UTC ou de précision sub-seconde ;
— de rotation de fichiers ;
— de buffering ou d'écriture asynchrone ;
— de garde contre un sink récursif qui rappelle lui-même le logger ;
— de détection automatique de terminal.

Pour la rotation, utilisez un outil externe comme `logrotate`. Pour des besoins plus avancés, enveloppez le module ou remplacez ses fonctions dans le script.

SIGNAL — arrêt propre, rechargement et signaux POSIX

`babet.signal` permet au script principal d'installer un callback Lua pour un petit ensemble de signaux POSIX, d'ignorer un signal ou de restaurer son comportement système par défaut.

Le module couvre :

— l'arrêt propre sur `TERM` et `INT` ;
— le rechargement de configuration sur `HUP` ;
— deux signaux applicatifs, `USR1` et `USR2` ;
— le choix explicite d'ignorer ou de restaurer un signal ;
— l'interruption de certains appels bloquants Babet ;
— le dispatch différé et sûr des callbacks dans le thread Lua principal.

Il ne permet pas d'envoyer un signal, de consulter une file de signaux, de configurer un masque POSIX ou d'installer des handlers depuis un worker.

Table des matières du module

— Conventions essentielles
— Vue d'ensemble de l'API
— Signaux pris en charge
— `handle(name, fn)` — installer ou remplacer
— `handle(name, nil)` — retirer un callback
— `ignore(name)` — ignorer
— `default(name)` — restaurer le comportement système
— Exécution des callbacks
  — Thread principal et contexte sûr
  — Ordre de dispatch
  — Coalescence des occurrences
  — Erreurs du callback
— Interaction avec les appels bloquants
— Interaction avec `debug.sethook`
— Exemples complets

    - Arrêt propre sur `TERM` et `INT`
    - Recharger une configuration avec `HUP`
    - Interrompre une attente socket
    - Ignorer temporairement `PIPE`
    - Observer les erreurs du callback
  - Contrat d'erreur
  - Workers, concurrence et limites
  - Fonctionnalités absentes

Conventions essentielles

Les succès renvoient une seule valeur

Les trois fonctions renvoient uniquement le booléen `true` en cas de succès. Elles ne renvoient pas un second `nil`.

```
local ok = babet.signal.ignore("PIPE")
assert(ok == true)
```

Une panne rare de `sigaction(2)` renvoie en revanche deux valeurs :

```
local ok, err = babet.signal.ignore("PIPE")
if not ok then
    io.stderr:write(err, "\n")
end
```

Les noms sont stricts

Un nom de signal doit être une véritable chaîne Lua, sans octet NUL, dans la casse indiquée par la documentation.

```
assert(babet.signal.handle("TERM", function() end))
```

Les formes suivantes sont invalides :

```
babet.signal.handle("term", function() end) -- casse incorrecte
babet.signal.handle(15, function() end)     -- nombre, pas string
babet.signal.handle("TERM\0X", function() end)
```

Un callback ne reçoit aucun argument

Le callback est appelé sans paramètre. Installe une fonction différente pour chaque signal lorsque le code doit savoir lequel est arrivé.

```
babet.signal.handle("USR1", function()
    print("USR1")
end)
```

Vue d'ensemble de l'API

```
local ok, err = babet.signal.handle(name, callback_or_nil)
local ok, err = babet.signal.ignore(name)
local ok, err = babet.signal.default(name)
```

Fonction	Succès	Échec système rare	Mauvais appel
`handle(name, fn)`	`true`	`(nil, err)`	erreur Lua levée
`handle(name, nil)`	`true`	`(nil, err)`	erreur Lua levée
`ignore(name)`	`true`	`(nil, err)`	erreur Lua levée
`default(name)`	`true`	`(nil, err)`	erreur Lua levée

`handle(name)` sans deuxième argument est volontairement refusé. Il faut écrire explicitement `handle(name, nil)` pour retirer un callback.

Signaux pris en charge

Nom Babet	Signal POSIX	Usage courant	Comportement système usuel
`"TERM"`	`SIGTERM`	demande d'arrêt propre, notamment par systemd	termine le processus
`"INT"`	`SIGINT`	interruption clavier, généralement Ctrl-C	termine le processus
`"HUP"`	`SIGHUP`	rechargement d'une configuration	termine souvent le processus
`"USR1"`	`SIGUSR1`	événement applicatif libre	termine le processus
`"USR2"`	`SIGUSR2`	second événement applicatif libre	termine le processus
`"PIPE"`	`SIGPIPE`	écriture sur un pipe ou socket fermé	termine le processus

Les noms `KILL` et `STOP` ne peuvent pas être exposés : POSIX interdit de les intercepter ou de les ignorer.

Les signaux dangereux ou réservés à d'autres mécanismes internes — par exemple `SEGV`, `BUS`, `FPE`, `ILL`, `CHLD` et `ALRM` — ne font pas partie de la liste blanche.

`handle(name, fn)` — installer ou remplacer

```lua
local ok, err = babet.signal.handle(name, fn)
```

- `name` est l'un des six noms supportés ;
- `fn` doit être une fonction Lua ;
- le handler POSIX précédent est remplacé ;
- un callback Lua déjà enregistré pour le même signal est remplacé ;
- le callback est conservé dans la registry Lua tant qu'il n'est pas retiré.

```lua
local stopping = false

assert(babet.signal.handle("TERM", function()
    stopping = true
end))
```

Remplacer le callback :

```lua
assert(babet.signal.handle("USR1", function()
    print("première version")
end))

assert(babet.signal.handle("USR1", function()
    print("nouvelle version")
end))
```

Après le second appel, seule la nouvelle fonction peut être exécutée.

`handle(name, nil)` — retirer un callback

```lua
local ok, err = babet.signal.handle(name, nil)
```

Cette forme :

1. efface le callback Lua associé ;
2. restaure `SIG_DFL`, le comportement système par défaut.

```lua
assert(babet.signal.handle("HUP", reload_config))

-- Plus tard : retrait explicite.
assert(babet.signal.handle("HUP", nil))
```

`handle("TERM")` sans second argument n'est **pas** un raccourci : il lève une erreur, afin d'éviter une désinstallation accidentelle due à un argument oublié.

`ignore(name)` — ignorer

```lua
local ok, err = babet.signal.ignore(name)
```

`ignore` installe `SIG_IGN` et efface tout callback Lua précédemment associé. Le noyau ignore ensuite le signal.

```lua
assert(babet.signal.ignore("PIPE"))
```

Cette forme est utile lorsqu'une application préfère recevoir les erreurs d'écriture normales plutôt que d'être terminée par `SIGPIPE`.

L'ignorance reste active jusqu'à un nouvel appel à `handle`, `default` ou `handle(name, nil)` pour ce même signal.

`default(name)` — restaurer le comportement système

```lua
local ok, err = babet.signal.default(name)
```

`default` installe `SIG_DFL` et efface tout callback Lua. Pour `TERM`, `INT`, `HUP`, `USR1`, `USR2` et généralement `PIPE`, le prochain signal correspondant terminera le processus selon les règles du système.

```lua
assert(babet.signal.ignore("PIPE"))
-- ... section où SIGPIPE doit être ignoré ...
assert(babet.signal.default("PIPE"))
```

`handle(name, nil)` et `default(name)` ont le même effet système. `default` est plus explicite lorsqu'aucun callback n'est en jeu.

Exécution des callbacks

Le vrai handler POSIX n'appelle jamais Lua. Il se contente de poser un flag de type `sig_atomic_t`, opération compatible avec le contexte asynchrone d'un signal.

Le callback Lua est exécuté plus tard, depuis un point sûr :

1. un signal supporté arrive ;
2. le handler C marque ce signal comme en attente ;
3. le thread principal remarque le flag depuis le hook d'instructions ou la sortie d'un appel bloquant interrompu ;
4. Babet remet le flag à zéro ;
5. Babet appelle le callback Lua avec zéro argument.

Thread principal et contexte sûr

Le callback tourne dans la coroutine Lua active du thread système principal, jamais dans le handler POSIX ou un worker. Les callbacks partagent la registry de cet état Lua et peuvent utiliser normalement Lua et les modules Babet.

```
babet.signal.handle("USR1", function()
    -- Ceci est exécuté dans un contexte Lua normal.
    local now = babet.time.now()
    print("USR1 à", now)
end)
```

Cette liberté ne signifie pas qu'un callback doive être long. Une fonction courte qui positionne un booléen ou enfile une action reste plus simple à raisonner.

Ordre de dispatch

Lorsque plusieurs signaux différents sont en attente au même point de dispatch, Babet les traite dans l'ordre fixe de sa liste blanche :

1. `TERM`;
2. `INT`;
3. `HUP`;
4. `USR1`;
5. `USR2`;
6. `PIPE`.

Ce n'est ni l'ordre d'enregistrement des callbacks ni une garantie de l'ordre exact d'arrivée au niveau du noyau.

Par exemple, si `USR1` puis `TERM` arrivent avant le prochain dispatch, le callback `TERM` est exécuté avant le callback `USR1`.

Coalescence des occurrences

Babet maintient un flag par type de signal, pas un compteur ni une file. Plusieurs occurrences identiques reçues avant le prochain dispatch sont donc coalescées en un seul appel du callback.

```
USR1, USR1, USR1 avant le dispatch -> un appel du callback USR1
```

Un nouveau signal identique reçu **pendant** l'exécution du callback peut reposer le flag et être traité lors d'un dispatch ultérieur.

N'utilise donc pas ce module comme compteur fiable d'événements. Pour ne perdre aucune occurrence métier, utilise une queue, un pipe, une socket ou un autre mécanisme de messages.

Erreurs du callback

Babet appelle le callback avec `lua_pcall`. Une erreur non attrapée est retirée de la pile et silencieusement ignorée ; elle ne remonte pas au code qui était en cours d'exécution.

```
babet.signal.handle("USR1", function()
    error("invisible pour l'appelant")
end)
```

Pour journaliser une erreur, protège explicitement le corps :

```
babet.signal.handle("USR1", function()
    local ok, err = pcall(function()
        refresh_metrics()
    end)
```

```lua
    if not ok then
        io.stderr:write("USR1: ", tostring(err), "\n")
    end
end)
```

Interaction avec les appels bloquants

Babet installe les handlers sans `SA_RESTART`. Certains appels bloquants internes peuvent donc être interrompus par un signal géré. Lorsque le thread principal constate qu'un signal Babet est en attente, il dispatche les callbacks puis renvoie généralement une erreur typée `"interrupted"`.

Les familles actuellement intégrées à ce mécanisme comprennent notamment :

- `babet.sleep(...)` ;
- `watcher:read(...)` du module Inotify ;
- les attentes TCP : connexion, acceptation et réception ;
- les handshakes et attentes TLS.

Exemple :

```lua
local data, err = peer:recv(4096, 30)
if not data then
    if err == "interrupted" then
        -- Un callback a déjà été dispatché : re-vérifier l'état du programme.
    elseif err == "timeout" then
        -- Aucun octet reçu avant l'échéance.
    else
        -- Fermeture ou erreur réseau.
    end
end
```

Toutes les fonctions Babet ne sont pas interruptibles par ce mécanisme. `babet.exec`, par exemple, applique son propre timeout et n'expose pas une sortie `"interrupted"` sur signal géré.

Un syscall peut aussi recevoir un `EINTR` sans qu'un signal géré par Babet soit en attente. Dans ce cas, les boucles internes reprennent normalement leur attente au lieu de produire un faux `"interrupted"`.

Interaction avec `debug.sethook`

En dehors des fonctions bloquantes intégrées, Babet utilise un hook Lua de type count, déclenché environ toutes les 10 000 instructions, pour traiter les flags et les événements du terminal. Il est installé à la demande par un `signal.handle(name, fn)` réussi ou par `curses.start()`. Au démarrage, `debug.gethook()` reste `nil`, y compris dans un contexte d'embedding recréé. Les scripts qui n'utilisent aucune de ces fonctions ne paient donc pas le coût du hook. Un hook count actif ralentit aussi le chemin d'interprétation entre ses callbacks ; augmenter seulement son intervalle ne supprime pas ce coût.

Les nouvelles coroutines héritent du hook courant de leur créatrice. Active les signaux ou curses avant de créer les coroutines qui doivent traiter les événements pendant une boucle Lua pure. Les coroutines déjà créées ne reçoivent pas rétroactivement le hook : un appel `signal.handle(name, fn)` depuis la coroutine équipe celle-ci et le thread Lua principal. Les appels bloquants intégrés conservent leurs points de traitement des signaux. Les workers n'installent pas ce hook réservé au thread système principal.

Lua ne permet qu'un hook actif par thread Lua. Par conséquent :

- `signal.handle(name, fn)` vérifie et restaure si nécessaire le hook Babet sur le thread Lua principal et la coroutine appelante, en remplaçant un éventuel hook personnalisé ;

— un `debug.sethook(...)` ultérieur remplace le hook de sa coroutine cible : le dispatch par instructions s'y arrête jusqu'à sa restauration, mais les appels bloquants intégrés peuvent encore dispatcher les signaux ;
— cette restauration ne modifie pas les autres coroutines déjà créées ; une coroutine dont le hook a été remplacé volontairement doit appeler elle-même `signal.handle(name, fn)` pour retrouver le dispatch automatique ;
— retirer les callbacks avec `handle(name, nil)`, `ignore` ou `default` laisse un hook déjà installé actif et ne restaure pas un ancien hook utilisateur ; ces appels n'installent pas de hook s'il n'y en avait pas. `curses.stop()` ne désinstalle pas non plus le hook partagé, dont le coût peut donc persister.

Évite de combiner `babet.signal` et un hook de debug personnalisé sauf si cette interaction est voulue. Le hook du runtime sert aussi les événements curses, même si aucun callback de signal n'a été enregistré.

Exemples complets

Arrêt propre sur `TERM` et `INT`

```
local running = true

local function request_stop()
    running = false
end

assert(babet.signal.handle("TERM", request_stop))
assert(babet.signal.handle("INT", request_stop))

while running do
    do_one_iteration()

    local slept, err = babet.sleep(250, "ms")
    if not slept and err ~= "interrupted" then
        io.stderr:write(tostring(err), "\n")
        break
    end
    -- En cas de "interrupted", le callback a déjà été exécuté et
    -- la boucle re-vérifie running.
end

close_resources()
```

Pour une boucle réseau, utilise directement le retour `"interrupted"` de la socket plutôt qu'un `pcall`.

Recharger une configuration avec `HUP`

Il est souvent préférable que le callback ne fasse que poser un drapeau. Le chargement complet s'effectue ensuite dans la boucle normale.

```
local running = true
local reload_requested = false

assert(babet.signal.handle("TERM", function()
    running = false
end))

assert(babet.signal.handle("HUP", function()
    reload_requested = true
```

```
end))

while running do
    if reload_requested then
        reload_requested = false
        local new_config, err = load_config()
        if new_config then
            config = new_config
        else
            io.stderr:write("reload: ", err, "\n")
        end
    end

    do_one_iteration(config)
end
```

Interrompre une attente socket

```
local running = true

assert(babet.signal.handle("TERM", function()
    running = false
end))

while running do
    local client, err = server:accept(60)
    if client then
        serve(client)
    elseif err == "interrupted" then
        -- Le callback TERM a pu mettre running à false.
    elseif err ~= "timeout" then
        io.stderr:write("accept: ", err, "\n")
    end
end

server:close()
```

Ignorer temporairement `PIPE`

```
assert(babet.signal.ignore("PIPE"))

local sent, err = socket:send(payload)
if not sent then
    io.stderr:write("send: ", err, "\n")
end

assert(babet.signal.default("PIPE"))
```

Attention : restaurer `SIGPIPE` à son comportement par défaut signifie qu'une écriture ultérieure sur un pipe fermé peut terminer le processus.

Observer les erreurs du callback

```
local function safe_handler(label, fn)
    return function()
        local ok, err = xpcall(fn, debug.traceback)
        if not ok then
```

```
            io.stderr:write(label, ": ", tostring(err), "\n")
        end
    end
end

assert(babet.signal.handle("USR2", safe_handler("USR2", function()
    rotate_logs()
end)))
```

Contrat d'erreur

Erreurs Lua levées

Les situations suivantes lèvent immédiatement une erreur Lua :

- appel depuis un worker ;
- `name` absent, non string ou contenant un NUL ;
- nom non supporté ou casse incorrecte ;
- `handle` sans deuxième argument ;
- callback différent d'une fonction ou de `nil`.

```
local ok, err = pcall(function()
    babet.signal.handle("TERM")
end)
assert(not ok)
```

Le nom non supporté lève également une erreur ; il ne produit pas `(nil, err)`.

```
local ok, err = pcall(function()
    babet.signal.ignore("QUIT")
end)
```

Erreur système renvoyée

Si `sigaction(2)` échoue, la fonction renvoie :

```
nil, "signal: sigaction failed: ..."
```

Ce cas est rare avec les six signaux autorisés, mais l'appelant peut conserver la convention `(ok, err)` lorsqu'il souhaite le traiter.

Workers, concurrence et limites

Les dispositions de signal sont globales au processus, pas propres à un thread. Babet impose donc les règles suivantes :

- `handle`, `ignore` et `default` sont réservés au thread principal ;
- leur utilisation dans un worker lève une erreur explicite ;
- les workers bloquent les six signaux supportés afin qu'ils restent gérés par le thread principal ;
- les workers ne peuvent ni voir ni consommer les flags en attente du parent.

Un worker doit être contrôlé par ses queues de messages, pas par `babet.signal` :

```
local job = assert(babet.workers.spawn([[
    while true do
        local ok, message = worker.recv()
        if not ok then
            return "closed"
        end
        if message == "stop" then
            return "stopped"
```

```
        end
    end
]]))

-- Le handler principal transforme le signal en message applicatif.
babet.signal.handle("TERM", function()
    job:send("stop", 0)
end)
```

Ce callback reste volontairement bref. Une queue pleine peut faire échouer l'envoi non bloquant ; une application robuste conserve également un drapeau de secours.

Fonctionnalités absentes

Le module ne fournit pas actuellement :

- `kill(pid, name)` pour envoyer un signal ;
- `list()` pour obtenir les noms supportés ;
- `is_pending()` ou un compteur d'occurrences ;
- `sigprocmask` ou un masque par thread ;
- `signalfd` ;
- les signaux autres que les six noms listés ;
- un hook de debug chaînable avec celui de l'utilisateur.

Pour envoyer un signal depuis Lua, un programme externe reste possible :

```
local result, err = babet.exec("kill", {
    "-TERM",
    tostring(pid),
})
assert(result, err)
assert(result.code == 0, result.stderr)
```

La réception reste soumise aux règles POSIX normales et au modèle de dispatch décrit dans cette page.

SOCKET — TCP et sockets Unix, flux binaires, lignes et timeouts

`babet.socket` fournit des sockets de flux synchrones, en TCP ou dans le domaine Unix (`AF_UNIX`), pour écrire des clients et serveurs de protocoles personnalisés : IRC, SMTP en clair avant STARTTLS, protocoles ligne par ligne, flux binaires ou petits services locaux.

Le module couvre :

- la connexion TCP à un hostname ou une adresse IP ;
- les connexions locales par chemin de socket Unix ;
- l'écoute sur une interface réseau ou sur un pathname Unix ;
- l'acceptation de clients ;
- l'envoi intégral d'une chaîne binaire ;
- la lecture par blocs, par lignes ou jusqu'à la fermeture distante ;
- des timeouts globaux par socket ou par appel ;
- l'obtention des adresses locale et distante ;
- la fermeture explicite et le nettoyage automatique par le GC ;
- l'interaction avec les signaux gérés par Babet.

Il ne fournit pas UDP, l'espace de noms abstrait Linux, l'I/O asynchrone, le multiplexage de milliers de connexions, ni un protocole applicatif prêt à l'emploi. Pour HTTP, utilise `babet.http`. Pour TLS direct ou STARTTLS, consulte `TLS`.

Table des matières du module

— Conventions essentielles
— Vue d'ensemble de l'API
— Créer une connexion cliente
— Créer un serveur TCP
— Se connecter à un socket Unix
— Créer un serveur Unix
— Accepter un client
— Configurer le timeout par défaut
— Envoyer des données
— Lire au plus N octets
— Lire une ligne
— Lire jusqu'à EOF
— Mélanger les méthodes de lecture
— Adresses locale et distante
— Fermer et laisser le GC nettoyer
— Signaux et interruptions
— Exemples complets
— Contrat d'erreur
— Sécurité, performances et limites

Conventions essentielles

Flux synchrones et bloquants

Chaque appel s'exécute dans le thread courant. Sans timeout, `connect`, `connect_unix`, `accept`, `send` et les lectures peuvent attendre indéfiniment.

Pour une application qui gère plusieurs connexions indépendantes, utilise des `workers` ou conçois une boucle autour de plusieurs processus. Babet n'expose pas `select`, `poll` ou `epoll` directement à Lua.

Chaînes binaires

Les données envoyées et reçues sont des chaînes Lua binary-safe. Elles peuvent contenir des octets NUL :

```
local sent = assert(sock:send("AB\0CD"))
local data = assert(sock:recv(5, 2))
assert(#data == 5 and data:byte(3) == 0)
```

Les hostnames et pathnames Unix utilisent en revanche une chaîne C et ne peuvent pas contenir de NUL.

Timeouts

Les timeouts sont exprimés en **secondes** et peuvent être fractionnaires.

— `nil` ou argument omis : utilise le timeout par défaut du socket ;
— `0` : attente infinie ;
— nombre positif : budget global pour l'appel ;
— valeur positive inférieure à 1 ms : arrondie à 1 ms ;
— nombre négatif, NaN, infini ou trop grand : erreur.

Une deadline est créée une seule fois par appel. Elle n'est pas renouvelée à chaque octet ou à chaque tentative interne.

Valeurs de retour

Une création ou une lecture réussie renvoie la valeur utile et `nil` comme seconde valeur implicite :

```
local sock, err = babet.socket.connect("127.0.0.1", 9000, 5)
local data, err = sock:recv(4096, 2)
```

Les actions `set_timeout`, `starttls` et `close` renvoient exactement :

```
true, nil
```

Une erreur d'usage positionnelle lève une erreur Lua. Une erreur de valeur, de transport ou de timeout renvoie généralement `(nil, err)`.

Vue d'ensemble de l'API

Constructeurs

```
local sock, err = babet.socket.connect(host, port, timeout?)
local server, err = babet.socket.listen(host, port, backlog?)
local local_sock, err = babet.socket.connect_unix(path, timeout?)
local local_server, err = babet.socket.listen_unix(path, opts?)
```

Argument	Type	Défaut	Rôle
`host` de `connect`	string stricte non vide	obligatoire	hostname ou adresse IP
`host` de `listen`	string stricte	obligatoire	interface ; `""` signifie toutes les interfaces
`port`	entier Lua `0..65535`	obligatoire	port TCP ; `0` est utile avec `listen` pour laisser le noyau choisir
`timeout`	nombre fini `>= 0`	`0`	timeout de connexion en secondes
`backlog`	entier Lua `1..INT_MAX`	`16`	taille demandée de la file d'attente

Les chaînes numériques comme `"443"` et les floats comme `443.0` sont refusés pour `port` et `backlog`, même si leur valeur mathématique est entière.

Pour `listen_unix`, `opts` accepte strictement :

Option	Défaut	Contrat
`backlog`	`16`	entier `1..INT_MAX`
`permissions`	`0600`	mode final Unix exact `0000..0777`, appliqué après `bind()`
`unlink_on_close`	`true`	supprime le pathname créé lors de `close()`/GC

Méthodes communes

```
local n, err = sock:send(data)
local data, err = sock:recv(count, timeout?)
local line, err, partial = sock:recv_line(timeout?)
local data, err = sock:recv_all(timeout?, max_bytes?)
local ok, err = sock:set_timeout(seconds)
local addr, err = sock:peer()
```

```lua
local addr, err = sock:sockname()
local ok, err = sock:close()
```

Méthode d'un socket d'écoute

```lua
local client, err = server:accept(timeout?)
```

Un socket d'écoute refuse `send`, `recv`, `recv_line`, `recv_all`, `peer` et `starttls`. Un socket connecté refuse `accept`.

babet.socket.connect(host, port, timeout?)

Ouvre une connexion TCP cliente.

```lua
local sock, err = babet.socket.connect("example.net", 9000, 5)
if not sock then
    error(err)
end
```

Résolution et adresses multiples

Babet appelle `getaddrinfo()` puis essaie les adresses retournées dans l'ordre. Le timeout est partagé entre toutes les tentatives IPv4/IPv6 : ce n'est pas un nouveau budget par adresse.

La résolution DNS elle-même est synchrone et se déroule avant cette deadline. Un résolveur lent peut donc faire durer l'appel plus longtemps que `timeout`. Pour éviter le DNS, utilise une adresse numérique :

```lua
local sock = assert(babet.socket.connect("127.0.0.1", 9000, 2))
```

Timeout de connexion non conservé

Le troisième argument borne uniquement la connexion. Le socket créé possède un timeout d'I/O par défaut de `0`, donc infini.

```lua
local sock = assert(babet.socket.connect("127.0.0.1", 9000, 2))
assert(sock:set_timeout(5)) -- nécessaire pour les I/O suivantes
```

Erreurs typiques

— hostname vide ou contenant un NUL : `(nil, err)`;
— port hors plage : `(nil, err)`;
— mauvais type positionnel : erreur Lua;
— refus de connexion, DNS, route ou autre erreur système : `(nil, "socket: …")`;
— expiration : `(nil, "timeout")`;
— signal géré pendant l'attente : `(nil, "interrupted")`.

babet.socket.listen(host, port, backlog?)

Crée un socket TCP d'écoute avec `SO_REUSEADDR`.

Écouter uniquement en loopback

```lua
local server = assert(babet.socket.listen("127.0.0.1", 9000))
```

Cette forme n'accepte pas les connexions provenant des autres interfaces.

Écouter sur toutes les interfaces

```
local server = assert(babet.socket.listen("", 9000))
```

Une chaîne vide active `AI_PASSIVE`. Selon l'adresse sélectionnée par le système, le socket peut être IPv4 ou IPv6. Pour imposer une famille, fournis explicitement `"0.0.0.0"` ou `"::"`.

> Écouter sur toutes les interfaces expose le service au réseau. Ajoute une authentification et des règles de pare-feu adaptées.

Demander un port libre au noyau

```
local server = assert(babet.socket.listen("127.0.0.1", 0))
local local_addr = assert(server:sockname())
print(local_addr.host, local_addr.port)
```

C'est la méthode recommandée pour les tests : elle évite les collisions de ports.

Backlog

```
local server = assert(babet.socket.listen("127.0.0.1", 9000, 128))
```

Le backlog est une demande faite au noyau. Le système peut le plafonner. Ce n'est ni un nombre maximal de clients simultanés ni une limite applicative.

`babet.socket.connect_unix(path, timeout?)`

Ouvre un flux local vers un socket Unix nommé dans le système de fichiers :

```
local sock, err = babet.socket.connect_unix("/run/my-service.sock", 2)
assert(sock, err)
```

Le pathname est une chaîne stricte, non vide, sans NUL, et doit tenir dans `sockaddr_un.sun_path` (107 octets utiles sur Linux). Le timeout suit le même contrat monotone que `connect` ; `0` signifie attente infinie. Le timeout du constructeur n'est pas conservé pour les I/O suivantes.

Les sockets abstraits Linux ne sont pas exposés : l'adresse est toujours un pathname réel. `peer()` renvoie `{ path = ... }` ; le `sockname()` d'un client non lié renvoie normalement `{ path = "" }`.

`babet.socket.listen_unix(path, opts?)`

Crée un listener local `AF_UNIX`/`SOCK_STREAM` :

```
local server = assert(babet.socket.listen_unix("/run/my-service.sock", {
    backlog = 32,
    permissions = tonumber("660", 8),
    unlink_on_close = true,
}))
```

Par sécurité, Babet refuse **toute entrée déjà présente** au pathname : socket obsolète, fichier régulier, symlink, FIFO ou dossier. Il ne supprime jamais silencieusement une entrée préexistante. Un socket obsolète doit être retiré explicitement par l'application après vérification.

Après `bind()`, les permissions finales sont appliquées exactement avec une opération sans suivi de symlink. Le défaut final `0600` limite l'accès au propriétaire. Entre `bind()` et cette mise en permissions, le pathname possède brièvement le mode créé par le noyau, donc filtré par l'`umask` du processus. Babet ne modifie pas cet `umask`, car il est global au processus et le runtime est multi-thread. Pour un service sensible, place donc le socket dans un répertoire parent privé, par exemple de mode `0700`. Ce parent doit déjà exister ; Babet ne le crée pas.

Avec `unlink_on_close = true`, `close()` et le GC retirent le pathname seulement si l'entrée courante est encore le même inode socket que celui créé par ce listener. Un fichier qui a remplacé le pathname

n'est jamais supprimé. Avec `false`, le pathname reste présent après fermeture et doit être retiré explicitement.

```
local server = assert(babet.socket.listen_unix("/tmp/demo.sock"))
local addr = assert(server:sockname())
assert(addr.path == "/tmp/demo.sock")
```

server:accept(timeout?)

Attend et accepte une connexion cliente.

```
local client, err = server:accept(10)
if not client then
    if err == "timeout" then
        print("aucun client")
    else
        error(err)
    end
end
```

Le timeout positionnel remplace celui défini par `server:set_timeout()` pour cet appel uniquement.

Le socket accepté :

- est un socket connecté, pas un socket d'écoute ;
- possède un timeout d'I/O par défaut de `0` ;
- n'hérite donc pas du timeout du serveur ;
- est fermé automatiquement à l'exécution d'un processus enfant grâce à `CLOEXEC`.

```
local client = assert(server:accept(10))
assert(client:set_timeout(5))
```

sock:set_timeout(seconds)

Définit le timeout par défaut des opérations suivantes :

- `send` ;
- `recv` ;
- `recv_line` ;
- `recv_all` ;
- `accept` sur un serveur.

```
assert(sock:set_timeout(3.5))
```

`0` rétablit les attentes infinies :

```
assert(sock:set_timeout(0))
```

Ce timeout ne s'applique pas rétroactivement à `connect`, ni au handshake de `starttls`. `starttls` utilise uniquement `opts.timeout`.

Une valeur positionnelle sur `recv`, `recv_line`, `recv_all` ou `accept` prime sur le défaut :

```
assert(sock:set_timeout(10))
local data, err = sock:recv(4096, 0.2) -- 200 ms pour cet appel
local data2, err2 = sock:recv(4096)   -- à nouveau 10 s
local data3, err3 = sock:recv(4096, 0) -- infini pour cet appel
```

sock:send(data)

Envoie la chaîne complète.

```lua
local count, err = sock:send("PING\r\n")
assert(count, err)
assert(count == 6)
```

Contrairement à l'appel POSIX `send(2)`, Babet boucle sur les écritures partielles. En succès, `count == #data`.

La chaîne peut être vide ou binaire :

```lua
assert(sock:send(""))
assert(sock:send("AB\0CD") == 5)
```

`send` n'accepte pas de timeout positionnel. Il utilise le timeout défini par `set_timeout()`. Sur TLS, les appels `SSL_write` sont découpés en blocs sûrs lorsque nécessaire.

Une erreur après l'envoi de certains octets ne fournit pas le nombre partiel à Lua. Pour un protocole nécessitant une reprise transactionnelle, ajoute un mécanisme applicatif d'identifiant ou d'acquittement.

Sur un socket TLS, un échec après la première tentative `SSL_write` ferme la connexion, y compris si OpenSSL n'a encore annoncé aucun octet envoyé. Le premier appel garde son diagnostic (`timeout`, `interrupted` ou erreur TLS), puis les appels suivants signalent un socket fermé. Un timeout avant toute tentative d'écriture TLS laisse la connexion utilisable.

`sock:recv(count, timeout?)`

Lit **au plus** `count` octets et renvoie dès qu'au moins un octet est disponible.

```lua
local chunk, err = sock:recv(4096, 5)
```

Ce n'est pas un équivalent de « lire exactement N octets ». Une lecture peut renvoyer moins que `count` sans que la connexion soit fermée.

`count` doit être un entier Lua strict compris entre `1` et `16 MiB`.

Lire exactement N octets

```lua
local function recv_exact(sock, wanted, timeout)
    local chunks = {}
    local total = 0

    while total < wanted do
        local chunk, err = sock:recv(wanted - total, timeout)
        if not chunk then
            return nil, err
        end
        chunks[#chunks + 1] = chunk
        total = total + #chunk
    end

    return table.concat(chunks)
end

local header = assert(recv_exact(sock, 12, 5))
```

Attention : dans ce helper, `timeout` est appliqué séparément à chaque appel. Pour imposer un budget global applicatif, calcule une échéance avec `babet.monotonic()` et passe le temps restant.

EOF et timeout

— fermeture distante avant tout octet : `(nil, "closed");`

— expiration : `(nil, "timeout")` ;
— interruption par signal : `(nil, "interrupted")`.

`sock:recv_line(timeout?)`

Lit jusqu'au prochain LF (`\n`) et renvoie la ligne sans le LF final.

```
local line, err = sock:recv_line(10)
```

Si le LF est précédé d'un CR, le CR est aussi retiré :

```
hello\n   -> "hello"
hello\r\n -> "hello"
```

Un CR situé ailleurs dans la ligne est conservé.

Les lignes complètes déjà conservées par `recv_all` sont traitées avant toute attente réseau. Un appel extrait une seule ligne et garde tous les octets suivants pour `recv_line`, `recv` ou `recv_all`.

Limite de ligne

Une ligne est limitée à **8 MiB**, avant retrait du CR éventuel et sans compter le LF. Cette limite s'applique aussi aux lignes déjà tamponnées par `recv_all`. Une ligne plus longue renvoie une erreur `line too long`. Les octets de cette ligne surdimensionnée ne sont pas exposés comme résultat partiel ; ferme généralement la connexion, car le protocole est alors désynchronisé. Lorsqu'une ligne complète trop longue est déjà tamponnée, elle est retirée jusqu'au LF inclus et les données suivantes sont conservées.

EOF avant LF

Si le pair ferme la connexion avant le LF, la méthode renvoie trois valeurs :

```
local line, err, partial = sock:recv_line(5)
if not line and err == "closed" then
    print("fragment final :", partial)
end
```

`partial` est toujours une chaîne, éventuellement vide.

Timeout avant LF

Lors d'un timeout ou d'une interruption, les octets déjà lus sont conservés internement. Ils ne sont pas renvoyés comme troisième valeur, mais seront livrés en premier lors de la lecture suivante.

```
local line, err = sock:recv_line(0.2)
if not line and err == "timeout" then
    -- un prochain recv_line(), recv() ou recv_all() reprend sans perte
end
```

`sock:recv_all(timeout?, max_bytes?)`

Lit jusqu'à la fermeture distante et renvoie tout le flux concaténé.

```
local body, err = sock:recv_all(10)
```

Ici, EOF est le terminateur normal et constitue un succès. Cette fonction est adaptée aux protocoles où la fermeture délimite le message. Elle est inadaptée à une connexion persistante qui reste ouverte après la réponse.

Limite mémoire

```
local body, err = sock:recv_all(10, 4 * 1024 * 1024)
```

- défaut : **64 MiB** ;
- valeur minimale : `1` ;
- maximum configurable : **2 GiB** ;
- type : entier Lua strict.

Au dépassement, le résultat est `(nil, err)` sans chaîne partielle. Les octets déjà consommés restent cependant dans le buffer interne. Un nouvel appel avec une limite supérieure peut reprendre le flux :

```
local body, err = sock:recv_all(2, 1024)
if not body and err:find("max_bytes", 1, true) then
    body = assert(sock:recv_all(2, 4096))
end
```

Le même principe s'applique après un timeout ou une interruption : les octets accumulés ne sont pas perdus.

Mélanger `recv`, `recv_line` et `recv_all`

Les trois méthodes partagent un buffer d'octets en attente. Cela garantit que changer de méthode après un timeout ne réordonne pas le flux.

```
-- Le pair a envoyé "abc" sans LF.
local line, err = sock:recv_line(0.1)
assert(line == nil and err == "timeout")

-- Les trois octets déjà consommés par recv_line restent disponibles.
local prefix = assert(sock:recv(3, 1))
assert(prefix == "abc")
```

La règle reste toutefois simple : utilise de préférence une seule stratégie de framing par phase de protocole. Mélanger volontairement les lectures peut être utile pour lire un header ligne par ligne puis un corps de taille connue, mais le code applicatif doit savoir exactement où se trouve la frontière.

Avant `starttls`, tout plaintext tamponné doit être consommé. Babet refuse le handshake si ce buffer n'est pas vide, afin de ne pas confondre des octets en clair avec des records TLS.

`sock:peer()` et `sock:sockname()`

Pour TCP, les deux méthodes renvoient une table :

```
{
    host = "127.0.0.1",
    port = 9000,
}
```

- `peer()` : endpoint distant d'un socket connecté ;
- `sockname()` : endpoint local du socket.

```
local remote = assert(sock:peer())
local local_addr = assert(sock:sockname())
print(remote.host, remote.port)
print(local_addr.host, local_addr.port)
```

Les adresses sont numériques : Babet ne réalise pas de reverse DNS. `peer()` est refusé sur un socket d'écoute. `sockname()` fonctionne aussi sur un serveur et permet de retrouver le port choisi après `listen(..., 0)`.

Pour un socket Unix, la table contient `path` à la place de `host`/`port` :

```
local addr = assert(sock:peer())
print(addr.path)
```

Un client Unix non lié possède un chemin local vide.

`sock:close()` et nettoyage automatique

`close()` est idempotent :

```
assert(sock:close())
assert(sock:close())
```

Après fermeture, les autres méthodes renvoient une erreur indiquant que le socket est fermé.

Le userdata possède le descripteur et l'éventuelle session TLS. Si le script oublie `close()`, le garbage collector ferme finalement les ressources. N'utilise pas ce mécanisme pour le contrôle normal du cycle de vie : le moment du GC n'est pas déterministe.

Les sockets créés et acceptés sont marqués close-on-exec. Un programme lancé par `babet.exec` n'hérite pas silencieusement des connexions ou du port d'écoute.

Pour un listener Unix, la fermeture applique aussi la politique `unlink_on_close` décrite plus haut, avec vérification de l'inode avant toute suppression.

Signaux et interruptions

Lorsque l'attente noyau est interrompue par un signal enregistré avec `babet.signal`, les opérations concernées peuvent renvoyer :

```
nil, "interrupted"
```

Le callback Lua est dispatché avant le retour. Les octets déjà retirés du flux par `recv_line` ou `recv_all` restent tamponnés.

```
local stopping = false
babet.signal.handle("TERM", function()
    stopping = true
end)

while not stopping do
    local client, err = server:accept(1)
    if client then
        -- traiter le client
    elseif err ~= "timeout" and err ~= "interrupted" then
        error(err)
    end
end
```

Consulte la page `SIGNAL` pour l'ordre, la coalescence et les appels effectivement interruptibles.

Exemples complets

Petit serveur ligne par ligne

```
local S = babet.socket
local server = assert(S.listen("127.0.0.1", 9000, 64))
assert(server:set_timeout(1))

print("écoute sur 127.0.0.1:9000")
```

```
while true do
    local client, err = server:accept()
    if client then
        assert(client:set_timeout(30))

        local line, read_err = client:recv_line()
        if line then
            assert(client:send("echo: " .. line .. "\n"))
        elseif read_err ~= "closed" then
            io.stderr:write(read_err, "\n")
        end

        client:close()
    elseif err ~= "timeout" then
        error(err)
    end
end
```

Petit service local par socket Unix

```
local path = "/tmp/babet-echo.sock"
local server = assert(babet.socket.listen_unix(path, {
    permissions = tonumber("600", 8),
}))
assert(server:set_timeout(1))

while true do
    local client, err = server:accept()
    if client then
        local line = client:recv_line(30)
        if line then assert(client:send(line .. "\n")) end
        client:close()
    elseif err ~= "timeout" and err ~= "interrupted" then
        server:close()
        error(err)
    end
end
```

Le pathname est retiré automatiquement par `server:close()` ou par le GC si l'entrée est toujours celle créée par Babet.

Client avec longueur préfixée

```
local function recv_exact(sock, n, timeout)
    local out, size = {}, 0
    while size < n do
        local part, err = sock:recv(n - size, timeout)
        if not part then return nil, err end
        out[#out + 1] = part
        size = size + #part
    end
    return table.concat(out)
end

local sock = assert(babet.socket.connect("127.0.0.1", 9000, 3))
assert(sock:set_timeout(5))
```

```lua
assert(sock:send(string.pack(">I4", 5) .. "hello"))
local raw_len = assert(recv_exact(sock, 4, 5))
local len = string.unpack(">I4", raw_len)
local payload = assert(recv_exact(sock, len, 5))
print(payload)
sock:close()
```

Lire une réponse délimitée par EOF

```lua
local sock = assert(babet.socket.connect("127.0.0.1", 9001, 3))
assert(sock:send("request\n"))
local response = assert(sock:recv_all(10, 8 * 1024 * 1024))
print(#response)
```

Cette forme suppose que le serveur ferme sa moitié d'écriture pour signaler la fin.

Contrat d'erreur

Erreurs Lua levées

Les mauvais types positionnels lèvent une erreur Lua, par exemple :

```lua
babet.socket.connect("localhost", "9000") -- port string
sock:send(42)                             -- données non string
sock:recv(4096.0)                         -- float, pas entier Lua
sock:set_timeout("5")                     -- string, pas nombre
```

Utilise `pcall` uniquement lorsque l'appel reçoit des valeurs dont le type n'a pas déjà été validé.

Erreurs renvoyées

Les valeurs hors plage et les erreurs d'exécution renvoient `(nil, err)` :

- port négatif ou supérieur à 65535 ;
- backlog nul ou trop grand ;
- pathname Unix vide, trop long, contenant un NUL ou déjà présent ;
- options Unix inconnues ou permissions hors `0000..0777` ;
- timeout invalide ;
- count de `recv` hors limite ;
- `max_bytes` invalide ;
- DNS, connexion, lecture, écriture ou fermeture ;
- méthode incompatible avec le type de socket ;
- socket déjà fermé ;
- `"timeout"`, `"closed"` ou `"interrupted"` lorsque ces états sont spécifiquement identifiés.

Ne compare une erreur complète que pour les chaînes stables et typées comme `"timeout"`, `"closed"` et `"interrupted"`. Les messages système peuvent varier selon la libc et le noyau.

Une exception C++ interne ne traverse jamais Lua. Un échec d'allocation est converti en `(nil, "socket: out of memory")` ; les autres exceptions internes reçoivent un diagnostic littéral stable. Cette protection couvre toutes les fonctions et méthodes publiques de `babet.socket`.

Sécurité, performances et limites

- **Pas d'authentification.** Un socket TCP brut ne prouve pas l'identité du pair et ne chiffre rien.
- **Pas de limite implicite de protocole.** `recv_line` et `recv_all` possèdent des caps, mais ton protocole doit aussi valider tailles, commandes et fréquences.

— **Timeouts recommandés.** Configure un timeout sur toute donnée contrôlée par un pair distant pour éviter les connexions lentes infinies.
— **Écoute réseau.** Préfère loopback lorsque le service n'a pas besoin d'être public.
— **Résolution DNS hors deadline.** La partie `getaddrinfo()` reste synchrone.
— **Un appel par thread.** N'utilise pas le même userdata simultanément depuis plusieurs états Lua. Les userdata ne sont de toute façon pas transportables par WORKERS.
— **Ordre TCP, pas messages.** TCP est un flux ; les frontières de `send` ne sont pas conservées à la réception.
— **Pas de half-close exposé.** Babet n'expose pas `shutdown(SHUT_WR)` ; seule la fermeture complète est disponible.
— **Pas de keepalive configuré.** Les options TCP avancées ne sont pas exposées.
— **Unix pathname uniquement.** Pas de sockets abstraits Linux, de datagrammes Unix, de `SCM_RIGHTS` ni de credentials.
— **Répertoire parent et `umask`.** Le mode final demandé est appliqué après `bind()`. Pendant le bref intervalle précédent, le pathname hérite du mode filtré par l'`umask` du processus. Babet ne change pas cet état global multithread ; pour un service sensible, place le socket dans un répertoire privé, idéalement `0700`. Les permissions finales du socket ne remplacent pas la sécurité du répertoire.
— **STARTTLS réservé au TCP.** Un flux Unix refuse `starttls()` sans fermer la connexion.
— **Propriété des ressources pendant la construction.** `connect`, `listen`, `connect_unix`, `listen_unix`, `accept` et `connect_tls` créent leur userdata Lua avant d'acquérir un FD ou un objet OpenSSL. Une erreur mémoire de Lua ne peut donc pas abandonner une ressource encore sans propriétaire. `starttls` emploie une garde séparée et ferme le flux dès que le handshake a commencé si une exception survient.
— **Limite des tests OOM.** Cette sûreté est établie par revue structurelle, compilation stricte et analyse statique, pas par injection déterministe de `bad_alloc` ; les tests d'intégration couvrent les cycles de vie et les chemins d'échec observables.

WEBSOCKET — client RFC 6455 pour `ws://` et `wss://`

`babet.websocket` fournit un **client** WebSocket synchrone. Il implémente nativement la négociation d'ouverture et les frames RFC 6455. Il convient aux protocoles qui ont besoin d'un transport bidirectionnel par messages, notamment WebDriver BiDi.

Babet reste volontairement générique : il connaît WebSocket, TLS, les timeouts et la fermeture, mais pas Selenium, WebDriver BiDi, JSON-RPC, les événements du navigateur ou les identifiants de commandes applicatifs.

Vue d'ensemble de l'API

```
local ws, err = babet.websocket.connect(url, opts?)

local n, err = ws:send_text(data, timeout?)
local n, err = ws:send_binary(data, timeout?)
local message, err = ws:recv(timeout?)
local ok, err = ws:ping(data?, timeout?)
local ok, err = ws:set_timeout(seconds)
local ok, err = ws:close(code?, reason?, timeout?)
```

`recv()` renvoie un message applicatif complet :

```
{ type = "text",   data = "..." }
{ type = "binary", data = "..." }
{ type = "close",  code = 1000, reason = "..." }
```

Les Ping reçus sont automatiquement suivis d'un Pong et ne remontent pas comme messages applicatifs. Les Pong sont consommés en interne.

babet.websocket.connect(url, opts?)

Seuls les schémas `ws://` et `wss://` sont acceptés ; la casse du schéma URI est ignorée en ASCII.

```
local ws = assert(babet.websocket.connect(
    "ws://127.0.0.1:9222/session/abc",
    { timeout = 5 }
))
```

Connexion chiffrée :

```
local ws = assert(babet.websocket.connect(
    "wss://example.net/events",
    { timeout = 5 }
))
```

Le parseur accepte les noms DNS, IPv4, IPv6 entre crochets, ports explicites, chemins et query strings. L'userinfo et les fragments sont refusés. Les espaces et octets de contrôle du target HTTP doivent être percent-encodés par l'appelant.

Options

`opts` est une table brute et stricte : les métaméthodes ne sont pas appelées, les noms d'options doivent être des chaînes et les champs inconnus sont refusés.

Option	Défaut	Contrat
`timeout`	`0`	secondes finies `>= 0` ; une deadline pour TCP, TLS et HTTP Upgrade
`verify`	`true`	vérifie la chaîne du certificat et l'identité pour `wss://`
`ca_cert`	aucun	fichier CA PEM ajouté aux autorités par défaut pour cette connexion
`ca_path`	aucun	dossier CA OpenSSL ajouté aux autorités par défaut pour cette connexion
`hostname`	hôte de l'URL	override d'identité TLS/SNI
`min_version`	`"1.2"`	`"1.2"` ou `"1.3"`
`max_message_bytes`	16 Mio	plafond du message complet réassemblé
`max_frame_bytes`	16 Mio	plafond des frames de données applicatives ; doit rester <= au plafond message ; les frames de contrôle gardent leur plafond RFC indépendant de 125 octets

Les deux plafonds sont des entiers stricts dans `1..2147483648`.

Le timeout du constructeur devient également le timeout d'I/O par défaut du WebSocket. `set_timeout()` permet ensuite de le modifier.

Exemples des options

Borner toute la séquence TCP/TLS/Upgrade après la résolution DNS :

```
local ws = assert(babet.websocket.connect("ws://127.0.0.1:9222/events", {
    timeout = 2,
}))
```

Utiliser un fichier CA privé pour un service `wss://` local ou interne :

```
local ws = assert(babet.websocket.connect("wss://automation.internal/events", {
    ca_cert = "/etc/myapp/automation-ca.pem",
}))
```

Pour WSS, `ca_cert` et `ca_path` **ajoutent** des autorités à celles déjà chargées depuis les chemins par défaut d'OpenSSL, ses variables d'environnement et les emplacements de distributions reconnus. Ils ne limitent pas la confiance au fichier ou dossier fourni. Une chaîne vide n'ajoute rien ; les CA d'un appel ne sont pas conservées par les connexions suivantes.

Cette politique correspond à `TLS socket` et diffère de `HTTP`, dont le fichier `ca_cert` non vide remplace les autorités par défaut. Ce n'est pas du pinning de certificat ou de clé : plusieurs certificats d'une même CA peuvent être acceptés. Avec `verify = true`, l'identité du serveur reste vérifiée.

Utiliser un dossier de certificats CA compatible avec OpenSSL :

```
local ws = assert(babet.websocket.connect("wss://automation.internal/events", {
    ca_path = "/etc/myapp/certs",
}))
```

Surcharger l'identité TLS de référence et le SNI tout en se connectant à une autre adresse. C'est utile pour un routage local ou des tests contrôlés ; le certificat reste vérifié pour `automation.internal` :

```
local ws = assert(babet.websocket.connect("wss://127.0.0.1:9443/events", {
    hostname = "automation.internal",
    ca_cert = "/etc/myapp/automation-ca.pem",
}))
```

Exiger TLS 1.3 au lieu du minimum TLS 1.2 par défaut :

```
local ws = assert(babet.websocket.connect("wss://example.net/events", {
    min_version = "1.3",
}))
```

Désactiver la vérification du certificat uniquement dans un environnement de test volontairement contrôlé :

```
local ws = assert(babet.websocket.connect("wss://127.0.0.1:9443/events", {
    verify = false,
}))
```

`verify = false` désactive à la fois la vérification de la chaîne et celle de l'identité de référence. Ne pas l'utiliser sur un réseau non fiable ni en production.

Borner séparément le message complet et les frames de données applicatives :

```
local ws = assert(babet.websocket.connect("ws://127.0.0.1:9222/events", {
    max_message_bytes = 8 * 1024 * 1024,
    max_frame_bytes = 1 * 1024 * 1024,
}))
```

Un `max_frame_bytes` volontairement faible ne réduit pas le plafond RFC des frames de contrôle : Ping, Pong et Close peuvent toujours transporter jusqu'à 125 octets.

Configuration sécurisée combinée :

```
local ws = assert(babet.websocket.connect("wss://127.0.0.1:9443/bidi", {
    timeout = 5,
    ca_cert = "/etc/myapp/automation-ca.pem",
```

```
    hostname = "automation.internal",
    min_version = "1.3",
    max_message_bytes = 8 * 1024 * 1024,
    max_frame_bytes = 512 * 1024,
}))
```

Négociation d'ouverture

Le client envoie une requête HTTP/1.1 Upgrade avec la version WebSocket 13 et un `Sec-WebSocket-Key` aléatoire. La réponse n'est acceptée que si :

— le statut est `101 Switching Protocols` ;
— `Upgrade` vaut `websocket` sans tenir compte de la casse ;
— `Connection` contient le token `Upgrade` ;
— `Sec-WebSocket-Accept` correspond exactement au SHA-1 + Base64 du GUID RFC ;
— le serveur n'a négocié aucune extension non demandée ;
— le serveur n'a sélectionné aucun sous-protocole non demandé.

Le bloc d'en-têtes de réponse est plafonné à 64 Kio.

La 2.22.0 n'expose pas encore les en-têtes personnalisés, cookies, sous-protocoles, proxies HTTP ni extensions WebSocket comme `permessage-deflate`.

Envoyer du texte

```
local n, err = ws:send_text('{"id":1,"method":"session.status","params":{}}')
assert(n, err)
```

Le texte doit être un UTF-8 valide. La valeur renvoyée est le nombre d'octets applicatifs et non la taille réellement envoyée sur le réseau.

Les gros messages sont automatiquement fragmentés en frames de continuation. Chaque frame cliente est masquée avec une nouvelle clé aléatoire 32 bits issue du générateur cryptographique OpenSSL, y compris en `wss://`.

Une fois l'émission d'une frame tentée, un abandon (timeout, interruption ou erreur de transport) ferme la connexion. Cette règle couvre aussi les messages fragmentés, Ping et les réponses Pong automatiques pendant `recv`. Le premier appel garde son erreur ; les opérations suivantes renvoient `closed` et `close()` reste idempotent. La fermeture précède tout callback de signal.

Les erreurs de validation avant émission (UTF-8, taille du message, arguments) laissent la connexion utilisable. Un timeout de réception qui n'abandonne aucune écriture conserve l'état de réception et reste reprenable.

Envoyer du binaire

```
local n = assert(ws:send_binary("A\0B\255"))
assert(n == 4)
```

Les messages binaires sont des chaînes Lua arbitraires : NUL et UTF-8 invalide sont autorisés.

Recevoir des messages

```
local message, err = ws:recv(2)
assert(message, err)

if message.type == "text" then
    print(message.data)
elseif message.type == "binary" then
    -- chaîne binaire
```

```
elseif message.type == "close" then
    print(message.code, message.reason)
end
```

`recv()` réassemble les messages texte/binaires fragmentés et accepte des frames de contrôle entre deux fragments. Le texte n'est validé en UTF-8 qu'après réassemblage complet.

Les frames du serveur doivent être non masquées. Bits RSV réservés, opcodes réservés, longueurs non minimales, contrôle fragmenté et Close mal formé sont des erreurs de protocole.

La longueur annoncée des frames de données applicatives est comparée à `max_frame_bytes` **avant** l'allocation ou la lecture du payload. Les frames de contrôle gardent leur plafond RFC fixe de 125 octets : une limite de données volontairement basse ne peut donc pas empêcher Ping/Pong ou Close. Le message réassemblé est plafonné séparément par `max_message_bytes`.

Ping et Pong

```
assert(ws:ping("health"))
```

Le payload d'un Ping est limité à 125 octets. Un Ping reçu pendant `recv()` est immédiatement renvoyé comme Pong avant la poursuite de l'attente du prochain message applicatif.

Fermeture

```
assert(ws:close(1000, "done", 2))
```

`close()` valide le code et la raison UTF-8, envoie une frame Close masquée, puis attend le Close distant avec la même deadline absolue. La raison est limitée à 123 octets pour que code + raison tiennent dans les 125 octets d'une frame de contrôle.

Si le pair ferme en premier, `recv()` valide sa frame, lui répond si nécessaire, ferme le transport et renvoie un message `type = "close"`.

Le GC reste volontairement best-effort : un userdata oublié ferme TCP/TLS mais ne lance pas une négociation WebSocket potentiellement bloquante.

Erreurs de protocole

Babet échoue de manière conservative :

— erreur de framing/protocole -> Close `1002` si possible ;
— UTF-8 invalide -> Close `1007` ;
— plafond frame/message dépassé -> Close `1009`.

Les pannes de transport et expirations restent `(nil, err)`. Un signal POSIX géré interrompt l'attente, déclenche le callback différé puis renvoie `(nil, "interrupted")`.

Sur `wss://`, les écritures TLS internes au handshake, à l'envoi et à la lecture sont protégées contre `SIGPIPE`. Une erreur de transport ne tue pas le programme et ne remplace pas le gestionnaire de signal de l'application.

Exemple WebDriver BiDi

WebDriver BiDi transporte ses messages sur WebSocket. Une session WebDriver renvoie un `webSocketUrl` ; Babet a uniquement besoin de cette URL :

```
local ws = assert(babet.websocket.connect(webSocketUrl, {
    timeout = 10,
    max_message_bytes = 8 * 1024 * 1024,
}))
```

```lua
local command = assert(babet.json.encode({
    id = 1,
    method = "session.status",
    params = {},
}))
assert(ws:send_text(command))

local message = assert(ws:recv())
assert(message.type == "text")
local decoded = assert(babet.json.decode(message.data))
print(decoded.id)
```

Le binding Selenium doit conserver en Lua les identifiants de commandes, les réponses en attente, abonnements et callbacks d'événements. `babet.websocket` reste uniquement la couche transport.

Concurrence et boucles graphiques

L'API est synchrone. Un `recv()` sans timeout dans le thread principal d'une boucle graphique bloque cette boucle. Utilise un worker Babet, une stratégie de thread dédiée dans l'hôte ou des attentes courtes et bornées selon le cas.

Chaque worker possède son propre état Lua et peut ouvrir sa propre connexion WebSocket. Un userdata WebSocket n'est pas sérialisable entre workers.

Sécurité et limites

- `wss://` vérifie les certificats par défaut et impose TLS 1.2 minimum.
- chaque frame cliente utilise une nouvelle clé de masquage cryptographiquement forte ;
- aucune extension de compression n'est négociée, ce qui évite l'amplification de décompression et l'état supplémentaire associé ;
- les limites frame/message empêchent un pair de provoquer des allocations non bornées via une longueur annoncée ;
- une frame serveur non conforme est refusée plutôt que normalisée ;
- la résolution DNS reste synchrone et n'est pas elle-même bornée par la deadline socket ; utilise une adresse numérique si ce point est critique.

`babet.sqlite` — base de données SQL embarquée

`babet.sqlite` embarque SQLite 3.53.1 et expose une API Lua complète mais volontairement compacte : connexions en lecture/écriture ou lecture seule, clés étrangères par connexion, compteurs de changements, exécution SQL directe, itérateurs paresseux, statements préparés réutilisables, transactions assistées, savepoints imbriqués, sauvegardes cohérentes en ligne et bind BLOB et `NULL` explicites.

Table des matières du module

- API
  - Ouverture et fermeture
  - Compteurs de connexion
  - Sauvegarde cohérente
  - Exécution directe
  - Statements préparés réutilisables
  - Transactions assistées
  - Savepoints imbriqués
  - Paramètres SQL
  - Valeurs NULL explicites

  - — BLOB explicites
  - — Texte SQL et octets NUL
- — Lecture des lignes et mapping de types
- — Cycle de vie
- — Contrat d’erreur
- — Exemples complets
- — Fonctions non exposées

API

Ouverture et fermeture

Fonction	Renvoie
`babet.sqlite.open(path, opts?)`	`db` (userdata) | `(nil, err)`
`babet.sqlite.blob(data)`	wrapper BLOB opaque
`babet.sqlite.NULL`	sentinelle lightuserdata exacte pour binder SQL `NULL`
`db:close()`	`(true, nil)` — idempotent
`db:in_transaction()`	boolean | `(nil, err)`
`db:transaction(callback, mode?)`	`true, ...résultats_callback` | `(nil, err)`
`db:savepoint(callback)`	`true, ...résultats_callback` | `(nil, err)`
`db:last_insert_rowid()`	entier | `(nil, err)`
`db:changes()`	entier | `(nil, err)`
`db:total_changes()`	entier | `(nil, err)`
`db:backup(path, opts?)`	`(true, nil)` | `(nil, err)`

Par défaut, `open` ouvre ou crée une base en lecture/écriture. Avec `readonly = true`, il ouvre une base existante sans la créer et SQLite refuse les écritures. Le chemin spécial `":memory:"` crée une base uniquement en mémoire, perdue à la fermeture. Le chemin doit être une chaîne sans octet NUL.

`opts` est une table facultative :

Champ	Type	Défaut
`wal`	boolean — demande `PRAGMA journal_mode=WAL`	`false`
`busy_timeout`	entier de `0` à `3600000` ms	`0` — aucun délai d’attente
`readonly`	boolean — ouverture sans création ni écriture	`false`
`foreign_keys`	boolean — impose les contraintes de clés étrangères	`false`

`wal = true` demande le mode WAL, mais SQLite peut conserver un autre mode lorsque WAL n’est pas applicable. C’est notamment le cas de `":memory:"`.

`busy_timeout` demande à SQLite de réessayer pendant la durée indiquée lorsque la base est verrouillée. La valeur `0` laisse remonter immédiatement `SQLITE_BUSY`.

`readonly = true` utilise le mode natif `SQLITE_OPEN_READONLY`. Une base absente produit `(nil, err)` et aucun fichier n’est créé. Les lectures restent autorisées ; une instruction d’écriture produit `(nil, err)`. Cette option se combine avec `busy_timeout` et `foreign_keys`, mais pas avec `wal = true` : la combinaison contradictoire lève une erreur Lua avant l’ouverture. Ce refus concerne uniquement la demande de **passer** la base en mode WAL. Il n’empêche pas `readonly = true` d’ouvrir une base qui utilise déjà WAL, sans fournir `wal = true`. SQLite doit alors pouvoir utiliser les fichiers compagnons

`-wal` et `-shm` ; un système de fichiers entièrement en lecture seule peut échouer si le fichier `-shm` nécessaire n'existe pas déjà ou n'est pas utilisable.

`foreign_keys = true` active l'intégrité référentielle avant la première instruction de la connexion. Le réglage est propre à chaque connexion et ne valide pas rétroactivement les lignes déjà stockées. La valeur par défaut `false` conserve le comportement SQLite historique de Babet.

Chaque option peut être utilisée seule. Pour définir uniquement l'attente sur verrou :

```
local db = assert(babet.sqlite.open("state.db", { busy_timeout = 2500 }))
```

Pour demander WAL en gardant l'attente par défaut :

```
local db = assert(babet.sqlite.open("state.db", { wal = true }))
```

Pour ouvrir uniquement en lecture :

```
local db = assert(babet.sqlite.open("state.db", { readonly = true }))
```

Pour activer les clés étrangères sur une connexion d'écriture :

```
local db = assert(babet.sqlite.open("state.db", { foreign_keys = true }))
```

Pour combiner WAL, attente sur verrou et clés étrangères :

```
local db = assert(babet.sqlite.open("state.db", {
    wal = true,
    busy_timeout = 2500,
    foreign_keys = true,
}))
```

La table `opts` est stricte : un champ inconnu, une clé d'option non chaîne, un mauvais type ou un délai hors limites lève une erreur Lua. Les options sont lues directement dans la table ; une métaméthode `__index` ne peut ni les fournir ni les masquer. Les types et combinaisons incompatibles sont contrôlés avant l'acquisition du handle natif. Une erreur de contrat ne crée donc ni fichier ni connexion.

Pour un lecteur strict combinant les options compatibles :

```
local db = assert(babet.sqlite.open("state.db", {
    readonly = true,
    busy_timeout = 2500,
    foreign_keys = true,
}))
```

`db:in_transaction()` renvoie `true` dès que la connexion se trouve dans une transaction, qu'elle ait été ouverte par `db:transaction()`, par un `db:savepoint()` externe ou par un `BEGIN` SQL manuel.

Compteurs de connexion

Les trois méthodes suivantes lisent directement l'état de la connexion. Elles n'exécutent aucun SQL, renvoient toujours un entier Lua sur une connexion ouverte et sont disponibles dans les workers.

`db:last_insert_rowid()` Renvoie le ROWID du dernier `INSERT` réussi sur cette connexion, ou `0` tant qu'aucun ROWID n'a été inséré. La valeur reste celle du dernier `INSERT` concerné après un `SELECT`, un `UPDATE`, un `DELETE`, et même après le rollback d'un `INSERT` qui avait réussi avant l'annulation.

```
local db = assert(babet.sqlite.open(":memory:"))
assert(db:exec("CREATE TABLE messages(id INTEGER PRIMARY KEY, body TEXT)"))
assert(db:exec("INSERT INTO messages(body) VALUES(?)", { "Bonjour" }))
local id = assert(db:last_insert_rowid())
print(id) -- 1
```

Une instruction réussie ne garantit pas qu'une ligne a été insérée. Avec `INSERT OR IGNORE`, une contrainte peut être ignorée et `last_insert_rowid()` conserve alors le ROWID précédent. Pour une insertion d'une ligne qui peut être ignorée, vérifier `changes() == 1` avant le ROWID :

```lua
assert(db:exec("INSERT OR IGNORE INTO messages(body) VALUES(?)",
    { "message unique" }))
assert(db:changes() == 1, "message non inséré")
local id = assert(db:last_insert_rowid())
```

`db:changes()` Renvoie le nombre de lignes modifiées par le dernier `INSERT`, `UPDATE` ou `DELETE` terminé sur cette connexion. Les modifications auxiliaires de triggers, d'actions de clés étrangères ou de la résolution `REPLACE` ne sont pas incluses dans ce compteur du dernier statement.

```lua
assert(db:exec(
    "UPDATE messages SET body = ? WHERE id IN (?, ?)",
    { "archivé", 1, 2 }
))
print(assert(db:changes())) -- 0, 1 ou 2 selon les lignes présentes
```

`db:total_changes()` Renvoie le cumul des lignes modifiées depuis l'ouverture de la connexion. Ce cumul inclut les changements produits par les triggers et les actions de clés étrangères, mais pas les suppressions internes de `REPLACE`. Il repart de zéro pour chaque nouvelle connexion.

```lua
local before = assert(db:total_changes())
assert(db:exec("INSERT INTO messages(body) VALUES ('un'), ('deux')"))
local delta = assert(db:total_changes()) - before
assert(delta == 2)
```

Pour utiliser les trois compteurs ensemble après une insertion :

```lua
assert(db:exec("INSERT INTO messages(body) VALUES(?)", { "nouveau" }))
local id = assert(db:last_insert_rowid())
local statement_rows = assert(db:changes())
local connection_rows = assert(db:total_changes())
print(id, statement_rows, connection_rows)
```

Ces valeurs appartiennent uniquement au handle `db` courant : une autre connexion, y compris dans un autre worker, possède ses propres compteurs. Après `db:close()`, les trois méthodes renvoient `(nil, "sqlite: connection closed")`.

Sauvegarde cohérente

```lua
local ok, err = db:backup(path, opts?)
```

`db:backup()` crée une sauvegarde cohérente de la base principale de la connexion ouverte. L'implémentation utilise l'API native `sqlite3_backup_init` / `sqlite3_backup_step` / `sqlite3_backup_finish` : elle ne copie jamais directement le fichier SQLite avec une primitive de système de fichiers.

La méthode est synchrone et renvoie :

- `(true, nil)` lorsque la sauvegarde a été copiée, fermée, synchronisée et publiée ;
- `(nil, err)` pour une erreur opérationnelle, un verrou persistant, un timeout ou une destination invalide ;
- une erreur Lua pour une arité, un type ou une option incorrecte.

Le chemin de destination doit désigner un fichier dans un dossier déjà existant. Il doit être une vraie chaîne Lua non vide, sans octet NUL et sans composant `..`. `":memory:"` n'est pas accepté comme destination : pour copier une base mémoire vers un fichier, appeler simplement `backup()` sur la connexion mémoire avec un vrai chemin.

Options

Champ	Type	Défaut	Contrat
`timeout`	nombre fini de `0` à `86400` secondes	`5.0`	deadline globale monotone
`pages_per_step`	entier de `1` à `INT_MAX`	`128`	pages maximales copiées par appel SQLite
`sleep`	nombre fini de `0` à `60` secondes	`0.01`	pause entre deux étapes ou tentatives
`overwrite`	boolean	`false`	remplace atomiquement une destination régulière existante

La table d'options est stricte et lue avec des accès bruts. Les champs inconnus, les clés non chaînes, les nombres non finis et les valeurs hors limites lèvent une erreur Lua. Une métaméthode `__index` ne peut pas fournir une option.

La deadline est calculée une seule fois avec une horloge monotone. Elle n'est jamais réinitialisée après une étape réussie, `SQLITE_BUSY` ou `SQLITE_LOCKED`. Le `busy_timeout` de la connexion source est temporairement désactivé pendant la sauvegarde puis restauré, afin qu'un handler SQLite ne puisse pas dépasser silencieusement la deadline globale.

`timeout = 0` effectue une vraie tentative non bloquante : Babet appelle une fois `sqlite3_backup_step()`, sans sommeil ni nouvelle tentative. Une petite base peut donc être sauvegardée immédiatement ; une base nécessitant une étape supplémentaire renvoie un timeout et ne publie aucun fichier partiel.

La deadline est vérifiée **entre** les appels à `sqlite3_backup_step()`. SQLite ne fournit pas d'interruption sûre au milieu d'un appel. Une grande valeur de `pages_per_step` réduit le nombre d'appels, mais réduit aussi la fréquence de vérification du délai. Pour une sauvegarde réactive, conserver une valeur modérée.

Publication atomique et destination existante La base n'est jamais écrite directement dans `path`. Babet :

1. ouvre le dossier parent sans suivre les liens symboliques ;
2. crée un fichier privé voisin nommé `.babet-sqlite-backup-*` ;
3. y exécute la sauvegarde native ;
4. appelle systématiquement `sqlite3_backup_finish()` ;
5. ferme SQLite, synchronise le fichier, puis le publie atomiquement.

Le fichier publié conserve le mode privé `0600` utilisé par le temporaire. Ce mode ne dépend pas de l'`umask` et remplace aussi les permissions d'une ancienne destination lors de `overwrite = true`. Le script peut ensuite élargir explicitement les permissions avec `babet.setMode()` lorsque son modèle de partage l'exige.

Sur erreur ou timeout, le fichier temporaire et ses éventuels compagnons `-journal`, `-wal` et `-shm` sont supprimés. Une destination existante reste inchangée.

Par défaut, une destination existante est refusée. Avec `overwrite = true`, elle est remplacée atomiquement seulement si elle est un fichier régulier qui n'est ni la base source, ni un lien symbolique, ni un hard link vers la source. Babet refuse aussi la publication lorsque des compagnons `path-journal`, `path-wal` ou `path-shm` existent : ils peuvent indiquer une connexion encore ouverte ou un état WAL qui ne doit pas être remplacé sous ses utilisateurs. Fermer et checkpoint-er proprement la destination avant de réessayer.

Les composants de dossier symboliques sont refusés. Les dossiers manquants ne sont pas créés. La méthode ne remplace jamais un dossier, FIFO ou autre fichier spécial.

Si le fichier a été publié mais que la synchronisation finale du dossier échoue, l'erreur l'indique explicitement. Dans ce cas particulier, la base publiée est complète et lisible, mais sa persistance après une panne système immédiate n'est pas garantie.

WAL et écritures concurrentes Une source en WAL est prise en charge sans traitement spécial côté Lua. SQLite ne maintient le verrou de lecture source que pendant les lectures nécessaires et permet aux autres connexions de continuer à lire ou écrire. Si une autre connexion modifie la source pendant la copie, SQLite redémarre automatiquement la sauvegarde lors d'une étape suivante afin de produire une base cohérente. La sauvegarde finale correspond à un état transactionnel complet ; elle ne mélange pas les pages de deux commits.

La destination publiée est autonome. Il suffit de l'ouvrir normalement avec `babet.sqlite.open()` ; aucune copie manuelle du fichier `-wal` source n'est nécessaire.

Exemples Sauvegarde simple, sans écrasement :

```
local db = assert(babet.sqlite.open("state.db"))
assert(db:backup("state-backup.db"))
assert(db:close())
```

Sauvegarde WAL bornée avec petites étapes :

```
local db = assert(babet.sqlite.open("state.db", {
    wal = true,
    busy_timeout = 2000,
}))

local ok, err = db:backup("state-backup.db", {
    timeout = 10,
    pages_per_step = 64,
    sleep = 0.005,
})
assert(ok, err)
```

Remplacement explicite d'une ancienne sauvegarde fermée :

```
assert(db:backup("state-backup.db", {
    overwrite = true,
    timeout = 5,
}))
```

Restaurer vers un nouveau fichier revient à ouvrir la sauvegarde comme source et à lancer une nouvelle sauvegarde :

```
local snapshot = assert(babet.sqlite.open("state-backup.db", {
    readonly = true,
}))
assert(snapshot:backup("state-restored.db"))
assert(snapshot:close())

local restored = assert(babet.sqlite.open("state-restored.db"))
```

Exécution directe

Fonction	Renvoie
`db:exec(sql)`	`(true, nil)` | `(nil, err)` — plusieurs instructions acceptées
`db:exec(sql, params)`	`(true, nil)` | `(nil, err)` — **une seule** instruction

Fonction	Renvoie
`db:query(sql, params?)`	`stmt, nil, nil, stmt` | `(nil, err)`
`stmt:close()`	`(true, nil)` — idempotent

`db:exec` Sans table `params`, `exec` accepte plusieurs instructions séparées par des points-virgules. Elles sont préparées et exécutées dans l'ordre. L'exécution s'arrête à la première erreur ; les instructions précédentes restent acquises si elles ne se trouvaient pas dans une transaction explicite.

Avec une table `params`, une seule instruction est acceptée. Les espaces, points-virgules et commentaires SQL finaux (`-- ...` ou `/* ... */`) ne comptent pas comme une seconde instruction.

Un `SELECT` passé à `exec` est exécuté jusqu'au bout, mais ses lignes sont ignorées. Une chaîne SQL vide ou composée uniquement de séparateurs et de commentaires est un no-op réussi avec `exec(sql)` ou `exec(sql, {})`.

`db:query` `query` prépare immédiatement une instruction et renvoie un **itérateur appelable à usage unique**. L'instruction n'est exécutée qu'au premier appel :

```
for row in db:query("SELECT id, name FROM users ORDER BY id") do
    print(row.id, row.name)
end
```

Chaque ligne est une table indexée par nom de colonne. L'itérateur renvoie `nil` et finalise l'instruction lorsqu'il est épuisé. En succès, `query` renvoie `stmt, nil, nil, stmt` : la quatrième valeur permet à Lua de fermer le curseur dès toute sortie du `for` générique (`break`, `return`, `goto`, erreur ou fermeture de coroutine). Les échecs renvoient toujours exactement `(nil, err)` ; `local iter, err = db:query(...)` garde donc son usage habituel.

Si seule la première valeur est conservée, `for row in iter do` ne reçoit pas de quatrième valeur à fermer. Utilise `local iter <close> = assert(db:query(...))` pour fermer à la sortie de portée, ou appelle explicitement `iter:close()`. Les parenthèses `(db:query(...))` ne conservent elles aussi que la première valeur. `__close` et `close()` sont idempotents : les références conservées restent des itérateurs épuisés valides. `__gc` reste le filet de sécurité des curseurs abandonnés. Le contrat de reset/réutilisation explicite des statements préparés `prepared:query` reste inchangé.

`query` n'est pas limité aux `SELECT`. Une instruction DDL ou DML est exécutée au premier appel, puis l'itérateur se termine sans produire de ligne.

Un SQL multi-instructions est refusé. Un SQL vide ou composé seulement de commentaires produit un itérateur déjà épuisé.

Statements préparés réutilisables

Fonction	Renvoie
`db:prepare(sql)`	`prepared` | `(nil, err)`
`prepared:exec(params?)`	`(true, nil)` | `(nil, err)`
`prepared:query(params?)`	le même userdata `prepared` | `(nil, err)`
`prepared:reset()`	`(true, nil)` | `(nil, err)`
`prepared:close()`	`(true, nil)` — idempotent
`prepared:finalize()`	alias de `close()`

`db:prepare()` accepte exactement **une instruction SQL non vide**. La préparation est immédiate : une table ou une colonne inconnue est donc signalée avant la première exécution.

Le statement peut ensuite être réutilisé autant de fois que nécessaire :

```lua
local insert = assert(db:prepare(
    "INSERT INTO files(path, digest) VALUES(?, ?)"
))

for _, file in ipairs(files) do
    assert(insert:exec({ file.path, file.digest }))
end

assert(insert:finalize())
```

`prepared:exec()` réinitialise automatiquement le statement avant le bind, l'exécute jusqu'à `SQLITE_DONE`, puis efface les bindings. Un `SELECT` est entièrement parcouru mais ses lignes sont ignorées.

`prepared:query()` réinitialise et bind le statement, puis renvoie **le même userdata**, rendu appelable par sa métatable :

```lua
local select_by_age = assert(db:prepare([[
    SELECT id, name FROM users
    WHERE age >= ?
    ORDER BY id
]]))

for row in select_by_age:query({ 18 }) do
    print(row.id, row.name)
end

for row in select_by_age:query({ 65 }) do
    print("senior", row.name)
end
```

À l'épuisement naturel, le statement est automatiquement réinitialisé et ses bindings sont effacés. Après un `break`, trois choix sont possibles :

— appeler `prepared:reset()` ;
— démarrer un nouveau `prepared:query(...)` ;
— appeler `prepared:exec(...)`.

Les deux dernières opérations réinitialisent aussi automatiquement l'itération précédente et abandonnent ses lignes restantes. Un statement préparé ne peut avoir qu'une seule itération active à la fois : réutiliser le même userdata dans une boucle imbriquée réinitialise donc la boucle extérieure.

Un appel direct `prepared()` sans `prepared:query()` actif renvoie simplement `nil` et n'exécute jamais le SQL avec des bindings effacés.

`prepared:reset()` interrompt l'exécution courante et efface les bindings. `close()` et `finalize()` finalisent définitivement le statement. Une fois fermé, son appel direct renvoie `nil`, tandis que ses méthodes d'exécution renvoient `(nil, "sqlite: statement closed")`.

Transactions assistées

```lua
local ok, result = db:transaction(function(tx)
    assert(tx:exec(
        "INSERT INTO accounts(name, balance) VALUES(?, ?)",
        { "alice", 100 }
    ))

    assert(tx:exec(
        "INSERT INTO audit(message) VALUES(?)",
        { "account created" }
```

```
    ))

    return "created"
end, "immediate")

assert(ok, result)
print(result) -- "created"
```

Signature :

```
ok, ...callback_results = db:transaction(callback [, mode])
-- ou
nil, err = db:transaction(callback [, mode])
```

Le callback reçoit la connexion `db` comme unique argument. `mode` vaut :

- `"deferred"` — défaut ;
- `"immediate"` ;
- `"exclusive"`.

Si le callback se termine normalement, Babet exécute `COMMIT` et renvoie `true`, suivi de toutes les valeurs du callback. Un retour normal `nil` ou `false` **ne demande pas un rollback** : seule une erreur Lua déclenche le rollback automatique.

Les opérations SQLite renvoyant habituellement `(nil, err)`, utilise `assert` dans le callback pour transformer une erreur opérationnelle en erreur Lua :

```
local ok, err = db:transaction(function(tx)
    assert(tx:exec("INSERT INTO unique_names VALUES(?)", { "alice" }))
    assert(tx:exec("INSERT INTO unique_names VALUES(?)", { "alice" }))
end)

-- Le second INSERT échoue, assert lève, et les deux INSERT sont annulés.
```

Une erreur du callback est interceptée : Babet tente `ROLLBACK` puis renvoie `(nil, "sqlite: transaction callback failed: ...")`. Elle n'est pas relancée. Un échec de `COMMIT` renvoie également `(nil, err)` après une tentative de rollback. Les valeurs éventuellement renvoyées par le callback sont alors abandonnées.

Après un `BEGIN` réussi, Babet possède explicitement la transaction jusqu'au `COMMIT` ou au `ROLLBACK`. La pile Lua nécessaire à l'appel du callback est réservée avant `BEGIN` ; une exception C++ interne déclenche également une tentative immédiate de rollback. Le diagnostic du callback n'est formaté qu'après cette tentative, afin qu'un objet d'erreur Lua inhabituel ne puisse laisser la connexion dans un état intermédiaire.

Si le rollback normal échoue, le message contient à la fois l'erreur initiale et l'erreur de rollback, puis Babet effectue une dernière tentative d'urgence. Après une erreur, `db:in_transaction()` permet de vérifier explicitement que la connexion est revenue en mode autocommit avant de poursuivre.

Exemple : échec au moment du commit Une contrainte différée peut n'être vérifiée qu'au `COMMIT` :

```
local db = assert(babet.sqlite.open(
    ":memory:", { foreign_keys = true }))
assert(db:exec([[
    CREATE TABLE parent(id INTEGER PRIMARY KEY);
    CREATE TABLE child(
        id INTEGER PRIMARY KEY,
        parent_id INTEGER NOT NULL,
        FOREIGN KEY(parent_id) REFERENCES parent(id)
            DEFERRABLE INITIALLY DEFERRED
```

```
    )
]]))

local ok, err = db:transaction(function(tx)
    assert(tx:exec(
        "INSERT INTO child(id, parent_id) VALUES(?, ?)",
        { 1, 999 }))
    return "ce résultat ne sera pas renvoyé"
end)

assert(ok == nil)
assert(type(err) == "string")
assert(db:in_transaction() == false)
```

L'`INSERT` est annulé et la connexion reste réutilisable si le rollback a réussi.

Exemple : réutiliser un statement préparé après rollback

```
local insert = assert(db:prepare(
    "INSERT INTO journal(id, valeur) VALUES(?, ?)"))

local ok, err = db:transaction(function()
    assert(insert:exec({ 1, "annulé" }))
    error("échec volontaire")
end)

assert(ok == nil and type(err) == "string")
assert(insert:exec({ 2, "conservé" }))
insert:finalize()
```

Le rollback réinitialise l'état transactionnel de SQLite ; le statement préparé peut ensuite être réutilisé normalement.

Limitations volontaires :

- les appels imbriqués à `db:transaction()` sont refusés ;
- la fonction est refusée si une transaction SQL manuelle est déjà active ;
- `db:close()` est refusé pendant le callback ;
- le callback ne doit pas exécuter lui-même `BEGIN`, `COMMIT` ou `ROLLBACK`.

Utilise `db:savepoint()` pour des portées imbriquées assistées. Les savepoints SQL bruts restent accessibles lorsqu'un identifiant explicite ou un contrôle manuel est nécessaire.

Savepoints imbriqués

Signature :

```
true, ...résultats_callback = db:savepoint(callback)
-- ou
nil, err = db:savepoint(callback)
```

Le callback reçoit la même connexion ouverte comme unique argument. Babet génère l'identifiant SQL en interne ; l'API n'accepte aucun nom et ne peut donc pas injecter de texte fourni par l'appelant dans `SAVEPOINT`, `ROLLBACK TO` ou `RELEASE`.

Si le callback se termine normalement, Babet exécute `RELEASE` et renvoie `true`, suivi de toutes les valeurs du callback. `nil` et `false` explicites sont des résultats normaux et ne demandent pas de rollback. Sur erreur Lua, Babet exécute `ROLLBACK TO` puis `RELEASE`, intercepte l'erreur et renvoie `(nil, "sqlite: savepoint callback failed: ...")`.

Exemple : savepoint autonome Hors transaction, le savepoint le plus externe démarre une transaction. Son `RELEASE` valide le travail isolé et rétablit le mode autocommit :

```
local ok, id = db:savepoint(function(tx)
    assert(tx:exec(
        "INSERT INTO jobs(name) VALUES(?)",
        { "index" }))
    return assert(tx:last_insert_rowid())
end)

assert(ok, id)
assert(db:in_transaction() == false)
```

Exemple : rollback sur erreur du callback Utilise `assert` sur les opérations SQLite dont l'échec doit annuler la portée :

```
local ok, err = db:savepoint(function(tx)
    assert(tx:exec(
        "INSERT INTO unique_names(name) VALUES(?)",
        { "alice" }))
    assert(tx:exec(
        "INSERT INTO unique_names(name) VALUES(?)",
        { "alice" }))
end)

assert(ok == nil)
assert(type(err) == "string")
-- Aucun INSERT ne reste et la connexion peut être réutilisée.
```

Exemple : récupérer l'échec d'une portée interne Les helpers peuvent être imbriqués. Une portée interne en échec revient à son propre marqueur ; le callback externe peut examiner l'erreur et continuer :

```
local ok, inner_err = db:savepoint(function(tx)
    assert(tx:exec("INSERT INTO audit(message) VALUES('avant')"))

    local inner_ok, err = tx:savepoint(function(inner)
        assert(inner:exec("INSERT INTO audit(message) VALUES('facultatif')"))
        error("annuler le travail facultatif")
    end)
    assert(inner_ok == nil)

    assert(tx:exec("INSERT INTO audit(message) VALUES('après')"))
    return err
end)

assert(ok, inner_err)
-- « avant » et « après » restent ; « facultatif » a été annulé.
```

Une erreur ultérieure du callback externe annule également tout savepoint interne déjà libéré. `RELEASE` fusionne le travail interne avec la portée englobante ; il ne rend jamais ce travail indépendant de la transaction externe.

Exemple : dans une transaction assistée `db:savepoint()` est autorisé dans `db:transaction()`, contrairement à un second helper de transaction :

```
local ok, err = db:transaction(function(tx)
    assert(tx:exec("INSERT INTO audit(message) VALUES('obligatoire')"))

    local optional_ok = tx:savepoint(function(inner)
        assert(inner:exec("INSERT INTO audit(message) VALUES('facultatif')"))
    end)
    if not optional_ok then
        assert(tx:exec("INSERT INTO audit(message) VALUES('échec facultatif')"))
    end

    assert(tx:in_transaction() == true)
end, "immediate")

assert(ok, err)
```

Le `RELEASE` interne ne valide **pas** la transaction assistée. Si le callback externe échoue ensuite, `db:transaction()` annule l'écriture obligatoire et toutes les écritures internes déjà libérées.

Exemple : dans une transaction manuelle

```
assert(db:exec("BEGIN"))

local ok, err = db:savepoint(function(tx)
    assert(tx:exec("UPDATE accounts SET balance = balance - 10 WHERE id = 1"))
end)
assert(ok, err)
assert(db:in_transaction() == true)

assert(db:exec("ROLLBACK")) -- annule aussi le travail du savepoint libéré
```

Exemple : échec du `RELEASE` du savepoint externe Une contrainte différée peut n'être contrôlée que lorsque le `RELEASE` externe tente de valider. Les résultats du callback sont abandonnés, Babet revient au savepoint, le retire et renvoie l'erreur du `RELEASE` :

```
local db = assert(babet.sqlite.open(
    ":memory:", { foreign_keys = true }))
assert(db:exec([[
    CREATE TABLE parent(id INTEGER PRIMARY KEY);
    CREATE TABLE child(
        id INTEGER PRIMARY KEY,
        parent_id INTEGER REFERENCES parent(id)
            DEFERRABLE INITIALLY DEFERRED
    )
]]))

local ok, err, leaked = db:savepoint(function(tx)
    assert(tx:exec("INSERT INTO child VALUES(1, 999)"))
    return "ne doit pas être renvoyé"
end)

assert(ok == nil and type(err) == "string")
assert(leaked == nil)
assert(db:in_transaction() == false)
```

Dans une transaction existante, libérer un savepoint interne n'exécute pas le commit externe : l'éventuelle erreur différée reste donc du ressort du futur `COMMIT` de cette transaction.

Le helper suit la profondeur sur chaque connexion et fonctionne dans les workers. `db:close()` est refusé pendant tout callback de savepoint. La pile Lua est réservée avant l'ouverture ; une garde C++ sans allocation tente un `ROLLBACK TO` puis un `RELEASE` d'urgence si une exception interne survient.

Ne mélange pas de commandes manuelles de transaction ou de savepoint dans le callback. En particulier, un `ROLLBACK` ou `COMMIT` complet détruit le savepoint assisté. Babet détecte le retour en autocommit, abandonne les résultats du callback, évite les commandes de nettoyage redondantes et renvoie une seule erreur stable sans exposer l'identifiant généré. Un `ROLLBACK` laisse la connexion réutilisable après avoir annulé les écritures du callback ; un `COMMIT` peut déjà les avoir validées avant que Babet ne diagnostique la violation du contrat. Utilise des appels `db:savepoint()` imbriqués pour les portées internes récupérables.

Paramètres SQL

Les formes suivantes sont prises en charge par `exec`, `query` et les statements préparés :

Placeholder	Valeur Lua
`?`	`params[1]`, `params[2]`, etc., selon l'ordre des `?`
`:name`, `@name`, `$name`	`params.name` — le préfixe est retiré

Les paramètres positionnels et nommés peuvent être mélangés :

```
assert(db:exec(
    "INSERT INTO events VALUES (?, :kind, ?)",
    { 42, 1700000000, kind = "start" }
))
```

La table doit correspondre exactement aux placeholders. Un paramètre manquant, en trop, un index numérique sparse, un index numérique non entier, un type de clé non supporté ou une valeur non

supportée lève une erreur Lua. Passer explicitement `nil` comme table équivaut à ne pas fournir de paramètres ; ce n'est pas une valeur de bind.

Les paramètres nommés sont lus directement dans la table. Une valeur fournie par `__index` ne compte pas et une erreur de cette métaméthode ne sera jamais déclenchée par le bind. Les placeholders numérotés `?NNN` sont explicitement refusés ; utilise des `?` anonymes dans leur ordre naturel.

Types acceptés au bind :

Type Lua	Valeur SQLite
boolean	INTEGER `0` ou `1`
integer	INTEGER, sur toute la plage signée 64 bits
number non entier fini	REAL
string	TEXT, y compris avec des octets NUL
`babet.sqlite.blob(data)`	BLOB binary-safe
`babet.sqlite.NULL`	SQL `NULL`

NaN et les infinis positif ou négatif sont refusés au lieu d'être transmis à SQLite avec un résultat dépendant de la plateforme.

Valeurs NULL explicites

Lua ne peut pas conserver `nil` dans une table de paramètres : lui affecter `nil` supprime la clé. Utilise l'unique sentinelle `babet.sqlite.NULL` lorsqu'un placeholder doit être bindé à SQL `NULL`.

Valeur optionnelle nommée :

```lua
local NULL = babet.sqlite.NULL

assert(db:exec([[
    INSERT INTO users(name, nickname)
    VALUES(:name, :nickname)
]], {
    name = "Ada",
    nickname = NULL,
}))
```

`NULL` positionnel au milieu d'une liste de paramètres :

```lua
assert(db:exec(
    "INSERT INTO events(id, payload, created_at) VALUES(?, ?, ?)",
    { 17, babet.sqlite.NULL, 1700000000 }
))
```

La sentinelle fonctionne de la même manière avec les statements réutilisables et peut alterner entre des valeurs ordinaires et `NULL` :

```lua
local update = assert(db:prepare(
    "UPDATE users SET nickname = ? WHERE name = ?"
))

assert(update:exec({ "Comtesse", "Ada" }))
assert(update:exec({ babet.sqlite.NULL, "Ada" }))
assert(update:finalize())
```

Le bind et la lecture sont volontairement asymétriques :

Valeur Lua au bind	Stockage SQLite	Valeur Lua lue
`true`	INTEGER `1`	integer `1`
`false`	INTEGER `0`	integer `0`
`babet.sqlite.NULL`	`NULL`	`nil` / clé absente

SQLite ne possède pas de classe de stockage booléenne : les entiers sont donc relus comme des entiers et Babet ne déduit pas un boolean de `0` ou `1`. Un résultat `NULL` ne crée aucune clé dans la table ligne : `row.colonne == nil` et la clé est également absente de `pairs(row)`.

`babet.sqlite.NULL` est accepté uniquement comme valeur de bind SQLite. Les autres lightuserdata sont refusés. L'encodage JSON et le transfert vers les workers ou les channels refusent aussi cette sentinelle avec un diagnostic explicite, afin qu'une valeur opaque ressemblant à un pointeur ne franchisse jamais silencieusement les frontières entre sous-systèmes.

BLOB explicites

Une chaîne Lua ordinaire reste **toujours** bindée avec `sqlite3_bind_text`, même si la colonne possède une affinité `BLOB`. `babet.sqlite.blob(data)` marque explicitement les mêmes octets comme BLOB :

```
local insert = assert(db:prepare(
    "INSERT INTO assets(name, payload) VALUES(?, ?)"
))

assert(insert:exec({
    "icon",
    babet.sqlite.blob("\x89PNG\r\n\x1a\n...")
}))
```

Le constructeur exige exactement une chaîne Lua et renvoie un userdata opaque. Les octets NUL sont conservés. `babet.sqlite.blob("")` produit bien un BLOB de zéro octet, et non `NULL`.

Le wrapper ne sert qu'au bind ; la lecture d'une colonne BLOB renvoie toujours une chaîne Lua binary-safe.

Texte SQL et octets NUL

Le texte SQL doit être une chaîne sans octet NUL. SQLite considérerait ce NUL comme une fin de chaîne et pourrait n'exécuter qu'un préfixe ; Babet refuse donc la requête avant toute préparation. La taille du SQL est également limitée à `INT_MAX` octets, conformément à `sqlite3_prepare_v2`.

Cette restriction concerne seulement le **texte SQL**. Les chaînes et BLOB bindés sont binary-safe.

Lecture des lignes et mapping de types

Type SQLite lu	Type Lua
INTEGER	integer
REAL	number (float)
TEXT	string
BLOB	string binary-safe, y compris pour un BLOB vide
NULL	clé absente de la table (`row.col == nil` et absente de `pairs`)

Si plusieurs colonnes portent le même nom, la dernière colonne gagne. Utilise des alias pour conserver toutes les valeurs.

La fonction SQL `length()` compte les caractères d'un TEXT et peut s'arrêter à un octet NUL. Pour compter ses octets lors d'un diagnostic, utilise par exemple `length(CAST(col AS BLOB))`.

Cycle de vie

`db:close()` est idempotent. Après fermeture, toute nouvelle opération comme `exec`, `query`, `prepare`, `transaction`, `savepoint` et `in_transaction` renvoient `(nil, "sqlite: connection closed")`.

Un itérateur ou un statement préparé créé **avant** `db:close()` reste cependant utilisable. Babet emploie `sqlite3_close_v2` : SQLite conserve une connexion « zombie » jusqu'à la finalisation du dernier statement actif.

```
local stmt = assert(db:prepare("SELECT id FROM users ORDER BY id"))
assert(db:close())

for row in stmt:query() do
    print(row.id)
end

stmt:finalize()
```

Les itérateurs temporaires de `db:query` et les statements préparés possèdent un `__gc` de secours. Les boucles directes `for row in db:query(...)` et les variables `<close>` finalisent les curseurs temporaires de façon déterministe. Pour les itérateurs conservés séparément et les statements préparés, utilise close/finalize explicitement pour libérer rapidement les ressources.

Les statements ne conservent volontairement ni référence Lua ni pointeur `Db*` brut vers le userdata parent. SQLite maintient lui-même la connexion native zombie en vie ; cela évite un pointeur C++ devenu invalide et permet au userdata `db` d'être collecté indépendamment.

Contrat d'erreur

Les erreurs se répartissent en deux catégories :

- les erreurs opérationnelles (`open`, préparation, `step`, transaction, connexion fermée) renvoient généralement `(nil, "sqlite: <description>")` ;
- les mauvais types, les arités invalides et les tables `params` incorrectes lèvent une erreur Lua.

Toutes les fonctions publiques vérifient leur arité exacte. Les erreurs de contrat des paramètres couvrent aussi `?NNN`, les REAL non finis, les lightuserdata étrangers et les tables d'options ou de paramètres contenant des clés non supportées.

`opts.readonly`, `opts.foreign_keys` et `opts.wal` exigent des booleans Lua stricts. La combinaison `readonly = true, wal = true` lève une erreur de contrat. Une base absente ouverte en lecture seule, une écriture via une connexion en lecture seule ou une violation de clé étrangère sont des erreurs opérationnelles et renvoient `(nil, err)`.

Les erreurs survenant pendant l'appel d'un itérateur (`db:query` ou `prepared:query`) lèvent une erreur Lua, car le protocole d'itération ne permet pas de renvoyer proprement `(nil, err)` en plus de la fin de séquence.

Sans table `params`, la présence d'un placeholder est détectée avant l'exécution. Babet refuse de laisser SQLite binder silencieusement `NULL`.

`db:transaction()` et `db:savepoint()` constituent des exceptions contrôlées : ils interceptent les erreurs Lua du callback, annulent la portée qu'ils possèdent et convertissent l'erreur en `(nil, err)`.

Exemples complets

Inserts préparés, NULL optionnel et BLOB

```
local db = assert(babet.sqlite.open("state.db", {
    wal = true,
    busy_timeout = 2000,
}))

assert(db:exec([[
    CREATE TABLE IF NOT EXISTS files (
        path TEXT PRIMARY KEY,
        digest BLOB NOT NULL,
        media_type TEXT
    )
]]))

local insert = assert(db:prepare([[
    INSERT INTO files(path, digest, media_type)
    VALUES(?, ?, ?)
    ON CONFLICT(path) DO UPDATE SET
        digest = excluded.digest,
        media_type = excluded.media_type
]]))

for _, file in ipairs(files) do
    assert(insert:exec({
        file.path,
        babet.sqlite.blob(file.digest),
        file.media_type or babet.sqlite.NULL,
    }))
end

insert:finalize()
```

Requête réutilisable et transaction

```
local by_prefix = assert(db:prepare([[
    SELECT path, digest
    FROM files
    WHERE path LIKE ?
    ORDER BY path
]]))

local ok, count = db:transaction(function(tx)
    local n = 0
    for row in by_prefix:query({ "images/%" }) do
        assert(tx:exec(
            "INSERT INTO audit(path) VALUES(?)",
            { row.path }
        ))
        n = n + 1
    end
    return n
end, "immediate")

assert(ok, count)
```

```
print(count, "rows audited")

by_prefix:finalize()
assert(db:close())
```

Fonctions non exposées

Les éléments suivants ne sont volontairement pas implémentés :

- le mode URI et les autres flags avancés de `sqlite3_open_v2` ;
- les APIs de progress handler et d'interruption ;
- l'API de streaming BLOB `sqlite3_blob_open` ;

Les savepoints SQL bruts et `VACUUM INTO` restent accessibles. FTS5 et R-Tree ne sont pas activés dans la compilation embarquée actuelle.

babet strings — découpage de chaînes

Babet expose une seule fonction de manipulation de chaînes : `babet.split`. Elle est enregistrée directement dans la table `babet`, sans sous-namespace, pour conserver l'API historique.

Table des matières du module

- API
- Mode séparateur
  - Limiter le nombre de coupes
- Mode octets
- Chaînes binaires et octets NUL
- Chaîne vide
- Contrat d'erreur
- Limites et choix de conception

API

```
local parts = babet.split(s [, sep [, max_splits]])
```

La fonction exige **entre un et trois arguments** et renvoie exactement une valeur : une table dense indexée de 1 à `n`.

Argument	Type et comportement
`s`	Chaîne Lua obligatoire. Les nombres ne sont pas convertis implicitement. Les octets NUL sont préservés.
`sep`	Chaîne Lua facultative. Omission ou chaîne vide : mode octets. Sinon, elle doit contenir exactement **un octet**, utilisé littéralement.
`max_splits`	Entier Lua facultatif, supérieur ou égal à `-1`. `-1` signifie illimité, `0` interdit toute coupe. Utilisé uniquement lorsque `sep` contient un octet.

`nil` n'est pas une valeur de remplacement pour un argument facultatif : `babet.split("abc", nil)` lève une erreur. Pour fournir `max_splits` en mode octets, il faut passer explicitement `""` comme deuxième argument.

Mode séparateur

Lorsque `sep` contient exactement un octet, chaque occurrence de cet octet provoque une coupe. Le séparateur est **littéral** : il ne s'agit ni d'un pattern Lua ni d'une expression régulière.

```
babet.split("a,b,c", ",")
-- { "a", "b", "c" }

babet.split("a.b.c", ".")
-- { "a", "b", "c" }  -- le point n'est pas un joker
```

Les entrées vides sont conservées, y compris en début, entre deux séparateurs et en fin de chaîne :

```
babet.split(",a", ",")
-- { "", "a" }

babet.split("a,,b", ",")
-- { "a", "", "b" }

babet.split("a,", ",")
-- { "a", "" }
```

Si le séparateur n'apparaît pas, le résultat contient la chaîne entière :

```
babet.split("hello", ",")
-- { "hello" }
```

Limiter le nombre de coupes

`max_splits` compte les **coupes**, pas le nombre d'éléments produits. Le reste non découpé est placé tel quel dans le dernier élément.

```
babet.split("a,b,c,d", ",", 2)
-- { "a", "b", "c,d" }

babet.split("a,b,c", ",", 0)
-- { "a,b,c" }

babet.split("a,b,c", ",", -1)
-- { "a", "b", "c" }
```

Toute valeur supérieure au nombre de séparateurs disponibles produit le même résultat que `-1`.

Mode octets

Si `sep` est omis ou vaut `""`, la chaîne est découpée octet par octet. Il n'existe donc **aucun séparateur espace par défaut**.

```
babet.split("ab c")
-- { "a", "b", " ", "c" }

babet.split("abc", "")
-- { "a", "b", "c" }
```

`max_splits`, lorsqu'il est fourni dans ce mode, est toujours validé mais n'est pas utilisé :

```
babet.split("abc", "", 0)
-- { "a", "b", "c" }
```

Ce mode travaille sur des **octets**, pas sur des points de code Unicode. Un caractère UTF-8 multi-octets est donc séparé en plusieurs chaînes d'un octet :

```lua
local bytes = babet.split("é")
-- #bytes == 2 en UTF-8
-- table.concat(bytes) == "é"
```

De même, un séparateur UTF-8 multi-octets comme `"é"` est refusé, car `sep` doit contenir zéro ou un octet. Pour découper sur une chaîne de plusieurs octets, il faut utiliser du Lua (`string.find`, `string.gmatch`, etc.) ou une fonction dédiée.

Chaînes binaires et octets NUL

Le sujet, le séparateur et les éléments produits sont manipulés avec leur longueur Lua exacte. Un octet NUL ne tronque donc rien :

```lua
local parts = babet.split("a\0b,c", ",")
-- { "a\0b", "c" }

local parts2 = babet.split("a\0b", "\0")
-- { "a", "b" }
```

Le contenu UTF-8 reste également intact lorsqu'il est séparé par un octet ASCII :

```lua
babet.split("été:ok", ":")
-- { "été", "ok" }
```

Chaîne vide

Deux comportements historiques différents sont conservés :

```lua
babet.split("", ",")
-- { "" }

babet.split("")
-- {}

babet.split("", "")
-- {}
```

En mode séparateur, zéro séparateur trouvé signifie un élément contenant la chaîne entière, même si elle est vide. En mode octets, une chaîne de zéro octet ne produit aucun élément.

Contrat d'erreur

Toutes les erreurs sont des erreurs Lua levées ; `split` ne possède pas de chemin de retour `(nil, err)`.

La fonction lève notamment dans les cas suivants :

- moins d'un ou plus de trois arguments ;
- `s` n'est pas une vraie chaîne Lua ;
- `sep` est présent mais n'est pas une vraie chaîne Lua ;
- `sep` contient plus d'un octet ;
- `max_splits` n'est pas un entier Lua ;
- `max_splits` est inférieur à `-1`.

Les conversions implicites nombre-vers-chaîne ne sont pas acceptées :

```lua
babet.split(123, ",")       -- erreur
babet.split("123", 2)       -- erreur
babet.split("a,b", ",", "1") -- erreur
```

Limites et choix de conception

— Ce n'est pas un parseur CSV : aucune gestion des guillemets, échappements ou lignes n'est effectuée.
— Les séparateurs multi-octets et les patterns Lua ne sont pas pris en charge.
— Les entrées vides sont volontairement conservées.
— Un équivalent `join` n'est pas exposé, car `table.concat` remplit déjà ce rôle dans la bibliothèque standard Lua.

SYS - processus, machine, environnement, exécutables, version et mémoire Lua

Les fonctions de ce chapitre sont exposées directement dans la table globale `babet`. Il n'existe pas de sous-table `babet.sys` : ce namespace plat est historique et fait partie de l'API stable.

Le chapitre SYS couvre :

— l'identité du processus Babet ;
— le nom de la machine et les informations du noyau ;
— la lecture et la modification de variables d'environnement ;
— la recherche d'un programme exécutable ;
— les constantes de version du runtime ;
— la mémoire utilisée par l'état Lua courant.

Le répertoire de travail courant est lui aussi un état global du processus, mais ses fonctions `currentDir` et `chdir` sont documentées dans FS - Répertoire courant, avec les autres fonctions de chemins.

Table des matières du module

— Conventions générales
— Vue d'ensemble de l'API
— Constantes de version
— Identifier le processus et la machine
  — `pid`
  — `hostname`
  — `uname`
— Rechercher un exécutable
  — Recherche dans `PATH`
  — Chemin direct
  — Programme absent ou non exécutable
— Variables d'environnement
  — `env`
  — `setenv`
  — Valeur vide et variable absente
  — Interaction avec les workers
— Mémoire de la VM Lua
  — `getMemoryUsage`
  — `getDetailedMemoryUsage`
  — Ce qui est mesuré et ce qui ne l'est pas
— Contrat d'erreur
— Décisions et limites

Conventions générales

Fonctions plates

Toutes les fonctions s'appellent directement depuis `babet` :

```
local pid = babet.pid()
local home = babet.env("HOME")
```

Les formes suivantes n'existent pas :

```
-- Incorrect : aucune sous-table babet.sys
-- babet.sys.pid()
-- babet.sys.env("HOME")
```

Chaînes strictes

`env`, `setenv` et `which` exigent de véritables chaînes Lua. Les nombres ne sont pas convertis automatiquement.

```
local ok, err = pcall(function()
    return babet.env(42)
end)

-- ok == false ; err décrit une mauvaise signature
```

Un octet NUL est toujours refusé. Cette règle évite qu'une chaîne Lua soit silencieusement tronquée lorsqu'elle est transmise à une API C.

État global du processus

L'environnement et le répertoire courant appartiennent au **processus entier**, pas à un script ou à un worker particulier.

- `setenv` modifie l'environnement vu par le runtime et les futurs programmes lancés avec `babet.exec` ;
- `chdir` modifie la résolution de tous les chemins relatifs ;
- ces deux mutations sont définitivement interdites après le premier `babet.workers.spawn()`.

Cette restriction évite des courses de données entre threads.

Vue d'ensemble de l'API

Fonction ou constante	Résultat
`babet.VERSION`	chaîne `major.minor.patch` du runtime
`babet.VERSION_MAJOR`	composante majeure entière
`babet.VERSION_MINOR`	composante mineure entière
`babet.VERSION_PATCH`	composante patch entière
`babet.pid()`	PID du processus, sous forme d'integer
`babet.hostname()`	hostname, ou `(nil, err)`
`babet.uname()`	table système, ou `(nil, err)`
`babet.which(name)`	chemin d'un exécutable, ou `(nil, err)`
`babet.env(name)`	valeur de la variable, ou `nil` si elle est absente
`babet.setenv(name, value)`	`(true, nil)` ou `(nil, err)`
`babet.getMemoryUsage()`	mémoire de l'état Lua courant en octets
`babet.getDetailedMemoryUsage()`	deux integers actuellement identiques

Constantes de version

Les quatre constantes décrivent le binaire Babet qui exécute le script.

```
print(babet.VERSION)       -- par exemple "2.22.0"
print(babet.VERSION_MAJOR) -- par exemple 2
print(babet.VERSION_MINOR) -- par exemple 15
print(babet.VERSION_PATCH) -- par exemple 0
```

`babet.VERSION` correspond à la sortie de :

```
babet --version
```

Les composantes entières permettent de comparer des versions sans parser une chaîne.

```
local function version_at_least(major, minor, patch)
    local current = {
        babet.VERSION_MAJOR,
        babet.VERSION_MINOR,
        babet.VERSION_PATCH,
    }
    local wanted = { major, minor, patch }

    for i = 1, 3 do
        if current[i] ~= wanted[i] then
            return current[i] > wanted[i]
        end
    end
    return true
end

assert(version_at_least(2, 9, 0), "Babet >= 2.9.0 requis")
```

La cohérence suivante est garantie par les tests :

```
assert(babet.VERSION == string.format(
    "%d.%d.%d",
    babet.VERSION_MAJOR,
    babet.VERSION_MINOR,
    babet.VERSION_PATCH
))
```

Identifier le processus et la machine

`babet.pid()`

Renvoie le PID du processus Babet sous forme d'integer strictement positif. L'appel ne peut pas échouer sur un système POSIX supporté.

```
local pid = babet.pid()
print("PID Babet :", pid)
```

Les workers sont des threads du même processus. Ils voient donc le même PID que le thread principal ; `pid()` ne permet pas d'identifier un worker particulier.

```
local main_pid = babet.pid()
local worker = assert(babet.workers.spawn([[
    return babet.pid()
]]))

local joined, worker_pid = worker:join()
assert(joined and worker_pid == main_pid)
```

`babet.hostname()`

Renvoie le nom d'hôte configuré pour la machine.

```
local host, err = babet.hostname()
assert(host, err)
print("Machine :", host)
```

Signature :

```
local host, err = babet.hostname()
```

Résultats :

— succès : `host` est une chaîne non vide et `err` vaut `nil` ;
— erreur système : `(nil, "hostname: ...")`.

Le hostname n'est pas nécessairement un nom DNS pleinement qualifié. Il peut être un nom court défini localement.

`babet.uname()`

Renvoie les cinq champs POSIX principaux de `uname(2)` dans une table.

```
local info, err = babet.uname()
assert(info, err)

print("Système :", info.sysname)
print("Noeud    :", info.nodename)
print("Noyau    :", info.release)
print("Version  :", info.version)
print("Machine  :", info.machine)
```

Table renvoyée :

```
{
    sysname  = "Linux",
    nodename = "station",
    release  = "6.12.0-arch1-1",
    version  = "#1 SMP PREEMPT_DYNAMIC ...",
    machine  = "x86_64",
}
```

Champ	Signification
`sysname`	nom du système d'exploitation ou du noyau
`nodename`	nom du noeud réseau, souvent proche du hostname
`release`	version de publication du noyau
`version`	chaîne de build détaillée du noyau
`machine`	architecture matérielle exposée par le noyau

En cas d'échec de `uname(2)`, la fonction renvoie `(nil, "uname: ...")`. Cette erreur est extrêmement rare, mais elle fait partie du contrat réel.

Rechercher un exécutable

Signature :

```
local path, err = babet.which(name)
```

`name` doit être une chaîne sans octet NUL.

Recherche dans `PATH`

Lorsque `name` ne contient aucun `/`, Babet parcourt la variable `PATH` de gauche à droite et renvoie le premier candidat qui est :

- un fichier régulier ;
- exécutable par le processus courant.

```
local shell, err = babet.which("sh")
assert(shell, err)
print(shell) -- généralement /usr/bin/sh ou /bin/sh
```

Le chemin renvoyé est normalisé en chemin absolu lorsque le système le permet.

Pour vérifier uniquement la présence d'un programme :

```
if babet.which("ffmpeg") then
    print("ffmpeg est disponible")
end
```

`PATH` est lu au moment de l'appel. Une modification effectuée avec `setenv` avant les workers et le chargement GTK est donc prise en compte par les appels suivants.

```
local old_path = babet.env("PATH")
assert(babet.setenv("PATH", "/opt/mon-app/bin:/usr/bin"))
local tool = babet.which("mon-outil")
```

Une composante vide de `PATH`, comme dans `:/usr/bin` ou `/bin::/usr/bin`, représente le répertoire courant conformément à la convention Unix.

Chemin direct

Lorsque l'argument contient un `/`, `PATH` n'est pas consulté. Le chemin est testé directement.

```
local sh, err = babet.which("/bin/sh")
assert(sh, err)
```

Un chemin relatif contenant `/` est également accepté :

```
local tool, err = babet.which("./bin/mon-outil")
assert(tool, err)
```

Un symlink valide vers un fichier régulier exécutable est accepté, car l'inspection suit sa cible.

Programme absent ou non exécutable

Si aucun exécutable valide n'est trouvé :

```
local path, err = babet.which("programme-introuvable")
-- path == nil
-- err  == "which: 'programme-introuvable' not found in PATH"
```

Un fichier présent mais non exécutable est traité comme introuvable :

```
local path, err = babet.which("./script-sans-bit-x")
assert(path == nil)
print(err)
```

Sont également refusés :

- les dossiers ;
- les symlinks cassés ;
- les fichiers non réguliers ;
- les fichiers auxquels le processus n'a pas le droit d'exécution.

Variables d'environnement

`babet.env(name)`

Lit une variable d'environnement du processus.

```
local home = babet.env("HOME")
if home then
    print("HOME :", home)
end
```

Signature :

```
local value = babet.env(name)
```

Résultats :

- variable définie : sa valeur sous forme de chaîne ;
- variable absente : `nil` seul, sans message d'erreur.

Cela permet l'idiome :

```
local port = babet.env("APP_PORT") or "8080"
```

`name` doit être une chaîne. Les formes suivantes lèvent une erreur Lua :

```
-- babet.env()       -- argument manquant
-- babet.env(42)     -- aucune conversion automatique
-- babet.env({})     -- mauvais type
-- babet.env("A\0B") -- NUL embarqué
```

`babet.setenv(name, value)`

Crée une variable d'environnement ou remplace sa valeur existante.

```
local ok, err = babet.setenv("APP_MODE", "production")
assert(ok, err)
assert(babet.env("APP_MODE") == "production")
```

Un second appel écrase la valeur :

```
assert(babet.setenv("APP_MODE", "test"))
assert(babet.env("APP_MODE") == "test")
```

`name` et `value` doivent être de vraies chaînes Lua.

Erreurs d'appelant, qui lèvent une erreur Lua :

```
-- babet.setenv()
-- babet.setenv("APP_MODE")
-- babet.setenv(42, "test")
-- babet.setenv("APP_MODE", 42)
```

Erreurs renvoyées sous la forme `(nil, err)` :

```
local ok, err = babet.setenv("BAD=NAME", "x")
-- ok == nil ; err indique un nom invalide
```

Un nom vide, un nom contenant `=` ou un octet NUL est refusé. Un octet NUL dans la valeur est également refusé.

Valeur vide et variable absente

Une valeur vide reste une valeur définie :

```
assert(babet.setenv("APP_OPTION", ""))
assert(babet.env("APP_OPTION") == "")
```

Elle est donc différente d'une variable absente :

```
local value = babet.env("VARIABLE_ABSENTE")
assert(value == nil)
```

Babet n'expose pas encore `unsetenv`. Il n'est pas possible de retirer une variable avec `setenv(name, nil)` : `nil` est un mauvais type et l'appel lève une erreur Lua.

Interaction avec les workers

`setenv` est autorisé uniquement avant le premier `workers.spawn()` et avant la première tentative de chargement GTK (`gui.available()` ou `gui.init()`).

```
assert(babet.setenv("APP_MODE", "production"))

local worker = assert(babet.workers.spawn([[
    return babet.env("APP_MODE")
]]))

local joined, value = worker:join()
assert(joined and value == "production")
```

Après le premier spawn, même si le worker est déjà terminé :

```
local ok, err = babet.setenv("APP_MODE", "test")
-- ok == nil
-- err explique que l'environnement partagé ne peut plus être modifié
```

L'interdiction est permanente pour la durée du processus, y compris si le premier spawn a échoué. Prépare donc `PATH`, les variables d'application et le répertoire courant avant de lancer les workers ou de charger GTK. Un échec de chargement ou d'initialisation GTK conserve aussi le verrouillage. Consulter les valeurs avec `babet.env` reste possible ; voir le contrat GUI.

Mémoire de la VM Lua

Les deux fonctions effectuent d'abord un cycle complet du ramasse-miettes Lua. Le résultat reflète donc la mémoire encore utilisée après collecte, et l'appel peut provoquer une pause proportionnelle à la taille de l'état Lua.

`babet.getMemoryUsage()`

Renvoie le nombre d'octets actuellement comptabilisés par le gestionnaire de mémoire de l'état Lua courant.

```
local bytes = babet.getMemoryUsage()
print(string.format("Lua utilise %d octets", bytes))
```

Le résultat est un integer positif.

Exemple de comparaison avant/après :

```
local before = babet.getMemoryUsage()

local values = {}
for i = 1, 10000 do
    values[i] = string.rep("x", 100)
end

local after = babet.getMemoryUsage()
print("Variation Lua :", after - before, "octets")
```

La mesure n'est pas un outil de profiling précis : l'allocateur, l'internement des chaînes et les optimisations du GC peuvent faire varier le résultat.

`babet.getDetailedMemoryUsage()`

Renvoie deux integers :

```
local used, total = babet.getDetailedMemoryUsage()
```

Dans la version actuelle, les deux valeurs sont **strictement identiques** :

```
assert(used == total)
```

La deuxième valeur est conservée uniquement pour la stabilité de l'API. Lua n'expose pas séparément une mesure fiable de fragmentation ou de mémoire réservée mais inutilisée.

Pour un nouveau code qui n'a besoin que d'une mesure, préfère :

```
local bytes = babet.getMemoryUsage()
```

Ce qui est mesuré et ce qui ne l'est pas

La mesure couvre la mémoire gérée par **l'état Lua qui exécute l'appel**.

Elle ne représente pas :

— le RSS total du processus ;
— la pile native ;
— les allocations C++ du runtime ;
— les buffers OpenSSL, SQLite, HTTP ou sockets ;
— le code de l'exécutable et les bibliothèques partagées ;
— les états Lua des autres workers.

Dans un worker, la fonction mesure uniquement l'état Lua isolé de ce worker. Pour mesurer la mémoire globale du processus, utilise un outil système comme `/proc`, `ps`, `smem`, Valgrind ou un profiler adapté.

Contrat d'erreur

Situation	Comportement
argument manquant ou mauvais type	erreur Lua, récupérable avec `pcall`
NUL dans `env`, `setenv` ou `which`	erreur Lua pour `env`, `(nil, err)` pour `setenv` et `which`
variable absente dans `env`	`nil` seul
exécutable introuvable	`(nil, err)`
erreur de `hostname` ou `uname`	`(nil, err)`
`setenv` invalide ou refusé après un worker	`(nil, err)`
`pid`	toujours un integer
fonctions mémoire	toujours un ou deux integers après un GC complet

Une exception C++ inattendue dans `which`, `env`, `setenv`, `hostname`, `uname` ou `pid` est arrêtée par la frontière commune et devient `(nil, "sys: out of memory")`, `(nil, "sys: internal failure")` ou `(nil, "sys: unknown internal failure")`.

Il s'agit d'une frontière de sûreté interne, pas d'un nouvel état runtime attendu. Le retour ordinaire d'une variable `env` absente reste un seul `nil`, et `pid` reste un seul integer.

Exemple de traitement d'un échec runtime :

```lua
local tool, err = babet.which("outil-optionnel")
if not tool then
    print("Fonction désactivée :", err)
end
```

Exemple de traitement d'une mauvaise signature :

```lua
local ok, err = pcall(function()
    return babet.env(42)
end)

if not ok then
    print("Erreur de programmation :", err)
end
```

Décisions et limites

- Les fonctions historiques restent directement sous `babet` pour ne pas casser les scripts existants.
- `which` renvoie le premier exécutable trouvé, pas la liste de tous les candidats.
- `which` respecte le `PATH` du processus et les droits du processus courant ; son résultat peut donc différer selon l'utilisateur ou l'environnement.
- `env` distingue une variable absente (`nil`) d'une variable définie à la chaîne vide (`""`).
- `setenv` écrase toujours une valeur existante.
- Aucune fonction `unsetenv` ni dump complet de l'environnement n'est exposé.
- `pid` est le PID du processus, commun à tous les workers.
- Les fonctions mémoire provoquent volontairement un GC complet et ne mesurent pas le processus entier.
- Pour les comptes système, consulte le module User - utilisateurs système.

babet tables — fusion et copie de tables Lua

Deux helpers historiques vivent directement dans la table `babet` :

- `babet.mergeTables(...)` construit une nouvelle table à partir de plusieurs sources ;
- `babet.deepCopyTable(t)` copie récursivement les **valeurs** qui sont des tables.

Ils travaillent sur les entrées réellement stockées dans les tables Lua. Les métaméthodes `__pairs` et `__index` ne fournissent donc pas de champs virtuels à copier ou à fusionner.

Table des matières du module

- API
- `babet.mergeTables(t1, t2, ...)`
  - Clés entières positives : concaténation
  - Toutes les autres clés : dernier écrivain gagnant
  - Fusion superficielle
- `babet.deepCopyTable(t)`
  - Valeurs et graphe partagé
  - Les clés ne sont pas copiées
  - Métatables et accès brut
  - Profondeur maximale
- Contrat d'erreur
- Limites et choix de conception
- Hors v1

API

Fonction	Renvoie
`babet.mergeTables(t1, t2, ...)`	une nouvelle table
`babet.deepCopyTable(t)`	une nouvelle table

Les deux fonctions renvoient exactement une valeur en cas de succès. Elles ne renvoient jamais `(nil, err)` : une erreur d'appel ou de profondeur est levée.

`babet.mergeTables(t1, t2, ...)`

Au moins deux arguments sont obligatoires et chacun doit être une table. Il n'existe pas de limite d'arité propre à l'API au-delà des limites générales de Lua.

Les sources ne sont pas modifiées. Le résultat est une table ordinaire sans métatable, même si les sources en possèdent une.

Clés entières positives : concaténation

Toute clé que Lua représente comme un entier `>= 1` est traitée comme une position de liste. Cela inclut une clé écrite `2.0`, que Lua canonicalise en entier.

Pour chaque source, ces clés sont parcourues par ordre numérique croissant, puis leurs valeurs sont ajoutées à la suite du résultat. Les trous et les indices d'origine sont donc compactés :

```lua
local r = babet.mergeTables(
    { [4] = "d", [2] = "b" },
    { [7] = "g", [1] = "a" }
)

-- r == { "b", "d", "a", "g" }
```

Une même position présente dans plusieurs sources n'est pas écrasée : chaque valeur est ajoutée.

Toutes les autres clés : dernier écrivain gagnant

Les clés chaînes, booléennes, tables, fonctions, threads, userdata, light userdata ainsi que les clés numériques flottantes, nulles ou négatives conservent leur identité. Une source ultérieure écrase la valeur précédente pour la même clé :

```lua
local key = {}
local r = babet.mergeTables(
    { mode = "safe", [key] = 1, [0] = "a" },
    { mode = "fast", [key] = 2, [0] = "b" }
)

-- r.mode == "fast"
-- r[key]  == 2
-- r[0]    == "b"
```

Fusion superficielle

Les valeurs ne sont jamais copiées. Une sous-table placée dans le résultat est la même table que dans la source :

```lua
local nested = { enabled = false }
local r = babet.mergeTables({ nested = nested }, {})
```

```lua
r.nested.enabled = true
-- nested.enabled == true
```

Pour obtenir ensuite un graphe indépendant pour les valeurs-table, utilise :

```lua
local isolated = babet.deepCopyTable(
    babet.mergeTables(defaults, overrides)
)
```

`babet.deepCopyTable(t)`

La fonction exige exactement une table. Elle crée une nouvelle table pour la racine et pour chaque **valeur** qui est elle-même une table.

Valeurs et graphe partagé

Les cycles parcourus par les valeurs sont pris en charge et l'identité du graphe est préservée :

```lua
local shared = { value = 42 }
local source = { a = shared, b = shared }
source.self = source

local copy = babet.deepCopyTable(source)

-- copy ~= source
-- copy.a ~= shared
-- copy.a == copy.b
-- copy.self == copy
```

Les valeurs non-table — nombres, chaînes, booléens, fonctions, threads, userdata et light userdata — sont réutilisées telles quelles.

Les clés ne sont pas copiées

Toutes les clés conservent leur valeur et leur identité d'origine. En particulier, une table utilisée comme clé reste la table source :

```lua
local key = { id = 1 }
local source = {
    [key] = "value",
    key_as_value = key,
}
local copy = babet.deepCopyTable(source)

-- copy[key] == "value"          -- clé originale conservée
-- copy.key_as_value ~= key      -- valeur-table copiée
-- copy[copy.key_as_value] == nil
```

La fonction est donc une copie profonde des **valeurs-table**, pas une copie structurelle des clés-table.

Métatables et accès brut

La métatable réelle de chaque table copiée est attachée à la copie **par référence**. Elle n'est pas dupliquée. Modifier cette métatable affecte donc la source et la copie.

Seules les paires réellement stockées sont parcourues. Un champ obtenu par `__index` ou inventé par `__pairs` n'est pas matérialisé dans la copie. Une fois la métatable partagée attachée, ses comportements restent naturellement actifs sur la copie.

Profondeur maximale

La racine est à la profondeur `0`. Jusqu'à **75 descentes vers de nouvelles valeurs-table** sont acceptées, soit au maximum 76 tables sur un chemin racine comprise. Une 76e descente vers une table encore inconnue lève :

```
Table is too deep to copy (max depth 75 exceeded)
```

Une référence vers une table déjà copiée ne crée pas un nouveau niveau : un cycle peut donc se refermer exactement à la limite.

Contrat d'erreur

Les erreurs sont levées via l'API Lua :

- `mergeTables` avec moins de deux arguments ;
- tout argument non-table de `mergeTables` ;
- nombre d'arguments différent de un pour `deepCopyTable` ;
- argument non-table de `deepCopyTable` ;
- profondeur maximale dépassée.

```lua
local ok, err = pcall(babet.deepCopyTable, 42)
-- ok == false ; err contient "Argument must be a table"
```

Limites et choix de conception

- `mergeTables` n'est pas un deep merge : une sous-table ultérieure remplace la référence précédente en bloc.
- Les métatables des sources sont ignorées par `mergeTables` ; son résultat n'en possède aucune.
- `deepCopyTable` partage les métatables et les clés-table avec la source.
- Une métatable faible (`__mode`) reste faible sur la copie puisqu'elle est partagée.
- L'ordre des clés de map n'est pas défini. Seul l'ordre des clés entières positives de `mergeTables` est garanti.
- `mergeTables` copie directement les clés entières positives denses `1..n`. Les clés positives creuses sont triées dans un stockage temporaire possédé par Lua, récupérable même après une erreur d'allocation. Pour une source de `m` entrées dont `k` clés entières positives, le parcours coûte O(m) si ces clés sont denses, sinon O(m + k log k), avec O(k) de stockage temporaire.

Hors v1

Des fonctions comme `deepMergeTables` ou une comparaison structurelle profonde pourraient être ajoutées séparément. Elles ne font pas partie de l'API actuelle.

babet time - horloges, sommeil, dates et durées

Babet expose deux horloges sub-seconde, un sommeil interruptible et des utilitaires simples pour les timestamps ISO 8601 et les durées compactes.

Les fonctions historiques restent disponibles à la racine :

- `babet.monotonic()` ;
- `babet.now()` ;
- `babet.sleep(amount [, unit])`.

Les mêmes fonctions existent sous `babet.time`, avec quatre utilitaires supplémentaires : `iso`, `parse_iso`, `parse_duration` et `format_duration`.

Table des matières du module

— API
— monotonic() et now()
— sleep(amount [, unit])
— babet.time.iso([ts])
— babet.time.parse_iso(text)
— babet.time.parse_duration(text)
— babet.time.format_duration(seconds)
— Contrat d'erreur récapitulatif
— Limites et choix de conception

API

Fonction	Résultat
`babet.monotonic()`	`number`, ou exceptionnellement `(nil, err)` si `clock_gettime` échoue
`babet.now()`	`number`, ou exceptionnellement `(nil, err)` si `clock_gettime` échoue
`babet.sleep(amount [, unit])`	`(true, nil)` ou `(nil, err)`
`babet.time.iso([ts])`	`string` UTC
`babet.time.parse_iso(text)`	`(integer, nil)` ou `(nil, "parse_iso: ...")`
`babet.time.parse_duration(text)`	`(integer, nil)` ou `(nil, "parse_duration: ...")`
`babet.time.format_duration(seconds)`	`string` compacte, par exemple `"1d2h3m4s"`

Les alias `babet.time.now`, `babet.time.monotonic` et `babet.time.sleep` utilisent exactement les mêmes bindings et le même contrat que leurs noms à la racine.

monotonic() et now()

```lua
local started = babet.monotonic()
do_something()
local elapsed = babet.monotonic() - started
print(string.format("durée : %.3f s", elapsed))

local timestamp = babet.now()
print(timestamp)
```

`monotonic()` repose sur `CLOCK_MONOTONIC`. Sa valeur part d'une origine arbitraire, généralement liée au démarrage du système. Elle sert à mesurer une durée, car elle ne subit pas les sauts de l'horloge civile.

`now()` repose sur `CLOCK_REALTIME`. Il renvoie le temps POSIX en secondes depuis le 1er janvier 1970 UTC, avec une partie fractionnaire. Il convient aux timestamps, mais pas à la mesure d'une durée : l'horloge civile peut être corrigée.

Les deux fonctions :

— n'acceptent aucun argument ;
— renvoient une seule valeur `number` en fonctionnement normal ;
— peuvent théoriquement renvoyer `(nil, err)` si l'appel système `clock_gettime` échoue.

sleep(amount [, unit])

```lua
assert(babet.sleep(250, "ms"))
assert(babet.time.sleep(0.5))       -- 0,5 seconde
```

```lua
assert(babet.sleep(1000, "us"))
```

`amount` doit être un véritable `number` Lua, fini et positif ou nul. Les floats sont acceptés. Une chaîne numérique comme `"100"` est refusée.

`unit` doit être une véritable `string` Lua :

Unité	Signification
`"s"`	secondes, valeur par défaut
`"ms"`	millisecondes
`"us"`	microsecondes

Une durée nulle réussit immédiatement. Une unité textuelle inconnue conserve le comportement historique suivant :

```lua
local ok, err = babet.sleep(1, "minutes")
-- ok == nil
-- err == "Invalid time unit"
```

Si un signal géré par `babet.signal` arrive pendant l'attente, son callback Lua est dispatché puis `sleep` renvoie `(nil, "interrupted")`. Un signal non géré provoquant `EINTR` ne raccourcit pas l'attente : Babet reprend avec le temps restant.

Une autre erreur de `nanosleep` renvoie `(nil, "sleep: <description>")`.

babet.time.iso([ts])

`iso` formate une seconde Unix en UTC :

```lua
print(babet.time.iso(0))
-- 1970-01-01T00:00:00Z

print(babet.time.iso())
-- heure courante, à la seconde entière
```

Sans argument, `iso()` utilise l'heure courante. Avec un argument :

- `ts` doit être un véritable `number` Lua ;
- un entier ou un float fini est accepté ;
- une fraction est arrondie vers moins l'infini ;
- la valeur doit tenir dans un entier signé 64 bits.

L'arrondi est important avant l'époque Unix :

```lua
babet.time.iso(0.9)   -- 1970-01-01T00:00:00Z
babet.time.iso(-0.5)  -- 1969-12-31T23:59:59Z
```

Pour les années usuelles, le résultat possède la forme `YYYY-MM-DDTHH:MM:SSZ`. Les timestamps extrêmes valides en `int64` peuvent produire une année signée de plus de quatre chiffres. Ce résultat étendu n'est pas forcément accepté par `parse_iso`, dont la grammaire impose une année sur exactement quatre chiffres.

`iso` n'applique ni locale ni fuseau local et ne conserve pas les fractions de seconde.

babet.time.parse_iso(text)

Le parseur accepte volontairement un sous-ensemble strict et déterministe :

```
YYYY-MM-DDTHH:MM:SSZ
YYYY-MM-DD HH:MM:SSZ
```

`YYYY-MM-DDTHH:MM:SS.fraction+HH:MM`
`YYYY-MM-DDTHH:MM:SS.fraction-HH:MM`

Règles exactes :

- `text` doit être une véritable `string` Lua ;
- l'année comporte exactement quatre chiffres et va de `0001` à `9999` ;
- le calendrier grégorien est validé, y compris les années bissextiles ;
- le séparateur est `T` majuscule ou un espace ;
- heures `00..23`, minutes `00..59`, secondes `00..59` ;
- les secondes intercalaires `:60` sont refusées ;
- la fraction est facultative, mais le point doit être suivi d'au moins un chiffre ; tous ses chiffres sont ignorés, sans arrondi ;
- la timezone est obligatoire : `Z` majuscule, `+HH:MM` ou `-HH:MM` ;
- les offsets acceptés vont de `00:00` à `23:59` ;
- `+0200`, `+02`, `z`, les espaces en début ou en fin et les données supplémentaires sont refusés ;
- un octet NUL embarqué n'est jamais tronqué : il rend la chaîne invalide.

Exemple :

```
local ts = assert(babet.time.parse_iso(
    "2026-06-17T10:00:00+02:00"))

assert(ts == babet.time.parse_iso("2026-06-17T08:00:00Z"))
```

Une chaîne syntaxiquement ou calendriquement invalide renvoie `(nil, "parse_iso: ...")`. Un mauvais type ou un mauvais nombre d'arguments lève une erreur Lua.

`babet.time.parse_duration(text)`

Le format est une suite sans espace de couples `nombre + unité` :

```
assert(babet.time.parse_duration("45s") == 45)
assert(babet.time.parse_duration("1h30m") == 5400)
assert(babet.time.parse_duration("2d12h") == 216000)
```

Unités disponibles :

Unité	Secondes
`d`	86400
`h`	3600
`m`	60
`s`	1

Les unités doivent apparaître au plus une fois et dans l'ordre strict `d > h > m > s`. Les signes, décimales, espaces et unités `w`, `ms`, `us` ou `ns` sont refusés. Un octet NUL embarqué n'est pas tronqué et provoque également une erreur de parsing.

Les composantes ne sont pas normalisées : `"1h90m"` est valide et vaut 9000 secondes. Les zéros de tête et les composantes nulles ordonnées sont également acceptés, par exemple `"0001s"` et `"0d0h0m0s"`.

Le total doit rester compris entre `0` et `math.maxinteger` sur la configuration 64 bits de Babet. Les dépassements renvoient une erreur de parsing.

Une chaîne invalide renvoie `(nil, "parse_duration: ...")`. Un mauvais type ou un mauvais nombre d'arguments lève une erreur Lua.

babet.time.format_duration(seconds)

```
babet.time.format_duration(0)      -- "0s"
babet.time.format_duration(90)     -- "1m30s"
babet.time.format_duration(90061)  -- "1d1h1m1s"
```

`seconds` doit être un véritable `number` Lua, entier et positif ou nul. Un float de valeur entière comme `3.0` est accepté ; `3.7`, `"3"`, NaN, Inf et les valeurs négatives lèvent une erreur Lua.

La sortie est la représentation canonique utilisée par Babet :

— unités dans l'ordre `d`, `h`, `m`, `s` ;
— composantes nulles omises ;
— zéro représenté par `"0s"`.

Pour tout entier valide `n >= 0` :

```
assert(babet.time.parse_duration(
    babet.time.format_duration(n)) == n)
```

L'inverse n'est pas textuellement garanti, car le parseur accepte aussi des formes non normalisées comme `"90m"` ou `"0h"`.

Contrat d'erreur récapitulatif

Situation	Comportement
Mauvais type ou mauvaise arité	erreur Lua
NaN, Inf, durée négative ou valeur numérique hors plage	erreur Lua
Unité textuelle inconnue dans `sleep`	`(nil, "Invalid time unit")`
Signal géré pendant `sleep`	`(nil, "interrupted")` après dispatch du callback
Texte ISO ou durée invalide	`(nil, "parse_iso: ...")` ou `(nil, "parse_duration: ...")`
Échec système d'horloge ou de sommeil	`(nil, err)`

Limites et choix de conception

— `monotonic()` sert aux durées ; `now()` sert aux timestamps.
— `iso` est un formateur UTC fixe, pas un remplaçant général de `os.date`.
— Aucun fuseau nommé, changement d'heure, locale ou calcul calendaire n'est fourni.
— Les fractions ISO sont ignorées et ne sont pas renvoyées par `iso`.
— `parse_duration` manipule des secondes entières uniquement.
— `CLOCK_BOOTTIME`, les durées négatives et les unités semaine/mois/année ne sont pas exposés.

TLS — connexion chiffrée directe, STARTTLS, certificats et SNI

Le support TLS de Babet repose sur les mêmes userdata que `babet.socket`. Après le handshake, les méthodes `send`, `recv`, `recv_line`, `recv_all`, `set_timeout`, `peer`, `sockname` et `close` restent identiques.

Deux modes sont disponibles :

— `babet.socket.connect_tls()` pour une connexion TLS dès le premier octet ;
— `sock:starttls()` pour transformer en place une connexion TCP après une négociation applicative en clair.

Le module couvre :

— TLS 1.2 minimum, avec TLS 1.3 négocié lorsqu'il est disponible ;

— la vérification du certificat activée par défaut ;
— la vérification du hostname ;
— SNI, indépendamment de l'activation de la vérification ;
— les autorités système et des CA supplémentaires par appel ;
— un timeout global connexion TCP + handshake pour `connect_tls` ;
— un comportement fail-closed après le début d'un STARTTLS raté.

Il ne fournit pas de serveur TLS, de certificat client, d'ALPN, de pinning de clé publique, d'OCSP applicatif ni de réglage fin des suites cryptographiques.

Table des matières du module

— Conventions essentielles
— Vue d'ensemble de l'API
— Options TLS
— Connexion TLS directe
— Transformer une connexion avec STARTTLS
— Vérification, hostname et SNI
— Autorités de certification
— Versions TLS
— Timeouts
— État du socket après une erreur
— Exemples complets
— Contrat d'erreur
— Sécurité et limites

Conventions essentielles

Vérification activée par défaut

Cette forme vérifie la chaîne de certification et l'identité du serveur :

```
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
```

`verify = false` est un opt-out explicite destiné aux tests ou à un protocole qui applique une autre authentification forte. Il désactive **à la fois** la validation de chaîne et la vérification du hostname :

```
local sock = assert(babet.socket.connect_tls("127.0.0.1", 8443, {
    verify = false,
    timeout = 5,
}))
```

Cette option chiffre le transport, mais ne protège pas contre un attaquant actif de type homme-du-milieu.

Même API de flux après le handshake

```
local sock = assert(babet.socket.connect_tls("irc.example.net", 6697, {
    timeout = 10,
}))
assert(sock:set_timeout(120))
assert(sock:send("PING :hello\r\n"))
local line = assert(sock:recv_line())
```

Le timeout du handshake et le timeout d'I/O sont distincts.

SNI n’est pas la vérification

SNI indique au serveur quel virtual host sélectionner. La vérification décide si le certificat reçu est digne de confiance et correspond au nom attendu.

Babet envoie SNI lorsqu’un hostname DNS est disponible, même avec `verify = false`. Désactiver la vérification ne doit pas empêcher un serveur multi-sites de choisir le bon certificat ou le bon endpoint.

Vue d’ensemble de l’API

```
local sock, err = babet.socket.connect_tls(host, port, opts?)
local ok, err = sock:starttls(opts?)
```

Les deux formes utilisent la même table d’options :

```
{
    verify = true,
    hostname = "example.com",
    ca_cert = "/path/ca-bundle.pem",
    ca_path = "/path/certs",
    min_version = "1.2",
    timeout = 10,
}
```

Champ	Type	Défaut	Rôle
`verify`	booléen strict	`true`	vérifie la chaîne et le hostname
`hostname`	string	host de `connect_tls`	nom attendu et nom SNI s’il est DNS
`ca_cert`	string	aucune CA supplémentaire	fichier PEM ajouté au trust store
`ca_path`	string	aucun dossier supplémentaire	dossier OpenSSL de CA hashées
`min_version`	`"1.2"` ou `"1.3"`	`"1.2"`	version TLS minimale
`timeout`	nombre fini `>= 0`	`0`	budget du handshake, en secondes

Les champs inconnus sont actuellement ignorés. Une faute comme `hostnme = "example.com"` ne produit donc pas d’erreur et ne remplace pas `hostname`.

Options TLS détaillées

`verify`

Doit être un vrai booléen Lua :

```
verify = true
verify = false
```

`verify = 1` et `verify = "yes"` sont refusés.

Avec `true`, Babet configure `SSL_VERIFY_PEER` et une vérification d’identité avec `SSL_set1_host`.

`hostname`

Sert à deux choses :

1. identité de référence pour le certificat lorsque `verify = true` ;
2. valeur SNI lorsqu'il s'agit d'un nom DNS.

```
local sock = assert(babet.socket.connect_tls("203.0.113.10", 443, {
    hostname = "api.example.com",
    timeout = 10,
}))
```

Cette forme connecte l'adresse IP fournie mais vérifie le certificat pour `api.example.com` et envoie ce nom en SNI.

Un hostname qui est lui-même une adresse IP est utilisé pour la vérification OpenSSL, mais n'est pas envoyé comme SNI, conformément au rôle DNS du champ `server_name`.

`ca_cert` et `ca_path`

Ces options ajoutent des autorités à celles déjà chargées pour cet appel. Elles ne remplacent pas le trust store système et ne constituent pas du pinning. Cela vaut pour `connect_tls` comme pour `starttls`. Une chaîne vide n'ajoute aucune autorité. À l'inverse, le `ca_cert` non vide de `HTTP` remplace les autorités par défaut pour la requête ; les options de même nom n'ont donc pas une politique identique entre les deux modules.

```
local sock = assert(babet.socket.connect_tls("service.internal", 443, {
    ca_cert = "/etc/myapp/internal-root.pem",
    timeout = 10,
}))
```

`ca_path` doit désigner un répertoire préparé selon les conventions OpenSSL, avec les liens hashés attendus. Un simple dossier rempli de PEM arbitraires ne suffit pas nécessairement.

`min_version`

`"1.2"` et `"1.3"` sont les seules valeurs acceptées. Il s'agit d'une version **minimale**, pas d'une version maximale.

```
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    min_version = "1.3",
    timeout = 10,
}))
```

`timeout`

`0` signifie aucun timeout de handshake. Une valeur positive est arrondie à au moins 1 ms et doit rester représentable en millisecondes système.

`babet.socket.connect_tls(host, port, opts?)`

Établit une connexion TCP puis réalise immédiatement un handshake TLS client.

```
local sock, err = babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
})
if not sock then
    error(err)
end
```

Hostname par défaut

Lorsque `opts.hostname` est absent, le premier argument `host` sert de nom de référence et de SNI s'il s'agit d'un nom DNS.

```lua
-- Vérifie et envoie example.com automatiquement.
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
```

Avec une connexion par IP vers un certificat DNS, passe explicitement le nom :

```lua
local sock = assert(babet.socket.connect_tls("192.0.2.15", 443, {
    hostname = "www.example.com",
    timeout = 10,
}))
```

Deadline globale

Après la résolution DNS, le même budget couvre :

— les tentatives de connexion TCP sur toutes les adresses ;
— le handshake TLS.

Le handshake ne reçoit pas un nouveau budget complet après la connexion. Comme pour `socket.connect`, la résolution DNS synchrone reste hors deadline.

Timeout d'I/O après succès

`opts.timeout` n'est pas conservé dans le socket. Après succès, le timeout d'I/O par défaut vaut `0` :

```lua
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
assert(sock:set_timeout(15))
```

`sock:starttls(opts?)`

Transforme en place un socket TCP connecté après la négociation STARTTLS du protocole applicatif.

```lua
local ok, err = sock:starttls({
    hostname = "mail.example.com",
    timeout = 10,
})
```

En succès, la méthode renvoie exactement `(true, nil)` et toutes les lectures et écritures suivantes passent par TLS.

Préconditions

Le socket doit être :

— ouvert ;
— connecté, pas en écoute ;
— encore en TCP clair ;
— sans octets plaintext conservés dans le buffer de lecture de Babet.

Hostname obligatoire avec vérification

Contrairement à `connect_tls`, `starttls` ne mémorise pas le hostname utilisé lors de la connexion TCP. Avec `verify = true`, `opts.hostname` est donc obligatoire :

```lua
assert(sock:starttls({
    hostname = "smtp.example.com",
```

```
    timeout = 10,
}))
```

Cette omission est refusée **avant** le début du handshake, et le socket TCP reste utilisable en clair.

Consommer le plaintext avant l'upgrade

Si `recv_line()` ou `recv_all()` a déjà retiré des octets du noyau puis a expiré, ces octets restent tamponnés. Babet refuse STARTTLS tant qu'ils ne sont pas consommés :

```
local line, err = sock:recv_line(0.2)
if not line and err == "timeout" then
    -- Finir de consommer/résoudre la réponse plaintext avant starttls.
end
```

Cela empêche de présenter des octets applicatifs en clair au décodeur TLS.

Déroulement applicatif

Babet n'envoie pas la commande `STARTTLS` à ta place. Le script doit suivre le protocole :

1. connexion TCP ;
2. dialogue initial ;
3. commande STARTTLS ;
4. vérification de la réponse positive ;
5. appel à `sock:starttls()` ;
6. reprise du protocole chiffré, souvent avec une nouvelle salutation.

Vérification, hostname et SNI

Les combinaisons importantes sont les suivantes :

Connexion	`verify`	`hostname`	Vérification	SNI
host DNS `example.com`	`true`	absent	`example.com`	`example.com`
IP `192.0.2.5`	`true`	`api.example.com`	`api.example.com`	`api.example.com`
IP `192.0.2.5`	`true`	absent	vérification de l'IP	aucun SNI
host DNS	`false`	absent	aucune	host DNS
IP	`false`	`api.example.com`	aucune	`api.example.com`
IP	`false`	absent	aucune	aucun SNI

Avec `verify = false`, fournir `hostname` reste utile pour un virtual host TLS, même si le certificat n'est pas vérifié.

SNI est envoyé avant que le serveur choisisse son certificat. La vérification du hostname intervient ensuite sur le certificat présenté. Les deux mécanismes sont donc liés dans l'usage courant, mais conceptuellement indépendants.

Autorités de certification

Babet utilise un OpenSSL embarqué statiquement. Le contexte TLS socket tente :

1. les chemins par défaut d'OpenSSL ;
2. les variables `SSL_CERT_FILE` et `SSL_CERT_DIR` prises en compte par OpenSSL ;
3. plusieurs emplacements de CA connus sous Linux et BSD ;

4. les CA supplémentaires `ca_cert` et `ca_path` de l'appel.

Les CA personnalisées sont isolées par connexion. Charger une CA pour un appel ne la rend pas fiable pour les connexions suivantes.

```
local a = assert(babet.socket.connect_tls("internal.example", 443, {
    ca_cert = "/tmp/test-root.pem",
    timeout = 5,
}))
a:close()

-- Cet appel ne réutilise pas /tmp/test-root.pem.
local b, err = babet.socket.connect_tls("internal.example", 443, {
    timeout = 5,
})
```

CA supplémentaire, pas certificat serveur piné

Si `ca_cert` contient une autorité capable de signer plusieurs certificats, tous les certificats valides pour le hostname et cette autorité peuvent être acceptés. Pour du pinning strict, ajoute une vérification applicative externe ; Babet n'expose pas actuellement le certificat pair ni son empreinte.

Versions TLS

Le contexte refuse les protocoles antérieurs à TLS 1.2.

— défaut : minimum TLS 1.2 ;
— `min_version = "1.3"` : minimum TLS 1.3 ;
— TLS 1.3 est négocié automatiquement avec le défaut si le serveur le propose.

Babet ne permet pas de forcer un maximum. La version finale dépend d'OpenSSL et du serveur.

Timeouts

`connect_tls`

```
local sock, err = babet.socket.connect_tls(host, port, {
    timeout = 5,
})
```

Le budget couvre TCP + handshake après DNS.

`starttls`

```
local ok, err = sock:starttls({
    hostname = host,
    timeout = 5,
})
```

Le budget couvre uniquement le handshake STARTTLS. Il ne reprend pas le timeout défini par `sock:set_timeout()`.

I/O après le handshake

```
assert(sock:set_timeout(30))
local line, err = sock:recv_line()
```

Les opérations TLS utilisent un descripteur non bloquant en interne, piloté par OpenSSL et `poll`, afin de respecter la deadline globale même lorsque `SSL_read` ou `SSL_write` réclame une autre direction d'I/O.

État du socket après une erreur

`connect_tls`

Une erreur ne renvoie aucun socket. Les ressources TCP/TLS intermédiaires sont fermées.

`starttls` avant le handshake

Les erreurs suivantes laissent le socket TCP inchangé :

- options invalides ;
- hostname manquant avec `verify = true` ;
- plaintext tamponné ;
- échec de préparation du contexte TLS ou de configuration avant `SSL_connect` ;
- échec de passage non bloquant avant le handshake.

Le script peut corriger le problème et continuer en clair si le protocole le permet.

`starttls` après le début du handshake

Dès que `SSL_connect` a commencé, tout échec ferme définitivement le socket :

- timeout ;
- interruption ;
- certificat invalide ;
- alerte TLS ;
- erreur de protocole ou d'I/O.

Le flux peut déjà contenir un ClientHello ou avoir consommé des octets du pair. Le réutiliser en clair serait ambigu et dangereux. Babet adopte donc un comportement **fail-closed**.

```
local ok, err = sock:starttls(opts)
if not ok then
    -- Si le handshake a commencé, les appels suivants signaleront "closed".
    sock:close() -- reste idempotent
end
```

`send` après une tentative d'écriture TLS

Après le premier `SSL_write` d'un appel `send`, tout abandon (timeout, interruption ou erreur) ferme le transport. OpenSSL peut avoir commencé un record sans encore annoncer d'octets applicatifs écrits : il ne suffit donc pas de tester si un compteur d'envoi est positif. Le diagnostic initial est conservé ; les appels suivants signalent un socket fermé et `close()` reste idempotent. La fermeture précède l'appel d'un éventuel callback de signal.

Un timeout avant toute tentative `SSL_write` laisse le socket utilisable.

Exemples complets

Client HTTPS minimal sur socket brut

Pour une requête HTTP classique, préfère `babet.http`. Cet exemple montre seulement l'API de flux :

```
local S = babet.socket
local sock = assert(S.connect_tls("example.com", 443, { timeout = 10 }))
assert(sock:set_timeout(10))

assert(sock:send(
    "GET / HTTP/1.1\r\n" ..
    "Host: example.com\r\n" ..
    "Connection: close\r\n\r\n"
```

```
))

local response = assert(sock:recv_all(nil, 8 * 1024 * 1024))
print(response)
sock:close()
```

Connexion par IP avec nom de certificat

```
local sock = assert(babet.socket.connect_tls("203.0.113.20", 443, {
    hostname = "api.example.com",
    ca_cert = "/etc/myapp/ca.pem",
    timeout = 5,
}))
```

SMTP STARTTLS simplifié

```
local S = babet.socket
local sock = assert(S.connect("mail.example.com", 587, 5))
assert(sock:set_timeout(10))

local banner = assert(sock:recv_line())
assert(banner:match("^220"), banner)

assert(sock:send("EHLO client.example\r\n"))
repeat
    local line = assert(sock:recv_line())
    if line:match("^250[ -]STARTTLS") then
        -- capability vue
    end
until line:match("^250 ")

assert(sock:send("STARTTLS\r\n"))
local reply = assert(sock:recv_line())
assert(reply:match("^220"), reply)

assert(sock:starttls({
    hostname = "mail.example.com",
    timeout = 10,
}))

-- SMTP demande généralement un nouvel EHLO après STARTTLS.
assert(sock:send("EHLO client.example\r\n"))
```

Cet exemple ne constitue pas un client SMTP complet : il faut parser toutes les réponses multi-lignes, authentifier et gérer les codes d’erreur.

Test local auto-signé

```
local sock = assert(babet.socket.connect_tls("127.0.0.1", 8443, {
    verify = false,
    hostname = "dev.local", -- SNI reste envoyé
    timeout = 3,
}))
```

N’utilise pas cette configuration en production.

Contrat d'erreur

Erreurs Lua levées

```
babet.socket.connect_tls({}, 443)
babet.socket.connect_tls("host", "443")
```

Le host doit être une string stricte et le port un entier Lua strict.

Erreurs `(nil, err)`

— host vide, NUL ou port hors plage ;
— table d'options invalide ;
— mauvais type ou mauvaise valeur d'un champ ;
— CA illisible ou invalide ;
— DNS ou connexion TCP ;
— timeout ou interruption ;
— échec du handshake ;
— vérification de chaîne ou de hostname ;
— précondition STARTTLS non satisfaite.

Les messages OpenSSL peuvent varier selon la version. Teste les états stables comme `"timeout"` ou `"interrupted"`, et journalise le message complet pour le diagnostic.

Les écritures internes à OpenSSL sont protégées contre `SIGPIPE`, y compris pendant le handshake, la lecture et la fermeture. Une erreur de transport est rapportée par l'API au lieu de tuer le programme ; `close()` reste une fermeture best-effort. Le gestionnaire `babet.signal` de l'application n'est pas remplacé.

Sécurité et limites

— **Ne désactive pas `verify` en production.** Le chiffrement sans authentification n'empêche pas un MITM.
— **Passe le vrai hostname attendu.** Ne vérifie jamais un nom fourni par le pair lui-même.
— **CA privée.** Protège le fichier CA et déploie une rotation maîtrisée.
— **STARTTLS stripping.** Le protocole applicatif doit exiger STARTTLS, pas simplement l'utiliser lorsqu'il est annoncé. Un attaquant pourrait supprimer la capability dans un canal clair si ton application accepte un downgrade.
— **Pas d'ALPN.** Babet socket n'est pas adapté à HTTP/2, qui négocie normalement `h2` via ALPN.
— **Pas de certificat client.** Les protocoles mTLS ne sont pas supportés par cette API.
— **Pas d'accès au certificat pair.** Impossible de faire du pinning ou une inspection avancée directement en Lua.
— **Pas de serveur TLS.** `listen()` et `accept()` créent des sockets TCP ; aucune méthode `accept_tls` n'est exposée.
— **Vérification de révocation.** Babet ne fournit pas de politique OCSP/CRL applicative supplémentaire au comportement d'OpenSSL.

`babet.toml` - décodage TOML

Le module `babet.toml` s'appuie sur **toml++ 3.4.0** et décode des documents conformes à **TOML 1.0.0**. Les extensions situées hors du profil TOML 1.0 et prises en charge optionnellement par cette version de toml++ sont désactivées.

La v1 expose uniquement le décodage d'une chaîne. Elle ne lit pas un fichier à ta place et ne fournit pas d'encodeur.

Table des matières du module

— API
— Correspondance des types
  — Entiers et flottants
— Chaînes, UTF-8 et clés
— Arrays et tables
  — Ambiguïté des conteneurs vides
— Dates et heures
— Ordre et informations perdues
— Contrat d'erreur
— Limites
— Non exposé en v1

API

Fonction	Contrat
`babet.toml.decode(text)`	Renvoie `table, nil` ou `nil, err`

`decode` exige exactement **un argument**, qui doit être une véritable chaîne Lua. Un nombre n'est pas converti implicitement en texte et tout argument supplémentaire est refusé.

Pour lire un fichier :

```lua
local file = assert(io.open("config.toml", "rb"))
local text = assert(file:read("*a"))
file:close()

local config, err = babet.toml.decode(text)
if not config then
    error(err)
end
```

Un document TOML possède toujours une table à sa racine. Une chaîne vide ou composée uniquement de commentaires produit donc une table Lua vide.

Correspondance des types

TOML	Lua
string	string
integer signé sur 64 bits	integer Lua
float	number Lua
boolean	boolean
array	table séquentielle `1..n`
table, inline table ou dotted table	table à clés chaînes
array of tables	table séquentielle de tables
local date/time et date-time	string normalisée

TOML ne possède pas de valeur équivalente à `nil` ou au `null` JSON.

Entiers et flottants

Les entiers TOML occupent la plage signée complète sur 64 bits, de `math.mininteger` à `math.maxinteger`, et sont conservés exactement. Les notations décimale, hexadécimale, oc-

tale et binaire produisent toutes un entier Lua. Une valeur hors de cette plage est une erreur de parsing.

Les flottants deviennent des nombres Lua. TOML autorise également les valeurs spéciales suivantes :

```
local values = assert(babet.toml.decode([[
positive = inf
negative = -inf
invalid  = nan
]]))

assert(values.positive == math.huge)
assert(values.negative == -math.huge)
assert(values.invalid ~= values.invalid) -- propriété de NaN
```

Comme en Lua, `NaN` ne se compare égal à aucune valeur, pas même à lui-même.

Chaînes, UTF-8 et clés

Le document TOML doit être en UTF-8 valide. Les chaînes basiques, littérales et multilignes sont décodées avant d'être renvoyées à Lua : les séquences d'échappement TOML deviennent les caractères correspondants.

Les chaînes Lua sont binary-safe. Une séquence TOML comme `\u0000` peut donc produire un octet NUL dans une valeur **ou dans une clé citée**, sans troncature :

```
local value = assert(babet.toml.decode([[
"a\u0000b" = "x\u0000y"
]]))

assert(value["a\0b"] == "x\0y")
```

Un octet NUL brut injecté dans le texte TOML n'est pas traité comme une fin de chaîne C : le parseur voit l'intégralité du buffer et refuse le document. Il en va de même pour une séquence UTF-8 invalide.

Les clés nues, citées et pointées suivent les règles TOML :

```
local value = assert(babet.toml.decode([[
"a.b" = 1
site."google.com" = true
physical.color = "orange"
]]))

assert(value["a.b"] == 1)              -- point littéral dans une clé citée
assert(value.site["google.com"] == true)
assert(value.physical.color == "orange")
```

Une clé citée vide (`""`) est valide en TOML et devient la clé Lua `""`.

Arrays et tables

Les arrays TOML deviennent des séquences Lua indexées à partir de 1. TOML 1.0 autorise les arrays hétérogènes :

```
local value = assert(babet.toml.decode([[
items = [1, "two", true, { name = "three" }]
]]))

assert(value.items[1] == 1)
```

```lua
assert(value.items[2] == "two")
assert(value.items[3] == true)
assert(value.items[4].name == "three")
```

Les sections, inline tables et dotted keys aboutissent toutes à des tables Lua ordinaires. La syntaxe d'origine n'est pas conservée.

Ambiguïté des conteneurs vides

Un array vide `[]` et une table vide `{}` deviennent tous deux une table Lua vide :

```lua
local value = assert(babet.toml.decode([[
a = []
b = {}
]]))

assert(type(value.a) == "table" and next(value.a) == nil)
assert(type(value.b) == "table" and next(value.b) == nil)
```

Aucune métatable ni sentinelle ne mémorise leur type TOML d'origine. Cette perte d'information est sans conséquence pour la lecture courante d'une configuration, mais elle empêche un round-trip fidèle.

Dates et heures

TOML distingue quatre types temporels. Lua n'ayant pas de type date natif, Babet les convertit en chaînes :

Type TOML	Forme Lua
local date	`YYYY-MM-DD`
local time	`HH:MM:SS[.fraction]`
local date-time	`YYYY-MM-DDTHH:MM:SS[.fraction]`
offset date-time	`YYYY-MM-DDTHH:MM:SS[.fraction]Z` ou avec `+/-HH:MM`

```lua
local value = assert(babet.toml.decode([[
date = 1979-05-27
time = 07:32:00.1234
local_dt = 1979-05-27T07:32:00
utc_dt = 1979-05-27T07:32:00Z
]]))
```

La valeur temporelle est conservée, mais sa **graphie** peut être normalisée par toml++ : séparateur canonique, fraction ou notation UTC. Les commentaires et l'orthographe exacte du document ne sont pas disponibles. Après conversion, une date TOML est également indiscernable d'une chaîne TOML contenant le même texte.

Ordre et informations perdues

Une table TOML n'a pas d'ordre sémantique garanti et une table Lua ne doit pas être parcourue en supposant l'ordre du fichier. Le décodage ne conserve pas :

— les commentaires ;
— l'ordre lexical des clés ;
— les espaces et retours à la ligne ;
— le choix entre section, inline table et dotted keys ;
— la graphie d'origine des nombres et des dates ;
— le type d'un conteneur vide.

Le module est donc destiné à **lire les valeurs**, pas à éditer puis réécrire un document TOML à l'identique.

Contrat d'erreur

Une erreur de syntaxe ou de valeur TOML renvoie :

```
local value, err = babet.toml.decode("answer = ")
assert(value == nil)
print(err) -- toml: ... (line ..., col ...)
```

Le message est préfixé par `toml:` et contient la ligne et la colonne indiquées par toml++.

Sont notamment renvoyés sous forme de `(nil, err)` :

— syntaxe invalide ou clé redéfinie ;
— entier hors de la plage signée 64 bits ;
— UTF-8 invalide ou caractère de contrôle interdit ;
— extension non disponible dans le profil TOML 1.0 utilisé.

Les erreurs d'utilisation de l'API lèvent en revanche une erreur Lua :

— argument absent ;
— argument qui n'est pas une chaîne ;
— argument supplémentaire.

Limites

Le document complet est parsé et converti en mémoire. Il n'existe pas d'API de streaming, de `decode_file`, de validation de schéma ni d'accès aux positions source des valeurs. La conversion récursive protège la pile Lua ; une profondeur impossible à convertir est renvoyée comme une erreur `toml:`.

Non exposé en v1

— `babet.toml.encode` ;
— `babet.toml.decode_file` ;
— options de parsing ou activation des extensions hors TOML 1.0 ;
— nœuds conservant commentaires, ordre et positions source ;
— sentinelles distinguant arrays et tables vides ;
— validation de schéma.

USER - rechercher des comptes système par nom ou UID

Le module `babet.user` interroge la base des utilisateurs du système via NSS. Il permet de :

— rechercher un compte par nom ;
— rechercher un compte par UID ;
— tester rapidement l'existence d'un compte ;
— récupérer le nom, l'UID, le GID principal, le champ GECOS, le home et le shell.

Il ne lit jamais `/etc/passwd` directement. Il utilise les mêmes mécanismes de résolution que `id` ou `getent passwd`, et peut donc voir les comptes fournis par LDAP, SSSD, NIS, FreeIPA ou d'autres sources configurées dans `/etc/nsswitch.conf`.

Table des matières du module

— Conventions générales
— Vue d'ensemble de l'API

— Table utilisateur renvoyée
— Rechercher un compte avec `get`
  — Recherche par nom
  — Recherche par UID
  — Compte absent
— Tester l'existence avec `exists`
  — Par nom
  — Par UID
  — Quand préférer `get`
— Validation des arguments
— NSS, workers et sécurité
— Contrat d'erreur
— Décisions et limites

Conventions générales

Sous-table dédiée

Contrairement aux fonctions historiques de SYS ou FS, les fonctions utilisateur sont regroupées dans `babet.user` :

```
local user = babet.user.get("root")
local exists = babet.user.exists("root")
```

Nom ou UID, sans conversion implicite

L'argument accepte exactement :

— une chaîne Lua : recherche par **nom** ;
— un integer Lua non négatif dans la plage de `uid_t` : recherche par **UID**.

Une chaîne numérique reste un nom :

```
-- Recherche un compte dont le nom est littéralement "1000"
local by_name = babet.user.get("1000")

-- Recherche l'UID numérique 1000
local by_uid = babet.user.get(1000)
```

Les floats, booléens, tables, `nil`, UID négatifs et UID trop grands lèvent une erreur Lua. Aucune troncature ni conversion silencieuse n'est effectuée.

Champs toujours présents

Lorsqu'un utilisateur est trouvé, la table contient toujours les six champs documentés. Les champs texte peuvent être la chaîne vide, mais jamais `nil`.

Vue d'ensemble de l'API

Fonction	Résultat
`babet.user.get(name_or_uid)`	table, `(nil, "user not found")` ou `(nil, "user: ...")`
`babet.user.exists(name_or_uid)`	booléen strict pour les résultats NSS ; `(nil, err)` uniquement sur échec inattendu de la frontière C++

Table utilisateur renvoyée

Exemple typique :

```
{
    name  = "www-data",
    uid   = 33,
    gid   = 33,
    gecos = "www-data",
    home  = "/var/www",
    shell = "/usr/sbin/nologin",
}
```

Champ	Type	Signification
`name`	string	nom canonique renvoyé par NSS
`uid`	integer	identifiant numérique de l'utilisateur
`gid`	integer	GID **principal** du compte
`gecos`	string	champ descriptif brut, souvent le nom complet
`home`	string	répertoire personnel déclaré
`shell`	string	shell déclaré pour le compte

`gid` n'est pas la liste des groupes secondaires. Le module ne réalise pas de lookup de groupes et n'expose pas encore les appartenances supplémentaires.

Le champ `gecos` est renvoyé tel quel. Son format historique peut contenir plusieurs valeurs séparées par des virgules, mais de nombreux systèmes y placent simplement un nom libre.

```
local u = assert(babet.user.get("root"))
print("Nom   :", u.name)
print("UID   :", u.uid)
print("GID   :", u.gid)
print("GECOS :", u.gecos)
print("Home  :", u.home)
print("Shell :", u.shell)
```

Rechercher un compte avec get

Signature :

```
local info, err = babet.user.get(name_or_uid)
```

Utilise `get` lorsque tu as besoin des informations du compte ou lorsque tu dois distinguer un utilisateur absent d'une panne NSS.

Recherche par nom

```
local root, err = babet.user.get("root")
assert(root, err)
assert(root.name == "root")
assert(root.uid == 0)
```

Exemple pour préparer un répertoire appartenant à un compte de service :

```
local account, err = babet.user.get("mon-service")
assert(account, err)

assert(babet.mkdir("/var/lib/mon-service"))
assert(babet.setAttributes(
    "/var/lib/mon-service",
```

```
    account.uid,
    account.gid,
    tonumber("750", 8)
))
```

Le nom est transmis à NSS tel quel. Il doit être une chaîne sans octet NUL. Une chaîne vide est une recherche valide au niveau de l'API, mais elle ne correspond normalement à aucun compte et renvoie `user not found`.

Recherche par UID

```
local root, err = babet.user.get(0)
assert(root, err)
assert(root.name == "root")
```

Exemple pour afficher le compte associé à un propriétaire de fichier :

```
local attrs, err = babet.getAttributes("rapport.txt")
assert(attrs, err)

local owner, user_err = babet.user.get(attrs.owner)
if owner then
    print("Propriétaire :", owner.name)
else
    print("UID sans compte résolu :", attrs.owner, user_err)
end
```

L'UID doit être un integer non négatif compatible avec le type système `uid_t`. Sur Linux, la limite est généralement `2^32 - 1`, mais le code utilise la limite réelle de la plateforme au moment de la compilation.

Compte absent

Un compte absent n'est pas une erreur Lua. La fonction renvoie :

```
local info, err = babet.user.get("compte-inexistant")
-- info == nil
-- err  == "user not found"
```

Traitement classique :

```
local account, err = babet.user.get("mon-service")
if not account then
    if err == "user not found" then
        print("Le compte doit être créé")
    else
        print("La résolution NSS a échoué :", err)
    end
end
```

Ne fais pas :

```
-- Mauvais si l'absence est un cas normal : assert lève immédiatement
-- local account = assert(babet.user.get("compte-optionnel"))
```

Tester l'existence avec `exists`

Signature :

```
local present = babet.user.exists(name_or_uid)
```

La fonction renvoie toujours un booléen pour un argument valide.

Par nom

```
if babet.user.exists("www-data") then
    print("Le compte www-data existe")
end
```

Exemple de précondition simple :

```
if not babet.user.exists("mon-service") then
    error("Le compte mon-service doit être créé avant le démarrage")
end
```

Par UID

```
assert(babet.user.exists(0)) -- root sur un système Unix normal
```

Un UID absent renvoie `false` :

```
local present = babet.user.exists(2000000000)
print(present)
```

Quand préférer `get`

`exists` assimile volontairement une erreur NSS à `false`.

Cela rend la fonction pratique pour une branche simple, mais elle ne permet pas de distinguer :

— un compte réellement absent ;
— un annuaire LDAP temporairement indisponible ;
— une erreur d'E/S ;
— un manque de mémoire dans le résolveur.

Pour une décision importante, notamment en administration ou en sécurité, utilise `get` :

```
local account, err = babet.user.get("mon-service")
if account then
    print("Compte disponible")
elseif err == "user not found" then
    print("Compte absent")
else
    error("Impossible d'interroger NSS : " .. err)
end
```

Validation des arguments

Les cas suivants lèvent une erreur Lua, récupérable avec `pcall` :

```
local invalid_calls = {
    function() return babet.user.get() end,
    function() return babet.user.get(nil) end,
    function() return babet.user.get(true) end,
    function() return babet.user.get({}) end,
    function() return babet.user.get(1.5) end,
    function() return babet.user.get(-1) end,
    function() return babet.user.get(8589934592) end,
    function() return babet.user.get("root\0autre") end,
}

for _, call in ipairs(invalid_calls) do
    local ok, err = pcall(call)
    assert(not ok)
```

```
    print(err)
end
```

`exists` applique exactement les mêmes validations :

```
local ok = pcall(function()
    return babet.user.exists(-1)
end)
assert(not ok)
```

Le rejet des NUL évite que `"root\0autre"` soit vu comme `"root"` par `getpwnam_r`.

NSS, workers et sécurité

Sources de comptes

Le résultat dépend de la configuration NSS de la machine. Selon `/etc/nsswitch.conf`, un compte peut provenir de :

- `/etc/passwd` ;
- LDAP ;
- SSSD ;
- NIS ;
- FreeIPA ;
- systemd-userdb ;
- un autre module NSS.

Un script exécuté sur deux machines peut donc obtenir des résultats différents sans que Babet ait changé.

Appels depuis des workers

Le module utilise `getpwnam_r` et `getpwuid_r`, les variantes réentrantes. Il peut être utilisé dans les workers :

```
local worker = assert(babet.workers.spawn([[
    local root, err = babet.user.get("root")
    if not root then
        error(err)
    end
    return root.uid
]]))

local joined, uid = worker:join()
assert(joined and uid == 0)
```

Chaque appel interroge NSS. Babet ne maintient pas de cache applicatif des utilisateurs.

Attention aux décisions de sécurité

Les champs `home` et `shell` sont des données de configuration, pas une preuve que le chemin existe ni que le programme est exécutable.

```
local u = assert(babet.user.get("mon-service"))
local home_is_dir = assert(babet.isDir(u.home))
local shell_path = babet.which(u.shell)
```

De même, la présence d'un compte ne prouve pas qu'il est autorisé à se connecter, qu'un mot de passe est valide ou qu'il possède un groupe secondaire particulier.

Contrat d’erreur

Situation	`get`	`exists`
compte trouvé	table utilisateur	`true`
compte absent	`(nil, "user not found")`	`false`
erreur NSS	`(nil, "user: ...")`	`false`
argument invalide	erreur Lua	erreur Lua
exception C++ inattendue	`(nil, "user: out of memory")`, `(nil, "user: internal failure")` ou `(nil, "user: unknown internal failure")`	même retour de frontière `(nil, err)`

La dernière ligne est distincte d’une erreur NSS. Elle sert uniquement à empêcher une exception C++ inattendue de traverser les frames C de Lua. Pour tous les résultats NSS ordinaires, `exists` renvoie toujours exactement un booléen strict comme indiqué plus haut.

Exemple générique :

```lua
local ok, info, err = pcall(function()
    return babet.user.get("mon-service")
end)

if not ok then
    print("Mauvais appel :", info)
elseif not info then
    print("Lookup impossible :", err)
else
    print("UID :", info.uid)
end
```

Décisions et limites

- — NSS est utilisé au lieu de parser `/etc/passwd`.
- — Les variantes réentrantes `_r` sont utilisées pour rester compatibles avec les workers.
- — Les champs texte sont toujours présents et remplacés par `""` si NSS fournit un pointeur nul.
- — Le buffer NSS est agrandi dynamiquement jusqu’à une limite interne de 64 Kio ; au-delà, `get` renvoie une erreur NSS.
- — `exists` privilégie une interface booléenne simple et masque les erreurs NSS ; utilise `get` lorsqu’un diagnostic est nécessaire. Seule la frontière de sûreté C++ interne peut produire son diagnostic exceptionnel `(nil, err)`.
- — Le module n’expose pas les groupes secondaires.
- — Il n’expose ni `/etc/shadow`, ni mots de passe, ni expiration de compte.
- — Il ne crée, ne supprime et ne modifie aucun utilisateur.
- — Pour créer un compte, un script peut appeler un outil système via `babet.exec`, avec les privilèges appropriés.

WORKERS — threads OS, états Lua isolés, queues et channels

`babet.workers` exécute du code Lua dans de vrais threads POSIX. Chaque worker possède son propre `lua_State`, reçoit une copie sérialisée de ses arguments et communique avec le parent par deux files de messages bornées. Des channels partagés peuvent également relier directement le parent et plusieurs workers, ou plusieurs workers entre eux, sans retransiter par l’inbox/outbox du parent.

Le module couvre :

— le lancement d'un chunk Lua dans un thread OS ;
— le passage d'une table d'arguments initiale ;
— la récupération bloquante ou non bloquante du premier résultat ;
— une inbox parent vers worker et une outbox worker vers parent ;
— des channels directs, bornés, FIFO, multi-producteurs et multi-consommateurs ;
— des timeouts et une contre-pression par capacité de queue ;
— la fermeture avec drainage et le réveil des appels bloqués ;
— le chargement des modules Babet, bundlés et utilisateur dans chaque état ;
— l'isolation de la mémoire Lua entre les threads ;
— un pool borné de workers persistants qui réutilise les pthreads et les états Lua entre plusieurs tâches.

Il ne fournit pas de mémoire Lua partagée, de terminaison forcée ni de channels entre processus OS distincts. L'annulation disponible est strictement coopérative.

Table des matières du module

— Conventions essentielles
— Vue d'ensemble de l'API
— `spawn(code, args?, opts?)`
  — `code`
  — `args` et `worker.args`
  — Capacités des queues
  — Transmission de channels par `opts.channels`
  — Moment où `setenv` et `chdir` sont verrouillés
— Valeurs transférables
  — Scalaires et chaînes
  — Tables listes et objets
  — Valeurs refusées
  — Copies, identité, profondeur et budgets
— Résultat final du worker
  — `status()` — observer sans consommer
  — `done()` — test booléen non consommant
  — `join(timeout?)` — attendre et consommer
  — `poll()` — tester et consommer
  — Ne pas combiner `poll()` terminé puis `join()`
— Messages parent vers worker
— Messages worker vers parent
— API `worker.send`, `worker.recv` et `worker.cancelled`
— Channels directs partagés
  — Créer un channel et choisir sa capacité
  — `send` et `recv`
  — `close` et `is_closed`
  — Concurrence, ordre et durée de vie
— Timeouts et raisons d'échec
— `close()` et cycle de vie des queues
— `cancel()` et annulation coopérative
— Chargement des modules et environnement du worker
— Nombre de CPU disponibles
— Pool borné réutilisable
  — Création et options
  — Soumission et tâches
  — Isolation et réutilisation des états
  — Contre-pression et timeouts
  — Fermeture, annulation et join
  — Channels partagés dans les tâches
— Exemples complets

    - Calcul simple avec `join`
    - Arguments structurés
    - Suivi non bloquant avec `poll`
    - Worker persistant requête/réponse
    - Envoyer et recevoir `nil`
    - Fermeture puis drainage de l'outbox
    - Contre-pression et timeout
    - Parent vers worker avec un channel
    - Worker vers worker sans relais du parent
    - Plusieurs producteurs et consommateurs
    - Annulation coopérative
    - Pool natif pour plusieurs tâches
- Interblocages à éviter
- Contrat d'erreur
- Garbage collector et destruction
- Signaux, environnement et concurrence
- Fonctionnalités absentes

Conventions essentielles

Chaque worker possède un état Lua distinct

Une table, une globale ou un module modifié dans le parent n'est pas partagé avec le worker. Les données traversent uniquement par sérialisation :

- `args` lors de `spawn` ;
- `job:send` et `worker.recv` ;
- `worker.send` et `job:recv` ;
- les méthodes `send` et `recv` d'un channel partagé ;
- la première valeur retournée par le chunk.

Il n'existe donc pas de data race sur les objets Lua eux-mêmes. Les effets externes — fichiers, sockets, bases de données, processus — restent en revanche partagés au niveau du système et doivent être coordonnés par l'application.

Les résultats utilisent une convention proche de `pcall`

`join(timeout?)` renvoie :

```
true, valeur_metier
false, message_erreur
nil, "timeout"
```

Le troisième état signifie uniquement que le worker est encore actif. Le résultat n'est alors ni rejoint ni consommé.

Cette convention permet au worker de retourner légitimement `nil` :

```lua
local ok, value = job:join(0.5)
if ok == true then
    -- value peut être nil.
elseif ok == false then
    io.stderr:write(value, "\n")
else
    assert(value == "timeout")
end
```

Les méthodes de queue renvoient également un booléen en première position, ce qui permet de transporter `nil` sans ambiguïté.

Le résultat final est consommable une seule fois

Dès que `join()` récupère le résultat, ou que `poll()` renvoie `"done"` ou `"error"`, ce résultat est consommé. Tout nouvel appel à `join()` ou `poll()` renvoie un diagnostic `result already consumed`.

Vue d'ensemble de l'API

Côté parent

```
local job, err = babet.workers.spawn(code, args?, opts?)
local channel, err = babet.workers.channel(opts?)
local cpu_count = babet.workers.cpu_count()
local pool, err = babet.workers.pool(opts?)

local state = job:status()
local finished = job:done()
local ok, value_or_reason = job:join(timeout?)
local state, value = job:poll()
local ok, err = job:cancel()

local ok, err = job:send(value, timeout?)
local ok, value_or_err = job:recv(timeout?)
local ok, err = job:close()

local ok, err = channel:send(value, timeout?)
local ok, value_or_reason = channel:recv(timeout?)
local ok, err = channel:close()
local closed = channel:is_closed()
```

API	Résultat principal	Bloquante
`spawn`	`job` ou `(nil, err)`	création seulement
`job:status()`	`"running"`, `"done"` ou `"error"`	non, ne consomme jamais
`job:done()`	booléen	non, ne consomme jamais
`job:join(t?)`	`(true, result)`, `(false, err)` ou `(nil, "timeout")`	selon `t`
`job:poll()`	`("running", nil)`, `("done", result)` ou `("error", err)`	non
`job:cancel()`	`(true, nil)`	non
`job:send(v, t?)`	`(true, nil)`, `(false, reason)` ou `(nil, err)`	selon `t`
`job:recv(t?)`	`(true, value)` ou `(false, reason)`	selon `t`
`job:close()`	`(true, nil)`	non
`workers.channel(opts?)`	`channel` ou `(nil, err)`	création seulement
`channel:send(v, t?)`	`(true, nil)`, `(false, reason)` ou `(nil, err)`	selon `t`
`channel:recv(t?)`	`(true, value)`, `(false, reason)` ou `(nil, err)`	selon `t`
`channel:close()`	`(true, nil)`	non
`channel:is_closed()`	booléen	non
`workers.cpu_count()`	entier positif	non
`workers.pool(opts?)`	`pool` ou `(nil, err)`	crée les workers persistants
`pool:submit(code, args?, t?)`	`task` ou `(nil, reason)`	selon `t`
`task:done()`	booléen	non, ne consomme jamais
`task:status()`	`"running"`, `"done"` ou `"error"`	non

API	Résultat principal	Bloquante
`task:join(t?)`	`(true, result)`, `(false, err)` ou `(nil, "timeout")`	selon `t`
`task:poll()`	`("running", nil)`, `("done", result)` ou `("error", err)`	non
`pool:close(t?)`	`(true, nil)` ou `(false, reason)`	selon `t`
`pool:cancel()`	`(true, nil)`	non ; annulation coopérative
`pool:join(t?)`	`(true, nil)`, `(false, err)` ou `(nil, "timeout")`	selon `t`
`pool:stats()`	`(table, nil)` ou `(nil, err)`	non

Côté worker

Le chunk reçoit une table globale `worker` :

```
worker.args
worker.channels
worker.send(value, timeout?)
worker.recv(timeout?)
worker.cancelled()
```

API	Résultat
`worker.args`	copie de `args`, ou `nil`
`worker.channels`	table des handles transmis par `opts.channels`
`worker.send(v, t?)`	`(true, nil)` ou `(false, reason_or_error)`
`worker.recv(t?)`	`(true, value)` ou `(false, reason)`
`worker.cancelled()`	booléen, sans effet de bord

spawn(code, args?, opts?)

```
local job, err = babet.workers.spawn(code, args?, opts?)
```

En succès, `spawn` renvoie uniquement le userdata `job`. Le thread peut être encore en cours, déjà terminé ou déjà en erreur au moment où le parent reçoit ce userdata.

Les erreurs de chargement ou d'exécution du chunk ont lieu **dans** le worker : `spawn` peut donc réussir, puis `join()` ou `poll()` rapporter l'erreur.

```
local job = assert(babet.workers.spawn("this is not valid Lua"))
local ok, err = job:join()
assert(not ok)
```

code

`code` est obligatoire et doit être une véritable chaîne Lua. Les nombres ne sont pas convertis implicitement.

```
local job = assert(babet.workers.spawn([[
    return 6 * 7
]]))
```

Le code est chargé comme un chunk texte dans un nouvel état Lua. Il ne s'agit pas d'une fonction capturant les upvalues du parent.

```
local factor = 3
```

```lua
-- factor n'existe pas dans le worker : il faut le transmettre dans args.
local job = assert(babet.workers.spawn([[
    return worker.args.value * worker.args.factor
]], {
    value = 14,
    factor = factor,
}))
```

Les octets NUL présents dans `code` ne sont pas tronqués par le binding, mais un NUL rend normalement le chunk invalide pour le parseur Lua. Utilise une chaîne source Lua ordinaire.

`args` et `worker.args`

`args` est facultatif. Il doit être une table ou `nil`.

```lua
local job = assert(babet.workers.spawn([[
    return worker.args.left + worker.args.right
]], {
    left = 20,
    right = 22,
}))
```

Dans le worker :

- `worker.args` contient une copie désérialisée de la table ;
- sans `args`, `worker.args == nil` ;
- modifier `worker.args` ne modifie jamais la table du parent ;
- les métatables et l'identité des sous-tables ne traversent pas ;
- la sérialisation lit les entrées brutes et n'invoque ni `__len` ni `__index`.

Un scalaire direct n'est pas accepté comme deuxième argument :

```lua
babet.workers.spawn("return worker.args", 42) -- erreur Lua levée
```

Emballe-le dans une table :

```lua
babet.workers.spawn("return worker.args.value", { value = 42 })
```

Capacités des queues

`opts` peut définir :

```lua
{
    inbox_capacity = 64,
    outbox_capacity = 64,
}
```

- `inbox_capacity` : nombre maximal de messages parent vers worker ;
- `outbox_capacity` : nombre maximal de messages worker vers parent ;
- défaut : 64 messages pour chaque queue ;
- plage autorisée : entier de `1` à `1 000 000` ;
- la valeur doit être un véritable entier Lua ; la chaîne `"64"` et le flottant `64.0` sont refusés ;
- la taille porte sur le **nombre de messages**, pas sur leur taille en octets.

```lua
local job = assert(babet.workers.spawn(code, nil, {
    inbox_capacity = 8,
    outbox_capacity = 32,
}))
```

Toute clé inconnue de `opts` est refusée par une erreur Lua. Cela détecte immédiatement les fautes de frappe :

```lua
babet.workers.spawn("return 1", nil, {
    inbox_capcity = 16, -- erreur : option inconnue
})
```

Les noms d'options doivent être de vraies chaînes sans suffixe NUL caché ; les clés numériques sont également refusées.

Une capacité plus grande consomme potentiellement davantage de mémoire et ne remplace pas une stratégie de drainage. Une petite capacité applique une contre-pression plus tôt.

Transmission de channels par `opts.channels`

Un channel est une ressource partagée spéciale. Il ne traverse jamais la sérialisation JSON de `worker.args` et ne peut pas être envoyé comme un message. Il doit être transmis explicitement dans le champ `channels` :

```lua
local tasks = assert(babet.workers.channel({ capacity = 16 }))
local results = assert(babet.workers.channel({ capacity = 16 }))

local job = assert(babet.workers.spawn([[
    local ok, task = worker.channels.tasks:recv(2)
    if not ok then return task end

    assert(worker.channels.results:send({
        id = task.id,
        result = task.value * 2,
    }, 2))

    return true
]], nil, {
    channels = {
        tasks = tasks,
        results = results,
    },
}))
```

Dans le nouvel état Lua :

- `worker.channels` existe toujours et vaut une table vide si aucun channel n'a été transmis ;
- chaque clé de `opts.channels` devient un champ de `worker.channels` ;
- plusieurs noms peuvent référencer le même channel ;
- le parent et tous les workers reçoivent des userdata distincts qui pointent vers la même queue C++ ;
- les noms doivent être des chaînes UTF-8 non vides et sans octet NUL ;
- chaque valeur doit être un channel créé par `babet.workers.channel()`.

Exemple avec le même channel sous deux noms :

```lua
local shared = assert(babet.workers.channel())

local job = assert(babet.workers.spawn([[
    assert(worker.channels.output:send("hello"))
    local ok, value = worker.channels.input:recv()
    return ok and value
]], nil, {
    channels = {
        input = shared,
        output = shared,
    },
}))
```

Un channel placé dans `args` reste un userdata non sérialisable et est refusé :

```
local channel = assert(babet.workers.channel())

local job, err = babet.workers.spawn("return true", {
    channel = channel,
})

assert(job == nil)
assert(err:find("userdata", 1, true))
```

Moment où `setenv`, `chdir` et `os.setlocale` sont verrouillés

Après validation des types et capacités, le premier appel à `spawn` marque le processus comme ayant utilisé les workers. À partir de cet instant :

- `babet.setenv(...)` est définitivement refusé ;
- `babet.chdir(...)` est définitivement refusé ;
- `os.setlocale(locale, catégorie)` lève une erreur Lua si `locale` n'est pas `nil`, y compris pour `""` et pour la locale déjà active ;
- la restriction reste active après `join()` ;
- elle reste active même si la sérialisation de `args`, l'initialisation des queues ou `pthread_create` échoue ensuite.

Un appel rejeté **avant** ce marquage — par exemple `spawn(42)` ou une capacité non entière — ne verrouille pas l'environnement.

La première tentative de chargement GTK par `gui.available()` ou `gui.init()` déclenche le même verrouillage, avant l'entrée dans la bibliothèque native, même si le chargement ou l'initialisation échoue. Cette règle couvre aussi les threads créés par GTK sans `workers.spawn` ; voir le contrat GUI.

Configure donc le répertoire courant, l'environnement et la locale avant le premier worker ou appel de chargement GUI. `os.setlocale()` et `os.setlocale(nil, catégorie)` restent disponibles en lecture dans tous les états. Avant le verrouillage, le contrat standard Lua est conservé : un nom de locale en cas de succès, ou un seul `nil` si la locale n'est pas disponible. La validation des arguments reste identique à celle de Lua.

Le verrouillage persiste aussi après destruction et recréation d'un contexte d'intégration. Ces règles protègent les points d'entrée Lua de Babet ; les plugins natifs et les programmes hôtes restent responsables de leurs propres mutations d'état global et de leurs threads.

Valeurs transférables

Le format de transport interne est JSON. Les mêmes règles s'appliquent à :

- `args` ;
- les messages dans les deux sens ;
- les messages envoyés dans les channels ;
- la première valeur de retour du worker.

Scalaires et chaînes

Valeur Lua	Transférable	Remarque
`nil`	oui	devient JSON `null`, puis redevient `nil`
booléen	oui	identité préservée
entier	oui	redevient normalement un entier Lua

Valeur Lua	Transférable	Remarque
nombre flottant fini	oui	`NaN`, `+Inf` et `-Inf` refusés
string texte	oui	doit passer le validateur UTF-8 et ne contenir aucun NUL

Contrairement à plusieurs APIs de fichiers ou de sockets, les messages workers ne sont **pas binary-safe**. Une chaîne contenant `\0` ou des octets non acceptés comme UTF-8 est refusée.

Tables listes et objets

Une table est transférable sous l'une de ces deux formes.

Liste dense non vide

```
{ "a", "b", "c" }
```

Toutes les clés doivent être les entiers consécutifs `1..n`, sans trou ni autre clé.

Objet à clés string

```
{
    name = "alice",
    age = 30,
}
```

Toutes les clés doivent être des strings acceptées par le validateur de texte.

Une table vide est sérialisée comme un **objet vide** `{}`, pas comme un array vide. Il n'existe pas de marqueur équivalent à `babet.json.empty_array` dans le transport workers.

Les tables mixtes ou creuses sont refusées :

```
{ [1] = "a", [3] = "c" }      -- trou
{ [1] = "a", label = "x" }   -- liste + map
{ [0] = "zero" }              -- clé numérique hors 1..n
```

Valeurs refusées

Ne traversent pas les états Lua :

- fonctions et closures ;
- userdata : socket, statement SQLite, watcher, job, fichier, etc. ;
- la sentinelle lightuserdata réservée à SQLite `babet.sqlite.NULL` ;
- coroutines/threads Lua ;
- tables cycliques ;
- tables mixtes ou creuses ;
- clés non string dans une table objet ;
- nombres `NaN` ou infinis ;
- chaînes avec NUL ou octets non acceptés comme UTF-8 ;
- structures imbriquées ou développées au-delà des limites de profondeur, de nœuds ou d'octets.

Un chemin, une URL ou une configuration sérialisable doit être transmis à la place de l'objet système lui-même. Le worker ouvre ensuite sa propre ressource.

La sentinelle NULL est refusée avec un diagnostic qui nomme explicitement `babet.sqlite.NULL`. Convertis-la en valeur métier comme `{ kind = "sql-null" }` si cette intention doit traverser une frontière worker ou channel.

```lua
-- Incorrect : socket est un userdata.
babet.workers.spawn(code, { socket = peer })

-- Correct : transmettre les paramètres de connexion.
babet.workers.spawn(code, {
    host = "127.0.0.1",
    port = 9000,
})
```

Copies, identité, profondeur et budgets

Le transport crée une copie par valeur :

- aucune référence Lua n'est partagée ;
- les métatables sont perdues ;
- deux références vers la même sous-table deviennent deux tables séparées ;
- les cycles sont refusés ;
- la profondeur maximale est de 32 niveaux selon le compteur interne de sérialisation ;
- une opération peut développer au plus **1 000 000 de valeurs JSON** ;
- la représentation estimée doit rester dans un budget conservateur de **64 Mio**.

```lua
local shared = { value = 1 }
local args = { a = shared, b = shared }

-- Dans le worker, worker.args.a et worker.args.b ont le même contenu,
-- mais ne sont pas la même table.
```

La limite de profondeur protège les piles C++ et Lua. Les deux budgets supplémentaires empêchent une petite structure Lua de produire une quantité exponentielle de données lorsque la même sous-table ou la même chaîne est référencée de nombreuses fois.

Chaque valeur développée consomme exactement un nœud et un coût fixe de 32 octets. Chaque occurrence d'une chaîne, y compris une clé d'objet, consomme également un majorant de `6 × taille_en_octets + 2`. Le facteur six couvre le pire échappement JSON d'un octet de contrôle. Les deux plafonds se recouvrent volontairement : ils bornent à la fois le nombre d'objets du DOM JSON et la charge textuelle développée.

Ces limites s'appliquent indépendamment à chaque opération : arguments de `spawn`, résultat final, `job:send`, `worker.send` ou `channel:send`. Une valeur refusée n'est pas ajoutée à la queue. Le contrôle identique effectué à la réception vérifie la cohérence du message ; la borne mémoire principale est appliquée à l'émission, avant la copie des chaînes et avant la publication du JSON interne.

Les plafonds font partie du contrat public. Les réduire dans une version ultérieure pourrait rendre invalides des transferts auparavant acceptés et devrait être annoncé comme un changement incompatible.

Résultat final du worker

Quand le chunk termine normalement, seule sa **première** valeur de retour est sérialisée.

```lua
return "first", "second", "third"
```

Le parent reçoit uniquement `"first"`.

Pour transporter plusieurs résultats, utilise une table :

```lua
return {
    value = 42,
    elapsed = 0.12,
}
```

Sans `return`, ou avec `return nil`, le résultat métier est `nil`.

Une erreur Lua non attrapée devient l'état `error`. Le texte transmis est un diagnostic, mais Babet n'ajoute pas automatiquement de traceback. Le worker peut utiliser `xpcall(..., debug.traceback)` s'il souhaite en construire un.

`status()` — observer sans consommer

```
local state = job:status()
```

`status()` renvoie exactement l'une des chaînes suivantes :

État	Signification
`"running"`	le chunk ou son nettoyage est encore en cours
`"done"`	le worker a terminé normalement
`"error"`	le worker a terminé avec une erreur

La méthode ne bloque pas, ne rejoint pas la pthread et ne consomme jamais le résultat. Elle reste donc utilisable avant et après `join()` :

```
while job:status() == "running" do
    update_interface()
    babet.sleep(10, "ms")
end

local ok, result = job:join()
assert(ok, result)
assert(job:status() == "done")
```

`status()` ne distingue volontairement pas un worker en cours d'annulation : tant que le chunk n'est pas terminé, l'état reste `"running"`.

`done()` — test booléen non consommant

```
if job:done() then
    print("le worker a terminé")
end
```

`done()` renvoie `false` tant que le statut interne vaut `"running"`, puis `true` pour `"done"` comme pour `"error"`. La méthode ne rejoint pas la pthread, ne désérialise pas le résultat et ne le consomme jamais. Elle est donc adaptée aux boucles d'événements qui ont seulement besoin de savoir si un `join()` ou un `poll()` final peut maintenant être tenté.

`join(timeout?)` — attendre et consommer

```
local ok, value_or_reason = job:join(timeout?)
```

Le timeout utilise les mêmes secondes que les queues :

- absent ou `nil` : attente indéfinie ;
- `0` : test immédiat ;
- nombre fini strictement positif jusqu'à `86400` : attente bornée ;
- valeur négative, NaN, infinie, chaîne numérique ou argument supplémentaire : erreur Lua levée.

Trois états de retour existent :

```
true, result       -- terminaison normale, résultat consommé
false, err         -- terminaison en erreur, résultat consommé
nil, "timeout"     -- worker encore actif, rien n'est consommé
```

Exemple avec reprise après expiration :

```
local ok, value = job:join(0.05)

if ok == nil then
    assert(value == "timeout")
    print("le worker continue")

    -- Le même job reste entièrement utilisable.
    ok, value = job:join(2)
end

if ok then
    print("résultat", value)
else
    io.stderr:write("worker: ", value, "\n")
end
```

Un timeout :

- ne ferme aucune queue ;
- ne demande aucune annulation ;
- ne rejoint pas la pthread ;
- ne consomme ni le résultat ni l'erreur ;
- permet encore `status`, `send`, `recv`, `close`, `cancel` et un nouvel appel à `join`.

Le timeout borne uniquement cet appel à `join()`. Si le worker reste actif puis que sa dernière référence est collectée, notamment pendant la fermeture de l'état Lua, son `__gc` doit encore effectuer un `pthread_join()` sans timeout et peut donc bloquer la fin du programme.

Même lorsqu'un worker retourne légitimement `nil`, un succès reste reconnaissable grâce à `ok == true`.

`poll()` — tester et consommer

```
local state, value = job:poll()
```

state	value	Consommé
`"running"`	`nil`	non
`"done"`	résultat, éventuellement `nil`	oui
`"error"`	message	oui

```
while true do
    local state, value = job:poll()
    if state == "running" then
        babet.sleep(10, "ms")
    elseif state == "done" then
        print("terminé", value)
        break
    else
        io.stderr:write("worker: ", value, "\n")
        break
    end
end
```

Lorsque l'état n'est plus `running`, `poll()` rejoint rapidement la pthread déjà terminée et consomme son résultat.

Ne pas combiner `poll()` terminé puis `join()`

Cette séquence est incorrecte :

```
local state = job:poll()
if state ~= "running" then
    local ok, result = job:join() -- résultat déjà consommé
end
```

Utilise directement les deux valeurs de `poll()` :

```
local state, value = job:poll()
if state == "done" then
    use_result(value)
elseif state == "error" then
    report_error(value)
end
```

Ou utilise `poll()` uniquement tant qu'il renvoie `running`, puis choisis une architecture dans laquelle un autre mécanisme indique qu'il faut appeler `join()` sans avoir consommé le résultat. Dans la plupart des boucles, utiliser `poll()` jusqu'au résultat final est plus simple.

Messages parent vers worker

```
local ok, err = job:send(value, timeout?)
```

`job:send` sérialise `value`, puis l'ajoute à l'inbox du worker en FIFO.

Résultats :

- `(true, nil)` : message accepté ;
- `(false, "full")` : queue pleine avec timeout `0` ;
- `(false, "timeout")` : queue restée pleine jusqu'à l'échéance ;
- `(false, "closed")` : inbox fermée par `close()` ou par la fin du worker ;
- `(false, "cancelled")` : annulation demandée par `cancel()` ;
- `(nil, err)` : valeur non sérialisable ou erreur interne de sérialisation.

```
local ok, err = job:send({ command = "scan", path = "/tmp" }, 1)
if not ok then
    io.stderr:write("send: ", err, "\n")
end
```

Le succès signifie seulement que le message est en queue, pas que le worker l'a déjà traité.

Messages worker vers parent

```
local ok, value_or_err = job:recv(timeout?)
```

`job:recv` retire le plus ancien message de l'outbox.

Résultats :

- `(true, value)` : message reçu ; `value` peut être `nil` ;
- `(false, "empty")` : queue vide avec timeout `0` ;
- `(false, "timeout")` : aucun message avant l'échéance ;
- `(false, "closed")` : queue fermée et entièrement drainée.

Lorsqu'une queue est fermée mais contient encore des messages, ceux-ci sont rendus avant `"closed"`.

```
while true do
    local ok, message = job:recv(0)
    if ok then
```

```
        process(message)
    elseif message == "empty" then
        break
    elseif message == "closed" then
        break
    else
        error(message)
    end
end
```

API `worker.send`, `worker.recv` et `worker.cancelled`

Dans le chunk :

```
local ok, message_or_err = worker.recv(timeout?)
local ok, err = worker.send(value, timeout?)
local cancelled = worker.cancelled()
```

`worker.recv` lit l'inbox alimentée par `job:send`. `worker.send` écrit dans l'outbox lue par `job:recv`. `worker.cancelled()` lit sans blocage le drapeau posé par `job:cancel()`.

Après une annulation :

- `worker.cancelled()` renvoie `true` ;
- le prochain `worker.recv()` renvoie `(false, "cancelled")` et ne livre plus les commandes qui attendaient encore dans l'inbox ; lorsque l'annulation devient visible juste après un `pop` réussi, cette commande extraite est volontairement abandonnée plutôt que livrée après la frontière de protocole ; une commande déjà rendue au Lua peut naturellement être en cours ;
- `worker.send()` reste disponible pour un dernier résultat, un diagnostic ou un accusé d'arrêt si l'outbox possède déjà une place ; un envoi bloqué sur une outbox pleine est réveillé et renvoie `(false, "cancelled")` ;
- le chunk décide lui-même quand et comment terminer.

Les règles FIFO, capacités, timeouts et fermeture restent symétriques en dehors de ce cas d'annulation.

Différence de convention sur une erreur de sérialisation :

- `job:send` renvoie `(nil, err)` ;
- `worker.send` renvoie `(false, err)`.

Cette différence conserve la convention booléenne du code worker.

```
local job = assert(babet.workers.spawn([[
    while not worker.cancelled() do
        local ok, message = worker.recv(0.2)

        if ok then
            process(message)
        elseif message == "cancelled" then
            break
        elseif message ~= "timeout" then
            return { error = message }
        end
    end

    worker.send({ stopped = true })
    return "cancelled"
]]))
```

Channels directs partagés

Un channel est une file bornée partagée entre le parent et autant de workers que nécessaire. Contrairement à l'inbox et à l'outbox d'un `job`, il n'est pas attaché à un worker particulier : tout handle du même channel peut produire ou consommer des messages.

Le channel est :

- thread-safe ;
- multi-producteurs et multi-consommateurs ;
- borné en nombre de messages ;
- FIFO selon l'ordre d'insertion effectif ;
- fermé explicitement avec drainage des messages déjà présents.

Créer un channel et choisir sa capacité

```
local channel, err = babet.workers.channel(opts?)
```

L'unique option est :

```
{
    capacity = 64,
}
```

- défaut : 64 messages ;
- minimum : 1 ;
- maximum : 1 000 000 ;
- la valeur doit être un entier Lua strict ;
- la capacité compte les messages, pas les octets.

```
local tasks = assert(babet.workers.channel({
    capacity = 16,
}))
```

Les options inconnues, les clés non textuelles, les noms contenant un octet NUL et les capacités hors limites lèvent une erreur Lua. Une erreur d'allocation ou d'initialisation renvoie `(nil, err)`.

`send` et `recv`

```
local ok, err = channel:send(value, timeout?)
local ok, value_or_reason = channel:recv(timeout?)
```

`send` renvoie :

- `(true, nil)` lorsque le message a été ajouté ;
- `(false, "full")` pour une tentative immédiate sur un channel plein ;
- `(false, "timeout")` lorsque l'échéance positive expire ;
- `(false, "closed")` après fermeture ;
- `(false, "cancelled")` dans un worker dont le job a été annulé ;
- `(nil, err)` si la valeur n'est pas sérialisable ou en cas d'erreur interne.

`recv` renvoie :

- `(true, value)` lorsqu'un message est retiré ; `value` peut être `nil` ;
- `(false, "empty")` pour une tentative immédiate sur un channel vide ;
- `(false, "timeout")` lorsque l'échéance positive expire ;
- `(false, "closed")` lorsque le channel est fermé **et entièrement drainé** ;
- `(false, "cancelled")` dans un worker dont le job a été annulé ;
- `(nil, err)` lors d'une anomalie interne de désérialisation.

```
local channel = assert(babet.workers.channel({ capacity = 1 }))

assert(channel:send(nil, 0))
```

```lua
local ok, value = channel:recv(0)
assert(ok == true)
assert(value == nil)

local got, reason = channel:recv(0)
assert(got == false and reason == "empty")
```

Les mêmes valeurs JSON que pour les workers sont acceptées. Un channel, un job, un socket, `babet.sqlite.NULL` ou tout autre userdata/lightuserdata ne peut pas être envoyé comme message. Un envoi refusé ne publie rien dans le channel. Pour une capture Selenium binaire, envoie de préférence son chemin, ou conserve sa forme Base64 puis utilise `babet.base64.decode()` au point de consommation.

Dans un worker, `job:cancel()` réveille également un `channel:send()` ou `channel:recv()` bloqué pour **ce worker uniquement**. L'appel renvoie alors `(false, "cancelled")` sans fermer le channel partagé ni perturber les autres participants. Après annulation, les nouveaux `send` et `recv` de ce worker renvoient aussi `"cancelled"` ; utilise `worker.send()` pour un dernier accusé d'arrêt vers le parent.

`close` et `is_closed`

```lua
local ok, err = channel:close()
local closed = channel:is_closed()
```

`close()` est global et idempotent. Dès son retour :

- aucun nouvel envoi n'est accepté ;
- `is_closed()` renvoie `true` ;
- tous les `send()` et `recv()` bloqués sont réveillés ;
- les messages déjà en file restent disponibles ;
- après le dernier message, `recv()` renvoie `(false, "closed")`.

```lua
local channel = assert(babet.workers.channel({ capacity = 2 }))
assert(channel:send("one"))
assert(channel:send("two"))
assert(channel:close())
assert(channel:close()) -- idempotent
assert(channel:is_closed())

local ok1, one = channel:recv()
local ok2, two = channel:recv()
local ok3, reason = channel:recv()

assert(ok1 and one == "one")
assert(ok2 and two == "two")
assert(not ok3 and reason == "closed")
```

Concurrence, ordre et durée de vie

Tous les handles créés pour un même channel référencent la même structure C++. Le parent et chaque état Lua possèdent néanmoins leur propre userdata.

- détruire un handle local ne ferme pas le channel pour les autres ;
- `close()` ferme explicitement la ressource pour tous les handles ;
- lorsque la dernière référence disparaît, Babet ferme la queue, réveille les éventuels waiters et détruit les messages restants ;
- la sortie d'un worker ne rend donc pas invalides les handles encore détenus par le parent ou d'autres workers.

Le FIFO décrit l'ordre d'insertion réel. Deux `send()` concurrents provenant de producteurs différents n'ont pas d'ordre déterministe garanti. En revanche, les envois successifs d'un même producteur restent ordonnés.

Une grande capacité n'impose aucune limite à la taille individuelle d'un message. Comme le transport produit une représentation JSON intermédiaire, les messages volumineux sont copiés et peuvent consommer beaucoup de mémoire. Pour des données binaires importantes, un fichier publié atomiquement et son chemin sont généralement préférables.

Timeouts et raisons d'échec

Les quatre méthodes de messages et `join(timeout?)` partagent les mêmes règles de validation des durées :

`timeout` Lua	Comportement
absent ou `nil`	attente indéfinie
`0`	tentative immédiate, non bloquante
`0 < t <= 86400`	attente d'au plus `t` secondes
négatif, NaN, Inf ou `> 86400`	erreur Lua levée
autre type, y compris `"1"`	erreur Lua levée

La résolution interne est la milliseconde. Une durée strictement positive est arrondie vers le haut : `0.0005` seconde attend au moins 1 ms et n'est pas transformé en mode non bloquant.

Les raisons de queue sont :

Raison	Signification
`"full"`	envoi immédiat, queue pleine
`"empty"`	réception immédiate, queue vide
`"timeout"`	échéance positive atteinte
`"closed"`	envoi sur queue fermée, ou réception sur queue fermée et vide
`"cancelled"`	`job:send`, `worker.recv` ou méthode de channel appelée dans un worker annulé

Des diagnostics internes comme `"out of memory"` ou `"internal mutex error"` peuvent exceptionnellement apparaître à la place.

`close()` et cycle de vie des queues

```lua
local ok, err = job:close()
```

`job:close()` :

- ferme uniquement l'**inbox** parent vers worker ;
- est idempotent ;
- renvoie `(true, nil)` ;
- débloque un `worker.recv()` en attente, qui obtient `(false, "closed")` une fois les messages déjà présents drainés ;
- rend les futurs `job:send()` impossibles avec `(false, "closed")` ;
- laisse l'outbox ouverte afin que le parent puisse encore recevoir les derniers messages du worker.

Il ne termine pas de force le thread et ne consomme pas son résultat final.

Le worker ferme automatiquement **les deux** queues lorsqu'il termine, en succès ou en erreur. Le parent peut néanmoins drainer les messages déjà placés dans l'outbox avant de recevoir `"closed"`.

Pour demander proprement à un worker qui attend des commandes de finir :

```
assert(job:close())
local ok, result = job:join()
```

Le worker doit traiter `worker.recv() == false, "closed"` comme une condition de sortie.

`cancel()` et annulation coopérative

```
local ok, err = job:cancel()
```

`job:cancel()` :

- pose un drapeau atomique visible par `worker.cancelled()` ;
- ferme l'inbox afin de réveiller immédiatement un `worker.recv()` bloqué ;
- réveille aussi le `channel:send()` ou `channel:recv()` actuellement bloqué dans ce worker, sans fermer le channel global ;
- fait renvoyer `(false, "cancelled")` aux futurs `job:send()` ;
- laisse l'outbox ouverte, accepte un dernier message si une place est immédiatement disponible et réveille un `worker.send()` bloqué sur une outbox pleine avec `(false, "cancelled")` ;
- ne rejoint pas le thread ;
- ne consomme pas le résultat final ;
- est idempotent et renvoie toujours `(true, nil)` pour un job valide.

```
assert(job:cancel())

local got, final = job:recv(1)
if got then
    print("dernier message", final)
end

local ok, result = job:join(2)
if ok == nil then
    error("le worker n'a pas coopéré dans le délai")
end
assert(ok, result)
```

Ce mécanisme n'utilise jamais `pthread_cancel()` et n'interrompt pas brutalement Lua, SQLite, un verrou C++ ou une transaction. Un worker doit donc consulter `worker.cancelled()` ou revenir régulièrement dans `worker.recv()`.

Un worker bloqué dans un appel système non interruptible ou une boucle qui ne vérifie jamais le drapeau peut rester actif. Les attentes de channel Babet et un `worker.send()` attendant une place dans l'outbox sont réveillés par l'annulation. `join(timeout)` permet au parent de rester borné dans les autres cas, mais Babet ne force pas la terminaison.

`close()` et `cancel()` sont distincts :

- `close()` signifie « aucune autre commande » et laisse drainer les commandes déjà en file ; `worker.recv()` finit par renvoyer `"closed"` ;
- `cancel()` signifie « abandonne le travail en cours dès que possible » ; les commandes encore en inbox ne sont plus livrées et `worker.recv()` renvoie `"cancelled"`. Une commande déjà extraite avant l'annulation peut être en cours de traitement : l'arrêt reste coopératif.

Chargement des modules et environnement du worker

Chaque thread crée un nouvel état Lua et :

- ouvre les bibliothèques standard Lua ;

— enregistre le namespace complet `babet` ;
— expose les modules bundlés via `require` ;
— configure le chargement des modules utilisateur de la même manière que le projet parent, en mode dossier ou embarqué ;
— expose `worker.args`, `worker.channels`, `worker.send`, `worker.recv` et `worker.cancelled` ;
— ne copie pas la table globale `arg` du parent : `arg == nil` dans le worker.

```
local job = assert(babet.workers.spawn([[
    local inspect = require("inspect")
    local mymod = require("mymod")
    return inspect({ answer = mymod.answer() })
]]))
```

Les états Lua sont isolés, mais les threads appartiennent au même processus :

— même PID ;
— même répertoire courant process-wide ;
— même environnement process-wide ;
— même locale C partagée par le processus ;
— même système de fichiers et mêmes ressources externes accessibles ;
— compte mémoire Lua mesuré séparément dans chaque état.

Babet interdit les mutations de cwd, d'environnement et de locale via ces API Lua après le premier `spawn` ou la première tentative de chargement GTK.

`os.exit(...)` lève une erreur Lua dans chaque worker, y compris ses coroutines et les workers imbriqués. Une erreur non interceptée fait renvoyer `(false, err)` à `job:join()` ; `pcall` peut l'intercepter sans fermer l'état Lua du worker. Utilise `return` pour terminer normalement un worker. Cette règle vaut aussi pour `require("os").exit` et les alias sauvegardés. Dans le CLI et les exécutables générés, `os.exit` conserve les arguments de Lua, restaure l'interface et termine le processus sans callbacks natifs `atexit`. Sans `close=true`, il n'attend pas les workers ; avec une valeur vraie de `close`, la fermeture Lua peut attendre leurs appels natifs non coopératifs. Voir le contrat de sortie, notamment pour l'embedding.

Nombre de CPU disponibles

```
local count = babet.workers.cpu_count()
assert(math.type(count) == "integer" and count >= 1)
```

`cpu_count()` renvoie le nombre de CPU que le processus courant peut utiliser. Sous Linux, Babet consulte d'abord l'affinité effective avec `sched_getaffinity()`, ce qui respecte les ensembles de CPU imposés au processus. Si cette information n'est pas disponible, `_SC_NPROCESSORS_ONLN` est utilisé, avec un dernier repli à `1`.

La fonction n'accepte aucun argument. Elle sert notamment de valeur par défaut à `workers.pool()`.

Pool borné réutilisable

`workers.pool()` construit un nombre fixe de workers persistants. Chaque worker crée une seule pthread et un seul `lua_State`, puis traite plusieurs tâches successives reçues par un channel interne. Le pool évite ainsi le coût d'un `workers.spawn()` complet pour chaque petite opération tout en conservant une borne stricte sur la concurrence et le nombre de tâches en vol.

Le pool est implémenté en Lua embarqué au-dessus des primitives natives `workers.spawn()` et `workers.channel()`. Il suit donc les mêmes limites de sérialisation, de profondeur, de budget mémoire et d'annulation coopérative.

Création et options

```
local pool, err = babet.workers.pool({
    size = math.min(babet.workers.cpu_count(), 8),
    queue_capacity = 64,
    channels = {
        progress = progress_channel,
    },
})
assert(pool, err)
```

Options acceptées :

Option	Défaut	Contrat
`size`	`min(cpu_count(), 1024)`	entier de 1 à `1024`
`queue_capacity`	`max(64, size * 4)`	entier de 1 à `1 000 000`
`channels`	aucune	table nom -> channel partagée avec chaque tâche

Les noms `__babet_pool_tasks` et `__babet_pool_results` sont réservés aux deux channels internes. Une option ou un nom inconnu est refusé immédiatement.

Le nombre maximal de tâches acceptées mais pas encore collectées vaut :

```
min(queue_capacity + size, 1 000 000)
```

Cette borne inclut les tâches en cours d'exécution et celles encore présentes dans la file.

Soumission et tâches

```
local task, err = pool:submit([[
    return worker.args.left + worker.args.right
]], { left = 20, right = 22 }, 1.0)
assert(task, err)

assert(task:done() == false or task:done() == true)
local ok, result = task:join(2)
assert(ok and result == 42)
```

`submit(code, args?, timeout?)` accepte le même type de chunk texte et la même table sérialisable que `workers.spawn()`. Le timeout porte sur l'admission dans le pool : il peut être nécessaire d'attendre qu'une place se libère dans la borne des tâches en vol ou dans le channel de travail.

Une tâche expose :

- `task:done()` : booléen non consommant ;
- `task:status()` : `"running"`, `"done"` ou `"error"` ;
- `task:poll()` : test non bloquant qui consomme un résultat terminé ;
- `task:join(timeout?)` : attente puis consommation du résultat.

Comme pour un job normal, la première valeur retournée est la seule transmise et le résultat se consomme une seule fois. Une erreur de chargement ou l'exception Lua d'une tâche devient l'erreur de cette tâche sans arrêter le worker persistant. Si la valeur retournée n'est pas sérialisable, le pool la convertit également en erreur de tâche puis continue à traiter les suivantes.

Isolation et réutilisation des états

Chaque tâche reçoit une table globale fraîche dont `_G` pointe vers elle-même. Une affectation globale ordinaire ne fuit donc pas vers la tâche suivante :

```lua
local first = assert(pool:submit("temporary = 42; return temporary"))
local second = assert(pool:submit("return temporary"))

assert(select(2, first:join()) == 42)
assert(select(2, second:join()) == nil)
```

Les bibliothèques et le cache `package.loaded` appartiennent néanmoins à l'état Lua persistant du worker. Modifier explicitement une table de module partagée, `package.loaded`, le registre C d'un module ou une ressource externe peut donc être visible par une tâche ultérieure exécutée sur le même worker. Le pool isole les globales ordinaires ; il ne recrée pas un état Lua complet par tâche.

Dans une tâche, la table `worker` expose volontairement seulement :

```lua
worker.args
worker.channels
worker.cancelled()
```

Les inbox/outbox privées `worker.send()` et `worker.recv()` ne font pas partie du contrat des tâches de pool. Utilise `opts.channels` pour une communication persistante supplémentaire.

Contre-pression et timeouts

```lua
local pool = assert(babet.workers.pool({
    size = 1,
    queue_capacity = 1,
}))

local a = assert(pool:submit("babet.sleep(1); return 'a'"))
local b = assert(pool:submit("babet.sleep(1); return 'b'"))
local c, reason = pool:submit("return 'c'", nil, 0)
assert(c == nil and reason == "timeout")
```

Avec un worker et une place de queue, deux tâches au maximum sont en vol : une en cours et une en attente. Un timeout nul est non bloquant. Une valeur strictement positive, au plus `86400`, utilise une deadline monotone et peut renvoyer `"timeout"`. Un timeout absent attend tant que les workers restent sains.

Le pool collecte les résultats pendant les attentes de soumission et de fermeture. La capacité du channel de résultats est égale à la borne des tâches en vol, de sorte qu'un worker ne reste pas bloqué simplement parce que le parent n'a pas encore appelé `join()` sur chaque tâche.

Fermeture, annulation et `join`

`pool:close(timeout?)` refuse les nouvelles soumissions et place un marqueur d'arrêt par worker **après** toutes les tâches déjà acceptées. Le FIFO du channel garantit donc leur traitement avant l'arrêt normal des threads.

`pool:join(timeout?)` appelle automatiquement `close()` si nécessaire, collecte tous les résultats en attente puis joint chaque worker persistant. Un timeout ne consomme ni les résultats de tâche encore disponibles ni les jointures de workers déjà terminées ; le même pool peut être rejoint à nouveau pour achever le nettoyage :

```lua
local task = assert(pool:submit("return 42"))
assert(pool:join(5))
assert(select(2, task:join(0)) == 42)
```

`pool:cancel()` ferme immédiatement l'admission, marque les tâches encore en vol comme `"cancelled"`, ferme le channel de travail et demande l'annulation de chaque worker. Une tâche déjà en cours ne peut s'arrêter que si son code revient ou consulte `worker.cancelled()`. Il n'existe ni `pthread_cancel()` ni arrêt forcé.

Un pool rejoint ne peut pas être rejoint une seconde fois. `pool:stats()` renvoie notamment `size`, `queue_capacity`, `max_pending`, `pending`, `accepting`, `closing`, `cancelled` et `joined`.

Channels partagés dans les tâches

```
local progress = assert(babet.workers.channel({ capacity = 16 }))
local pool = assert(babet.workers.pool({
    size = 2,
    channels = { progress = progress },
}))

local task = assert(pool:submit([[
    assert(worker.channels.progress:send({ percent = 100 }))
    return "done"
]]))

local received, message = progress:recv(2)
assert(received and message.percent == 100)
assert(task:join(2))
assert(pool:join(2))
```

Les handles sont partagés avec tous les workers du pool et restent soumis au contrat FIFO, borné et multi-producteurs/multi-consommateurs des channels normaux. Le pool ne ferme jamais automatiquement les channels fournis par l'utilisateur.

Exemples complets

Calcul simple avec `join`

```
local job, err = babet.workers.spawn([[
    return 21 * 2
]])
assert(job, err)

local ok, result = job:join()
assert(ok, result)
print(result) -- 42
```

Arguments structurés

```
local job = assert(babet.workers.spawn([[
    local total = 0
    for _, value in ipairs(worker.args.values) do
        total = total + value
    end
    return {
        name = worker.args.name,
        total = total,
    }
]], {
    name = "batch A",
    values = { 10, 20, 12 },
}))

local ok, result = job:join()
assert(ok, result)
print(result.name, result.total)
```

Suivi non bloquant avec `poll`

```
local job = assert(babet.workers.spawn([[
    babet.sleep(200, "ms")
    return "ready"
]]))

while true do
    local state, value = job:poll()

    if state == "running" then
        update_ui()
        babet.sleep(10, "ms")
    elseif state == "done" then
        print(value)
        break
    else
        error(value)
    end
end
```

Il ne faut pas appeler `join()` après la branche `done` ou `error` : le résultat a déjà été consommé par `poll()`.

Worker persistant requête/réponse

```
local job = assert(babet.workers.spawn([[
    while true do
        local ok, request = worker.recv()
        if not ok then
            return "parent closed inbox"
        end

        if request.op == "stop" then
            return "stopped"
        elseif request.op == "square" then
            local sent, err = worker.send({
                id = request.id,
                value = request.value * request.value,
            })
            if not sent then
                return { error = err }
            end
        end
    end
]], nil, {
    inbox_capacity = 16,
    outbox_capacity = 16,
}))

for i = 1, 5 do
    assert(job:send({ id = i, op = "square", value = i }))
end

for _ = 1, 5 do
    local ok, response = job:recv(2)
    assert(ok, response)
```

```
    print(response.id, response.value)
end

assert(job:send({ op = "stop" }))
local ok, result = job:join()
assert(ok, result)
print(result)
```

Envoyer et recevoir `nil`

Le booléen de statut distingue un vrai message `nil` d'une queue vide.

```
local job = assert(babet.workers.spawn([[
    local ok, value = worker.recv()
    assert(ok)
    assert(value == nil)

    assert(worker.send(nil))
    return "done"
]]))

assert(job:send(nil))

local got, value = job:recv()
assert(got == true)
assert(value == nil)

local ok, result = job:join()
assert(ok and result == "done")
```

Fermeture puis drainage de l'outbox

```
local job = assert(babet.workers.spawn([[
    assert(worker.send("ready"))

    local ok, reason = worker.recv()
    assert(not ok and reason == "closed")

    assert(worker.send("closing"))
    return "finished"
]]))

local ok, first = job:recv(1)
assert(ok and first == "ready")

assert(job:close()) -- ferme seulement l'inbox

local got, last = job:recv(1)
assert(got and last == "closing")

local joined, result = job:join()
assert(joined and result == "finished")
```

Contre-pression et timeout

```
local job = assert(babet.workers.spawn([[
    babet.sleep(500, "ms")
```

```
    while true do
        local ok, message = worker.recv()
        if not ok then return "closed" end
        if message == "stop" then return "done" end
    end
]], nil, {
    inbox_capacity = 2,
}))

assert(job:send("one", 0))
assert(job:send("two", 0))

local ok, reason = job:send("three", 0)
assert(ok == false and reason == "full")

local waited, why = job:send("three", 0.1)
assert(waited == false and why == "timeout")

job:close()
job:join()
```

Parent vers worker avec un channel

```
local tasks = assert(babet.workers.channel({ capacity = 8 }))

local job = assert(babet.workers.spawn([[
    local ok, task = worker.channels.tasks:recv(2)
    assert(ok, task)
    return task.left + task.right
]], nil, {
    channels = { tasks = tasks },
}))

assert(tasks:send({ left = 20, right = 22 }, 2))

local ok, result = job:join(2)
assert(ok and result == 42)
```

Worker vers worker sans relais du parent

```
local tasks = assert(babet.workers.channel({ capacity = 16 }))
local results = assert(babet.workers.channel({ capacity = 16 }))

local producer = assert(babet.workers.spawn([[
    for i = 1, 10 do
        assert(worker.channels.tasks:send({ id = i, value = i * 10 }, 2))
    end
    return "producer done"
]], nil, {
    channels = { tasks = tasks },
}))

local consumer = assert(babet.workers.spawn([[
    for _ = 1, 10 do
        local ok, task = worker.channels.tasks:recv(2)
        assert(ok, task)
```

```
        assert(worker.channels.results:send({
            id = task.id,
            result = task.value * 2,
        }, 2))
    end
    return "consumer done"
]], nil, {
    channels = { tasks = tasks, results = results },
}))

for expected = 1, 10 do
    local ok, item = results:recv(2)
    assert(ok, item)
    assert(item.id == expected)
    assert(item.result == expected * 20)
end

assert(producer:join(2))
assert(consumer:join(2))
assert(tasks:close())
assert(results:close())
```

Le message producteur vers consommateur ne transite jamais par le parent.

Plusieurs producteurs et consommateurs

```
local tasks = assert(babet.workers.channel({ capacity = 32 }))
local results = assert(babet.workers.channel({ capacity = 32 }))
local jobs = {}

for producer_id = 1, 2 do
    jobs[#jobs + 1] = assert(babet.workers.spawn([[
        for i = 1, 100 do
            assert(worker.channels.tasks:send({
                producer = worker.args.id,
                sequence = i,
            }, 2))
        end
        return true
    ]], { id = producer_id }, {
        channels = { tasks = tasks },
    }))
end

for _ = 1, 2 do
    jobs[#jobs + 1] = assert(babet.workers.spawn([[
        while true do
            local ok, task = worker.channels.tasks:recv(2)
            if not ok then
                assert(task == "closed")
                break
            end
            assert(worker.channels.results:send(task, 2))
        end
        return true
    ]], nil, {
        channels = { tasks = tasks, results = results },
```

```
    }))
end

assert(jobs[1]:join(3))
assert(jobs[2]:join(3))
assert(tasks:close())

local seen = {}
for _ = 1, 200 do
    local ok, item = results:recv(3)
    assert(ok, item)
    local key = item.producer .. ":" .. item.sequence
    assert(not seen[key])
    seen[key] = true
end

assert(results:close())
assert(jobs[3]:join(3))
assert(jobs[4]:join(3))
```

Avec plusieurs consommateurs, la répartition des messages dépend de l'ordonnancement des threads ; elle n'est pas prédictible, mais chaque message n'est retiré que par un seul consommateur.

Annulation coopérative native

```
local job = assert(babet.workers.spawn([[
    local total = 0

    for i = 1, worker.args.limit do
        total = total + expensive_step(i)

        if i % 1000 == 0 and worker.cancelled() then
            worker.send({
                stopped = true,
                partial = total,
            })
            return {
                cancelled = true,
                partial = total,
            }
        end
    end

    return { cancelled = false, total = total }
]], { limit = 1000000 }))

-- Plus tard :
assert(job:cancel())

-- L'outbox reste drainable après cancel().
local got, progress = job:recv(0.5)
if got then
    print("partiel", progress.partial)
end

local ok, result = job:join(2)
```

```
if ok == nil then
    error("annulation non observée dans les deux secondes")
end
assert(ok, result)
```

Exemple combinant `worker.recv()` et le drapeau :

```
local job = assert(babet.workers.spawn([[
    while not worker.cancelled() do
        local ok, command = worker.recv(0.1)

        if ok then
            execute(command)
        elseif command == "cancelled" then
            break
        elseif command ~= "timeout" then
            return { error = command }
        end
    end

    worker.send({ stopped = true })
    return "cancelled"
]]))

assert(job:cancel())
local ok, result = job:join(2)
assert(ok and result == "cancelled")
```

L'annulation reste coopérative : elle ne coupe pas arbitrairement un appel système ni du code qui ne consulte jamais le drapeau.

Pool natif pour plusieurs tâches

```
local pool = assert(babet.workers.pool({
    size = math.min(4, babet.workers.cpu_count()),
    queue_capacity = 16,
}))

local tasks = {}
for index = 1, 100 do
    tasks[index] = assert(pool:submit([[
        return worker.args.value * worker.args.value
    ]], { value = index }))
end

assert(pool:close(5))

local results = {}
for index, task in ipairs(tasks) do
    local ok, value = task:join(5)
    assert(ok, value)
    results[index] = value
end

assert(pool:join(5))
assert(results[10] == 100)
```

Le nombre de pthreads reste fixe pendant les cent tâches. `close()` peut être appelé avant la col-

lecte des résultats : le pool traite toutes les soumissions acceptées, tandis que chaque `task:join()` récupère son résultat par identifiant.

Interblocages à éviter

Worker en attente de l'inbox, parent dans `join()`

```
-- Worker
local ok, message = worker.recv() -- attend indéfiniment

-- Parent
job:join() -- attend le worker
```

Les deux côtés s'attendent. Ferme l'inbox, envoie une commande, ou demande l'annulation :

```
job:cancel()
local ok, result = job:join(2)
assert(ok ~= nil, result) -- nil signifie que le worker n'a pas encore coopéré
```

Worker bloqué sur une outbox pleine

Si le worker utilise `worker.send()` sans timeout, que l'outbox est pleine et que le parent appelle `join()` sans la drainer, aucun côté ne peut progresser.

Solutions :

— le parent lit régulièrement `job:recv()` ;
— le worker utilise un timeout fini ;
— l'outbox possède une capacité adaptée ;
— le protocole sépare clairement la phase de messages et la phase de join ;
— lors d'une annulation, le parent appelle `job:cancel()` avant le `join()` borné : l'annulation réveille désormais l'envoi bloqué sans supprimer les messages déjà placés dans l'outbox.

Producteurs bloqués sur un channel plein

Un channel borné peut créer le même cycle d'attente qu'une outbox : les producteurs attendent de la place, tandis que le parent appelle `join()` avant que les consommateurs aient drainé la queue.

Prévoir explicitement qui consomme, qui ferme le channel et à quel moment. Les boucles longues doivent utiliser des timeouts finis ou une stratégie de fermeture afin qu'une erreur d'un participant ne bloque pas tous les autres.

Oublier que `poll()` consomme

Un code qui attend `poll() ~= "running"`, puis appelle `join()`, ne se bloque pas mais perd le vrai résultat au profit de `already consumed`. Utilise les deux valeurs de `poll()`.

GC potentiellement bloquant

Abandonner la dernière référence à un worker encore actif peut déclencher un `__gc` qui ferme les queues puis attend le thread. Le GC débloque les attentes de queue, mais ne peut pas interrompre un calcul infini ou un syscall externe non borné.

Contrat d'erreur

Erreurs Lua levées par mauvais usage

Sont notamment levés :

— `code` absent ou non string ;

— `args` non table et non `nil` ;
— `opts` non table et non `nil` ;
— option inconnue ou nom d'option non chaîne dans `opts` ;
— capacités non numériques, non entières, hors `1..1 000 000` ;
— timeout non numérique, négatif, non fini ou supérieur à 86 400 secondes ;
— argument supplémentaire à `join`, `status`, `cancel` ou `worker.cancelled` ;
— appel d'une méthode sur un objet qui n'est ni le job ni le channel attendu ;
— arité, option ou capacité invalide de `workers.channel()` ;
— table `opts.channels` invalide, nom vide/non UTF-8/avec NUL ou valeur qui n'est pas un channel.

```
local ok, err = pcall(function()
    babet.workers.spawn(42)
end)
assert(not ok)
```

Échecs renvoyés par `spawn`

`spawn` renvoie `(nil, err)` pour les erreurs d'exécution rencontrées avant ou pendant la création :

— `args` non transférables ;
— dépassement d'un budget de sérialisation ;
— échec de sérialisation JSON ;
— initialisation d'une queue ou du signal de terminaison impossible ;
— échec de `pthread_create`.

```
local job, err = babet.workers.spawn("return 1", {
    fn = function() end,
})
assert(job == nil)
```

Erreurs finales du worker

`join()` renvoie `(false, err)` et `poll()` renvoie `("error", err)` pour :

— syntaxe Lua invalide ;
— erreur Lua non attrapée ;
— valeur de retour non transférable ;
— exception C++ interne interceptée ;
— corruption ou erreur interne lors de la désérialisation.

Le diagnostic ne contient pas automatiquement de traceback complet.

Sérialisation des messages

— `job:send` : `(nil, err)` si la valeur ne peut pas être sérialisée ;
— `worker.send` : `(false, err)` pour la même situation ;
— `channel:send` : `(nil, err)` pour la même situation ;
— `job:recv` et `worker.recv` : `(false, err)` sur une anomalie interne ;
— `channel:recv` : `(nil, err)` sur une anomalie interne.

Garbage collector et destruction

Le userdata possède un `__gc` de sécurité. Lorsqu'il est collecté :

1. le drapeau d'annulation est posé ;
2. l'inbox et l'outbox sont fermées ;
3. les attentes de queue sont réveillées ;
4. Babet appelle `pthread_join` si nécessaire ;

5. les primitives pthread, le signal de terminaison et les chaînes internes sont détruits.

Cette séquence évite de libérer un état encore utilisé par un thread. Elle peut toutefois bloquer si le worker ne peut pas terminer.

Ne compte pas sur le GC comme mécanisme normal de synchronisation. Conserve le job, termine le protocole, puis appelle `join()` ou consomme le résultat avec `poll()`.

Un pool est un objet Lua qui possède plusieurs userdata workers. Abandonner les dernières références au pool et à ses tâches sans `pool:join()` délègue à terme chaque thread aux finaliseurs de ces userdata et peut donc bloquer la collecte ou la fermeture de l'état Lua. Le code normal doit appeler explicitement `close()` ou `cancel()`, puis rejouer `pool:join()` jusqu'à sa réussite.

Les handles de channel possèdent également un `__gc`, mais sa portée est locale : il libère uniquement la référence détenue par cet état Lua. Il ne remplace jamais `channel:close()` et ne ferme pas la ressource tant que d'autres handles existent.

`tostring(channel)` renvoie `WorkerChannel(open)` ou `WorkerChannel(closed)` selon l'état global instantané.

`tostring(job)` renvoie une indication comme :

```
Worker(running)
Worker(done)
Worker(error)
```

Cette chaîne reflète l'état interne instantané, pas le fait que le résultat a déjà été consommé.

Signaux, environnement et concurrence

Signaux

Les six signaux gérés par `babet.signal` sont bloqués dans chaque pthread worker. `babet.signal.handle`, `ignore` et `default` lèvent une erreur dans un worker. Le thread principal reste le seul propriétaire des callbacks POSIX.

`os.execute` et `io.popen` dans un worker

Dans les états workers, ces deux fonctions lancent `/bin/sh -c` avec un masque de signaux vide dans l'enfant. Les signaux bloqués pour protéger les threads workers ne restent donc pas bloqués dans la commande, même si le shell conserve normalement le masque hérité. Le masque du worker lui-même reste inchangé. Les dispositions explicitement ignorées restent héritables selon les règles habituelles d'exécution des programmes.

Le lancement et l'attente de `os.execute` dans un worker ne modifient pas les dispositions globales de `SIGINT` et `SIGQUIT` : ils ne désactivent donc pas temporairement le gestionnaire de signaux du thread principal.

Les contrats Lua restent disponibles : `os.execute()` renvoie un booléen de disponibilité du shell ; une commande terminée renvoie `true` ou `nil`, puis `"exit"` ou `"signal"` et le code correspondant. Un échec natif de lancement ou d'attente renvoie `nil, message, errno`. `io.popen` accepte `"r"` et `"w"` et renvoie un fichier Lua ordinaire, ou `nil, message, errno` si le lancement échoue. Ses méthodes, `io.type`, `close`, `<close>` et le GC continuent de fonctionner ; la fermeture attend et récolte l'enfant. Les extrémités de pipe conservées par le parent ne sont pas héritées après un autre `exec`.

Ces appels restent bloquants. Ni `job:cancel()` ni l'expiration d'un `job:join(timeout)` ne terminent automatiquement la commande ; la fermeture d'un fichier `io.popen`, y compris pendant le GC, peut attendre sa fin. Les fonctions de l'état Lua principal conservent leur implémentation standard.

Environnement et cwd

`setenv` et `chdir` modifient un état process-wide qui ne peut pas être muté en sécurité pendant que d'autres threads l'utilisent. Babet les interdit donc après le premier `spawn` ou la première tentative de chargement GTK.

Ressources externes

Deux workers peuvent ouvrir des fichiers, bases SQLite ou sockets distincts. Lorsqu'ils ciblent la même ressource externe, la sérialisation workers ne fournit aucun verrou automatique.

Pour SQLite, chaque worker doit ouvrir sa propre connexion. `wal = true` et un `busy_timeout` adapté peuvent améliorer la concurrence, mais ne remplacent pas la conception correcte des transactions.

Coût

Chaque `spawn` crée :

— une pthread ;
— un état Lua complet ;
— les modules Babet ;
— deux queues et leurs buffers ;
— les handles des channels explicitement transmis ;
— des copies JSON des données transférées.

Évite de créer un worker pour une opération minuscule. Regroupe le travail ou utilise quelques workers persistants lorsque le protocole le permet.

Fonctionnalités absentes

Le module ne fournit pas actuellement :

— une terminaison forcée d'un worker ;
— l'annulation individuelle et forcée d'une tâche de pool ;
— le redimensionnement dynamique d'un pool après sa création ;
— des channels entre processus OS distincts ;
— de la mémoire Lua partagée ;
— le transfert de fonctions, userdata ou coroutines ;
— un format binaire pour les messages ;
— le transfert automatique de plusieurs valeurs de retour.

Le pool 2.18 est local à l'état Lua qui l'a créé. Il ne constitue pas un ordonnanceur global partagé entre plusieurs processus Babet et ne déplace pas une tâche déjà commencée d'un worker vers un autre.

Cookbook

Recettes concrètes pour des tâches de scripting courantes. Chaque recette est courte et autonome ; copie, adapte, déploie.

Surveiller un dossier et envoyer les nouveaux fichiers par email

Un dossier de dépôt qui mailise chaque fichier dès qu'il arrive. Utilise `babet.inotify` pour le réveil instantané (pas de polling), et écoute `close_write` + `moved_to` pour garantir que le fichier est complet.

```
local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))
```

```
babet.signal.handle("TERM", function()
    w:close()
    os.exit(0)
end)

while true do
    local events, err = w:read()
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            -- queue saturée : rescanne le dossier
            -- (des événements ont été perdus)
        else
            local path = "/srv/incoming/" .. ev.name
            send_email(path)
            os.remove(path)
        end
    end
end
```

S'assurer qu'un user système existe, puis basculer dessus

Check typique à l'install avant de lancer un daemon sous un utilisateur non-root.

```
local function ensure_user(name)
    if babet.user.exists(name) then return true end
    local ok, err = babet.exec("useradd", {
        "--system",
        "--home-dir", "/var/lib/" .. name,
        "--create-home",
        "--shell", "/usr/sbin/nologin",
        name,
    })
    if not ok then
        return nil, "useradd a échoué : " .. tostring(err)
    end
    return true
end

assert(ensure_user("myapp"))
local u = assert(babet.user.get("myapp"))
babet.chdir(u.home)
```

Récupérer beaucoup d'URLs en parallèle

`babet.workers` fait tourner du Lua sur plusieurs cœurs. Le temps total est proche du fetch le plus lent, pas de la somme.

```
local urls = { "https://a.example/", "https://b.example/",
               "https://c.example/", "https://d.example/" }

local jobs = {}
for i, url in ipairs(urls) do
```

```
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { error = err } end
        return { status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    print(i, ok, result and result.status or result.error)
end
```

Utiliser WebSocket comme transport WebDriver BiDi

Une fois que la session WebDriver a renvoyé son `webSocketUrl`, Babet peut transporter les messages JSON BiDi sans rien connaître de Selenium. Ce petit dispatcher séquentiel résout une commande à la fois et sépare les événements spontanés :

```
local ws = assert(babet.websocket.connect(webSocketUrl, {
    timeout = 5,
    max_message_bytes = 8 * 1024 * 1024,
}))

local next_id = 0
local function bidi_command(method, params)
    next_id = next_id + 1
    local id = next_id
    assert(ws:send_text(assert(babet.json.encode({
        id = id,
        method = method,
        params = params or {},
    }))))

    while true do
        local message = assert(ws:recv())
        assert(message.type == "text", "BiDi attend des messages JSON texte")
        local packet = assert(babet.json.decode(message.data))
        if packet.id == id then
            return packet
        end
        if packet.method then
            print("Événement BiDi :", packet.method)
        end
    end
end

local status = bidi_command("session.status", {})
print(assert(babet.json.encode(status)))
assert(ws:close())
```

Un binding Selenium complet remplacera cette boucle séquentielle par sa propre table de commandes en attente et son dispatcher d'événements. Le transport WebSocket reste générique.

Arrêt propre d'un script long-running

Attraper `SIGTERM` / `SIGINT` pour que le script puisse fermer ses ressources proprement. Fonctionne avec `systemctl stop` et Ctrl-C.

```
local log = require("logging")
log.set_level("info")

local running = true
babet.signal.handle("TERM", function()
    log.info("SIGTERM, arrêt en cours")
    running = false
end)
babet.signal.handle("INT", function()
    log.info("Ctrl-C, arrêt en cours")
    running = false
end)

while running do
    -- boucle principale ; les appels bloquants renvoient
    -- (nil, "interrupted") quand un signal géré arrive, donc la
    -- boucle sort rapidement.
    local line, err = sock:recv_line()
    if not line then
        if err == "interrupted" then break end
        log.error("recv : ", err); break
    end
    process(line)
end

sock:close()
log.info("arrêt propre")
```

Lire une config TOML, valider, persister en SQLite

```
-- Charger le TOML
local f = assert(io.open("config.toml", "r"))
local body = f:read("a"); f:close()
local cfg, perr = babet.toml.decode(body)
if not cfg then error("config.toml : " .. perr) end

-- Valider (checks manuels simples ; pour un vrai schéma, bâtis-le en Lua)
assert(type(cfg.server) == "table", "section [server] absente")
assert(type(cfg.server.host) == "string", "server.host absent")
assert(math.type(cfg.server.port) == "integer", "server.port doit être un entier")

-- Ouvrir la db, persister
local db = assert(babet.sqlite.open("state.db", { wal = true }))
db:exec([[CREATE TABLE IF NOT EXISTS config (k TEXT PRIMARY KEY, v TEXT)]])
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
        { "host", cfg.server.host })
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
        { "port", tostring(cfg.server.port) })
db:close()
```

Cache avec TTL lisible, échéances en timestamps ISO

Parser un TTL depuis une config (`"15m"`, `"2h30m"`, `"1d"`), calculer une échéance, et la logger dans un format UTC uniforme sur lequel `grep` à travers les fichiers de log marche vraiment.

```
-- Config depuis TOML, CLI, env, peu importe.
local raw_ttl = "15m"

local ttl, err = babet.time.parse_duration(raw_ttl)
if not ttl then error("TTL invalide '" .. raw_ttl .. "' : " .. err) end

local deadline = babet.now() + ttl
print(string.format("[%s] entrée cache créée, expire à %s (TTL %s)",
    babet.time.iso(),
    babet.time.iso(deadline),
    babet.time.format_duration(ttl)))

-- ...plus tard, ailleurs dans le script :
if babet.now() >= deadline then
    print("[" .. babet.time.iso() .. "] cache miss : entrée expirée")
end
```

Propriétés utiles de ce pattern :

- `babet.time.iso()` est toujours UTC, toujours 20 caractères, toujours la même forme. `grep` et `sort` marchent directement sur les timestamps.
- `parse_duration("15m")` renvoie un entier, donc `now() + ttl` est exact (pas de drift float sur les longues durées).
- `format_duration(ttl)` est aller-retour : `parse_duration( format_duration(n)) == n` pour tout `n >= 0`. Sûr à utiliser comme représentation canonique dans les logs ou les bases.

Pretty-print d'une valeur Lua

Le module `inspect` (embarqué) donne une sortie lisible pour les tables imbriquées.

```
local inspect = require("inspect")
print(inspect({ a = 1, b = { c = 2, d = { 3, 4, 5 } } }))
-- { a = 1, b = { c = 2, d = { 3, 4, 5 } } }
```

Hello there

Le script Babet minimum :

```
babet.helloThere()
```